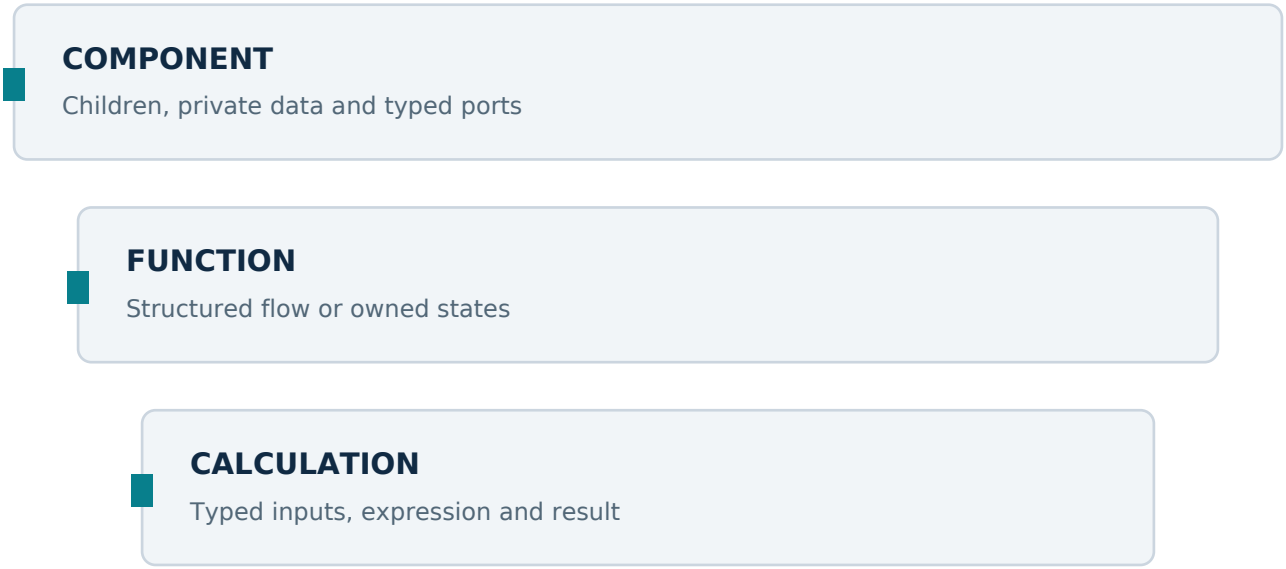
SOFTWARE ARCHITECTURE MODELING LANGUAGE

SWAML

Complete language specification

Specification 1.3.0 / Language 1.3 / Reference simulation 1.3



Typed interfaces at every software boundary.

Embedded realtime systems, services and event-driven architectures.

16 October 2026 / Catalin Ilie and SWAML contributors

Independent language design on a proposed KerML semantic foundation.

github.com/voidaliot/swaml

About this edition

VERSION 1.3.0 / RELEASE EDITION

This book contains the complete SWAML 1.3 language specification, a concern guide and every keyword in alphabetical order, formal execution rules, flat notation examples, an ontology and view guide, a reference simulation binding, the complete grammar and runnable model examples. The specification and package revision is 1.3.0. Source files begin with packages; hosts select and pin the 1.3 grammar. Explicit type definitions retain `def`; component usages form execution roots and functions own their states directly. Reusable QoS definitions reduce repeated connector policy without weakening explicit routes.

Normative chapters define syntax, validity, interface isolation, function/state behavior and mapping obligations. Tutorials and figures illustrate these rules. This independently authored language is not an OMG UML release or an AUTOSAR implementation. KerML export remains a proposed semantic mapping contract.

The reference parser, validator and deterministic logical-time simulator provide executable evidence. Production timing, runtime and platform mappings require the target binding and evidence defined in the normative chapters.

Navigation	How to use this PDF
Contents and bookmarks	Select a contents entry or use the PDF outline to open a chapter or section.
Cross-references	Links to included documents jump inside this book. Other repository links target version v1.3.0.
Code and tables	Code is selectable, numbered and wrapped without omitting source text. Tables repeat their headers across pages.
Source record	The build manifest lists every included source and its SHA-256 digest. Original source files are embedded as PDF attachments. The release includes a PDF checksum.

Original text and code: MIT license, copyright 2026 Catalin Ilie. Referenced standards retain their own terms; no standards PDFs are redistributed. Full license text appears at the end.

Contents

About this edition 2

Language reference by concern 17

Architecture selection and platform correspondence . . . 17
Behavior and state ownership . . . 17
Cases, requirements and evidence . . . 17
Communication and interface boundaries . . . 17
Conformance and formal accuracy . . . 18
Graphical views and navigation . . . 18
Kernel foundation and interchange . . . 18
Names, structure and reusable types . . . 18
Simulation and runtime failures . . . 19
Timing, deployment and concurrency . . . 19
Values and calculation . . . 19
Versioning and language evolution . . . 19

Alphabetical keyword reference 20

Keyword: after . . . 20
Keyword: and . . . 20
Keyword: as . . . 20
Keyword: assert . . . 21
Keyword: atLeastOnce . . . 21
Keyword: atMostOnce . . . 21
Keyword: attribute . . . 21
Keyword: bestEffort . . . 21
Keyword: bool . . . 21
Keyword: bound . . . 22
Keyword: byte . . . 22
Keyword: calc . . . 22
Keyword: call . . . 22
Keyword: capacity . . . 22
Keyword: client . . . 23
Keyword: clientServer . . . 23
Keyword: completion . . . 23
Keyword: component . . . 23
Keyword: concurrency . . . 23
Keyword: connect . . . 24
Keyword: cv . . . 24
Keyword: data . . . 24
Keyword: datatype . . . 24
Keyword: deadline . . . 24

Keyword: def . . . 25
Keyword: delegate . . . 25
Keyword: delivery . . . 25
Keyword: deploy . . . 25
Keyword: do . . . 25
Keyword: doc . . . 26
Keyword: dropNewest . . . 26
Keyword: dropOldest . . . 26
Keyword: else . . . 26
Keyword: entry . . . 26
Keyword: enum . . . 26
Keyword: event . . . 27
Keyword: every . . . 27
Keyword: exit . . . 27
Keyword: fail . . . 27
Keyword: false . . . 27
Keyword: fifo . . . 27
Keyword: final . . . 28
Keyword: for . . . 28
Keyword: forEach . . . 28
Keyword: function . . . 28
Keyword: fv . . . 28
Keyword: grv . . . 29
Keyword: gv . . . 29
Keyword: idempotent . . . 29
Keyword: if . . . 29
Keyword: import . . . 29
Keyword: in . . . 30
Keyword: include . . . 30
Keyword: initial . . . 30
Keyword: int . . . 30
Keyword: interface . . . 30
Keyword: interruptible . . . 31
Keyword: iv . . . 31
Keyword: label . . . 31
Keyword: manual . . . 31
Keyword: maxAge . . . 31
Keyword: maxLatency . . . 32
Keyword: mixed . . . 32
Keyword: ms . . . 32
Keyword: not . . . 32
Keyword: ns . . . 32
Keyword: of . . . 33
Keyword: on . . . 33

Keyword: optional . . . 33
Keyword: or . . . 33
Keyword: ordering . . . 33
Keyword: otherwise . . . 34
Keyword: overflow . . . 34
Keyword: package . . . 34
Keyword: port . . . 34
Keyword: priority . . . 34
Keyword: provides . . . 35
Keyword: publish . . . 35
Keyword: publisher . . . 35
Keyword: publishSubscribe . . . 35
Keyword: qos . . . 35
Keyword: queued . . . 36
Keyword: real . . . 36
Keyword: realtime . . . 36
Keyword: receive . . . 36
Keyword: receiver . . . 36
Keyword: record . . . 37
Keyword: recursive . . . 37
Keyword: redefines . . . 37
Keyword: reentrant . . . 37
Keyword: reject . . . 37
Keyword: reply . . . 37
Keyword: requirement . . . 38
Keyword: requires . . . 38
Keyword: return . . . 38
Keyword: s . . . 38
Keyword: sampled . . . 38
Keyword: satisfy . . . 38
Keyword: send . . . 39
Keyword: sender . . . 39
Keyword: senderReceiver . . . 39
Keyword: serial . . . 39
Keyword: server . . . 39
Keyword: service . . . 40
Keyword: specializes . . . 40
Keyword: start . . . 40
Keyword: state . . . 40
Keyword: string . . . 40
Keyword: subscriber . . . 41
Keyword: sv . . . 41
Keyword: timing . . . 41
Keyword: transition . . . 41

Keyword: true . . . 41
Keyword: uint . . . 41
Keyword: unordered . . . 42
Keyword: us . . . 42
Keyword: useCase . . . 42
Keyword: var . . . 42
Keyword: verificationCase . . . 42
Keyword: verify . . . 42
Keyword: version . . . 43
Keyword: via . . . 43
Keyword: view . . . 43
Keyword: void . . . 43
Keyword: wcet . . . 43
Keyword: when . . . 44
Keyword: where . . . 44
Keyword: while . . . 44
Keyword: within . . . 44
Punctuation and operator reference . . . 44

SWAML language specification 46

1. Normative document set . . . 46
2. Normative terms and precedence . . . 46
3. Design invariants . . . 47
4. Abstract model inventory . . . 47
5. Conformance summary . . . 48
Requirements and formal documentation . . . 48
Extended function flow and timing . . . 48

1. Structure, names and types 49

1.1 Lexical model . . . 49
1.2 Namespaces and resolution . . . 49
1.3 Definitions, instances and specialization . . . 50
1.4 Value types . . . 51
1.5 Expression rules . . . 52
1.6 Requirements for tools . . . 52

2. Interfaces and communication 53

2.1 Contract identity and members . . . 53
2.2 Ports and boundary exposure . . . 53
2.3 Emission and reception ownership . . . 54
2.4 Exchange semantics . . . 55
2.5 Queue and delivery policy . . . 55
2.6 Timing of messages . . . 56

3. Functions, flow and states 57

3.1 Function identity and activation . . . 57
3.2 Flow execution . . . 57
3.3 Scheduling and concurrency . . . 59
3.4 Stateful functions . . . 60
3.5 Transition triggers and selection . . . 60
3.6 Transition execution . . . 61

4. Root components, timing, cases and conformance 62

4.1 Root components and profiles . . . 62
4.2 Deployment . . . 63
4.3 Requirements and cases . . . 64
4.4 View declarations . . . 65
4.5 Static validity rule groups . . . 66
4.6 Conformance levels . . . 67

5. KerML semantic mapping 68

5.1 Foundation and boundary . . . 68
5.2 Semantic specializations . . . 68
5.3 Normalization algorithm . . . 69
5.4 Required extension predicates . . . 70
5.5 Interchange and round-trip obligations . . . 70

6. Operational contract and target bindings 71

6.1 Configuration and observable occurrences . . . 71
6.2 Initialization and release . . . 71
6.3 Dispatch, backlog and snapshots . . . 72
6.4 Queue offers and fanout . . . 72
6.5 RPC completion and races . . . 73
6.6 State reaction ordering . . . 74
6.7 Execution binding contract . . . 75

7. Normalized semantic model 76

7.1 Identity, ownership and source trace . . . 76
7.2 Declaration graph and links . . . 76
7.3 Contract projection and route normalization . . . 77
7.4 Executable normalization . . . 78
7.5 Kernel representation and SWAML library responsibilities . . . 79
7.6 Case and view normalization . . . 80
7.7 Equivalence and completeness . . . 80
Requirement and presentation extension . . . 80

8. Conformance obligations and evidence 81

8.1 Static acceptance . . . 81
8.2 Runtime obligations . . . 83
8.3 Analysis claims . . . 83

8.4 Kernel, editor and graphical acceptance . . . 84
8.5 Scope and future evolution . . . 84
Requirement and presentation extension . . . 84

9. Formal core and simulation relation 85

9.1 Domains and judgments . . . 85
9.2 Static rule schemas . . . 85
9.3 Local execution rules . . . 87
9.4 Communication transition schemas . . . 88
9.5 Stateful function transition schemas . . . 89
9.6 Time, traces and refinement . . . 89
Requirement and presentation extension . . . 89

SWAML graphical notation 90

1. Scope, authority and coverage . . . 90
2. Canonical drawing vocabulary . . . 91
3. Names, ownership, types and values . . . 92
4. Interfaces, members and ports . . . 92
5. Architectural edges, direction and communication policy . . . 93
6. Function signatures, triggers and control flow . . . 94
7. Directly owned state behavior . . . 95
8. Timing, profiles and deployment . . . 96
9. Cases, requirements and sequence exchanges . . . 96
10. Views and navigation . . . 96
11. Canonical assets and reproduction . . . 97
12. Editing, accessibility and graphical conformance . . . 98
Formal documentation and ordinary notes . . . 98

Canonical graphical notation catalog 99

Concept index . . . 99
Cases and evidence . . . 102
Notation: case-causal-order . . . 102
Notation: case-data-exchange . . . 102
Notation: case-event-exchange . . . 103
Notation: case-request . . . 103
Notation: case-reply . . . 104
Notation: view-selection . . . 104
Notation: formal-documentation . . . 105
Notation: sequence-lifeline . . . 105
Notation: ordinary-comment . . . 106
Notation: requirement . . . 106
Notation: requirement-attribute . . . 107
Notation: satisfies-link . . . 107
Notation: case . . . 108
Notation: verification-case . . . 108

Notation: verification-link . . . 109
Function flow . . . 109
Notation: assert . . . 109
Notation: assignment . . . 110
Notation: bounded-loop . . . 110
Notation: conditional . . . 111
Notation: for-loop . . . 112
Notation: send . . . 112
Notation: publish . . . 113
Notation: function . . . 113
Notation: fail . . . 114
Notation: for-each-loop . . . 115
Notation: return . . . 115
Notation: interruptible-region . . . 116
Notation: local-declaration . . . 117
Notation: local-call . . . 117
Notation: trigger-manual . . . 118
Notation: trigger-periodic . . . 118
Notation: trigger-receive . . . 119
Notation: do-while-loop . . . 119
Notation: calculation . . . 120
Notation: routed-call . . . 120
Notation: trigger-call . . . 121
Notation: trigger-start . . . 121
Function states . . . 122
Notation: completion-transition . . . 122
Notation: final-state . . . 122
Notation: stateful-function . . . 123
Notation: initial-state . . . 123
Notation: timer-transition . . . 124
Notation: state . . . 124
Notation: state-entry . . . 125
Notation: state-exit . . . 125
Notation: transition . . . 126
Notation: transition-guard . . . 126
Notation: transition-priority . . . 127
Interfaces and connections . . . 127
Notation: delegation . . . 127
Notation: client-server-interface . . . 128
Notation: data-member . . . 128
Notation: port-receiver . . . 129
Notation: port-sender . . . 129
Notation: delivery-policy . . . 130
Notation: event-member . . . 130

Notation: port-publisher . . . 131
Notation: port-subscriber . . . 131
Notation: connector . . . 132
Notation: function-signature . . . 132
Notation: freshness-bound . . . 133
Notation: qos-policy-reference . . . 133
Notation: optional-port . . . 134
Notation: overflow-policy . . . 134
Notation: publish-subscribe-interface . . . 135
Notation: capacity-policy . . . 135
Notation: ordering-policy . . . 136
Notation: qos-policy-definition . . . 136
Notation: port-client . . . 137
Notation: port-server . . . 137
Notation: sender-receiver-interface . . . 138
Notation: port-requires . . . 138
Notation: service-interface . . . 139
Notation: port-provides . . . 139
Notation: latency-bound . . . 140
Structure and values . . . 140
Notation: component-definition . . . 140
Notation: component-specialization . . . 141
Notation: component-usage . . . 141
Notation: enumeration . . . 142
Notation: enum-value . . . 142
Notation: expression . . . 143
Notation: array-value . . . 143
Notation: inline-root . . . 144
Notation: member-redefinition . . . 144
Notation: type-relationship . . . 145
Notation: package . . . 145
Notation: import . . . 146
Notation: parameter . . . 146
Notation: primitive-types . . . 147
Notation: private-variable . . . 147
Notation: record-value . . . 148
Notation: datatype . . . 148
Notation: root-component . . . 149
Timing and deployment . . . 149
Notation: concurrency-mode . . . 149
Notation: deployment . . . 150
Notation: duration . . . 150
Notation: deployment-target . . . 151
Notation: timing-contract . . . 151

Notation: function-timing-diagram . . . 152
Notation: execution-profile . . . 153
View projections . . . 153
Notation: view-browser . . . 153
Notation: view-cv . . . 154
Notation: view-fv . . . 154
Notation: view-gv . . . 155
Notation: view-grv . . . 155
Notation: view-iv . . . 156
Notation: view-sv . . . 157
Coverage and interpretation . . . 157
Keyword coverage . . . 157
Grammar production coverage . . . 162

A. Complete concrete grammar **167**

B. Grammar-to-semantics inventory **170**

C. Model examples **176**

01 hello . . . 176
02 realtime controller . . . 177
03 service pubsub . . . 178
04 stateful function . . . 180
05 values and flow . . . 181
06 mixed gateway . . . 182
07 client server . . . 184
08 service field . . . 185
09 component roots . . . 187
10 reusable qos . . . 188
11 autosar classic . . . 189
12 autosar adaptive . . . 191
13 web application . . . 193
14 cloud service . . . 195
15 component specialization . . . 198
16 requirements and views . . . 199
17 function timing . . . 200
18 control loops . . . 201
19 interruptible function . . . 202
catalog . . . 203
control flow . . . 205

SWAML at a glance **206**

The graphical ontology . . . 206
1. Begin with the use case . . . 207
2. Describe the message flow . . . 207

3. Open the function flow . . . 208
4. Assign clear software ownership . . . 209
5. Inspect the final structure . . . 210
Try the complete model . . . 211
Requirements and formal documentation . . . 212

SWAML in five minutes **213**

1. Define the exchanged contract . . . 213
2. Give behavior an owner . . . 213
3. Compose and connect . . . 213
4. Validate the model . . . 214
5. Review through views . . . 214
Requirements and formal documentation . . . 215

Architecture patterns **216**

AUTOSAR Classic: a climate software component . . . 216
AUTOSAR Adaptive: an ARA application . . . 217
Service-oriented web application: checkout and inventory . . . 218
Cloud service: ingestion and event distribution . . . 218
Validate and run the models . . . 219

SWAML for UML architects **220**

Translate the modeling concepts . . . 220
Choose the interface by interaction . . . 221
Build the model incrementally . . . 221
Read the seven surfaces . . . 222
Deliberate language limits . . . 222

SWAML 1.2 architecture usability review **223**

Findings and changes . . . 223
Language decision . . . 224
Review evidence . . . 224

SWAML ontology and flat views **225**

Ownership and reuse . . . 225
Communication contracts . . . 226
How the views look . . . 226
Start small . . . 227
Requirements and formal documentation . . . 227

Graphical notation examples **228**

Model Browser: actual ownership . . . 228
gv: general structure and allocation . . . 229
iv: explicit interconnection and delegation . . . 230
cv: cases and claimed requirement coverage . . . 231

sv: interface exchanges and partial order . . . 232
grv: inspect properties without changing semantics . . . 233
fv: function statement flow . . . 234
fv: function-owned states . . . 235
Reproduction and semantic checks . . . 235

Reference simulation profile 1.3 **236**

Run a model . . . 236
Binding parameters . . . 236
Deterministic scheduling and time . . . 237
Transport and fault interpretation . . . 238
Values, flow and state coverage . . . 238
Evidence and limitations . . . 239
Extended control flow and interruption . . . 239

Migrating from SWAML 1.0 to 1.1 **240**

Select the language edition in the host . . . 240
Keep definitions explicit; make roots component occurrences . . . 240
Put states directly in their operation . . . 241
Trim expressions and no-result annotations . . . 241
Preserve architecture and check evidence . . . 242

Migrating to SWAML 1.2 **243**

Migrate 1.2.0 to 1.2.1 . . . 243
Convert the model . . . 243
Reuse a component type . . . 244
Keep data and communication explicit . . . 244
Update hosts and check fixtures . . . 244
Final 1.2 authoring vocabulary . . . 245

Migrating to SWAML 1.3 **246**

Source changes . . . 246
Interrupts and traces . . . 246
Timing presentation . . . 246

Loops, interrupts and function timing **247**

Choose a loop . . . 247
Interrupt a protected region . . . 247
Read an FV timing diagram . . . 248

ADR 0011 - Loops, interruptible flow and function timing **250**

Control flow . . . 250
Interruptible regions . . . 250
Function timing diagrams . . . 251
Edition and evidence . . . 251

SWAML 1.3 user guide 252

1. Install and check a model . . . 252
2. Write the contract first . . . 252
Reuse a communication policy . . . 253
3. Choose component and component boundaries . . . 253
4. Describe function control flow . . . 254
5. Model a service call and its failure . . . 254
6. Use publish/subscribe without a hidden event bus . . . 254
7. Put states inside a function . . . 255
8. Model realtime constraints . . . 255
9. Separate deployment from architecture . . . 256
Specialize a reusable component . . . 256
Add a pure calculation . . . 256
10. Organize a larger model . . . 256
11. Navigate through views . . . 256
12. Resolve diagnostics . . . 257
Service fields, repeated consumers and checked traces . . . 257
Find a construct or run an example . . . 258
Requirements and formal documentation . . . 258
Extended loops, interrupts and timing . . . 258

Software architecture rationale 259

AUTOSAR Classic and Adaptive . . . 259
UML and object-oriented implementation . . . 259
SysML 2.0 and KerML . . . 260
Analysis and runtime boundaries . . . 260

Language tooling and integration 261

Package contract . . . 261
Reproducible commands . . . 261
Services and source ownership . . . 262
Language-server integration . . . 262
Browser hosts . . . 262
Graphical architecture . . . 262
Kernel backend boundary . . . 263
Validation and release gates . . . 263
Simulation API for a separate host . . . 263
Component-model integration . . . 263

Implementation status - 1.3.0 264

Implemented and tested . . . 264
Limits of static acceptance . . . 266
Specified, not implemented . . . 266
Intentionally outside 1.3 . . . 266
Requirements and selected views in 1.2.1 . . . 267

Relation syntax in 1.2.1 . . . 267

SWAML 1.3.0 release notes **268**

SysML-style specification review **269**

ADR 0001 - Ownership and communication boundaries **270**

Decision 0002: Complete semantics before claiming complete tooling **271**

Decision 0003: stabilize the language with reproducible execution evidence **272**

ADR 0004 - Compact component language **273**

Decision . . . 273

Consequences . . . 273

ADR 0005 - Canonical graphical notation catalog **274**

Decision . . . 274

Consequences . . . 274

ADR 0006 - Reusable QoS and flat architecture views **275**

Decision . . . 275

Consequences . . . 275

ADR 0007 - Components, functions and calculations **276**

Ownership . . . 276

Reuse . . . 276

Deployment and types . . . 276

Migration and evidence . . . 277

ADR 0008 - Function-local calculations and one call syntax **278**

ADR 0010 - Consistent subject relations **279**

Decision . . . 279

Migration and version selection . . . 279

Audit and evidence . . . 279

Requirements, documentation and views **280**

ADR 0009 - Requirements, documentation and selected views **282**

PDF specification and release publication **283**

Version identifiers . . . 283

Rebuild the book . . . 283

Quality and release gate . . . 284

Contributing to SWAML **285**

Development setup . . . 285
Propose a language change . . . 285
Implement and test . . . 285
Pull requests and review . . . 286
Documentation and standards . . . 286
Resolve specification issues . . . 286
Report a defect . . . 286
Graphical notation changes . . . 286
Publication changes . . . 287

Instructions for agents working on SWAML **288**

Read first . . . 288
Preserve the language invariants . . . 288
SWAML 1.3 authoring contract . . . 288
Source and architecture . . . 289
Required checks . . . 289
Working practices . . . 289
Reference execution and publication . . . 289
Requirements and presentation . . . 290

D. External references **291**

E. License **292**

USER REFERENCE / CONCERN AND KEYWORD NAVIGATION

Language reference by concern

Use this alphabetical map to navigate SWAML 1.3 by architectural question. The alphabetical keyword reference provides the other reading order; the complete Markdown edition contains both maps, all normative chapters, user guidance, grammar and model examples in one document. Rules are normative where their source chapter says so; a tutorial or illustration never overrides those rules.

Architecture selection and platform correspondence

Start with the architecture comparison. SWAML borrows strict interface boundaries from component/service architectures and familiar structural/behavioral concepts from UML/SysML. Its authoring model is deliberately software-specific. A Classic-like control loop uses sampled senderReceiver members, periodic functions and explicit timing. A service architecture uses function/event contracts, correlated requests, timeout handling and static subscriber connections. A gateway explicitly translates between these through its own component ports.

Choose a root component profile (`realtime`, `service` or `mixed`) according to the contracts you intend to supply. The choice does not turn the model into AUTOSAR configuration or prove a scheduling property. Use an adapter component when hardware or middleware behavior matters to the architecture. External devices cannot enter a case through an untyped actor shortcut.

Behavior and state ownership

See functions, flow and states and the operational contract. Components own executable functions directly. A calculation evaluates a pure expression from typed inputs; an untriggered flow function is a local helper with an optional result; a triggered function is a root activation. The same component may also compose child components; each child retains its own private store and activation lock.

Choose a statement body for sequential calculation and requests. Choose a state body when the function remains in named states between inputs. States, guards and transitions remain inside that function. There are no independent actions or performers. The function's flow/state body is the subject of fv. Entry/exit cannot read a transition payload; save a needed value in private component data in the effect.

Cases, requirements and evidence

See root components and cases. Requirements are named natural-language obligations. A case describes permitted interface exchanges in a chosen root component and may claim requirement coverage. Case steps reference actual top-level connectors and member paths. `after` gives causality; independent steps remain unordered.

A reply needs one unanswered matching request in its causal ancestry. Route validity is checked statically. A simulation or analysis witness is separate evidence that the function bodies can realize the described case. Neither a `satisfy` keyword nor a green line in a diagram establishes that evidence. cv shows coverage claims; sv shows the causal exchanges.

Communication and interface boundaries

See interfaces and communication. Choose senderReceiver for data, clientServer for RPC, publishSubscribe for events, or service for RPC plus provider events. Contracts are nominal and versioned. Component ports bind contracts to local functions or delegate them through the hierarchy. Connect owned or immediate-child ports at each composition level. An explicit route may also connect two complementary ports of the same instance.

A sender/publisher can fan out. A subscriber can receive multiple publisher routes, each with its own queue. A receiver/client/requires port has at most one route. Required ports must be connected in a selected closed root component; optional unbound RPC reports Unavailable. Reading another component's fields, invoking its function directly, sending a free event, or adding a topic string outside a contract is invalid.

Queue delivery, order, capacity and overflow are mandatory. Data/event admission can partially succeed across fanout, so atomic broadcast needs an explicit protocol. Requests wait for one correlated result and require a failure branch. Timeout does not cancel server effects. A field-like service property is modeled as getter, setter and changed event, with private storage; see the setting example.

Conformance and formal accuracy

Read the obligation classification, formal rules and implementation matrix. S means grammar support, V static validity, K kernel mapping, E execution and G graphical editing. Their obligations are distinct. The reference implementation supplies parsing, broad static checks and a defined logical simulation binding; independent complete certification is not asserted.

A parsed but invalid model is not executable. A valid model may deliberately fail an assertion, overflow a queue, exceed a loop bound, time out, or violate a timing contract at runtime. Report those observations; do not weaken the language to make every accepted model look successful. Unknown syntax and unsupported requested editions must be rejected. The host pins the parser edition; historical file headers are not 1.2 syntax.

Graphical views and navigation

See graphical notation and the canonical concept catalog, with original SVG/PNG drawings and graphical/textual notation tables. The Model Browser exposes ownership and definition/occurrence identity. gv shows general structure, iv connections, cv cases/coverage, sv exchanges, grv tabular properties, and fv function flow or states. A selected occurrence retains its instance path when its shared declaration is displayed.

A diagram is a projection of the same semantic model as the text. Layout belongs in a sidecar. Port/member paths and connector identities must remain discoverable; a drawing cannot create a hidden channel. Source edits require parse/link/validation and undo.

Kernel foundation and interchange

Read KerML mapping and the normalized model. They specify identities, ownership, value types, routes, behavior lowering, metadata and reverse traces. The mapping inventory covers every grammar rule and type alias, including syntax that contributes to an owner's semantics rather than creating its own kernel node.

SWAML is defined over a proposed KerML-based semantic contract. This release does not export a kernel-validated artifact or publish a formal SWAML KerML library. The Langium AST is an authoring representation. A backend must preserve SWAML-specific queues, copying, snapshots, correlation and reaction order through a versioned library; kernel metadata alone is not execution support.

Names, structure and reusable types

See structure, names and types. A package owns top-level declarations; `A::T` selects a package member, while dots follow structural or local-value paths. Imports are direct, not transitive. Short-name ambiguity is an error across declaration kinds. Local functions resolve in their owning component; calculations resolve only in their enclosing function.

`component def` explicitly declares a reusable type. Use `specializes` and compatible `redefines` members when a related type needs controlled variation. Instances create distinct boundaries, stores and activations. A root component is a named occurrence, either inline composition or an instance of a component definition. Both root and nested components may contain components. Only a root carries an execution profile and external

deployment binding. Containment is acyclic. A component variable belongs to one occurrence and is initialized before any startup work runs. Reusing a definition never creates shared singleton state.

Simulation and runtime failures

Use the reference execution profile. Every example has a checked run configuration. Supply the entire validated workspace, selected root component and finite horizon. The output records exact sources, options, chronological observations and final stores/states. Exercise adverse conditions with the offer-indexed fault schedule and inspect whether the intended offer occurred.

Simulation fixes one deterministic ordering among core-permitted schedules, uses bounded resources and zero local CPU cost. It can expose behavioral failures, queue effects and elapsed-time violations; it cannot prove all schedules safe or establish hardware schedulability. Keep evidence linked to exact source versions and target assumptions.

Timing, deployment and concurrency

See timing and deployment and dispatch rules. WCET is active execution, deadline is elapsed time from release, maxLatency ends at destination readiness, and maxAge measures original message age until dispatch. Periodic deadlines cannot exceed their periods. A state machine's contract applies per reaction, not to its whole lifetime.

Serial activation roots retain their component lock across RPC waits; a callback to that same serial component may have to wait. Reentrant activation roots use read-only dispatch-time snapshots and may not mutate private component state through helpers. Allocation inherits from the nearest explicitly deployed ancestor. An external system/part reference does not supply a scheduler or bypass interface isolation.

Values and calculation

See value and expression rules. Values are bool, bounded int, finite binary64 real, string, nominal records/enums and fixed arrays. Parameters, locals and messages contain copies. There are no pointers, dynamic collections, implicit remote references or global mutable data.

Use whole-value assignment. Record field and final array-index reads are local value operations; field/index mutation is absent. Calculations cannot read component variables. Numeric widening is scalar; uint/byte assignments are range checked. Division returns real; integer remainder follows the dividend's sign. Arithmetic range, general array bounds and finite loop/microstep behavior are runtime obligations.

Versioning and language evolution

The source is headerless; its parser/host is explicitly pinned to SWAML 1.3. This specification/package revision is 1.3.0. Interface versions remain independent nominal contract labels. Version 1.2 replaces separate units and operations with component-owned functions, adds calculations and explicit specialization, and uses external deployment references. Follow the migration guide and preserve historical release artifacts.

Multiple inheritance, generics, dynamic instance/subscription/discovery lifecycle, unbounded arrays, reference payloads, hierarchical/orthogonal states, unrestricted fork/join and host-language code are outside this closed version. A proposal adding them must define syntax, ownership, typing, operational meaning, graphical projection, mapping, migration and tests before it is accepted. These concerns must not appear as undocumented backend shortcuts.

USER REFERENCE / CONCERN AND KEYWORD NAVIGATION

Alphabetical keyword reference

This user-facing reference covers every reserved word in the SWAML 1.3 grammar. Entries are sorted alphabetically, ignoring case for sorting only; source keywords remain case-sensitive. Example fragments illustrate a use and are not standalone models. Read the linked normative concern for full constraints. Punctuation and operator precedence follow the final section.

after · and · as · assert · atLeastOnce · atMostOnce · attribute · bestEffort · bool · bound · byte · calc · call · capacity · client · clientServer · completion · component · concurrency · connect · cv · data · datatype · deadline · def · delegate · delivery · deploy · do · doc · dropNewest · dropOldest · else · entry · enum · event · every · exit · fail · false · fifo · final · for · forEach · function · fv · grv · gv · idempotent · if · import · in · include · initial · int · interface · interruptible · iv · label · manual · maxAge · maxLatency · mixed · ms · not · ns · of · on · optional · or · ordering · otherwise · overflow · package · port · priority · provides · publish · publisher · publishSubscribe · qos · queued · real · realtime · receive · receiver · record · recursive · redefines · reentrant · reject · reply · requirement · requires · return · s · sampled · satisfy · send · sender · senderReceiver · serial · server · service · specializes · start · state · string · subscriber · sv · timing · transition · true · uint · unordered · us · useCase · var · verificationCase · verify · version · via · view · void · wcet · when · where · while · within

Keyword: after

Concern: Behavior and states. **Grammar:** `Transition`, `ScenarioStep`.

In a transition, creates a timer measured from source-state entry completion; leaving the state cancels its generation. In a case step, names one earlier causal predecessor, not a delay or source-order convention.

Example fragment: `on after 10 ms / via link after sent label received;`

Graphical notation: Residence timer transition, Case causal order. Residence timer transition; Case causal order.

Keyword: and

Concern: Values and calculation. **Grammar:** `AndExpression`.

Short-circuit Boolean conjunction. Both operands must be bool; the right operand is evaluated only when the left is true.

Example fragment: `ready and enabled`

Graphical notation: Expressions and value paths. Expressions and value paths.

Keyword: as

Concern: Communication. **Grammar:** `RemoteCall`, `LocalCall`.

Names a fresh local-call result or awaited RPC result. Local void calls omit it; routed void calls reserve a completion name without a readable value.

Example fragment: `call api.read() as value within 1 s otherwise { return; };`

Graphical notation: Routed function call. Routed function call.

Keyword: assert

Concern: Behavior and states. **Grammar:** `Assert`.

Evaluates a bool; false fails the activation without rollback. A failed RPC handler returns RemoteFailure through its correlation. Assertions are executable checks, not static proofs.

Example fragment: `assert measured <= 100;`

Graphical notation: Assertion. Assertion.

Keyword: atLeastOnce

Concern: Communication. **Grammar:** `QosBody`.

Permits retries and duplicate delivery subject to liveness, overflow and freshness. RPC members on such a connector must declare idempotent behavior. It never implies exactly-once effects.

Example fragment: `delivery atLeastOnce;`

Graphical notation: Delivery policy. Delivery policy.

Keyword: atMostOnce

Concern: Communication. **Grammar:** `QosBody`.

Allows loss but no duplicate delivery for a logical message. A lost request or response can still lead to timeout. This policy does not guarantee admission or successful execution.

Example fragment: `delivery atMostOnce;`

Graphical notation: Delivery policy. Delivery policy.

Keyword: attribute

Concern: Cases and evidence. **Grammar:** `Attribute`.

Immutable typed requirement metadata.

Example fragment: `attribute priority: uint = 1;`

Graphical notation: Requirement attribute. Requirement attribute.

Keyword: bestEffort

Concern: Communication. **Grammar:** `QosBody`.

Permits loss without an eventual-delivery guarantee and without duplicate delivery. Queue capacity, overflow and declared timing bounds still apply.

Example fragment: `delivery bestEffort;`

Graphical notation: Delivery policy. Delivery policy.

Keyword: bool

Concern: Values and calculation. **Grammar:** `TypeRef`.

Primitive two-valued Boolean type. Conditions and logical operators require bool; numbers and strings are not truthy substitutes.

Example fragment: `var ready: bool = true;`

Graphical notation: Primitive types. Primitive types.

Keyword: bound

Concern: Behavior and states. **Grammar:** `While`, `For`, `DoWhile`.

Sets the positive safe-integer maximum number of body executions for while, for and do-while loops. A still-true condition after the final permitted body fails before another body. forEach instead uses its copied array cardinality as the bound.

Example fragment: `while (i < 3) bound 3 { i = i + 1; }`

Graphical notation: Bounded loop, Counted for loop, Post-test do-while loop. Bounded loop; Counted for loop; Post-test do-while loop.

Keyword: byte

Concern: Values and calculation. **Grammar:** `TypeRef`.

Exact unsigned eight-bit values from 0 through 255. Integral assignment checks the destination range statically for known constants and at runtime for computed values.

Example fragment: `var payload: byte[2] = [0, 255];`

Graphical notation: Primitive types. Primitive types.

Keyword: calc

Concern: Values and calculation. **Grammar:** `Calculation`.

Declares a pure calculation at the start of a function body. Its typed inputs and named result expression are visible only within that owning function, including its states and guards.

Example fragment: `calc twice { in x: int; return result: int = x * 2; }`

Graphical notation: Pure calculation. Pure calculation.

Keyword: call

Concern: Behavior and states. **Grammar:** `Trigger`, `RemoteCall`, `LocalCall`, `ScenarioStep`.

Calls an untriggered local function, awaits a routed port.member function, marks an incoming RPC trigger, or describes a case RPC step. Local non-void calls require as name; routed calls require a result/completion name, timeout and terminating failure path.

Example fragment: `call normalize(input) as value; / call api.read() as reply within 1 s otherwise { return; };`

Graphical notation: RPC handler activation, Local function call, Routed function call, Case request. RPC handler activation; Local function call; Routed function call; Case request.

Keyword: capacity

Concern: Communication. **Grammar:** `QosBody`.

Positive safe-integer retained-message limit per connector occurrence, direction and member. Transport-pending messages count; running handlers do not. Sampled data requires capacity 1.

Example fragment: `capacity 8;`

Graphical notation: Queue capacity. Queue capacity.

Keyword: client

Concern: Communication. **Grammar:** `Port`.

Port role for the requesting side of a clientServer interface. A closed client port has at most one route; replies return through the same route and correlation.

Example fragment: `port api: Health client;`

Graphical notation: RPC client port. RPC client port.

Keyword: clientServer

Concern: Communication. **Grammar:** `Interface`.

Interface kind containing function signatures only. A client issues request; a server implements on call handlers. Events belong in service or publishSubscribe contracts.

Example fragment: `interface clientServer Health version "1.0.0" { function check(): bool; }`

Graphical notation: Client-server interface. Client-server interface.

Keyword: completion

Concern: Behavior and states. **Grammar:** `Transition`.

Local state stimulus reconsidered after entry and each reaction until quiescent. It is not an interface event. Unconditional cycles are invalid; guarded cycles have a 1,024-edge runtime limit.

Example fragment: `transition next: A -> B on completion {}`

Graphical notation: Completion transition. Completion transition.

Keyword: component

Concern: Structure and names. **Grammar:** `Component`, `Instance`, `RootComponent`.

Defines a reusable software boundary with def, or a concrete instance. Components own child components, ports, private data and executable functions. Functions own calculations. Each instance receives distinct state and activation locks.

Example fragment: `component def Controller { } / component controller: Controller;`

Graphical notation: Component definition, Component usage, Occurrence typing relationship, Root component, Inline root composition. Component definition; Component usage; Occurrence typing relationship; Root component; Inline root composition.

Keyword: concurrency

Concern: Timing and deployment. **Grammar:** `Timing`.

Selects serial or reentrant function activation policy in a timing contract. Helpers inherit the activation mode. Stateful functions never overlap their own reactions.

Example fragment: `concurrency serial;`

Graphical notation: Concurrency mode. Concurrency mode.

Keyword: connect

Concern: Communication. **Grammar:** `Connect`.

Creates an explicit connector between owned or immediate-child ports. Contracts must be identical and roles complementary. Every connector supplies a complete QoS selection.

Example fragment: `connect samples: sensor.output -> controller.input qos { ... };`

Graphical notation: Interface connector. Interface connector.

Keyword: cv

Concern: Graphical views. **Grammar:** `View`.

Case View kind: displays cases, requirements and claimed coverage for a case or root component. Verified coverage requires separate linked evidence.

Example fragment: `view coverage cv of Shop;`

Graphical notation: Case View. Case View.

Keyword: data

Concern: Communication. **Grammar:** `DataMember`.

Declares one senderReceiver member with a value type and sampled or queued mode. Outgoing use is send; incoming handling is on receive with one typed argument.

Example fragment: `data sample: real sampled;`

Graphical notation: Data member. Data member.

Keyword: datatype

Concern: Values and calculation. **Grammar:** `DataType`.

Declares a nominal finite record. Field names are unique, every literal supplies all fields once, and recursive value containment is invalid. Empty records have one value.

Example fragment: `datatype Point { x: real; y: real; }`

Graphical notation: Record datatype. Record datatype.

Keyword: deadline

Concern: Timing and deployment. **Grammar:** `Timing`.

Positive elapsed-time bound from activation release to completion, including queueing and request suspension. For stateful functions it applies per reaction. WCET must not exceed it.

Example fragment: `deadline 5 ms;`

Graphical notation: Function timing contract, Function timing diagram. Function timing contract; Function timing diagram.

Keyword: def

Concern: Structure and names. **Grammar:** `Component`, `QosPolicy`.

Declares a reusable component type or complete package-owned QoS policy. Component definitions may specialize one base; definitions create no runtime singleton.

Example fragment: `component def Controller { } / qos def LocalLink { ... }`

Graphical notation: Component definition, Reusable QoS policy. Component definition; Reusable QoS policy.

Keyword: delegate

Concern: Communication. **Grammar:** `Delegate`.

Aliases a component boundary port to one immediate child port of the same role and interface. A port has at most one delegation and cannot also have local behavior owners. Delegation adds neither conversion nor a queue.

Example fragment: `delegate api -> worker.api;`

Graphical notation: Boundary delegation. Boundary delegation.

Keyword: delivery

Concern: Communication. **Grammar:** `QosBody`.

Chooses bestEffort, atMostOnce or atLeastOnce for all member queues of a connector. Delivery is separate from FIFO ordering, admission and application success.

Example fragment: `delivery atMostOnce;`

Graphical notation: Delivery policy. Delivery policy.

Keyword: deploy

Concern: Timing and deployment. **Grammar:** `Deployment`.

Associates a component instance path, or the root name itself, with an external system/part identity string. Descendants inherit the nearest binding. The host resolves the external destination.

Example fragment: `deploy controller on "sysml:Vehicle::ecu";`

Graphical notation: Deployment binding. Deployment binding.

Keyword: do

Concern: Behavior and states. **Grammar:** `DoWhile`.

Starts a post-test loop. Its body runs at least once, then its Boolean while condition is evaluated. A mandatory positive safe-integer bound limits body executions; another requested body fails with LoopBound. Body-local declarations do not escape to the condition.

Example fragment: `do { count = count + 1; } while (count < 3) bound 3;`

Graphical notation: Post-test do-while loop. Post-test do-while loop.

Keyword: doc

Concern: Cases and evidence. **Grammar:** `Documentation`, `DOC_COMMENT`.

One owned formal multiline specification.

Example fragment: `doc /* Formal text. */`

Graphical notation: Formal documentation. Formal documentation.

Keyword: dropNewest

Concern: Communication. **Grammar:** `QosBody`.

On a full queue, discards the offered message observably. Admission does not fail the producer; an RPC can later time out. Existing retained messages stay.

Example fragment: `overflow dropNewest;`

Graphical notation: Overflow behavior. Overflow behavior.

Keyword: dropOldest

Concern: Communication. **Grammar:** `QosBody`.

On a full queue, discards the oldest retained message and admits the new one. Required with capacity 1 for sampled data. It does not undo effects of already dispatched handlers.

Example fragment: `overflow dropOldest;`

Graphical notation: Overflow behavior. Overflow behavior.

Keyword: else

Concern: Behavior and states. **Grammar:** `If`.

Optional alternative block of an if statement. Only the chosen branch executes; branch-local declarations do not escape that branch.

Example fragment: `if (ready) { return; } else { fail "not ready"; }`

Graphical notation: Conditional branch. Conditional branch.

Keyword: entry

Concern: Behavior and states. **Grammar:** `State`.

State-entry block executed after that state becomes current. It has no incoming payload scope. Residence timers begin after entry finishes; return/fail terminates the owning function.

Example fragment: `state Idle { entry { count = 0; } }`

Graphical notation: State entry behavior. State entry behavior.

Keyword: enum

Concern: Values and calculation. **Grammar:** `EnumType`.

Declares a nominal set of one or more unique literals. Values use Type.Literal or Package::Type.Literal. A live value first segment in the current expression environment takes precedence over an unqualified enum name, with no fallback after a failed field lookup; qualify to select the enum explicitly.

Example fragment: `enum Status { Ready, Failed } / Status.Ready / Contracts::Status.Ready`

Graphical notation: Enumeration. Enumeration.

Keyword: event

Concern: Communication. **Grammar:** `EventMember`.

Declares a typed asynchronous member in publishSubscribe or service. It must be emitted by a function through a port and owned by one receiving function; free global events are absent.

Example fragment: `event changed(value: real);`

Graphical notation: Event member. Event member.

Keyword: every

Concern: Behavior and states. **Grammar:** `Trigger`.

Periodic parameterless void activation at epoch plus k periods, k starting at 1. Releases do not merge when earlier activations are busy; pending work is bounded by the binding.

Example fragment: `function sample(): void on every 10 ms { ... }`

Graphical notation: Periodic activation. Periodic activation.

Keyword: exit

Concern: Behavior and states. **Grammar:** `State`.

State-exit block runs after timer cancellation and before the transition effect. It lacks payload scope. Return/failure skips remaining transition phases.

Example fragment: `state Running { exit { active = false; } }`

Graphical notation: State exit behavior. State exit behavior.

Keyword: fail

Concern: Behavior and states. **Grammar:** `Fail`.

Raises a local failure with a string literal reason, unwinds synchronous callers, and preserves committed effects. An RPC caller receives the correlated RemoteFailure outcome.

Example fragment: `fail "invalid operating mode";`

Graphical notation: Failure. Failure.

Keyword: false

Concern: Values and calculation. **Grammar:** `Literal`.

The false bool literal. Distinct from numeric zero, empty strings or absent values; no implicit coercion exists.

Example fragment: `var ready: bool = false;`

Graphical notation: Expressions and value paths. Expressions and value paths.

Keyword: fifo

Concern: Communication. **Grammar:** `QosBody`.

Dispatches retained messages in queue order for one connector/direction/member. Independent queues and handler completion order are not globally ordered.

Example fragment: `ordering fifo;`

Graphical notation: Queue ordering. Queue ordering.

Keyword: final

Concern: Behavior and states. **Grammar:** `State`.

Marks a state that completes its owning function after entry. It has no outgoing transition. Future input to that function is undeliverable, not rerouted.

Example fragment: `state Stopped final {}`

Graphical notation: Final state. Final state.

Keyword: for

Concern: Behavior and states. **Grammar:** `For`.

Counted pretest loop with one typed local initializer, Boolean condition, assignment update and mandatory positive safe-integer bound. The header variable belongs to the loop; body locals are fresh each iteration. The update follows each completed body. A true condition after the bound fails before another body.

Example fragment: `for (var i: int = 0; i < 3; i = i + 1) bound 3 { count = count + 1; }`

Graphical notation: Counted for loop. Counted for loop.

Keyword: forEach

Concern: Behavior and states. **Grammar:** `ForEach`.

Iterates in array order over one copied fixed-array value evaluated once on entry. Every iteration binds a fresh typed copy of its item. Array cardinality is the finite bound; changing the source or item cannot change the iteration snapshot.

Example fragment: `forEach (item: int in values) { total = total + item; }`

Graphical notation: Fixed-array forEach loop. Fixed-array forEach loop.

Keyword: function

Concern: Behavior and states. **Grammar:** `Signature`, `FunctionDef`.

Declares an executable component-owned behavior or an interface contract signature. A body contains optional local calculations followed by statements or directly owned states. An omitted result means void; triggers select activation.

Example fragment: `function startUp() on start { return; }`

Graphical notation: Interface function signature, Executable function, Function with states. Interface function signature; Executable function; Function with states.

Keyword: fv

Concern: Graphical views. **Grammar:** `View`.

Function view of existing executable functions. It projects statement flow, directly owned states or an observed function-timing diagram using the same view kind. Timing contracts remain distinct from observed trace intervals.

Example fragment: `view control fv of Controller.regulate;`

Graphical notation: Flow View, Function timing diagram. Flow View; Function timing diagram.

Keyword: grv

Concern: Graphical views. **Grammar:** `View`.

Grid View of a package or named semantic declaration other than a view. Presents inventory, contracts, timing, allocation or trace tables; sorting does not change model meaning.

Example fragment: `view inventory grv of Architecture;`

Graphical notation: Grid View. Grid View.

Keyword: gv

Concern: Graphical views. **Grammar:** `View`.

General View of a package or resolvable named semantic declaration other than a view. Shows ownership, types, attributes and relations, retaining definition/occurrence distinction.

Example fragment: `view overview gv of Product;`

Graphical notation: General View. General View.

Keyword: idempotent

Concern: Communication. **Grammar:** `Signature`.

Contract assertion required for RPC under atLeastOnce delivery. Repeated requests must have idempotent effects including published side effects. The keyword alone is not a proof.

Example fragment: `function read(): int idempotent;`

Graphical notation: Interface function signature. Interface function signature.

Keyword: if

Concern: Behavior and states. **Grammar:** `If`.

Evaluates one bool and executes the selected lexical block. It supplies no concurrency or implicit variable merging. Value-returning functions must cover both paths or fail.

Example fragment: `if (value > limit) { return false; } else { return true; }`

Graphical notation: Conditional branch. Conditional branch.

Keyword: import

Concern: Structure and names. **Grammar:** `Import`.

Imports top-level names from one package using namespace::*; imports are not transitive. A short name must be unique across local and directly imported declarations.

Example fragment: `import Common::*;`

Graphical notation: Package import. Package import.

Keyword: in

Concern: Values and calculation. **Grammar:** `Calculation`, `ForEach`.

Declares a typed input of a pure calculation, or separates the typed item binding from the fixed-array expression of forEach. Calculation inputs are copied; array iteration snapshots its collection once and binds copied items.

Example fragment: `in measured: real;`

Graphical notation: Pure calculation, Fixed-array forEach loop. Pure calculation; Fixed-array forEach loop.

Keyword: include

Concern: Graphical views. **Grammar:** `ViewSelection`.

Adds a subject selection to a view.

Example fragment: `include Demo;`

Graphical notation: Filtered view selection. Filtered view selection.

Keyword: initial

Concern: Behavior and states. **Grammar:** `StateBody`.

Introduces the state body after optional function-local calculation declarations. References exactly one state owned by that function and distinguishes state behavior from a statement block without a wrapper keyword. Initialization enters it once per run epoch before input consumption.

Example fragment: `initial Idle;`

Graphical notation: Function with states, Initial state reference. Function with states; Initial state reference.

Keyword: int

Concern: Values and calculation. **Grammar:** `TypeRef`.

Exact signed safe integers from â^'(2^53â^'1) through 2^53â^'1. Integral arithmetic produces int, with exact intermediate checks; division returns real.

Example fragment: `var delta: int = -2;`

Graphical notation: Primitive types. Primitive types.

Keyword: interface

Concern: Communication. **Grammar:** `Interface`.

Declares a nominal, versioned contract of one of four kinds. Port endpoints must reference the same declaration; matching names or version strings alone are insufficient.

Example fragment: `interface service Orders version "1.0.0" { ... }`

Graphical notation: Sender-receiver interface, Client-server interface, Publish-subscribe interface, Service interface. Sender-receiver interface; Client-server interface; Publish-subscribe interface; Service interface.

Keyword: interruptible

Concern: Behavior and states. **Grammar:** `Interruptible`.

Names a cooperatively cancellable region in a triggered serial flow function. An explicitly routed data/event input cancels protected work at a statement boundary or suspended request and runs the typed handler in the same activation. Pending requests are abandoned locally without rolling back prior or remote effects. The region supplies no hardware preemption or pause/resume.

Example fragment: `interruptible work { call job.perform() as answer within 1 s otherwise { return; }; } on receive control.stop { cancelled = true; }`

Graphical notation: Interruptible flow region. Interruptible flow region.

Keyword: iv

Concern: Graphical views. **Grammar:** `View`.

Interconnect View of a root component, component definition or component instance. Shows owned and immediate-child ports, connectors and delegation with exact interface paths.

Example fragment: `view wiring iv of Product;`

Graphical notation: Interconnect View. Interconnect View.

Keyword: label

Concern: Cases and evidence. **Grammar:** `ScenarioStep`.

Assigns a unique name to a scenario step. Later after clauses use that label as a causal predecessor; source position alone imposes no ordering.

Example fragment: `via orderLink label submitted;`

Graphical notation: Case data exchange, Case event exchange, Case request, Correlated case reply, Case causal order. Case data exchange; Case event exchange; Case request; Correlated case reply; Case causal order.

Keyword: manual

Concern: Behavior and states. **Grammar:** `Trigger`.

Parameterless void function trigger released by a harness. It is control of execution, not a route for injecting external model payloads or bypassing interfaces.

Example fragment: `function refresh(): void on manual { ... }`

Graphical notation: Manual activation. Manual activation.

Keyword: maxAge

Concern: Communication. **Grammar:** `QosBody`.

Maximum message age measured from original emission through retries. Eligibility is inclusive; greater age discards before dispatch. Required for sampled data in realtime/mixed roots.

Example fragment: `maxAge 20 ms;`

Graphical notation: Message freshness bound. Message freshness bound.

Keyword: maxLatency

Concern: Communication. **Grammar:** `QosBody`.

Positive bound from admitted emission to destination readiness, excluding dispatch wait. Exceeding it is an observable contract violation. Required on realtime/mixed connectors.

Example fragment: `maxLatency 1 ms;`

Graphical notation: Transport latency bound. Transport latency bound.

Keyword: mixed

Concern: Timing and deployment. **Grammar:** `RootComponent`.

Root component profile covering embedded/service combinations. All used functions carry timing and all connectors latency; sampled data also carries age. It does not silently exempt cloud-facing paths.

Example fragment: `component Product mixed { ... }`

Graphical notation: Root execution profile. Root execution profile.

Keyword: ms

Concern: Timing and deployment. **Grammar:** `Duration`.

Millisecond duration unit; one ms equals 1,000,000 ns. Duration magnitude must be a positive integer and normalize exactly within the safe range.

Example fragment: `within 25 ms`

Graphical notation: Duration value. Duration value.

Keyword: not

Concern: Values and calculation. **Grammar:** `UnaryExpression`.

Unary bool negation with precedence above multiplication. No numeric or string coercion is allowed.

Example fragment: `not ready`

Graphical notation: Expressions and value paths. Expressions and value paths.

Keyword: ns

Concern: Timing and deployment. **Grammar:** `Duration`.

Nanosecond duration unit and the reference simulation clock's integral base. All specified durations are positive, with safe exact conversion.

Example fragment: `after 1 ns`

Graphical notation: Duration value. Duration value.

Keyword: of

Concern: Graphical views. **Grammar:** `Satisfaction`, `Scenario`, `View`.

Selects the root context of a useCase/verificationCase, the design source of a satisfy claim, or the subject of a compact view. These are references, not changes of containment. Satisfaction remains directed from design to requirement.

Example fragment: `view flow fv of Product.controller.worker.run;`

Graphical notation: Use case, Satisfaction relation, General View, Interconnect View, Case View, Sequence View, Grid View, Flow View, Verification case. Use case; Satisfaction relation; General View; Interconnect View; Case View; Sequence View; Grid View; Flow View; Verification case.

Keyword: on

Concern: Behavior and states. **Grammar:** `Deployment`, `Trigger`, `Interruptible`, `Transition`.

Introduces a function activation trigger, state transition trigger, interruptible-region input or deployment destination. Deployment is a location association; activation and interruption are incoming control bindings.

Example fragment: `function tick() on every 10 ms { return; } / deploy app on "system:production";`

Graphical notation: Startup activation, Manual activation, Periodic activation, Port receive activation, RPC handler activation, State transition, Deployment binding, Interruptible flow region. Startup activation; Manual activation; Periodic activation; Port receive activation; RPC handler activation; State transition; Deployment binding; Interruptible flow region.

Keyword: optional

Concern: Communication. **Grammar:** `Port`.

Allows a port to remain unconnected in a closed root component. It does not change contract, role or handler rules. Unbound data/event sends succeed; unbound RPC reports Unavailable.

Example fragment: `port diagnostics: Notices publisher optional;`

Graphical notation: Optional port. Optional port.

Keyword: or

Concern: Values and calculation. **Grammar:** `OrExpression`.

Short-circuit Boolean disjunction. The right bool operand is evaluated only when the left is false.

Example fragment: `ready or retryAllowed`

Graphical notation: Expressions and value paths. Expressions and value paths.

Keyword: ordering

Concern: Communication. **Grammar:** `QosBody`.

Selects fifo or unordered independently of delivery reliability. Ordering applies per connector occurrence, direction and member, not across an entire interface or root.

Example fragment: `ordering fifo;`

Graphical notation: Queue ordering. Queue ordering.

Keyword: otherwise

Concern: Communication. **Grammar:** `RemoteCall`.

Mandatory request failure block with implicit copied failure:string. It must return or fail on every path and cannot fall through to successful result use.

Example fragment: `within 1 s otherwise { lastError = failure; return; };`

Graphical notation: Routed function call. Routed function call.

Keyword: overflow

Concern: Communication. **Grammar:** `QosBody`.

Required admission policy when a member queue is full: dropOldest, dropNewest or reject. Policies apply independently to every route in fanout.

Example fragment: `overflow reject;`

Graphical notation: Overflow behavior. Overflow behavior.

Keyword: package

Concern: Structure and names. **Grammar:** `Package`.

Top-level, non-nested namespace for declarations and direct imports. A package is declared once in the loaded workspace; multiple packages may share a file.

Example fragment: `package Control { ... }`

Graphical notation: Package. Package.

Keyword: port

Concern: Communication. **Grammar:** `Port`.

Owned communication boundary typed by one interface with a compatible role and optionality. A port may bind local functions to members or delegate to a child while retaining one resolved handler owner.

Example fragment: `port api: Orders requires;`

Graphical notation: Data sender port, Data receiver port, RPC client port, RPC server port, Event publisher port, Event subscriber port, Service provider port, Service consumer port. Data sender port; Data receiver port; RPC client port; RPC server port; Event publisher port; Event subscriber port; Service provider port; Service consumer port.

Keyword: priority

Concern: Behavior and states. **Grammar:** `Transition`, `Attribute`.

Nonnegative safe-integer tie-break among matching transitions for the same state and stimulus; larger wins, default 0. It is not a function scheduling priority.

Example fragment: `on receive commands.begin priority 2 {}`

Graphical notation: Transition priority. Transition priority.

Keyword: provides

Concern: Communication. **Grammar:** `Port`.

Provider role of a service interface. Handles incoming RPC, returns correlated results/failures and publishes its declared events in reverse connector direction.

Example fragment: `port api: Orders provides;`

Graphical notation: Service provider port. Service provider port.

Keyword: publish

Concern: Communication. **Grammar:** `Send`, `ScenarioStep`.

Emits an outgoing event through a publisher/provides port. In a case, describes a routed event step. Arguments are evaluated once and copied per route; no free event syntax exists.

Example fragment: `publish api.changed(current);`

Graphical notation: Event publication, Case event exchange. Event publication; Case event exchange.

Keyword: publisher

Concern: Communication. **Grammar:** `Port`.

Outgoing role for a publishSubscribe contract. One port may fan out through several connectors; every outgoing member is tied to an emitting function.

Example fragment: `port events: Notices publisher;`

Graphical notation: Event publisher port. Event publisher port.

Keyword: publishSubscribe

Concern: Communication. **Grammar:** `Interface`.

Event-only contract kind with publisher/subscriber ports. Subscriptions are explicit static connectors. Runtime subscribe/unsubscribe and wildcard topics are outside 1.3.

Example fragment: `interface publishSubscribe Notices version "1.0.0" { event changed(); }`

Graphical notation: Publish-subscribe interface. Publish-subscribe interface.

Keyword: qos

Concern: Communication. **Grammar:** `QosPolicy`, `Qos`.

Introduces a complete inline connector policy, an explicit named-policy reference, or a package-owned qos def declaration. A named policy reuses values without sharing connector queues. Every body declares delivery, ordering, capacity and overflow, then optional latency and age.

Example fragment: `qos def LocalLink { delivery atMostOnce; ordering fifo; capacity 8; overflow reject; } / qos LocalLink`

Graphical notation: Delivery policy, Reusable QoS policy, Named connector policy. Delivery policy; Reusable QoS policy; Named connector policy.

Keyword: queued

Concern: Communication. **Grammar:** `DataMember`.

SenderReceiver data mode representing a sequence of retained values under queue policy. It does not imply losslessness or unbounded history.

Example fragment: `data text: string queued;`

Graphical notation: Data member. Data member.

Keyword: real

Concern: Values and calculation. **Grammar:** `TypeRef`.

Finite IEEE 754 binary64 values. Scalar byte, uint and int may widen to real; real does not implicitly narrow to an integral type.

Example fragment: `var gain: real = 0.5;`

Graphical notation: Primitive types. Primitive types.

Keyword: realtime

Concern: Timing and deployment. **Grammar:** `RootComponent`.

Root component profile requiring function timing, connector latency and sampled freshness contracts. Those declarations require target evidence before any hard realtime claim.

Example fragment: `component Heating realtime { ... }`

Graphical notation: Root execution profile. Root execution profile.

Keyword: receive

Concern: Behavior and states. **Grammar:** `Trigger`, `Interruptible`, `Transition`.

Consumes an incoming data/event member through a receiver/subscriber/requires port. An ordinary function receives copied parameters; a transition or interruptible handler receives a copied data payload or event parameters. Boundaries remain explicit.

Example fragment: `on receive commands.begin`

Graphical notation: Port receive activation, State transition, Interruptible flow region. Port receive activation; State transition; Interruptible flow region.

Keyword: receiver

Concern: Communication. **Grammar:** `Port`.

Incoming senderReceiver role. Every data member has one handler owner; at most one route feeds a closed receiver port. Multiple sources require an explicit merge/adaptor component.

Example fragment: `port input: Samples receiver;`

Graphical notation: Data receiver port. Data receiver port.

Keyword: record

Concern: Values and calculation. **Grammar:** `RecordLiteral`.

Constructs a nominal datatype value by assigning each declared field exactly once. Field order in the literal does not change record equality; fields remain copied values.

Example fragment: `record Point { x = 1.0; y = 2.0; }`

Graphical notation: Record construction. Record construction.

Keyword: recursive

Concern: Graphical views. **Grammar:** `ViewSelection`.

Includes descendants before filtering.

Example fragment: `include Demo recursive;`

Graphical notation: Filtered view selection. Filtered view selection.

Keyword: redefines

Concern: Structure and names. **Grammar:** `Instance`, `Port`, `Variable`, `Connect`, `Delegate`, `FunctionDef`.

Explicitly replaces an inherited member while preserving its category and compatible contract. Unmarked inherited-name collisions and missing inherited targets are errors.

Example fragment: `redefines function startUp() on start { return; }`

Graphical notation: Member redefinition. Member redefinition.

Keyword: reentrant

Concern: Timing and deployment. **Grammar:** `Timing`.

Allows overlapping function activations with independent dispatch-time component snapshots. Direct or helper writes to component state are statically invalid. Local variables remain mutable; one state reaction cannot overlap itself.

Example fragment: `concurrency reentrant;`

Graphical notation: Concurrency mode. Concurrency mode.

Keyword: reject

Concern: Communication. **Grammar:** `QosBody`.

On full queue, rejects the offer. Data/event producers fail after all fanout offers; requests take otherwise with Rejected. Previously accepted offers and effects are preserved.

Example fragment: `overflow reject;`

Graphical notation: Overflow behavior. Overflow behavior.

Keyword: reply

Concern: Cases and evidence. **Grammar:** `ScenarioStep`.

Describes a response step in a case, reversing an existing connector/member request. It must have one unanswered causal request ancestor. Executable handlers reply by returning, not by a reply statement.

Example fragment: `reply backend.api.get -> client.api.get via link after asked label answered;`

Graphical notation: Correlated case reply. Correlated case reply.

Keyword: requirement

Concern: Cases and evidence. **Grammar:** `Requirement`.

Declares a named obligation with immutable typed attributes and at most one optional formal doc. satisfy links design to the requirement; only verificationCase owns verify links. These relations do not establish a passed result.

Example fragment: `requirement Safe { doc /* Duty shall remain within the allowed range. */ }`

Graphical notation: Requirement. Requirement.

Keyword: requires

Concern: Communication. **Grammar:** `Port`.

Consumer role of a service interface. Issues RPC and handles reverse-direction provider events. It has at most one route in a closed root component.

Example fragment: `port orders: Orders requires;`

Graphical notation: Service consumer port. Service consumer port.

Keyword: return

Concern: Behavior and states. **Grammar:** `Calculation`, `Return`.

Completes a function activation or local call with its required result; bare return completes void. In a calc declaration it introduces the one named typed output and pure expression.

Example fragment: `return true; / return result: real = value * gain;`

Graphical notation: Pure calculation, Function return. Pure calculation; Function return.

Keyword: s

Concern: Timing and deployment. **Grammar:** `Duration`.

Second duration unit; one s equals 1,000,000,000 ns. A duration must still fit the exact safe nanosecond range.

Example fragment: `within 2 s`

Graphical notation: Duration value. Duration value.

Keyword: sampled

Concern: Communication. **Grammar:** `DataMember`.

SenderReceiver data mode retaining the latest pending sample: capacity 1 and dropOldest are mandatory. Dispatch removes the sample; the core does not define a remote shared-variable read.

Example fragment: `data temperature: real sampled;`

Graphical notation: Data member. Data member.

Keyword: satisfy

Concern: Cases and evidence. **Grammar:** `Satisfaction`.

Design-to-requirement satisfaction claim.

Example fragment: `satisfy Reliable of Demo;`

Graphical notation: Satisfaction relation. Satisfaction relation.

Keyword: send

Concern: Communication. **Grammar:** `Send`, `ScenarioStep`.

Emits an outgoing senderReceiver data member through a sender port with exactly one value. In a case it describes a data exchange. It cannot publish events or bypass another component's interface.

Example fragment: `send output.temperature(23.5);`

Graphical notation: Data send, Case data exchange. Data send; Case data exchange.

Keyword: sender

Concern: Communication. **Grammar:** `Port`.

Outgoing senderReceiver role. May fan out through multiple explicit connectors; each member has at least one syntactically associated emitting function.

Example fragment: `port output: Samples sender;`

Graphical notation: Data sender port. Data sender port.

Keyword: senderReceiver

Concern: Communication. **Grammar:** `Interface`.

Data-only contract kind with sampled/queued members and sender/receiver roles. No RPC functions or event members may be declared inside it.

Example fragment: `interface senderReceiver Samples version "1.0.0" { data value: real sampled; }`

Graphical notation: Sender-receiver interface. Sender-receiver interface.

Keyword: serial

Concern: Timing and deployment. **Grammar:** `Timing`.

Default activation mode: acquire the component's one serial lock until completion/failure, including RPC waits. Local invokes share the activation lock. Independent component activations may still interleave.

Example fragment: `concurrency serial;`

Graphical notation: Concurrency mode. Concurrency mode.

Keyword: server

Concern: Communication. **Grammar:** `Port`.

Provider role for clientServer contracts. Each function signature has exactly one on call handler; results/failures follow the request correlation.

Example fragment: `port health: Health server;`

Graphical notation: RPC server port. RPC server port.

Keyword: service

Concern: Communication. **Grammar:** `Interface`, `RootComponent`.

As an interface kind, combines RPC functions and events with requires/provides roles. As a root component profile, makes timing contracts optional while retaining interfaces, explicit QoS and static validity; it is the default when no profile is written.

Example fragment: `interface service Orders version "1.0.0" { ... } / component Shop service { ... }`

Graphical notation: Service interface, Root execution profile. Service interface; Root execution profile.

Keyword: specializes

Concern: Structure and names. **Grammar:** `Component`.

Names one visible base component definition. Effective inherited members are reused; specialization and effective containment must be acyclic.

Example fragment: `component def Tuned specializes Controller { }`

Graphical notation: Component specialization. Component specialization.

Keyword: start

Concern: Behavior and states. **Grammar:** `Trigger`.

Once-per-run startup trigger for a parameterless void function. Stateful functions require it. All stores/routes/handlers exist before startup work runs; readiness protocols still need explicit ports.

Example fragment: `function boot(): void on start { ... }`

Graphical notation: Startup activation. Startup activation.

Keyword: state

Concern: Behavior and states. **Grammar:** `State`.

Declares a named state directly inside a function after its initial reference. Optional entry/exit blocks contain ordinary function statements. State names are function-local; no separate state allocation or performer exists.

Example fragment: `state Idle { entry { active = false; } }`

Graphical notation: State, Final state. State; Final state.

Keyword: string

Concern: Values and calculation. **Grammar:** `TypeRef`.

Finite Unicode text with exact equality and no normalization. Strings support portable escapes; implicit concatenation, ordering and bounded transport size are absent.

Example fragment: `var lastError: string = "";`

Graphical notation: Primitive types. Primitive types.

Keyword: subscriber

Concern: Communication. **Grammar:** `Port`.

Incoming publishSubscribe role. Multiple publishers may feed one subscriber port through distinct queues. Each event has one handler owner; cross-source ordering is unspecified.

Example fragment: `port input: Notices subscriber;`

Graphical notation: Event subscriber port. Event subscriber port.

Keyword: sv

Concern: Graphical views. **Grammar:** `View`.

Sequence View of a case or a root component's selected case. Lifelines are occurrences; arrows show port/member/connector and request correlation. Independent steps must remain visibly unordered.

Example fragment: `view checkout sv of Checkout;`

Graphical notation: Sequence View. Sequence View.

Keyword: timing

Concern: Timing and deployment. **Grammar:** `Timing`.

Function contract containing wcet, deadline and concurrency in that order. Required on used functions in realtime/mixed roots; stateful function timing applies per reaction.

Example fragment: `timing { wcet 1 ms; deadline 5 ms; concurrency serial; }`

Graphical notation: Function timing contract, Function timing diagram. Function timing contract; Function timing diagram.

Keyword: transition

Concern: Behavior and states. **Grammar:** `Transition`.

Named state edge owned by one function, with source/target, stimulus, optional guard/priority and effect. Execution orders exit, effect, state update, entry, then completion processing.

Example fragment: `transition beginRun: Idle -> Running on receive commands.begin {}`

Graphical notation: State transition. State transition.

Keyword: true

Concern: Values and calculation. **Grammar:** `Literal`.

The true bool literal, with no implicit equivalence to numeric one or other nonempty values.

Example fragment: `assert true;`

Graphical notation: Expressions and value paths. Expressions and value paths.

Keyword: uint

Concern: Values and calculation. **Grammar:** `TypeRef`.

Exact nonnegative integers from 0 through 2^53âˆ’1. Scalar widening is supported; assigning an integral expression to uint checks its range instead of wrapping.

Example fragment: `var count: uint = 0;`

Graphical notation: Primitive types. Primitive types.

Keyword: unordered

Concern: Communication. **Grammar:** `QosBody`.

No source-language dispatch ordering guarantee beyond other causal rules. A deterministic backend may still choose FIFO; this choice must not be mistaken for a model guarantee.

Example fragment: `ordering unordered;`

Graphical notation: Queue ordering. Queue ordering.

Keyword: us

Concern: Timing and deployment. **Grammar:** `Duration`.

Microsecond duration unit; one us equals 1,000 ns. The source spelling is ASCII us, not the micro sign.

Example fragment: `wcet 100 us;`

Graphical notation: Duration value. Duration value.

Keyword: useCase

Concern: Cases and evidence. **Grammar:** `Scenario`.

Intended use with optional routed exchanges.

Example fragment: `useCase Start of Demo {}`

Graphical notation: Use case. Use case.

Keyword: var

Concern: Values and calculation. **Grammar:** `Variable`, `LocalVariable`.

Declares mutable private component data or a block-local copied value. Ownership determines scope; local declarations cannot shadow live values.

Example fragment: `var count: int = 0;`

Graphical notation: Private component variable, Local declaration. Private component variable; Local declaration.

Keyword: verificationCase

Concern: Cases and evidence. **Grammar:** `Scenario`.

Verification intent with explicit verify links.

Example fragment: `verificationCase Check of Demo { verify Reliable; }`

Graphical notation: Verification case. Verification case.

Keyword: verify

Concern: Cases and evidence. **Grammar:** `Verification`.

Links a verification case to a requirement.

Example fragment: `verify Reliable;`

Graphical notation: Verification relation. Verification relation.

Keyword: version

Concern: Communication. **Grammar:** `Interface`.

Required interface semantic-version string major.minor.patch. Compatibility is nominal declaration identity, not automatic version-range matching or structural subtyping.

Example fragment: `interface clientServer API version "1.0.0" { ... }`

Graphical notation: Sender-receiver interface, Client-server interface, Publish-subscribe interface, Service interface. Sender-receiver interface; Client-server interface; Publish-subscribe interface; Service interface.

Keyword: via

Concern: Cases and evidence. **Grammar:** `ScenarioStep`.

Names the root-level connector used by a case exchange. Endpoint-member paths must resolve on root-owned or immediate-child component ports with a legal orientation.

Example fragment: `via orderLink label submitted;`

Graphical notation: Case data exchange, Case event exchange, Case request, Correlated case reply. Case data exchange; Case event exchange; Case request; Correlated case reply.

Keyword: view

Concern: Graphical views. **Grammar:** `View`.

Declares a named gv/iv/cv/sv/grv/fv projection. Compact of syntax selects one subject; a block combines include selections, optional recursive traversal and pure where filters into a stable union of occurrence paths. It carries no executable behavior or diagram geometry.

Example fragment: `view wiring iv of Product;`

Graphical notation: General View, Interconnect View, Case View, Sequence View, Grid View, Flow View. General View; Interconnect View; Case View; Sequence View; Grid View; Flow View.

Keyword: void

Concern: Values and calculation. **Grammar:** `TypeRef`.

No result value. It is the default function/signature result and cannot type data, input parameters, fields, arrays or calculation outputs.

Example fragment: `function reset() { return; }`

Graphical notation: Primitive types. Primitive types.

Keyword: wcet

Concern: Timing and deployment. **Grammar:** `Timing`.

Positive active-execution-time bound including local helpers and state reaction microsteps, excluding request wait. Must not exceed deadline; physical evidence belongs to a target binding.

Example fragment: `wcet 1 ms;`

Graphical notation: Function timing contract, Function timing diagram. Function timing contract; Function timing diagram.

Keyword: when

Concern: Behavior and states. **Grammar:** `Transition`.

Optional pure bool transition guard evaluated against the component snapshot/live environment and stimulus payload as applicable. False consumes/ignores the stimulus when no other transition matches.

Example fragment: `on receive commands.begin when (enabled) {}`

Graphical notation: Transition guard. Transition guard.

Keyword: where

Concern: Graphical views. **Grammar:** `ViewSelection`.

Filters selected candidates with a Boolean predicate.

Example fragment: `include Demo recursive where kind == "function";`

Graphical notation: Filtered view selection. Filtered view selection.

Keyword: while

Concern: Behavior and states. **Grammar:** `While`, `DoWhile`.

Checks the Boolean condition before each while iteration or after each do-loop body. Both forms require a positive safe-integer bound and a fresh lexical body scope. A true condition after the bound fails instead of silently exiting.

Example fragment: `while (i < 3) bound 3 { i = i + 1; }`

Graphical notation: Bounded loop, Post-test do-while loop. Bounded loop; Post-test do-while loop.

Keyword: within

Concern: Communication. **Grammar:** `RemoteCall`.

Required positive RPC timeout measured from issuance. Only a response observed strictly before expiry can win; equal-time expiry wins and late server effects are not canceled.

Example fragment: `within 100 ms otherwise { return; };`

Graphical notation: Routed function call. Routed function call.

Punctuation and operator reference

Token	Meaning
`;`	Declaration/statement terminator where the production requires it.
`{ }`	Owned declarations, block scope, record assignments, QoS or timing block according to context.
`( )`	Arguments, parameters, conditions or precedence grouping.
`[ ]`	Fixed type extent, array literal or one final index read.
`:`	Name/type separator, function result or connector/transition header separator.
`,`	List separator; no implicit trailing comma.
`::`	Package qualification; imports append `::*`.

Token	Meaning
.	Structural/member path, local record field, enum literal separator or real-literal decimal point.
->	Connector, delegation, state transition or case exchange direction.
=	Initialization, assignment, calculation result expression or record field assignment; never equality.
== !=	Typed recursive equality/inequality; numeric scalar promotion permitted.
< <= > >=	Numeric ordering; no string ordering.
+ - * / %	Checked numeric arithmetic; slash always real, remainder integral only; minus also unary.
// /* */	Line / non-nested block comments outside strings.

Precedence, lowest first: or; and; equality; ordered comparison; addition/subtraction; multiplication/division/remainder; unary not/minus; primary. Binary operators associate left. Only and/or short-circuit. Identifiers, strings, numerics and durations have the exact lexical rules in chapter 1; the complete grammar is authoritative for concrete syntax.

NORMATIVE / LANGUAGE CONTRACT

SWAML language specification

Version: 1.3.0 · **Source:** headerless packages; parser/host pinned to 1.3 · **Status:** released · **Date:** 2026-10-18

SWAML means Software Architecture Modeling Language. This document and its normative chapters define a software architecture language with a proposed KerML semantic mapping. SWAML is independently authored and does not denote an OMG standard or UML compatibility level. Architects familiar with UML can begin with the informative UML transition guide; implemented capabilities are identified separately from mapping and execution contracts.

1. Normative document set

Read these chapters together. Their requirements have equal standing:

1. Structure, names and types.
2. Interfaces and communication.
3. Function and state semantics.
4. Timing, deployment, cases and conformance.
5. KerML semantic mapping.
6. Graphical notation and canonical concept catalog.
7. Concrete grammar.
8. Operational contract and target bindings.
9. Normalized semantic model.
10. Conformance obligations and evidence.
11. Formal core and simulation relation.
12. Machine-readable mapping inventory.

The user guide, architecture comparison, examples and design decisions are informative. The implementation matrix reports current tool support; a missing implementation does not waive a language rule.

2. Normative terms and precedence

MUST and MUST NOT state conformance requirements. SHOULD states a recommendation with an explicitly documented justification needed for deviation. MAY grants permission. “Error” means a model is not valid under this version. An implementation may retain an invalid model for editing but MUST NOT report it as fully valid.

The grammar determines tokens and parse structure. Prose determines validity and meaning. If grammar accepts text that violates a semantic rule, that text is invalid. If a discovered conflict makes the two impossible to satisfy, record a specification defect; do not silently choose a runtime meaning. Unknown syntax and unsupported requested language versions MUST be rejected. The host selects a pinned 1.3 parser before loading source; a file does not guess its own edition. See the 1.3 migration guide and historical 1.1 migration guide for the intentional syntax migration from 1.0.

The language is closed at version 1.3: constructs not listed in the grammar are not implicitly inherited from UML, SysML or KerML. Component definitions support instantiation, single specialization and compatible explicit member redefinition. Generics, unrestricted KerML embedding and dynamic creation remain outside this version. External system identities support deployment integration without importing another modeling language.

3. Design invariants

- **I1 - Boundary:** every exchange between distinct component instances crosses port instances typed by the same interface contract and a permitted connector/delegation path.
- **I2 - Ownership:** components own child components, ports, private data and functions; functions own calculations and statement flow or states. No performer element, behavior allocation, free action, free signal, or global variable exists.
- **I3 - Event provenance:** after resolving any delegation, every outgoing data/event member has at least one syntactically associated emitting function in the resolved owning component. Every incoming member has exactly one handling function in that context, including a stateful function or interruptible-region owner where applicable. A delegated boundary has no additional local handler or emitter.
- **I4 - Type safety:** payloads and results conform to nominal contracts. Messages transfer values, never aliases to another component's private state.
- **I5 - Explicit communication policy:** every connector declares delivery, ordering, capacity and overflow. Time-sensitive connections add latency/freshness bounds.
- **I6 - Views preserve meaning:** views reference existing semantic elements. Drawing a message cannot create a hidden communication channel.

Local calls, calculation values, timer expiration, initial activation and state completion do not constitute exchanges between components. They are the only exceptions needed for local computation and lifecycle. An external actor or device that exchanges model data MUST be represented through a component port; a case actor label cannot bypass I1.

4. Abstract model inventory

Element	Owns	References / meaning
Model	packages	language edition selected by the parser/host
Package	imports and named declarations	namespace
DataType / EnumType	typed fields / literals	immutable nominal value shape
Interface	data, event or function signatures	nominal versioned communication contract
Component definition	child component instances, ports, variables, functions, connectors and delegation	reusable software boundary; optional base definition
Component instance	distinct private stores and expanded effective members	one component definition; multiplicity one
Root component	inline members or a component-definition instance; deployment bindings	closed architecture selected for execution; default service profile
Port	role and optionality	one interface
QoS policy definition	complete policy values	package-owned reuse without queue sharing
Connector / delegation	policy selection / boundary alias	explicit typed route / same-contract forwarding
Function	typed parameters/result, trigger, timing, calculations; statements **or** states	executable behavior owned by one component

Element	Owns	References / meaning
Calculation	typed inputs, named non-void output and expression	pure value computation owned by one function
State / transition	entry/exit blocks / trigger, guard, priority and effect	behavior directly owned by one function
Deployment	no system definition	component-instance path and external system/part identity
Requirement	immutable typed attributes and one optional formal doc	named obligation; incoming satisfy/verify links
Use / verification case	optional doc, verification links where permitted, and exchange steps	one root context selected with of; intended use / verification intent
View	kind, optional doc and one or more subject selections	stable union of occurrence paths; pure optional filters; no runtime effects

Definitions and instances retain different identities. Specialization and redefinition reference source declarations; neither shares private state across instances. Tools SHOULD persist a separate identity map for editing. A rename preserves identity only when the tool updates that map and all references atomically.

5. Conformance summary

A parser supports concrete syntax. A semantic validator additionally enforces static rules. A KerML mapper additionally produces the normalized representation in chapter 5. An execution backend additionally realizes chapter 3 and communication policies. A graphical editor additionally satisfies the view/editing contract. A tool MUST state which levels it implements; success at one level MUST NOT imply the others.

This repository supplies a parser, LSP foundation, documented static validation and a reference simulator. The specification defines more than that implementation, especially runtime delivery, idempotency, target snapshot realization, timing proof, interchange and graphical editing. See the exact support matrix.

Requirements and formal documentation

Requirements own typed attributes and one optional `doc /* ... */` specification. Design elements link through `satisfy`; verification cases link through `verify`. Use cases describe intent. Ordinary comments remain independent notes. View kinds use lower camel case, and view blocks combine multiple selections and Boolean filters. See the guide and complete model.

Extended function flow and timing

The 1.3 edition adds counted for, copied-array forEach, bounded do loops and typed interruptible regions. FV includes trace-backed function timing. See the control-flow guide, 1.3 migration and design decision.

NORMATIVE / LANGUAGE CONTRACT

1. Structure, names and types

Normative chapter of the SWAML 1.3 specification.

1.1 Lexical model

Source files MUST use UTF-8 and the `.swaml` extension. Whitespace and ordinary `//` line comments and `/* … */` block comments are insignificant to execution outside strings. `doc /* … */` is a formal owned specification, retained in the semantic model. Block comments do not nest. Identifiers match `[_a-zA-Z][a-zA-Z0-9_]*` and are case sensitive. Every quoted word in the grammar and the lexical doc marker are reserved. All keywords use lower camel case, including primitive types and view kinds. This version has no escaped identifiers.

Strings use double quotes and backslash escapes. A conforming implementation MUST support `\"`, `\\`, `\n`, `\r`, and `\t`; other escape sequences are invalid in conforming source. Raw control characters U+0000-U+001F, including line breaks and tabs, MUST NOT occur inside strings; use the permitted escapes. The lexer enforces this portable subset.

Unsigned decimal integers and decimal real literals are supported; a minus sign is a unary operator. Decimal real literals require digits on both sides of the point. Hexadecimal, exponent and suffix forms are absent. Duration literals consist of a positive integer followed by `ns`, `us`, `ms` or `s`. A duration MUST normalize to an exact integer nanosecond count no larger than 2^53-1. Time has no implicit unit.

The document contains zero or more packages, with no language header. The parser and its host MUST select SWAML 1.3 explicitly before loading the workspace; the CLI in this edition supports exactly 1.3. A host MUST NOT silently select whichever language version is newest. Historical `swaml "1.0";` headers are rejected by this grammar. An empty model is valid as an editing/library artifact, but contains no root component to execute.

1.2 Namespaces and resolution

Packages are top-level and non-nested in 1.3. A package name MUST be unique in the loaded model workspace. A package is declared in one file, although one file may contain several packages. Named elements in the same namespace MUST be unique; overload resolution is not supported.

`A::Type` identifies a package declaration. `import A::*;` exposes A's top-level declarations by short name in the importing package. Imports are not transitive. A short reference MUST resolve to exactly one declaration across the local package and its direct imports. Ambiguity is an error even when one candidate is local. Fully qualified references do not require an import.

Periods express containment/feature paths, not package qualification. A structural instance path follows actual child instances. A behavior communication path has exactly two identifiers: `localPort.member`. Directly following another component's data or function is forbidden. Pure record field paths follow only values already in the function's local environment.

Local function references resolve only in the owning component. Calculation references resolve only among calculations declared by the enclosing function, including its state guards and action blocks. State names resolve only inside the owning function's state body. Deployment destinations are external system identities; they do not participate in source-name lookup. No lookup fallback may expose a similarly named declaration from another component.

1.3 Definitions, instances and specialization

`component def C` declares a reusable software type. `component c: C;` declares an instance of that type, with multiplicity exactly one. Repeated instances require separate declarations. A component may own child component instances, ports, connectors, delegation, private variables, executable functions. Functions own calculations. A component with no children uses exactly the same ownership and execution rules as a composite component. `def` distinguishes a reusable definition from a concrete top-level instance.

A package may declare a **root component instance**: `component Demo { … }` supplies inline composition and local behavior; `component Demo: Assembly;` instantiates a reusable definition. A typed root may have a deployment-only body, for example `component Demo: Assembly realtime { deploy controller on "sysml:Vehicle::ecu"; }`. Its architecture and behavior come entirely from the definition; that binding body MUST NOT extend or redefine them. Inline roots may own the same structural and behavioral members as a definition, plus deployment bindings. All roots are closed execution contexts: every required port, including a root-owned port, is checked for closure.

A root has optional `service`, `realtime` or `mixed` profile following its name or type. Omission means `service`. A root is a named instance, never a definition, and cannot be used as a child's type. Definitions create no runtime participants until instantiated.

`component def Specialized specializes Base { … }` declares single inheritance from a visible component definition. The specialization graph MUST be acyclic. Effective members consist of inherited members plus newly declared members, with explicit compatible replacements. Reusing an inherited name without `redefines` is an error; `redefines` without a matching inherited member is an error. For example, `redefines function initialize() on start { … }` replaces that inherited function. Redefinition replaces the whole member rather than merging bodies or private values. Inherited behavior resolves local member names in the effective specialized component.

Redefinition MUST retain the member category and its exposed contract. A port keeps its interface identity, role and optionality; a variable keeps its exact value type; a child instance keeps its declared component type or specializes that type; a function keeps its parameter names/types, result, statement/state body category, trigger kind/endpoint/period and concurrency contract, and does not increase a declared WCET or deadline. Connectors and delegations keep their endpoint paths. A replacement may change an initializer, compatible function body (including its local calculations) or connector policy, subject to every ordinary validity rule. Specialization supplies bounded source reuse; it does not introduce dynamic creation, reference-valued objects or class dispatch.

The effective containment graph MUST be acyclic, including inherited child instances. Every instance receives distinct private storage, local locks, function activations and child instances. Two instances of the same definition require typed ports and explicit routes to communicate; sharing a definition never shares private state.

Library definitions may leave required ports unconnected until instantiated. A closed root MUST connect every nonoptional port after resolving delegation. A component port can be handled by that component's local functions or delegated to a child, but cannot simultaneously acquire two behavior owners. An optional port cannot weaken a required alias elsewhere on its route.

Component state is private to its instance. A variable has a declared type and a pure initializer independent of mutable variables and communication. Function/calculation parameters and local variables MUST NOT shadow component variables or other live values in their scope. Blocks create lexical scopes. Parameters are copied input values; assignment changes the local copy only.

1.4 Value types

Type	Domain / semantics
`bool`	`true` or `false`
`int`	exact signed integers between -(2^53-1) and 2^53-1
`uint`	exact integers from 0 through 2^53-1
`byte`	exact integers from 0 through 255
`real`	finite IEEE 754 binary64 values, with nearest-even arithmetic
`string`	finite Unicode text; no implicit encoding or transport size guarantee
`void`	no result value; the default when a function/signature omits `:` Type, never a variable, parameter, field, array or calculation result
`datatype`	a finite record with named typed fields
`enum`	a nominal set of one or more unique literals
`T[n]`	a fixed array of exactly n values, n a positive safe integer

Datatypes and enums are nominal: two declarations with identical members are distinct types. Recursive records, including recursion through arrays, MUST be rejected. There are no pointers, references, nullable values, dynamic collections, aliases or generics in 1.3. `byte` is an unsigned eight-bit value domain. Other fixed-width encodings and bounded strings are target-binding choices; they are not inferred from these primitives.

`record T { field = expression; … }` MUST assign every field of T exactly once and no others. `T.Literal` MUST name a declared enum literal; a package-qualified form is `Package::T.Literal`. `[e1, e2, …]` constructs a nonempty fixed array; its elements MUST have one nominal type or a numeric common type. Identical numeric element types are retained; mixed numeric types yield real if any is real, otherwise int if any is int, otherwise uint. A literal assigned to a declared array type checks each element against that element type, so `[0, 255]` can initialize `byte[2]`. Existing array values have invariant element type and extent. Arrays of differing lengths are incompatible.

Enum and local-value paths resolve deterministically. If the first segment of an unqualified dotted path is a live lexical/private value accessible in the current expression environment, the whole path is a local value read; a missing field is an error and MUST NOT fall back to an enum. Otherwise a two-segment `T.Literal` path resolves T among declarations in the current package and direct imports, checking ambiguity across all declaration kinds before requiring an enum. `Package::T.Literal` explicitly selects a loaded package enum without using the local-value environment. A wrong declaration kind, unknown literal, ambiguous name or array index applied to an enum is invalid. Calculations expose only parameters to that environment, and variable initializers expose no mutable values. A component variable therefore does not shadow an enum where that variable is inaccessible. No variant permits cross-component private-value lookup.

Record fields may be read by `value.field.subfield`. `arrayValue[index]` reads one array element, with an integral index. Only one final index suffix is available in 1.3; a local variable can hold an intermediate array value. Bounds are zero inclusive to length exclusive. Out-of-bounds access fails the current function at runtime and MUST NOT produce an undefined value. Whole-value assignment is supported; mutating a record field or array element in place is not part of 1.3.

Assignment, argument passing and return preserve nominal types and exact array lengths. Scalar numeric widening follows `byte → uint → int → real`. Assigning an integral value to `uint` or `byte` performs a range check: a known out-of-range constant is a static error, and a computed out-of-range value fails execution.

real values do not implicitly narrow to integral types. Arrays do not implicitly widen or narrow. Enum/integer conversion is absent.

Numeric overflow, non-finite results, division by zero and invalid indexing fail execution. The parser/validator rejects out-of-range literal integers, but proving arithmetic ranges requires execution or additional analysis.

1.5 Expression rules

From lowest to highest precedence: `or`, `and`, `== !=`, `< <= > >=`, `+ -`, `* / %`, unary `not -`, primary expressions. Binary operators associate left. Parentheses override precedence. `and` and `or` short-circuit left-to-right. Other operands and call arguments are evaluated left-to-right.

Boolean operators require bool operands. Ordered comparison and arithmetic require numeric operands. Equality compares equal nominal value types recursively (records by declared fields, arrays by ordered values), or two numerics after promotion. string equality is exact code-point equality with no normalization; no ordering or concatenation operator is defined. `%` is integer remainder with the sign of the dividend and requires integral operands. `/` always returns real. Other arithmetic returns real if either input is real and int otherwise.

A calculation is declared only at the beginning of a function body, before its statement sequence or initial-state declaration. It is not a component member and cannot be independently inherited or redefined. Redefining the function replaces its whole calculation scope. A calculation has the syntax `calc twice { in x: real; return result: real = x * 2.0; }`. It declares zero or more ordered typed inputs and exactly one named non-void output defined by a pure expression. Its environment consists only of parameters, literals, record/enum/array construction and other calculations in the same function. It MUST NOT capture component variables, enclosing function parameters, block locals or state payloads, call functions, or communicate. Needed values must be passed as explicit arguments. Local call graphs MUST be acyclic in 1.3. `name(arguments)` evaluates the expression once with copied arguments and returns its value. The output name labels the result; it is not a mutable local or an input to its own expression. `twice(3.0)` supplies ordered copied inputs and yields the output value. Calculations can be used in local variable initializers, conditions, state guards/actions and other expressions of their enclosing function. Component variable initializers cannot call calculations. To reuse work across functions, define a local value-returning function and use a call statement. A calculation has no statement block, trigger, state or communication effect.

1.6 Requirements for tools

A reference that cannot be resolved, has the wrong target kind, or is ambiguous is an error. Tools MUST distinguish declaration paths, instance paths and local value paths even when their spelling is similar. A tool supporting rename MUST update declaration references and structural/member paths together, preserving the identity of each referenced occurrence.

NORMATIVE / LANGUAGE CONTRACT

2. Interfaces and communication

Normative chapter of the SWAML 1.3 specification.

Target profiles own security, authorization, integrity, serialization, encryption and transport while preserving the interface boundaries below. Chapter 6 specifies admission, timeout races, lifetime and failure.

2.1 Contract identity and members

An interface declaration has a kind, package-qualified name, explicit `major.minor.patch` version string, and at least one member. Its identity is its declaration plus that version. Member names are unique across the entire interface; function overloading is absent.

Kind	Permitted members	Port roles
`senderReceiver`	`data name: T sampled;` or `queued;`	`sender`, `receiver`
`clientServer`	function signatures	`client`, `server`
`publishSubscribe`	event signatures	`publisher`, `subscriber`
`service`	function and event signatures	`provides`, `requires`

A function signature declares ordered input parameters, an optional result annotation, and optionally `idempotent`. Omitting `: Type` means void, exactly as an explicit `: void`. An event signature declares zero or more ordered payload parameters and no result. Data is one typed value. A notification without payload uses a zero-parameter event. No event may be declared directly in a package, component or state.

A service field is modeled explicitly as a getter function, optional setter function and changed event. This avoids a second implicit communication mechanism. A setter returns an application result; its implementation owns the decision whether and when to publish the changed event. Property storage never becomes remotely writable data.

Version strings document compatibility intent: a breaking change SHOULD increase major; additive changes SHOULD increase minor; a meaning-preserving correction SHOULD increase patch. However, 1.3 performs **exact nominal matching**, not automatic version negotiation. Use a differently named declaration for another version in the same package and an adapter between versions. Adding an event may require a new receiver and is not automatically compatible with an existing complete implementation.

2.2 Ports and boundary exposure

A port belongs to one component and references exactly one interface. Its role must be legal for that interface kind. `optional` permits the port to have zero connections in a closed root component; it does not relax type, direction, handler or emission requirements.

A connector joins owned or immediate-child ports: `connect name: child.port -> child.port qos { … };` or `connect name: localPort -> child.port qos { … };`. Its owner is a component definition or inline root; a typed root uses its definition's connectors. Each endpoint has one or two segments and references a port, not a member, function, variable, state or grandchild. The interface declarations MUST be identical. Explicit routes may connect complementary ports of the same component instance. Such exchanges still use the connector queue and handler dispatch; local calls and calculations remain available for synchronous local work.

Legal connector orientations are sender → receiver, client → server, publisher → subscriber, and requires → provides. Service roles also support provider-to-consumer event traffic over the same contract; the arrow denotes the request orientation, not a unidirectional transport restriction.

The following table restates the direction and closure rules. Counts refer to resolved leaf port connections, not interface member count or UML part multiplicity. Optional ports permit zero connections; otherwise the minimum is one. Multiple connections always require distinct explicit connectors.

Kind	Connector orientation	Permitted traffic	Maximum connections per leaf port
senderReceiver	sender → receiver	data in the arrow direction	sender: many; receiver: one
clientServer	client → server	request forward; correlated result/failure reverse	client: one; server: many
publishSubscribe	publisher → subscriber	events in the arrow direction	publisher: many; subscriber: many
service	requires → provides	request forward; correlated result/failure and events reverse	requires: one; provides: many

A role identifies interaction direction, not visibility or optionality. In particular, `requires` is the service consumer role; `optional` independently controls whether its port may remain unbound. Interfaces describe complete contracts: a consumer cannot silently omit handlers for unwanted incoming members. Separate contracts or an explicit adapter are needed when those responsibilities differ.

A component port may be handled locally or have exactly one delegation to an immediate child port: `delegate outer -> child.port;`. Delegation MUST preserve role and nominal interface. It is a forwarding boundary, with no implicit queue, payload conversion, delivery retry or additional source. Child components can delegate again. To cross a hierarchy, expose a port at each boundary; no connector may tunnel to a grandchild.

At root closure, recursively expand instances, resolve each delegation to a leaf component port, and inspect actual connector endpoints. Every nonoptional port, including boundary aliases, MUST resolve to a connected leaf. Receiver ports have exactly one source; client/requires ports have exactly one provider. Sender, publisher, server and provides ports can participate in multiple connectors. Subscriber ports may receive from multiple publishers. A duplicate connector resolving to the same directed leaf pair is an error; it must not create an accidental duplicate subscription. A component may not expose the same child port under two aliases in 1.3.

A locally handled port and a delegated alias cannot acquire conflicting handlers. Delegation forwards a boundary; it does not grant private access to the child.

2.3 Emission and reception ownership

A locally handled component port's outgoing data/event member MUST occur in at least one `send`/`publish` statement inside one of its functions, including a state entry, exit or transition effect. This is a static provenance relationship, not a proof of reachability or a promise that the function always emits.

Each incoming data/event member MUST have exactly one handler owner: an ordinary function triggered by that member, or one stateful function containing one or more receiving transitions for it. A function signature on a server/provides port MUST have exactly one ordinary function with `on call port.member`. Several transitions in one stateful function count as one owner. Two different functions receiving the same member are invalid. To intentionally fan out processing, use a local coordinating handler that invokes helpers, or explicit subscriber components.

An ordinary handler MUST reproduce the member's ordered parameter types. Names may differ. A data handler has one parameter of the data type; an event handler has the event's parameter list. Both return void. An RPC handler MUST also reproduce the signature's result type.

2.4 Exchange semantics

`send p.data(value);` emits one typed data value through a sender port. `publish p.event(args);` emits one typed event through a publisher/provides port. Both paths are exactly `localPort.member`. The transfer copies the value; a target implementation may use zero-copy storage only if it preserves isolation and observable value semantics.

For each outgoing permitted connector, the communication system independently offers the message to that connector's destination queue. Emission does not execute a remote handler on the caller's stack. All fanout offers are attempted. Any `reject` fails the emitting activation after those decisions; accepted offers cannot be rolled back. No interconnector offer order is implied. Programs needing atomic broadcast must model a protocol explicitly.

`call p.member(args) as result within duration otherwise { … };` issues one correlated RPC to the single provider selected by its connector. The result name becomes available in the current block only after successful completion. The caller is suspended until a result or failure; this is an awaited request in the model, independent of whether generated code uses a blocking call or a future.

The communication system creates a fresh correlation identity for each request and carries its response back through the same interface/port/connector route. That response is the function's declared result, not an independent event or unmodeled channel. A server return and a service event are distinct occurrences and have no implicit order beyond their function's control flow.

The `otherwise` block executes on timeout, unavailable optional provider, admission rejection, transport failure or server `fail`. Its local `failure: string` is respectively `Timeout`, `Unavailable`, `Rejected`, `Transport`, or `RemoteFailure`. Remote implementation error strings are not automatically disclosed. Every path of the failure block MUST return from or fail the enclosing function. No result is bound on failure. The implicit failure name must not shadow a live value. A void success name labels completion only and cannot be used as data.

The timeout starts at issuance and includes admission, queueing, service execution and return transport. Timeout does not cancel committed server effects. A late response to that correlation is discarded. Automatic application-level retry is absent. A backend using retries to satisfy delivery policy MUST preserve correlation and timeout budgets. Application-specific unsuccessful results remain typed results, not communication failures.

An outgoing optional unbound data/event port accepts an emission with no receivers; an unbound request fails as Unavailable. Incoming optional ports retain their handlers even if no external source is connected.

2.5 Queue and delivery policy

Every connector MUST explicitly select its complete policy, either with an inline `qos { ... }` block or `qos PolicyName`. A reusable policy is declared at package scope as `qos def PolicyName { ... }`. The name follows ordinary package/import/qualified-name and cross-kind ambiguity rules. There are no implicit policies, inheritance, overrides or partial bodies. An unresolved reference is an error.

Every inline or named policy body MUST specify all four base QoS fields, in the grammar's order:

Field	Meaning
`delivery bestEffort`	messages may be lost, no retransmission obligation; accepted unique transmissions are not deliberately duplicated
`delivery atMostOnce`	delivery may be absent, but a given logical message is presented to a handler no more than once
`delivery atLeastOnce`	admitted messages may be retried and duplicated; eventual delivery is required only under a live connection, available receiver and unexpired lifetime/budget

Field	Meaning
`ordering fifo`	per connector and per member, presented messages preserve sender emission order; duplicates may repeat an item before the next item
`ordering unordered`	presentation order is unspecified
`capacity n`	positive bound on buffered messages per connector, direction and member; executing activations are not in this queue
`overflow dropOldest`	remove the oldest buffered message and admit the new one
`overflow dropNewest`	discard the incoming message
`overflow reject`	refuse admission and signal the emitting activation

Overflow and expiration govern admission/retention and may intentionally discard messages even when delivery is atLeastOnce. This policy is therefore not a promise of lossless application execution. Reliability is separate from handler success; processing that fails is not silently repeated. AtLeastOnce RPC is permitted only when every function on the interface is declared idempotent. A runtime/implementation MUST substantiate idempotency (including event side effects); the declaration alone is not a proof.

Named policy bodies are checked even when unused. Each use is checked against its actual interface and root: sampled buffering, idempotent RPC and realtime bounds are never waived by a reference. Reuse copies policy values conceptually; it does not share queues or correlations. The connector retains the named declaration identity as provenance. `qos OtherPackage::PolicyName` selects a qualified declaration; short names require local or directly imported visibility and must be unambiguous.

Exactly-once delivery is deliberately not a QoS option. Application-level deduplication and transactional processing can be modeled as functions and state, but depend on storage and failure assumptions outside the connector.

Sampled data retains only the latest pending value for that member. Any connector on an interface containing sampled members MUST use capacity 1 and dropOldest; this single connector policy also applies to other members of that interface. Use separate interfaces if mixed buffering policies are needed. Queued data preserves individual samples subject to explicit capacity/overflow. Receiving a sampled value is an activation with that value; there is no hidden cross-component shared variable or implicit read function.

Subscriptions are active while their endpoint components are alive. Pub/sub does not require a central broker; a logical connector describes the allowed subscription. Broker placement and behavior can be modeled explicitly. Runtime discovery, topic wildcards, content filters, retention persistence and dynamic subscription management are outside 1.3 and MUST NOT be assumed. Use explicit alternate root components or adapter components to document such deployment variants.

2.6 Timing of messages

`maxLatency` is a bound from accepted emission to availability for destination dispatch, excluding handler execution. `maxAge` is a maximum age since the producing emission timestamp; expired messages are discarded before dispatch. Clock translation across external deployment targets is an implementation obligation. If a backend cannot establish a sufficiently bounded common time basis, it cannot claim the corresponding bound.

Request and response each have their own directional queue. A service event has its provider-to-consumer event queue. The connector QoS applies to each independently. Capacity is not shared between request, response and event traffic. A timed-out correlation does not keep its response queue alive indefinitely.

`maxLatency` and `maxAge` constrain execution; they supply neither simulated delays nor measured timing evidence. A conforming checking executor MUST report violations.

NORMATIVE / LANGUAGE CONTRACT

3. Functions, flow and states

Normative chapter of the SWAML 1.3 specification.

3.1 Function identity and activation

A function belongs directly to a component definition or an inline root and executes in its owning component instance. It has a name, ordered parameters, optional result annotation, optional trigger, optional timing contract, zero or more function-local calculations, and exactly one executable body form: a statement block or directly owned states. Omitting `: Type` means void; an explicit `: void` has identical meaning. Both leaf and composite components may execute their own functions. Each function executes against its owning component instance, independently of that component's children.

An **activation root** is one dispatched function activation or state reaction and its synchronous local-call stack. It owns the execution lock/snapshot context inherited by helpers. This runtime term is distinct from a **root component**, which selects architecture and may also own local functions.

Trigger	Activation and permitted signature
omitted	local `call`; untriggered flow functions may return void or a typed value
`on manual`	parameterless void activation activated by a test/administrative harness, carrying no model data
`on start`	parameterless void function, once after component initialization and connector readiness
`on every D`	parameterless void function released at component start + kD, for positive integer k
`on receive p.m`	copied data/event arguments, void result
`on call p.m`	copied RPC arguments and exact contract result

Manual activation is a control of the modeling environment, not a way to inject an external data exchange without a port. A modeled UI command, interrupt payload, device sample, network event or external client MUST use an interface. Timers and initial activation are local lifecycle mechanisms.

Several ordinary functions may exist in one component. An interface signature declares an RPC contract; the component function triggered by that member implements it. A stateful function directly contains its states. Reuse of algorithms is through calculations or local functions, and reuse of architecture is through definitions and instances.

3.2 Flow execution

An activation holds its copied parameters, locals, a program position, its component reference and any request correlation. Execute statements in source order. Blocks introduce lexical scope and discard block-local variables on leaving the block.

Statement	Semantics
`var x: T = e;`	evaluate e, typecheck, create a local mutable value
`x = e;`	replace an existing local/parameter/component-variable value; no field or remote assignment

Statement	Semantics
`call helper(args);` / `call helper(args) as result;`	execute an untriggered local flow function synchronously; void omits the binding, a value result requires a fresh binding
`return;` / `return e;`	complete the current function; return a copied value when required
`if (e) { … } else { … }`	evaluate a bool once and execute the selected block; else is optional
`while (e) bound n { … }`	evaluate the bool before each iteration; at most n body executions
`for (var i: T = e; condition; i = update) bound n { … }`	initialize once, test before each iteration, update after each normally completed body; at most n body executions
`forEach (item: T in values) { … }`	evaluate and copy one fixed-size array; execute once per copied element with a fresh typed binding
`do { … } while (e) bound n;`	execute the body once before testing; at most n body executions
`interruptible name { … } on receive port.member { … }`	protect a region and transfer to its typed interrupt handler within the current activation
`assert e;`	require a bool true; false fails the current activation
`fail "reason";`	terminate the activation with a local failure reason
`send` / `publish` / `call port.member(args) as result within D otherwise { … };`	execute chapter 2's typed interface exchanges

Reaching the end of a void function completes it. A value-returning function MUST return a value or fail on every path. If the loop condition remains true after n iterations, execution fails with a bound violation; the nth iteration is allowed. Local recursion is forbidden for both functions. Functions own calculations. `call name(args)` can only target an untriggered flow function in the same component. A non-void result MUST bind a fresh local with `as name`; void MUST omit that binding. Arguments and results are copied, with declared type and range checks. The result is available only after successful return and cannot shadow an existing live value. The helper borrows its caller’s lock and snapshot; failure unwinds the caller without rollback. Functions are never expression calls. `call port.member(args)` selects a routed exchange with mandatory timeout and failure clauses. This two-segment form cannot bypass a component boundary.

Failure unwinds the current activation and its synchronous local callers. It does not undo earlier variable assignments, published messages, responses, or external effects. For an RPC handler, failure produces a RemoteFailure response through the originating interface. For a periodic/start/manual/receive activation it produces a local diagnostic occurrence, not an implicit broadcast. To notify another component, model a fault event and publish it explicitly on a declared interface in an explicit branch before failing.

Conditionals, bounded loops, interruptible regions, expression evaluation and sequential local calls are the full flow language in 1.3. There is no generic fork/join statement, exception hierarchy, arbitrary host-language body or independent action node. Concurrent architectural behavior is expressed by separately activated functions and connected components. A later fork/join extension must define data-race and cancellation semantics before entering the grammar.

Loop scopes and limits

while, for and do loops require a positive safe-integer literal bound. After n body executions, a true condition fails with LoopBound before another body execution. A counted loop's initializer executes once; its local binding is visible in the condition, update and body, and disappears after the loop. Its update sees the header scope, not locals declared in the body. Body locals are fresh for every iteration. A return/failure skips later update and condition evaluation while unwinding its function/frame.

forEach requires a nonempty fixed-size array and an explicit element type. The collection is evaluated and copied exactly once before the first iteration. Each element is copied and checked/widened into a fresh mutable iteration binding. Writes to the source array or iteration binding do not alter the iteration snapshot or other iterations. Array length is the iteration bound; no bound clause is permitted. do executes at least one body, then evaluates its Boolean condition in the enclosing scope; body locals cannot escape into that condition.

Nested loops have independent counters and scopes. A for/while loop may run zero times, so it cannot alone establish a required return. A do or nonempty forEach body that definitely terminates establishes termination on its first iteration. Bounds constrain repetitions, not elapsed physical time.

Interruptible regions

An interruptible region is a named statement owned by a triggered serial flow function. It may carry one doc before its protected body. Its incoming endpoint MUST be a data/event member of a port owned by that component with receiver, subscriber or requires role. The endpoint uses the existing interface and connector/delegation route. Only one receiving function owns that member; it cannot also activate the same function or select another region. Region names are unique in the function. Reentrant functions, untriggered helpers and state actions cannot declare regions; stateful functions retain their receive transitions.

The region is armed when execution enters it and disarmed when the protected body exits or an interrupt is accepted. Delivery while armed cancels the remaining protected work and enters the handler at a statement boundary, including while the activation is suspended on a request. Nested protected regions and local call frames inside the cancelled region are unwound. The handler retains enclosing values and gains copied data payload or named event parameters; these bindings must not shadow enclosing values. Protected-body locals are absent. The same region is disarmed during its handler; an enclosing region may remain armed. After a normally completed body or handler, control continues after the region.

The handler retains the activation identity, serial lock, snapshot and deadline. Previous assignments, emissions and remote effects remain committed. A pending request continuation and its otherwise branch are cancelled; its correlation becomes terminal with an Interrupted reason. A response already terminal remains terminal with its original outcome. Server execution is not cancelled. Future replies cannot resume the unwound continuation. Available interrupt input outside an armed interval is explicitly discarded. One input cannot create a fresh activation.

Return and runtime failure unwind/disarm regions normally; a region catches only its selected interrupt. For value-returning functions, a region establishes definite termination only when both its protected body and handler definitely terminate. See chapter 6 for correlation races and statement-boundary scheduling. This construct does not specify processor interrupt service, pause/resume or physical preemption latency.

3.3 Scheduling and concurrency

All ordinary functions and state-machine reactions are serial by default. The component has one logical serial dispatch lock. A serial activation holds this lock until completion, including while suspended on a request; the logical lock is not required to consume a physical thread. A local call borrows its caller's lock and does not acquire a second lock.

A callback that starts another serial activation in the same component cannot execute until the current serial activation ends. A matching interrupt instead transfers control within the existing serial activation and does not acquire a second lock. Cycles of requests can therefore time out or deadlock absent timeouts; all requests have finite timeouts. The language does not silently release the lock and allow reentrant state mutation. Authors SHOULD avoid synchronous request cycles.

`concurrency reentrant` permits overlapping activations with separate local frames. Such a function MUST NOT mutate component data directly or through local helpers, and MUST NOT call a function that mutates shared state. It may read the component's state snapshot captured at dispatch, and calculations receive copied parameters. A conforming backend must preserve that snapshot or reject the model if its target cannot realize it. The reference validator follows local helper calls to check this prohibition. Snapshot realizability remains a backend obligation.

Input queues are per connector/member. FIFO constrains order within one queue. Across independent ready queues, the scheduler may choose any ready activation consistent with deadlines and the target profile; no global total order is inferred. Periodic activations released while the serial component is busy remain pending; a target profile MUST bound that activation backlog; exceeding the bound records ActivationOverflow and stops the selected root execution, as defined in chapter 6. Deadlines are measured from release, not eventual dispatch. Merely listing WCET/deadline does not define thread priorities.

The core semantics is therefore a set of allowed execution traces. An execution backend may add a deterministic scheduling policy that narrows this set but must document it. A model requiring determinism across sources must explicitly order exchanges or provide a coordinating function.

3.4 Stateful functions

A stateful function MUST have no parameters, result void (explicit or omitted), and `on start`. Its body begins with zero or more calculation declarations, then `initial StateName;`, followed by one or more states and zero or more transitions. There is no `machine` declaration or keyword. The grammar's `StateBody` only groups the owned syntax; it has no separate authored name, definition, allocation or runtime identity. `initial` MUST precede states and transitions after the optional calculations, so the executable body is unambiguously distinguished from statement flow. Calculations remain visible in all state guards, entries, exits and effects. The behavior is mathematically a flat state machine with exactly one initial reference. State and transition names are unique within that function's state body. Hierarchical/orthogonal states, history pseudostates, entry/exit points and state inheritance are absent in 1.3.

Starting the function enters its initial state and executes that state's entry block once. The function remains alive between reactions without holding the serial component lock. Each entry/exit/transition reaction is scheduled as a serial activation unless its timing contract explicitly says reentrant; a stateful function nevertheless MUST have at most one reaction in progress and cannot reenter its own state machine.

A state owns an optional entry block and optional exit block. A `final` state completes the function after its entry finishes and MUST have no outgoing transition. Ordinary completion is not an event emitted to another component. A notification on completion must be published explicitly before completing.

3.5 Transition triggers and selection

A transition names source and destination states in the same function. It has one trigger and optional bool guard and nonnegative safe integer priority. Missing priority means 0; larger values take precedence.

- `on receive port.member` consumes one incoming data/event for the machine. Event parameter names are available as copied values in the guard and effect. Data has the local name `payload`. These names MUST NOT collide with component variables. RPC functions are not allowed as state transition triggers.
- `on after D` becomes eligible after D elapsed time since completing the source state's entry block. Leaving the source cancels its timer. Reentering starts a fresh timer.
- `on completion` is eligible after the state's entry and any prior reaction have finished, before waiting for a new external message.

For one stimulus, first select transitions from the current state with matching trigger, then evaluate guards, then choose the highest-priority enabled transition. Potentially competing transitions with the same source and identical trigger MUST have distinct priorities even when an author believes their guards are mutually exclusive. A missing guard means true. Unrelated queued stimuli are dispatched one at a time, in the scheduler order permitted by 3.3.

A received message with no enabled transition is consumed and ignored. A timer with no enabled transition expires once and is not retriggered until reentry. Completion transitions are reevaluated after each successful state entry and each machine reaction, including an ignored stimulus. Models MUST NOT contain unconditional completion cycles. Guarded cycles must reach quiescence; a checking executor MUST fail a reaction after 1,024 completion microsteps rather than spin forever. This limit is a semantic protection, not a proof of useful behavior.

3.6 Transition execution

A selected transition executes atomically with respect to this component's other serial reactions:

1. Consume the matching stimulus, if any, and cancel all timers for the source state's entry generation.
2. Run the source state's exit block.
3. Run the transition effect with the stimulus payload in scope.
4. Update the current-state identity to the destination.
5. Run the destination state's entry block.
6. Complete if final; otherwise evaluate completion transitions or await a stimulus.

Entry and exit blocks do not inherit event parameters; only the guard/effect does. Copy a needed payload to component data in the effect if entry behavior needs it. A self-transition exits and reenters, including entry/exit effects and resetting timers.

`return;` in any state block terminates the owning stateful function immediately and cancels pending timers; remaining transition phases are skipped. `fail` terminates it with failure and also cancels timers. Neither rolls back prior side effects. Other queued messages still belong to the component, but a terminated machine no longer dispatches its members; a target runtime must record these as undeliverable until component restart.

Every state MUST be reachable from the initial state in the transition graph, ignoring guards for this static check. Reachability through satisfiable guards and eventual completion are stronger analysis obligations, not consequences of graph reachability. No state, function or component may use an unbound global event to change another component's behavior.

The operational contract further defines initialization barriers, snapshots, timer generations, stimulus ties, failure and resource bounds.

NORMATIVE / LANGUAGE CONTRACT

4. Root components, timing, cases and conformance

Normative chapter of the SWAML 1.3 specification.

4.1 Root components and profiles

A root component is the closed occurrence selected for validation, deployment and execution. It uses the same component concept as reusable architecture: an inline `component Demo { … }` owns component children and connectors, while `component Demo: Assembly;` obtains them from a component definition. Several named roots in one package may instantiate the same definition independently or describe alternative assemblies. Root selection is explicit in the simulator/host by package-qualified name, and is not inferred from file order.

A typed root's optional body is a deployment binding only: deployments may reference existing instances; structural members and behavior MUST NOT be added there. An inline root may contain child components and its own private data, functions and ports. All paths in cases, deployment and simulation are relative to the selected root; the root name is not an extra child segment.

Instantiation recursively creates children and local variable storage, resolves delegation and connectors, checks closure, and then enables startup/periodic activation. No component may publish during a partially connected startup phase.

The root profile is `service`, `realtime` or `mixed`; omission means `service`. A root profile selects timing obligations independently of interface kinds: a `service` root may use any interface kind, and a `realtime` root may use a `service` interface. Only the service profile defaults; QoS, interface versions, timeouts and realtime obligations remain explicit:

Obligation	service	realtime	mixed
Explicit connector QoS	required	required	required
Function WCET/deadline/concurrency	optional	required for all used functions	required for all used functions
Connector maxLatency	optional	required	required
Sampled data maxAge	optional	required	required
Interface isolation and handler checks	required	required	required
Production scheduling evidence	target dependent	required for hard realtime claims	required for realtime claims

The mixed profile is intentionally conservative: all used functions and connectors carry timing contracts. It does not infer that a cloud-facing function has no timing requirement. A future subassembly profile can relax this with explicit partitioning; no such implicit relaxation exists in 1.3.

A timing contract declares positive `wcet`, positive `deadline`, and `concurrency serial|reentrant`. WCET MUST be no larger than deadline. For periodic functions, deadline MUST be no larger than the period. A stateful function's timing contract applies separately to each run-to-completion reaction, including entry/exit/effect and any completion microsteps, not to the lifetime of the machine.

WCET is active CPU execution time for the activation including synchronous local helpers. Deadline is elapsed time from activation release to completion, including waiting and interference. Connector

maxLatency excludes destination dispatch wait. For a chain of functions, the end-to-end bound must account for release delay, queue delay, interference, transport, WCET and response delay; simply summing WCET is insufficient.

A conforming validator checks arithmetic consistency. A conforming target backend must additionally prove or measure its declared timing assumptions with an appropriate execution model. The core specifies no rate-monotonic/EDF policy, task mapping, interrupt priorities, memory size, hardware access mechanism or clock synchronization algorithm. Such a backend MUST supply a named external deployment profile, and MUST NOT claim timing conformance without evidence for those obligations.

4.2 Deployment

`deploy instance.path on "sysml:Vehicle::ecu";` associates an existing component instance with an external system or part identity. `system:production::checkout` is another valid reference form. The quoted destination MUST be nonempty and use a URI-like scheme followed by a nonempty identity. The scheme and identity are preserved exactly for a host or target binding to resolve. SWAML does not declare a second hardware/system ontology or load external models through this statement.

A deployment references an instance, not a reusable definition. The root name may be used as its own deployment target, for example `deploy Demo on "system:production";`, to place root-owned behavior and establish its subtree default. If an immediate child has the same name as the root, using that ambiguous name as a deployment target is invalid (SW070). The same explicit instance path may occur in only one deployment. A component deployment applies to that instance and its descendants by default; a more specific descendant deployment overrides the inherited destination. Without a deployment on an instance or ancestor, its effective destination is Unallocated. A valid logical model may be unallocated; a target must supply and validate the placement needed for execution.

For `sysml:` references, the target integration must resolve the identified SysML part/system, preserve its external identity and establish allocation semantics. The string alone is not a SysML import, a validated cross-model link or evidence of SysML/KerML conformance. SWAML's parser checks its local path and reference form; the host checks external existence, kind, availability and execution suitability.

Deployment leaves interfaces, connector identity, handler ownership and private-data visibility unchanged. Serialization, addresses, credentials, task schedules and physical channels belong to the target binding. Every exported mapping must retain the SWAML instance identity and external destination.

Architecture and target scope

The same component and interface constructs describe embedded, service-oriented, web and cloud architectures. Choose each component boundary around a coherent responsibility with private state, functions and optional child components. External deployment references connect this software model to system architecture without embedding a hardware metamodel.

Architecture	Core model	Target binding supplies
AUTOSAR Classic	components for atomic software-component behavior; components for compositions; sender-receiver and client-server contracts; periodic or input-triggered functions; MCU placement	software-component and port mapping, runnable/event mapping, RTE access semantics, task scheduling, ECU data representation and communication configuration
AUTOSAR Adaptive	application components; required/provided service contracts with methods and events; process placement	application and execution manifests, service-instance identities, discovery and availability policy, `ara::com` binding and transport configuration

Architecture	Core model	Target binding supplies
Web / SOA	clients and services with explicit request, reply and notification routes; private service state; process/container placement	API and serialization mapping, endpoints, authentication, lifecycle, persistence and failure handling
Cloud services	independently placed services; explicit broker/storage adapters where they participate in the model; connector queues and retry contracts	broker and storage guarantees, infrastructure placement, resource limits, restart policy and replica routing

These mappings are target obligations, not additional core syntax. A function is a behavior owner; mapping it to an AUTOSAR runnable or a web request handler requires an explicit activation and concurrency mapping. An external deployment reference alone does not establish that mapping. An interface's nominal version identifies its SWAML contract; it does not imply wire compatibility with a platform interface.

The 1.3 instance tree and connector topology are fixed for a selected root. Multiple replicas are separate authored occurrences with separate private stores. Discovery, replacement or scaling in a target system MUST preserve the modeled endpoint identities, routes and state isolation; changes to the modeled topology require a new root configuration. A deployment binding cannot introduce a new peer, a shared store or an unmodeled exchange.

The architecture patterns provide complete examples and explain the additional platform decisions for each mapping. The execution obligations are defined in chapter 6.

4.3 Requirements and cases

`requirement Name { doc /* obligation */ attribute id: string = "REQ-001"; }` declares a named obligation with optional formal documentation and immutable typed attributes. Requirements have no subject. Attribute names are unique; values use the ordinary nominal type rules and cannot read private state, functions or other attributes. `priority` is also accepted as an attribute name.

`satisfy Requirement of Design.path;` is a package-owned relation from a resolvable design element to a requirement. Sources may be software declarations or occurrences in the same package, but not requirements, cases, views or the package itself. Requirement references use the ordinary import/qualification rules. Duplicate source/requirement pairs are invalid.

`useCase Name of Root { ... }` describes intended use. `verificationCase Name of Root { verify Requirement; ... }` describes verification intent. Only verification cases own verify relations, before their exchanges; duplicate targets are invalid. Both kinds may own one doc and zero or more exchanges. Neither satisfaction nor verification links establish fulfilled status or execute a test. Evidence belongs to a versioned external simulation, test, analysis or review result.

A case step has a kind (`send`, `publish`, `call`, `reply`), two endpoint-member paths, a named connector at the selected root composition level, an optional earlier-step predecessor and a unique label. Case endpoints must be visible ports of the root itself or its immediate child components. A typed root resolves those children and connectors through its definition. Internal interactions at a deeper level require a separate root component assembled at that abstraction level. This limitation keeps case paths subject to the same hierarchy rule as structural connectors.

Both endpoints MUST identify the same member of the connector's interface. Sender-receiver steps use data, publish steps use events, and request/reply steps use function signatures. A request follows client → server or requires → provides orientation; a reply reverses the matching request. A service publish goes from provides → requires; ordinary pub/sub follows publisher → subscriber.

`after label` imposes a happens-before dependency on an earlier declared step. No predecessor means no ordering is imposed relative to other steps. Source order is only a stable display tie-break, not an implicit total order. Thus unrelated notifications can remain concurrent in sv. A reply MUST have exactly one

unanswered matching request among its transitive `after` ancestors, with reversed endpoints and the same connector/member. Zero or multiple candidates is an error (SW083); each request has at most one reply. A request may remain unanswered in a partial trace. A prior reply closes a candidate only when that reply is itself an ancestor of the current step. A reply on an independent causal branch cannot resolve ambiguity merely by appearing earlier in the file. Source order never chooses between ambiguous correlations. Realization by function bodies is a separate analysis claim requiring a witness; route-valid cases remain unverified until that evidence exists.

Cases describe representative traces, not exhaustive behavior. They do not execute a second program, introduce independent event definitions, or loosen function guards. A complete behavioral verification must check traces against function semantics and environmental assumptions.

Relation vocabulary

Subject-assignment relations consistently use `of`: a case of a root, a satisfaction claim of a design element, or a compact view of a subject. The relation does not change containment: cases, views and satisfaction relations remain package-owned, and satisfaction still points from design to requirement. Case `for` and satisfaction `by` are rejected in 1.3.1; see the migration guide.

Syntax	Meaning
`useCase C of Root` / `verificationCase C of Root`	Root context for routed case exchanges.
`satisfy R of Design.path;`	Design.path claims satisfaction of requirement R.
`view V iv of Subject;` / `include Subject;`	Select existing subjects for a projection.
`on`	Activation stimulus or external deployment location.
`as`	Local/routed call result or completion binding.
`:` / `=`	Type/header separator / value initialization or assignment.
`via` / `after` / `->`	Existing route / causal predecessor or timer / directed edge.

`for` is used only by the counted-loop construct; it is unavailable for subject relations. The loop forms and their bounds are defined in chapter 3.

4.4 View declarations

A view declaration names a lower camel case view kind and one or more subjects in its own package. `view Name iv of Subject;` selects one subject. `view Name gv { include Subject; include Other recursive where kind == "function" and name != "debug"; }` selects a stable union. Each include resolves its subject before optional recursive expansion and filtering. Paths identify occurrences even when definitions are reused. Repeated paths are deduplicated in first-selection order. Empty results are valid; blocks require at least one include. Unknown or ambiguous paths are invalid. Filters use name, kind and path as strings; string/Boolean literals; ==, !=, and, or, not and parentheses. The whole predicate must be Boolean. Traversal is independent of filtering. Evaluation is pure presentation and cannot read stores or call behavior. gv and grv may reference any resolvable named semantic declaration or the owning package, except another view; iv requires a root component, a component definition or a nested component occurrence; cv references a case or root component, sv references a case or root component (for case selection), and fv references a function. A view of an instance resolves through its definition while retaining the occurrence path and identity.

View declarations carry no executable behavior. Layout, color and collapsed regions are presentation data. Authored include/filter clauses have the deterministic selection semantics above; they do not alter the selected model. The full graphical contract defines representation and source-edit invariants.

Documentation and comments

`doc /* multiline text */` is the owning declaration's formal specification. At most one is permitted. The grammar defines owner positions: first in a body, before a leaf semicolon, before a transition, or before a QoS policy body. Text is preserved literally, including line breaks and backslashes; blocks are non-nested and end at the first */ delimiter. Ordinary comments remain separate hidden lexical tokens: each // line and each /* */ block is one comment. Neither annotation affects execution. The lexical doc token includes its prefix and block; doc is included in the reserved-word inventory.

A comment belongs to the narrowest named declaration containing its source extent. A trailing comment on the same line as a completed declaration belongs to that declaration. File-level comments belong to the model. A formal doc belongs only to its grammar owner. Graphical tools draw each comment as a plain note inside the owner or connected with a plain dashed line. Formal doc is a labeled compartment or a table column; it must not be duplicated as a second owned specification. Hiding annotations never removes their source or ownership.

4.5 Static validity rule groups

The following groups collectively define semantic validity. Diagnostic numbers identify the present implementation's related checks, not a license to omit unimplemented clauses.

Group	Required checks	Diagnostic family
Reuse	acyclic single specialization; explicit compatible redefinition	SW014-SW015
Names	pinned language edition, uniqueness, imports, reference target kind, ambiguity, lexical scopes	SW002-SW004, parser/linker
Contracts	interface kind/member compatibility, role compatibility, exact interface identity	SW010-SW013, SW031-SW032
Values	nominal typing, arity, field completeness, enum membership, array shape, literal ranges, purity, no recursive value containment	SW020-SW028
Architecture	acyclic specialization/composition, compatible explicit redefinition, local/immediate-child boundaries, one delegation, no duplicate links/aliases, closure/cardinality	SW030-SW035
Time/QoS	positive/bounded values, explicit policy, sampled buffering, WCET/deadline/period, realtime completeness	SW040-SW045
Behavior	local-only calls/data, outgoing provenance, signatures, unique handlers, return coverage, terminating error paths, bounded loops, no recursion	SW050-SW059
States	legal stateful signature, final-state rules, priority, reachability, completion-cycle restrictions	SW060-SW064

Group	Required checks	Diagnostic family
Allocation	existing instances, valid external destination identities, no duplicate exact deployments	SW070
Requirements	immutable attribute typing, valid design sources, verification-case-only verify links, unique relations	SW090-SW091
Presentation/cases	subject kind, connector/member paths, happens-before and uniquely correlated replies	SW080-SW083

4.6 Conformance levels

Syntax conformance (S): the complete concrete grammar, pinned host/parser version, tokenization and AST construction. Recovery during editing must retain syntax diagnostics; an incomplete AST is not a valid model.

Static conformance (V): S plus every static validity rule in this specification, classified explicitly in chapter 8. A partial validator MUST enumerate checks that it lacks. It must not hide unresolved links or a validator crash as success.

KerML mapping conformance (K): V plus chapter 5's normalized elements and preserved SWAML semantic constraints, with a tested mapping against KerML 1.0 and its selected libraries. Export must retain a reverse trace map and stable identity. K does not automatically imply SWAML execution.

Execution conformance (E): V plus the operational, correlation, queue, timing-observation, failure and concurrency semantics. Target binding assumptions must be explicit. A simulator may abstract physical time while still reporting which timing obligations it does not check.

Graphical conformance (G): S and the graphical specification's projection and edit rules. G may present invalid models with diagnostics, but must not invent valid connectivity from layout. fv projects a function's statement flow or owned states.

A tool claiming `SWAML 1.3 S, partial V` accurately describes this repository's current core. It must not advertise K, E or G until those implementations and checks exist. A language host must require an explicit supported edition, and incompatible changes require documented migration. Version 1.1 is an intentional syntax revision; it is not backward-compatible with 1.0 despite the minor version number. Migration tooling must not change the meaning of models silently.

NORMATIVE / LANGUAGE CONTRACT

5. KerML semantic mapping

Normative SWAML mapping contract, with implementation status stated explicitly. Part of the SWAML 1.3 specification.

5.1 Foundation and boundary

SWAML defines software-specific specializations and constraints over a normalized KerML model. Its concrete syntax is independent: a `.swaml` file is not automatically valid `.kerml` text. The Langium AST is an authoring representation, not itself the KerML abstract syntax.

The selected foundation is OMG KerML 1.0. KerML supplies structural classifiers/features, behavior classifiers/steps, connectors, successions and expressions. The relevant normative kernel concepts are in clauses 7.3-7.4 and their abstract-syntax/semantic counterparts. SWAML's mapping below is this project's proposed specialization contract, not an OMG-approved extension. KerML specification.

The mapping inventory is a machine-readable design contract. This release does not implement or validate a KerML exporter, does not bundle a kernel semantic library, and does not claim K conformance. Implementers must test the normalized representation against the selected KerML metamodel and library before making that claim.

5.2 Semantic specializations

SWAML element	Proposed KerML base/representation	SWAML restriction or additional meaning
Package	Package	flat authoring namespace
Component definition	Structure	composition, encapsulated state and local executable features; optional single specialization
Specialization / redefinition	Specialization / Redefinition	explicit source reuse with SWAML compatibility checks and provenance
Root component	composite Feature typed by a referenced or synthetic Structure	selected named occurrence, closed topology and effective execution profile; typed binding cannot extend topology
Component instance	composite Feature typed by its Structure	exactly-one multiplicity and acyclic composition
Value datatype / enum	DataType	finite value semantics; enum domain restriction
Interface contract	Classifier	nominal version, legal members and communication mode
Port	Feature typed by contract Classifier	direction/role projection and boundary ownership
Function signature	Behavior plus typed parameter Features	no implementation; copied request/result contract
Function implementation	Behavior and an owning-component executable Step typed by it	direct ownership, activation, serialization and timing
Calculation	Function	parameter-only environment and no side effects
QoS policy definition	named MetadataFeature with typed policy fields	reusable values and reference provenance; never shared queue state

SWAML element	Proposed KerML base/representation	SWAML restriction or additional meaning
Connector	Connector between port Features	permitted interface exchange and QoS obligations
Delegation	Connector plus identity-preserving forwarding constraint	same role and contract; no extra queue
Ordered flow	contained Steps and Successions	structured branches/loops, bounded execution
Message emission	Step typed by a SWAML-library communication Behavior	source activation, route and correlation metadata
State / transition	contained behavior steps, state predicate and succession relations	SWAML state-selection and reaction semantics
Requirement / case / view	immutable attributes, formal docs, directed satisfy/verify memberships and selected occurrence identities	SWAML-specific interpretation

The table establishes a mapping target, not a claim that names such as “SWAML state” already exist as native KerML metaclasses. A full exporter needs a versioned SWAML semantic library for the additional predicates, activation and communication constraints. Chapter 7 completes the information model, reference edges and lowering rules for every syntax family; the inventory covers every current grammar rule/type name.

5.3 Normalization algorithm

A mapper MUST perform the following in dependency order:

1. Validate the complete workspace and resolve every textual reference. Allocate stable IDs for package declarations and each named semantic element. Retain file URI and source range as provenance, not as identity.
2. Create value types and interface classifiers. Materialize every data/event/function parameter and result as a typed feature with copied-value semantics. Preserve interface version and member order.
3. Create component structures and owned instance features, plus synthetic structures for inline root composition and behavior, with membership and typing links. Each root is a named occurrence of its referenced or synthetic structure; a typed root has no extra wrapper child. Preserve source profile omission in provenance and effective service in semantics. Keep definition identity separate from every instance-path identity. A reused definition must not merge the states of its occurrences.
4. Create each port feature in its owning structure. Apply role-specific direction to each member projection: RPC inputs/results have opposite directions; service events run provider-to-consumer. Do not assign a single simplistic “out” bit to an entire bidirectional service contract.
5. Create function behaviors within their component's ownership. Add a synthetic owned executable step typed by that behavior and a component-instance context binding. This lowering artifact gives execution a structural context; it is not a user-visible performer or a second editable allocation model.
6. Lower statement blocks to structured steps with explicit sequencing. A branch has a Boolean selection predicate and branch result control; a bounded loop has condition, body, count and failure edge. Lower expressions as typed expressions/calculations. A mapper must preserve source order and short-circuit evaluation.
7. Lower each stateful function to one state identity feature per activation, nested state-entry/exit/effect behaviors, trigger predicates, priority selection and succession relationships. Preserve finality, timer start/cancel and the run-to-completion relation. A bare succession graph without these constraints is not an equivalent mapping.

8. Lower communication statements to message occurrences linked to the emitting activation, sending port, interface member, connector/delegation route and destination port. Requests additionally carry correlation, timeout and response/error continuation. An event cannot become an unowned signal merely because a kernel target permits it.
9. Materialize queues, policy constraints, timing/deployment annotations and case happens-before relations in the SWAML semantic library vocabulary. Keep queue identity per connector/direction/member.
10. Produce a reverse mapping from every generated element to its authoring element, including synthetic lowering elements. Validate kernel typing/membership constraints and SWAML constraints separately. Report failures at the source element, not only at generated IDs.

5.4 Required extension predicates

The semantic library MUST define, or a mapper MUST carry as explicit constrained metadata, at least:

- `ownsExecutable(component, function)` and instance-local `contextOf(activation)`;
- `contractOf(port)`, `roleOf(port)`, `memberOf(contract, member)`;
- `route(sourcePort, destinationPort, connectorPath)`;
- `emits(activation, port, member, copiedPayload)` and `handles(function, port, member)`;
- `correlates(request, response)`, `expiresAt(request)` and `lateResponseDiscarded(response)`;
- `queueOf(connector, direction, member)`, capacity, overflow, ordering and delivery policy;
- `currentState(activation)`, `enabled(transition, stimulus)`, priority and final-state completion;
- `wcet(function)`, `deadline(function)`, `period(function)` and instance allocation;
- `happensBefore(caseStepA, caseStepB)`, `satisfy(designElement, requirement)` and `verify(verificationCase, requirement)`.

These predicates are part of SWAML meaning even if an exchange format stores them as metadata. A consumer lacking their interpretation MUST report a partial import and MUST NOT execute a “best guess.” A generic KerML consumer can preserve structural content while lacking SWAML runtime semantics.

5.5 Interchange and round-trip obligations

A future exporter SHOULD use the standard KerML interchange representation supported by its chosen backend, with a declared SWAML semantic library version. No new unregistered MIME type, XMI metamodel URI, or fake OMG namespace is assigned by this document. SWAML library identifiers must use a project-controlled namespace when published.

Round-trip tests MUST compare semantic identities, typing, owned instances, routes, triggers, guards, flow ordering, queue policies, timing and trace dependencies. byte-for-byte textual equality is not required. Comments, formatting and diagram layouts are separate preservation requirements for an authoring tool.

Dropping a guard, converting a queued event to a sampled value, merging per-subscriber queues, or omitting a request's response path is a semantic loss and must be reported as an error for K conformance. The mapping must not infer a performer allocation that changes the function's direct owning component.

NORMATIVE / LANGUAGE CONTRACT

6. Operational contract and target bindings

Normative chapter of the SWAML 1.3 specification. This chapter makes the allowed traces precise; it does not prescribe an implementation thread model. The reference simulation binding makes a deterministic logical-time choice within this contract; production backends require their own binding and evidence.

6.1 Configuration and observable occurrences

A running configuration consists of: the selected root component instance tree; private variable stores per component instance; live activation frames and their local call stacks; one serial lock per component; the state and timer generation of each live stateful function; message queues keyed by connector occurrence, direction and member; pending request correlations; released lifecycle activations; and a monotonic logical time.

Each message has a logical identity, original emission time, producing activation identity, typed copied payload, source/destination port occurrences, interface member, connector occurrence and delegation route. A retry keeps the logical identity and original timestamp. A transport attempt has a separate identity. An RPC response additionally references its request correlation. Occurrence identities MUST be unique within a run and cannot be declaration IDs reused for every emission.

The minimum observable trace alphabet is initialization, release, dispatch, statement completion, emission, admission, drop, delivery, request completion, timeout, state entry/exit, activation completion/failure, and contract violation. Each record includes logical time, occurrence identity and relevant source identities. A backend may omit detailed statement traces during normal execution, but MUST provide sufficient checking instrumentation to diagnose violated obligations. Trace records and diagnostics are observations, not implicit interface messages.

A step of execution is either a local statement/expression step, communication admission/delivery, dispatch, reaction microstep, or time advancement. Independent components may step in either order. No text-file order between components creates a happens-before edge. Elapsed physical execution time must be represented when checking deadlines; treating each statement as zero-time cannot establish production timing compliance.

6.2 Initialization and release

Initialization has a barrier with the following phases:

1. Expand the acyclic instance tree, establish effective deployments, create private stores from pure initializers, resolve all routes and install all queues/handlers.
2. Install each stateful function's initial state identity and arm its initialization reaction. It is not yet ready to consume a stimulus. Register all ordinary startup activations.
3. Establish one run epoch `t0`. Release every startup/initialization activation at `t0`; activate periodic releases at `t0 + k * period`, with k starting at 1. All ports may now admit traffic.

The scheduler may interleave startup activations across components and within a component subject to its lock. Messages arriving before a machine finishes initial entry remain in their connector queue; they are not discarded merely because initial entry is pending. Timers start when entry finishes, not when the initialization reaction was released. A startup function may request a service whose component's startup activation has not run: handlers and initialized data already exist. If an application needs an additional readiness protocol, it MUST model that protocol through an interface.

Each startup activation is released exactly once per root run. The core defines no automatic component restart. A deployment implementation restarting a failed component starts a new run epoch for the affected

binding and MUST report its reset, queue-retention and correlation policy; it cannot imply uninterrupted E conformance for a reset it has not modeled.

Periodic releases are independent of earlier completions. Serial dispatch does not merge or silently skip them. Manual activations are parameterless local harness releases. Input-triggered work remains in the input queue until dispatch is possible; it is not copied into an unbounded second activation queue. Admission of an RPC response resumes its suspended frame without acquiring another component lock.

6.3 Dispatch, backlog and snapshots

A serial activation root acquires its component's lock at dispatch and releases it on completion/failure. Local calls execute on the activation root's stack and inherit its concurrency mode and snapshot, regardless of a helper's own timing declaration. A reentrant activation root may overlap other activation roots, but neither it nor any reachable helper may assign component data. Assigning its own parameters/locals is allowed.

A reentrant activation root captures an atomic value snapshot of all component variables at dispatch, before its first expression. All reads by that activation and its helpers use that snapshot throughout suspension and resumption. An assignment by a serial activation is atomic at the whole-value level; snapshots observe some consistent store between assignments. A machine's next reaction captures a fresh snapshot if it is reentrant. Reentrant does not allow overlapping reactions of the same machine.

Between independent ready queues and lifecycle releases, the scheduler chooses any enabled activation root. For a FIFO queue, dispatch preserves queue order; completion order of reentrant handlers need not match dispatch order. A target backend MUST specify fairness and scheduling policy, and MUST NOT introduce starvation while claiming eventual at-least-once delivery under continuously enabled receivers.

An executable target binding MUST bound pending start/manual/periodic activations separately from connector queues. On exceeding that bound it MUST record `ActivationOverflow` and stop the affected root execution; silently dropping or coalescing releases is not an implementation of this core. A simulator may expose an explicitly unbounded mathematical backlog, but cannot use it as evidence for a bounded production deployment. The binding also bounds simultaneous reentrant activation roots; when full, subsequent work remains pending rather than exceeding the limit.

WCET includes local helpers and active reaction microsteps; request wait consumes deadline budget but no active CPU budget. The activation root's deadline is measured from release. For input reactions release occurs when the message is ready in its destination queue. A timer reaction is released at its due time even if dispatch is later. Local helpers have no independent release; a backend checks their declared execution bounds over the invocation and also includes them in the activation root's bounds.

6.4 Queue offers and fanout

`send` and `publish` evaluate all arguments once and snapshot the matching active connector set. They offer a distinct per-connector logical message to **every** connector in that set, even if an earlier offer rejects. Offers are independent and have no specified interconnector order. The statement finishes after all local admission decisions. If any offer rejects, the activation fails with an admission failure; successful offers remain committed. Loss through dropOldest/dropNewest is not rejection and does not fail the sender. Atomic broadcast requires a modeled protocol.

For a given queue, offer and dispatch are atomic decisions. Capacity counts retained messages waiting for dispatch, including ones whose transport availability is pending; it excludes running handlers. Removing an old message or discarding a new one MUST be observable to a checking executor. An optional port with no connectors produces no offers and succeeds for data/events.

Condition	Data/event emission	RPC request	RPC response
No live endpoint	optional unbound succeeds; failed bound route reports transport failure	`Unavailable` if unbound; otherwise `Transport` when detected	requester gets `Transport` if detectable, otherwise timeout

Condition	Data/event emission	RPC request	RPC response
Full queue with reject	producer activation fails after all fanout offers	caller executes `otherwise` with `Rejected`	requester gets `Rejected` if the binding can report it; otherwise timeout; server's completed effects remain
dropOldest/dropNewest	dropped occurrence is observable; statement can succeed	dropped request waits until timeout unless transport reports failure	dropped response leaves request waiting until timeout
Age expired	discard before dispatch	no service dispatch; caller eventually times out unless already failed	discard; no successful completion
Receiver handler fails	receiver records local failure; sender is not retrospectively failed	correlated `RemoteFailure`	not a second handler; resume request failure path

Control failures such as admission rejection use the same request route/correlation. They are transport outcomes of that interface exchange, not arbitrary model event channels. A binding unable to signal a failure cannot fabricate a successful result.

An admitted message becomes ready no later than emission plus maxLatency when such a bound is claimed. Deadline miss is a violation even if the message is later delivered; it is not itself a new delivery policy. `maxAge` eligibility is inclusive: age <= maxAge may dispatch; age > maxAge is expired. Discarded/expired messages have no eventual-delivery obligation. Without maxAge, asynchronous messages have no core expiry before connection teardown; a backend must maintain its delivery obligation or explicitly report loss of availability. Retry policy, finite memory and resource exhaustion behavior belong in the target binding, not undocumented defaults.

6.5 RPC completion and races

A request correlation has states Pending, Succeeded or Failed. Creation precedes the first request offer. Its expiry is issuance time plus timeout. At most one terminal completion may win. A response can succeed only if it is observed at the requester at a time **strictly earlier** than expiry and the correlation is Pending. At expiry timeout wins ties. Terminal state is immutable; later responses and duplicate replies are discarded and observable.

AtLeastOnce may dispatch multiple handler activations with the same logical request/correlation. Idempotency must cover storage writes and published side effects; returning the same value alone is insufficient. The first eligible response completes the caller. Retrying must never create a new caller-visible correlation or extend its timeout. A binding may deduplicate repeated requests while preserving required behavior.

The mandatory `as name` on a void RPC labels successful completion without introducing an inhabitable value. It reserves that name in the enclosing lexical scope but it MUST NOT be read or assigned as data. A void handler uses bare `return;`. The failure block has an implicit copied `failure: string`; an existing live value with that name is a static error. Failure blocks cannot fall through to the successful continuation.

A timeout does not retract messages already sent or cancel a server frame. Pending requests cannot be used as detached futures; each statement awaits its own outcome. Synchronous dependency cycles remain expressible but may time out. A static validator is not required to prove their absence; an execution/timing claim must account for them.

Interruption of pending continuations

An accepted region interrupt resumes its existing activation at a statement boundary, retaining the serial lock and activation deadline. Its input is admitted, delayed, ordered, expired or dropped through the same per-route queue rules as other data/events. It is delivered only while its selected region is armed; available input outside that interval is discarded with an undeliverable-interrupt observation. It does not release an independent activation or create an unbounded handler queue.

Cancellation unwinds the protected stack to that region, disarms it, and enters its handler with copied payload bindings and the enclosing lexical environment. If a caller-side request is Pending, cancellation makes it terminal Failed with an Interrupted reason and records requestCancelled. Its success and otherwise continuations are both abandoned. Scheduled resume work for the unwound stack is invalidated. Server execution and committed effects remain unchanged. A later reply observes the terminal correlation and is discarded.

A response or timeout already terminal keeps its original outcome. In particular, expiry still wins the response/timeout tie. A subsequent interrupt may transfer control before the already queued terminal continuation runs; this does not change the recorded correlation outcome. A normally completed interrupt handler continues after the region. An outer interrupt can unwind an inner region or its handler, including a second suspended request. Return/failure still disarms all regions unwound with the frame. The original activation deadline applies through cancellation and handler completion.

A conforming trace identifies region entry/exit, interrupt delivery and cancelled correlations with their source region, activation, member and route identities. Reference simulation uses one-shot region watches and continuation generations to prevent stale scheduling from resuming cancelled frames. Physical cancellation latency remains a target-binding obligation.

6.6 State reaction ordering

A machine has one initialization reaction and then a sequence of reactions. Each reaction includes all immediately enabled completion transitions until quiescent/final/failure. Completion selection precedes dispatch of the next external stimulus. If no completion guard holds, the machine becomes quiescent. A write by another function does not itself awaken completion transitions; they are reconsidered after the next machine reaction, including one that consumed an unhandled message or expired a timer with no enabled transition.

On completing a state's entry, create one timer stimulus per distinct `after` duration on its outgoing transitions, tagged with the current state-entry generation. All transitions with that duration compete using guard/priority. Distinct durations produce distinct stimuli. Leaving a state cancels every timer from that generation. A queued timer from an obsolete generation is discarded. Reentering, including a self-transition, creates a new generation. Entry code time is outside the residence timer; the timer measures time since entry completion.

A due timer and a ready input are independent stimuli. Their selection order is unspecified unless the target scheduler constrains it. Transition priority selects among transitions for the **same** stimulus; it is not a task/queue priority. A consumed stimulus is not retained merely because all guards were false. Entry/exit blocks have no event payload scope. Exit and effect run before state identity changes; destination entry follows that change.

The completion microstep counter is zero at the start of a reaction and increases once per taken completion transition. Taking a 1,025th completion transition fails the reaction before executing that transition. External dispatch does not occur between those microsteps. Guards are pure expressions and cannot send, mutate, or consume another message. A guard evaluation failure fails the owning activation.

Final entry completion, bare return or failure terminates the machine, cancels all its timers and disables its dispatch. Future messages to its members become undeliverable occurrences; they are not rerouted to another function. Other functions in the same component may remain alive. No implicit exit behavior runs on return/failure unless it has already been reached in the specified transition phases.

6.7 Execution binding contract

Every backend claiming E MUST identify its backend version, supported SWAML version, selected root component, model digest and target configuration. It MUST supply explicit values or a justified symbolic abstraction for:

- effective deployment of every component; clock basis, precision and cross-target clock-error bounds;
- scheduler, fairness, serial locking, snapshot implementation and reentrant limits;
- lifecycle backlog limits and overflow observation;
- transport, serialization, copying/isolation, queue storage and retry/deduplication policy;
- deadline/latency/age measurement points and what happens after a reported violation;
- external adapter behavior and assumptions, startup readiness, shutdown/reset and queue retention;
- resource failure handling, correlation uniqueness, observability and evidence artifacts.

Timing/resource failure policies may stop execution after recording a violation. Continuing is permitted only with the violation visible, and cannot count as a satisfying execution for the violated contract. An execution trace is conforming when all observed steps obey core transitions; satisfying a particular model additionally requires its declared assertions and contracts to hold. A runtime that correctly reports a failed assertion is not thereby a broken interpreter.

Security and target-specific bus/OS policies can strengthen this contract, but cannot add hidden model exchanges. AUTOSAR Classic task/RTE mappings, Adaptive service bindings and broker configurations must preserve the same identities, member directions and copied-value boundaries.

For each mapped function the binding MUST identify its activation source and preserve component-wide serial exclusion or the declared reentrant snapshot semantics. For each mapped interface member it MUST identify the data conversion and transport route, including response correlation where applicable. A platform read/write API, shared-memory optimization or callback must preserve the observable copied-value and queue semantics of the modeled exchange.

For AUTOSAR Classic, the binding MUST state how modeled activations map to runnable events and tasks, and how sampled or queued members map to RTE communication. For Adaptive, it MUST state how the fixed modeled endpoints map to service instances and how discovery, unavailability and application startup affect pending exchanges. For web and cloud targets, it MUST state how requests, notifications, broker acknowledgments, durable state and restarts relate to modeled completion and failure. A transport acknowledgment is not automatically a successful application response, and a connector queue is not automatically durable storage.

Primitive values retain the domains in chapter 1. A binding to fixed-width platform integers, bounded strings, schema fields or wire encodings MUST define conversion limits and observable failure for unsupported values, or provide evidence that the model stays within the supported subset. It MUST NOT silently truncate, wrap, reinterpret or share mutable payloads. Target-specific restrictions and evidence belong with the mapping; accepting a core model does not certify a platform implementation.

NORMATIVE / LANGUAGE CONTRACT

7. Normalized semantic model

Normative SWAML contract extending chapter 5. This chapter defines the complete information a mapper must preserve. It specifies SWAML normalization, not a new OMG metamodel or an implemented serializer.

7.1 Identity, ownership and source trace

Normalization consumes a resolved workspace and emits a finite declaration graph plus instance expansions for selected root components. Each semantic node has an immutable identity within that normalized artifact, a semantic kind, at most one owning node, ordered owned children where order matters, typed reference edges, and source provenance. Except for model roots, every semantic node has exactly one owner. Reference edges never imply containment.

The deterministic textual identity of a declaration is its package plus owned-name path, for example `Settings::SettingStore.update`. Anonymous syntax nodes use the owning declaration ID plus a structural slot path such as `/body/statements/2/condition`; these IDs are stable for an unchanged syntax tree, not across arbitrary edits. Synthetic elements append a reserved role segment such as `/@execution` or `/@failure`. No source identifier contains `/` or `@`, so generated roles cannot collide with source names.

An occurrence identity consists of the root component declaration ID and the child-instance path. A port occurrence adds the owned port name. A function occurrence additionally refers to its shared function declaration and component occurrence. Runtime activations add a run ID and unique activation ordinal. A mapper MUST NOT collapse occurrence paths merely because they share a definition. Persistent editor UUIDs are a separate identity map with explicit rename/move history; source locations and in-memory AST addresses are never persistent IDs.

A reverse trace relation maps every generated element to one or more authoring node identities and their URI/ranges. Synthetic nodes retain their source owner and generation role. No generated constraint may be untraceable. An import can have an import-declaration source location while referencing a package located in another document.

7.2 Declaration graph and links

Normalized family	Required information and constraints
Workspace/package	language version, packages, direct import edges, unique declaration namespaces
Record/enum	nominal identity, ordered fields or ordered unique literal names, finite domain/containment
Type use	primitive or nominal target, optional array extent; never encode a type solely by a display label
Interface	kind, version, ordered data/event/RPC members, signatures, data mode and idempotency claims
Structure definition	component kind, owned ports, contained instance features, private variables, owned executable declarations as applicable
Instance feature	component kind, exactly one definition target, composite ownership, multiplicity one
Port feature	interface target, role, optionality and derived member directions
QoS policy	package-owned declaration identity, complete ordered body, source provenance; no runtime queues

Normalized family	Required information and constraints
Connection/delegation	owner, endpoint feature references, explicit path provenance, interface identity, queue policy for a connection; no queue for delegation
Variable/local/parameter	declared type, ownership/scope, initializer where present, mutable copied-value semantics
Executable	signature, trigger, concurrency/timing, function-local calculations and one statement or state body, context bound to the owning component
Root component/binding	named occurrence with inline composition or component-definition reference, external destination identities, explicit/effective deployment and effective service/realtime/mixed profile
Requirement/case/view	immutable attributes and doc, satisfy/verify edges, case kind and step graph, view kind and ordered selected occurrence paths

All textual references MUST resolve before normalization. A `TypeRef` is a use, not a new nominal declaration. Array extents are carried as shape constraints and do not create element aliasing. Empty records have one value. void results create no output-value feature. Parameters preserve order, independent of parameter names at a handler implementation.

A typed root expands its component definition, and an inline root expands its authored children; no extra wrapper occurrence is inserted. Its normalized profile records service even when the source omits that default, while provenance preserves the omission. A typed binding body contributes only deployments, never topology. Every component child expands its definition's child features recursively. Expansion preserves each connector's owner path, so two uses of the same component produce distinct connector occurrences and queues. Effective deployment is computed by the nearest explicitly deployed ancestor, including the occurrence itself. An absent ancestor yields an explicit Unallocated value, not an invented external system.

Specialization normalizes to an explicit base-definition reference and an ordered effective member set. Each inherited member retains declaration provenance; each replacement additionally references the inherited member it redefines. Expansion instantiates the effective set separately for every component instance, including root-owned functions and variables. Local references in inherited bodies resolve in that effective instance context. A mapper MUST preserve redefinition compatibility checks and MUST NOT merge stores merely because they share an inherited declaration.

External deployment destinations retain their exact source identity string and an optional host-provided resolved external identity. Normalization must distinguish unresolved external references from local path errors; it cannot fabricate an imported SysML part.

7.3 Contract projection and route normalization

For every port occurrence and interface member, derive a member projection with contract/member identity and role-specific directions:

Role	Data/event projection	RPC request	RPC result/failure
sender / receiver	out / in data	absent	absent
publisher / subscriber	out / in event	absent	absent
client / server	absent	out / in	in / out
requires / provides	in / out event	out / in	in / out

Contract signatures remain shared; these projections belong to specific port occurrences. One bit on a service port cannot replace this table. After following delegation to its resolved locally handled port, every

receiving projection references exactly one handling function in that component context. Each outgoing data/event projection references the nonempty set of syntactic emission sites in its resolved context. Those links do not assert reachability.

Resolve a connection by following each boundary port's unique delegation chain to a locally owned handling/emitting component port. Preserve the ordered original boundary chain as provenance. The normalized route is `(connectorOccurrence, sourceLeafPort, destinationLeafPort, interface)`. Delegation aliases do not add storage, queues, dispatch, retries or observable messages. Distinct connectors resolving to the same directed leaf pair are invalid. Optionality is checked at every port along a boundary chain; one optional declaration cannot erase a required declaration.

A connector policy normalizes to its complete effective body, whether selected inline or by a named `QosPolicy`. For a named selection, preserve the declaration reference and both declaration/use source locations; for an inline selection, preserve its body/use provenance. `Qos` is the authoring selection and `QosBody` is its field grouping, not a runtime object. Policies cannot contain policy references or overrides. Multiple uses of one declaration still produce independent connector occurrences.

A queue key is `(connectorOccurrence, direction, memberIdentity)` with direction relative to the connector's declared orientation. RPC responses and requests use opposite directions. Service events use the response direction but their distinct event member identities prevent queue sharing with results. Correlation state is separate from queue storage.

7.4 Executable normalization

Every function has a behavior declaration and one synthetic executable feature owned by its component structure. The feature is typed by that behavior and binds an activation to a component occurrence. It MUST NOT create a user-visible performer, an independently editable implementation allocation, or permission to call it across a component boundary.

A statement body normalizes to an entry, ordered steps, exit/failure terminals and lexical scopes. Every step records its source construct and ordered operands. The following lowerings are required:

Source construct	Normalized meaning
var / assignment	evaluate expression, then create/replace copied local or private value
send / publish	operand evaluation followed by per-route message offers, emission provenance and success/rejection continuation
routed call	issuance/correlation creation, suspended frame, timeout, typed success binding or terminating failure branch
local call	synchronous same-component call edge, copied arguments and results, fresh non-void result binding, borrowed activation context/snapshot and caller continuation
return / fail	terminate the current frame; unwind callers on failure; correlated result/failure for RPC activations
if	evaluate bool once, mutually exclusive selected branch and merge of control only
while	repeated condition, body, iteration counter and bound-failure edge; no implicit unbounded loop
for	loop-owned initializer scope, initial test, ordered body/update/test, counter and bound-failure edge; update is an assignment grouping
forEach	one copied fixed-array snapshot, finite element order, fresh typed element/body scope for each iteration
do	initial body execution followed by condition, counter and bound-failure edge; condition excludes body locals

Source construct	Normalized meaning
interruptible	named armed region, typed input reference, normal/interrupt exits, handler payload scope and explicit stack/correlation cancellation
assert	bool evaluation and failure edge when false
calc	function-owned typed inputs, named output and expression tree with parameter-only environment and explicit acyclic calculation call edges

Expressions carry inferred type and evaluation order. Binary short-circuit nodes conditionally skip their right operand. A record value maps fields to declared identities, not input order; an array preserves value order and fixed extent; enum values retain enum and literal identity. Numeric failures and invalid index edges are part of the expression contract. A mapper MUST NOT silently substitute mathematical unbounded integers for SWAML bounded int execution.

The directly owned state body is syntax grouping, not an independent behavior declaration. It contributes to its function's normalization: state identities, initial reference, entry/exit/effect step groups, trigger matchers, pure guards, priorities, finality, current-state feature, timer-generation features and the chapter 6 dispatch/reaction rules. A transition is not merely an unconstrained edge: its priority selection, source-state condition, payload environment and ordered exit/effect/update/entry phases are required semantics.

7.5 Kernel representation and SWAML library responsibilities

The `kernel` field of the mapping inventory lists proposed KerML bases; the `semantic` field names the SWAML meaning to preserve. Empty kernel lists mean a lexical/grouping node contributes to its owner's representation, not that its meaning may be discarded. The inventory covers every authored grammar rule and type alias; punctuation and reserved keywords are covered by their productions.

Kernel ownership, membership, typing, parameter, feature-value and reference relationships MUST be instantiated according to the selected KerML backend. The inventory is not a recipe to instantiate abstract metaclasses blindly. A backend must choose suitable concrete subclasses for abstract bases such as Expression and validate all resulting relationships. Source grouping rules do not generate duplicate nodes for the same AST element.

The SWAML library supplies typed semantics for activation/context, copied values, versioned port contracts, queue offers, request correlation, failure outcomes, bounded flow and machine reactions. A communication lowering uses a Step typed by the library's send/publish/request behavior, with typed features for its ports, member and payload. It MUST NOT serialize a made-up kernel class called `SWAML-library-message-interaction`. The kernel Step is the base; the library behavior is the additional meaning.

Requirements, timing, QoS, deployment, case and view annotations use project-owned metadata definitions with typed fields and explicit references. Cases additionally carry a scenario behavior/step graph and happens-before edges; they do not become an independently executing program. A view's selections use semantic identities and occurrence contexts; geometry/layout stays in a separate sidecar.

A K-conforming artifact MUST include or resolvably identify its exact SWAML library version and declare which constraints are checked. Carrying uninterpreted metadata is preservation, not proof that a generic kernel tool understands or executes it. The project has not yet selected/published a library namespace or implemented this backend; these obligations define its acceptance criteria.

7.6 Case and view normalization

Normalize each case label to a distinct step identity, resolve its route/member and orientation, and translate `after` to an explicit happens-before edge. The transitive closure determines causal ancestry. A reply links to its unique unanswered request ancestor with reversed endpoints and the same connector/member. A request is already answered for candidate selection only if its reply is also a causal ancestor; an independent branch cannot settle the choice. Ambiguous candidates and multiple replies to one request are invalid. Source position contributes display order only. An unanswered request is allowed in a partial trace; no synthetic reply is inserted.

A case realization claim records a witness execution and assumptions, including payload values absent from the diagram, dispatch choices and function paths. Mere route validity does not supply that witness. A graphical renderer can display an unverified case but must not show it as proven.

Normalize views to `(kind, selectedOccurrences[])`, retaining the authored selection/filter clauses for round trips. Compact of syntax contributes one selection. Resolve include subjects in the owning package, expand recursive descendants, then evaluate typed Boolean predicates and deduplicate occurrence paths in first-selection order. gv/grv may select any named semantic element or package except a view; iv selects components; cv/sv select cases or roots; fv selects executable functions. Paths traverse effective component members, owned named behavior elements and interface members exposed through ports. A package/declaration name collision is ambiguous.

An FV timing presentation additionally references an identified run/trace and its selected function occurrences. Preserve activation identities, event ordinals, correlations and observed phase boundaries separately from declared timing contracts. Horizon/stopped intervals remain open; zero-cost execution cannot be normalized as measured physical CPU time. Timing geometry and interactive filters remain presentation data. No new executable view kind or scheduling participant is introduced.

7.7 Equivalence and completeness

Two normalized artifacts are equivalent when an identity-respecting graph correspondence preserves ownership, types, value shapes, interface versions, instance contexts, routes, evaluation order, triggers, state selection, policies, deployments, case dependencies/correlations and ordered view selections. Requirement attributes, formal docs and satisfy/verify identities must be preserved. Whitespace, ordinary comments, source offsets and diagram geometry are excluded from execution equivalence but remain required authoring data. Failure reasons remain observable diagnostic values.

A round trip may rename anonymous lowering IDs while retaining their owner/role trace. It may not remove guarded branches, reorder effects, merge queues, weaken timing contracts, convert copied payloads into remote references, or drop SWAML metadata because a receiver lacks support. Unsupported meaning requires an explicit partial-import result. This information-preservation contract is complete for the closed 1.3 syntax; implementing and validating a KerML backend is a separate milestone.

Requirement and presentation extension

The rules in chapter 4 preserve requirements as named obligations with immutable attributes and at most one owned formal doc. Satisfaction references a design identity and requirement identity; verification references a verificationCase identity and requirement identity. Use cases cannot own verification links. Ordinary comments retain source extents and owner identities separately from formal documentation. Cases and links express intent, with no automatic proof status. View include/filter clauses resolve to a stable union of occurrence paths and have no execution steps or store effects. These constructs remain part of the proposed mapping contract; no kernel exporter is implemented.

NORMATIVE / LANGUAGE CONTRACT

8. Conformance obligations and evidence

Normative classification of the SWAML 1.3 specification. This chapter separates static language validity, runtime behavior and proof claims. It prevents acceptance by one tool from being mistaken for evidence supplied by another.

8.1 Static acceptance

Static validity is determined from the complete loaded source workspace without executing a model. The grammar plus the following obligations define the static core. The reference core implements these groups, but remains advertised as partial V pending an independent clause-by-clause conformance suite.

ID	Static obligation	Reference-core evidence
V01	Pinned edition, headerless package syntax; portable tokens, escapes and safe literal domains	parser/SW025; lexical and migration tests
V02	Unique package/scoped names; imports exist; references resolve uniquely before kind filtering	SW002/SW004, owner-aware linker; multi-file tests
V03	Finite nominal records/enums/arrays; no void data; complete typed constructions	SW020-SW028; values/flow and audit fixtures
V04	Pure initializer/calculation environments, no cross-component value lookup, compatible operands/arguments/results	SW022-SW024; isolation and type tests
V05	Legal interface members/port roles, exact nominal matching and signature implementations	SW010/SW011/SW031/SW032/SW052
V06	Acyclic specialization and effective composition, compatible explicit redefinition, immediate-child routes, one delegation per boundary, no duplicate aliases/routes	SW012/SW013/SW014/SW015/SW030/SW033/SW035
V07	Required-port closure through every alias; single source/provider where required	SW034/SW035; closure fixtures
V08	Every outgoing member has syntactic emission provenance; every incoming member has one handler owner	SW050/SW051/SW055
V09	Positive safe durations, capacities, array extents and loop bounds; nonnegative safe priorities; complete inline/named QoS including unused declarations; sampled buffering and idempotent RPC declaration	SW020/SW040/SW041/SW043/SW059/SW062

ID	Static obligation	Reference-core evidence
V10	WCET <= deadline; periodic deadline <= period; realtime/mixed required timing/latency/age metadata	SW042/SW044/SW045
V11	Local acyclic calls; legal call targets; copied input parameters; no live-name shadowing	linker, SW002/SW056/SW058
V12	Value returns and request failure paths terminate; void completion is not a value	SW020/SW022/SW023/SW053/SW057
V13	Reentrant activation roots do not write component data directly or through any reachable helper	SW054; transitive-effect audit tests
V14	Stateful function signature, initial reference, graph reachability, final-state rules, competing priorities and no unconditional completion cycle	SW060-SW064
V15	Valid instance deployment paths and external destination syntax; no duplicate exact allocation	SW070
V16	View subject kind, case member/route/direction, earlier predecessor and uniquely correlated reply	SW080-SW083
V17	Triggered serial interrupt-region owner, local incoming data/event route, unique receiving owner/member/name, payload scope and definite-return branches	SW002/SW033/SW051/SW055/SW065

“Every path returns” in V12 is structural definite termination: a sequential block contains return/fail, an if with both branches definitely terminating, a do/nonempty forEach body that definitely terminates on its first iteration, or an interruptible region whose protected body and handler both definitely terminate. while/for alone are insufficient because they may run zero times or hit their bound. A request failure block obeys the same rule. This rule is intentionally conservative and does not require solving guard predicates. A request's success continuation is checked independently from its failure block.

Literal array indices, including unary sign expressions, are checked statically. General expression-derived bounds and arithmetic overflow are execution checks; a validator may additionally reject a proven inevitable failure with an explained analysis diagnostic. Syntactically present but unreachable emission still satisfies provenance; proving that an interface is useful is a stronger claim.

Imported declarations and string-valued semantic paths MUST be revalidated after a dependency change, addition or deletion. Correctness includes negative lookups: a newly added declaration can make an existing short reference ambiguous. A conservative implementation may rebuild all loaded language documents. Source recovery may suspend semantic checks on a syntactically invalid document, but syntax errors and unresolved references remain visible and the document cannot pass validation.

8.2 Runtime obligations

ID	Obligation	Required evidence for E
E01	Distinct instance stores, copied payloads and activation frames	isolation traces for reused definitions and transports
E02	Startup barrier, release epochs, serial lock, snapshots and bounded activation resources	scheduler traces including suspended RPCs and reentrant readers
E03	Per-route queues, sampled replacement, overflow, ordering, age and delivery policies	adversarial queue/duplicate/drop tests
E04	Unique correlations, response route, timeout race, failure outcome and late-response suppression	concurrent request/timeout/reply traces
E05	Structured statement order, bounded loops, failure unwinding and numeric/index checks	reference evaluator cases and failure traces
E06	State reaction order, timer generations, priority, completion bound and termination	competing input/timer and self-transition traces
E07	Timing observation and explicit deployment/resource assumptions	target binding plus measured/proved scheduling evidence
E08	Same-activation interrupt transfer, no rollback, pending-continuation cancellation and terminal-correlation immutability	nested/suspended cancellation, inactive/duplicate input, deadline and late-reply traces

The runtime must detect dynamic out-of-bounds access, non-finite arithmetic, integer overflow, division by zero and loop/microstep-limit violations. V does not have to prove that these failures never occur. A static accepted model can describe a failing execution. Likewise V checks the idempotent declaration where required; a deployment claim that it is true requires implementation evidence.

8.3 Analysis claims

A **case realization** claim requires at least one function-level witness trace consistent with all case dependencies, route/member identities and model assumptions. A route-valid case is only a candidate until such evidence exists. Invalid correlation is statically rejectable; guard feasibility is not implied by correlation validity.

A **requirement satisfied** claim links named requirements to design elements and verification cases plus test/analysis/review evidence; satisfy and verify relations alone denote intent. A **schedulable** claim identifies the target scheduler, release assumptions, execution costs, transport/queue bounds and proof or measurement envelope. A **deterministic** claim accounts for independent stimulus ordering and reentrant completion. A **deadlock-free** claim accounts for synchronous request dependency cycles and retained serial locks.

These analyses may be conservative, bounded or incomplete, but MUST state assumptions and limitations. Failure to find a witness is not automatically proof that no witness exists. Runtime snapshot realizability is a backend obligation, not something syntax validation can establish.

8.4 Kernel, editor and graphical acceptance

K requires the complete normalized model, an implemented versioned semantic library/exporter, kernel validation and round-trip evidence. The production inventory checks coverage of the mapping design; it does not validate generated KerML because no such exporter exists yet.

G requires Browser and all six view projections, instance-aware identity, accessible notation, validated source edits, layout separation and atomic undo/redo. An LSP with generated TextMate tokens is an editor foundation, not G. Full IDE refactoring additionally requires path-aware rename/reference/navigation and persistent identity handling.

A tool's conformance report MUST identify language version, implementation version, levels claimed, rule groups checked, unsupported capabilities and evidence references. It MUST NOT promote S or partial V to K/E/G by implication. Implementation status is this repository's report.

8.5 Scope and future evolution

Completeness of this specification means every 1.3 construct has syntax, ownership, reference/type rules, static constraints, operational or descriptive meaning, a normalization contract and a conformance category. It does not mean every future software architecture concern is a built-in keyword or that all backends are implemented.

The finite boundary remains explicit: single inheritance with compatible explicit redefinition; no multiple inheritance, generics, reference-valued payloads, dynamic object creation/subscriptions/discovery, unrestricted fork/join, hierarchical/orthogonal state machines or arbitrary host-language code. Model adapters and protocols can express supported interactions; a new construct needs a versioned proposal with the same obligations. Patch clarifications ordinarily preserve valid-model source compatibility. The 1.2.1 relation-spelling correction preserves model meaning but requires migrating case for and satisfaction by to of. ADR 0010 records this source compatibility change and the rejected old forms. The 1.2 host selected a language family plus installed package revision. The 1.3 parser explicitly selects the new edition and profile; reproduction must pin the package revision too. Other incompatible changes require an explicitly versioned edition and migration notes. The historical 1.1 release intentionally migrates syntax from 1.0 despite using a minor version; it provides no silent legacy aliases.

Requirement and presentation extension

The rules in chapter 4 preserve requirements as named obligations with immutable attributes and at most one owned formal doc. Satisfaction references a design identity and requirement identity; verification references a verificationCase identity and requirement identity. Use cases cannot own verification links. Ordinary comments retain source extents and owner identities separately from formal documentation. Cases and links express intent, with no automatic proof status. View include/filter clauses resolve to a stable union of occurrence paths and have no execution steps or store effects. These constructs remain part of the proposed mapping contract; no kernel exporter is implemented.

NORMATIVE / LANGUAGE CONTRACT

9. Formal core and simulation relation

Normative chapter of the SWAML 1.3 specification. These judgments constrain implementations alongside chapters 1-8. They make explicit the distinction between static validity, allowed executions, and a particular target's choice among them. They do not assert a mechanically proved metatheory.

9.1 Domains and judgments

Let D be the resolved declaration graph; O the finite expanded occurrence tree; Γ a lexical environment mapping names to types; σ an occurrence-indexed store of copied values; ρ an activation's parameter/local store; Q the connector queues; A the live activations; M the machine states and timer generations; R the correlation table; L the component serial locks; B the released lifecycle/timer work; and t an integer nanosecond time. A runtime configuration is:

```text
C = (D, O, σ, Q, A, M, R, L, B, t)
D ⊢ O valid                  closed assembly validity
D; Γ ⊢ e : T                expression typing
D; Γ ⊢ s ⇒ Γ'               statement typing and resulting lexical scope
D; σ; ρ ⊢ e ⇓ v : T         successful pure expression evaluation
D; σ; ρ ⊢ e ⇑ f             expression failure
C  - label→ C'                one permitted observable/local step
trace(C) = label₀ label₁ …   finite or infinite execution trace
```

A value is a bool, bounded byte/uint/int, finite real, string, nominal enum literal, finite nominal record of values, or fixed array of values. void has no inhabitants. Record identity includes its qualified declaration identity; arrays include element type and extent. Values never contain references into O, σ, A or Q. An environment binding also carries its declared type; widening an int into a real binding preserves real type even when its mathematical value is integral.

Assignment compatibility includes identity and scalar widening byte ≤ uint ≤ int ≤ real. Integral assignment into uint or byte additionally requires a destination-domain range check, statically for known constants and at runtime for computed values. real never implicitly narrows. Integral arithmetic yields int; division or a real operand yields real. Numeric array literals preserve a common identical type; mixed types yield real when any element is real, otherwise int when any is int, otherwise uint. A literal checked against a declared array type checks each element against that destination domain. Existing array values retain invariant element type and length. A scope extension Γ[x:T] requires x absent from the live environment. Block-local extensions are discarded at block exit; assignments to existing cells remain.

9.2 Static rule schemas

The schemas below apply recursively, together with the name, ownership, route and contract restrictions in chapters 1-4. A failed premise rejects the construct, rather than adding an implicit conversion or exchange.

Construct	Premises	Conclusion
Reference x	x:T in Γ	Γ ⊢ x:T
Enum T.literal	no first-segment value T in the current expression environment Γ; one unambiguous accessible enum T; literal in its domain	nominal value:T
Qualified enum P::T.literal	loaded package P declares enum T and its literal	nominal value:P::T, independent of lexical T

Construct	Premises	Conclusion
Field e.f	e has nominal record T; T declares f:U	e.f:U
Index e[i]	e:T[n], i:byte/uint/int	e[i]:T; runtime requires 0 ≤ i < n
Record T	one value for every declared field; each value assignable to that field	constructed value:T
Array	nonempty values have one nominal type or numeric common type T	value:T[n]
Arithmetic	numeric operands, % additionally integral operands	/:real; %:int; other operators:real when either operand is real, otherwise int
Boolean/comparison	chapter 1 operand conditions	result:bool
f(args)	calculation f owned by enclosing function; signature-compatible copied args; pure acyclic graph	result has f's declared non-void type
var x:T=e	e:U, U assignable to T with required range check, fresh x	Γ ⇒ Γ[x:T]
x=e	x:T in Γ, e:U, U assignable to T with required range check; activation effect rules permit write	Γ ⇒ Γ
call f(args) [as x]	same-component untriggered flow function; compatible copied args; acyclic graph; fresh x required exactly for non-void	Γ ⇒ Γ[x:result] for value return, otherwise Γ ⇒ Γ
send/publish p.m	p owned by current component; correct outward member and verb; compatible args	Γ ⇒ Γ
call p.m as x	p owned by component; outward RPC; compatible args; positive timeout; fresh x and failure name; failure block cannot fall through	Γ ⇒ Γ[x:result] on success; void x is reserved but uninhabited
return e	e assignable to owning function result	current function terminates
if / while / assert	condition:bool; loop bound positive safe integer	each nested block checked in a fresh child scope
for	fresh typed initializer local; condition:bool; typed update in header scope; positive literal bound	header scope encloses fresh body scopes and is discarded after the loop
forEach	copied nonempty fixed array; compatible explicit element type; fresh binding	one fresh copied element/body scope per array position
do	positive literal bound; body checked first; enclosing condition:bool	at least one body; body locals absent from the condition
interruptible	triggered serial flow owner; unique local incoming member; typed payload bindings	normal and interrupt branches share outer scope, activation and deadline; protected locals cannot escape

Construct	Premises	Conclusion
transition	states in same function; legal stimulus; bool guard; effect typed with payload; distinct competing priorities	transition belongs directly to its function

For root validity, an inline root expands its authored component children, connections and local behavior; a typed root expands exactly its referenced component definition. Both require an acyclic expansion, distinct occurrence stores and port closure. A typed root binding contains only deployments and introduces no topology. An omitted profile normalizes to service. An omitted function/signature result normalizes to void; parameter/data/calculation result types cannot be omitted. These are defined elaborations, not inference from body behavior.

A connector selecting `qos P` requires one unambiguous package-visible QoS policy declaration P. Its effective policy equals P.body; an inline selection uses its own body. The base-field, sampled, idempotency and realtime premises are checked on that effective body for every use. Queue identity is still the connector occurrence, direction and member, independent of policy identity.

Before expansion, compute effective members by following the acyclic single-specialization relation. Every duplicate inherited name requires compatible explicit redefinition. Resolve inherited local references against the effective component context. Expansion then assigns a distinct store and lock to every instance, including a root with local behavior. Type reuse never identifies stores.

Let F(op) be the set of component-variable assignments in op and recursively in every locally invoked helper. For every reentrant activation root, F(op) = ∅. Calculations have no component-variable environment and no function edges. These finite graphs are acyclic, so both checks terminate.

For every normalized connector (p,q), contracts(p) = contracts(q) by declaration identity and roles(p,q) are complementary in the orientation defined in chapter 2. Delegation preserves both identity and role and introduces no queue. Every cross-component effect is an offer to a member projection of such a connector. Thus a successful path lookup into a remote private store is never an execution rule.

The integral domains are `byte = {0,…,255}`, `uint = {0,…,2^53-1}` and `int = {-(2^53-1),…,2^53-1}`. Scalar widening preserves value along byte → uint → int → real. A checked integral assignment into byte or uint either yields an in-range value or raises a range failure; it never wraps. Integral arithmetic produces int unless division or a real operand determines real. A calculation evaluates its one expression in a copied input-only environment and returns its named output value; it creates no component activation or shared store.

9.3 Local execution rules

Expressions evaluate operands left to right, except short-circuit `and`/`or`. A checked arithmetic operation either produces a value in its result domain or raises a local Arithmetic failure. Integer arithmetic is mathematically exact before checking the bounded result; implementations MUST NOT let floating-point rounding turn an out-of-range integer into a valid one. Remainder uses truncation toward zero. real arithmetic uses binary64. Equality recursively compares typed values; integral/real numerics compare after scalar promotion.

A local var creates a new local cell initialized by a copied, widened value. An assignment evaluates first, then atomically replaces the target cell. A call pushes a copied parameter frame on the existing activation root, preserving its lock and snapshot. A helper return copies and typechecks its result, pops only that frame, and binds the successful non-void result in the caller’s lexical environment. Returning from the outermost activation frame completes the activation. A failure unwinds all frames of that activation root without rollback. A loop tracks completed body iterations: after n executions, it evaluates the condition once more and fails if still true.

A counted loop evaluates its initializer once in a new header scope. Each true test permits one body while the count is below the bound, followed by its assignment update. A do loop starts with its body and then

tests; a true test after the nth body fails before any extra execution. forEach evaluates/copies its array once and processes its ordered elements in fresh scopes; collection and element mutations do not change the saved sequence. Return/failure propagates without later update or condition effects.

An armed region adds a typed receive watch to its current serial activation. Delivery disarms the selected watch and transfers control by unwinding to that region's handler boundary. Outer-scope cells and committed effects remain; protected locals and abandoned local-call frames disappear. A Pending caller correlation becomes terminal Interrupted; terminal correlations are unchanged. The new handler continuation retains the activation lock/snapshot/deadline, while all queued resumes of the abandoned continuation are invalid. The server has no cancellation transition implied by this local transfer. Region exit, return and failure remove the associated watch; unavailable watches produce explicit input discard observations.

A reentrant dispatch snapshots σ for its component. All component reads by that activation, including helpers after suspension, use that snapshot. A serial dispatch reads the live store and holds its lock through requests. Local statements may interleave with other enabled activation roots at statement boundaries, but no competing serial activation root can acquire the same lock.

9.4 Communication transition schemas

Let q be a route/direction/member queue, capacity k, m a copied message, and c a fresh correlation. `offer(q,m)` is atomic. Each accepted logical message retains its original timestamp through all attempts.

Rule	Preconditions	State change and observation
Admit	retained(q) < k	append/admit m; emit admission
Reject	full(q), overflow=reject	q unchanged; report rejection to producer/correlation
Drop newest	full(q), overflow=dropNewest	q unchanged; emit drop(m); producer admission succeeds
Drop oldest	full(q), overflow=dropOldest	discard oldest retained logical message; admit m; observe both
Ready	message has reached destination transport boundary	set ready time; check maxLatency against original emission
Expire	t - emitted(m) > maxAge	discard before dispatch; no receiver effect
Dispatch	m ready, live handler, lock/limit available; FIFO head when required	remove dispatchable occurrence; copy args into an activation frame; record delivery/release/dispatch
Request	outward RPC statement, no other awaited statement in that frame	create Pending c with expiry=t+timeout; offer request; suspend caller
Respond	handler returned/failed	offer copied result/RemoteFailure on reverse route referencing c
Complete	response ready, R(c)=Pending, t < expiry(c)	R(c)=Succeeded/Failed exactly once; resume caller
Timeout	R(c)=Pending, t ≥ expiry(c)	R(c)=Failed(Timeout); resume failure block
Late response	R(c) terminal	discard; no caller state change

Fanout applies offer independently to every selected route before propagating any rejection. Running handlers are not queue entries. A dispatch or response cannot fabricate another member, route or correlation. Lifecycle releases, observations and local failures are not message offers. All queue loss and duplicate dispatch must be visible in a checking trace. Delivery policies constrain allowed traces, not source-language branches.

9.5 Stateful function transition schemas

M(op) contains (state, generation, ready, live, reacting). One reaction at a time may execute. Initialization runs initial entry before setting ready. For stimulus z in state s, form candidates with matching source and trigger, evaluate their guards in the appropriate copied payload environment, and select the enabled candidate of maximum priority. No candidate means consume/ignore z. Selection is not applied across unrelated stimuli.

For selected transition s→u: invalidate s's timers; execute exit(s); execute effect with z's payload; change state to u and increase generation; execute entry(u); install one timer per distinct outgoing duration. Any return/failure stops these phases immediately. Final entry terminates the machine. Completion selection then repeats until no transition is enabled or the machine terminates, with at most 1,024 taken completion edges per reaction. Entry/exit lack payload bindings. Completion reconsideration also follows an ignored external stimulus.

A timer stores its machine generation. A stale timer has only a discard transition. A valid timer is released at entry-completion time plus duration, and competes with independent work under the target scheduler. A terminated machine never regains liveness inside the same run.

9.6 Time, traces and refinement

Logical time never decreases. A target may advance to the next release, transport availability, deadline or timer event when its scheduling policy permits. Time addition MUST remain exact; a simulator encountering an unrepresentable time stops with an explicit resource/clock failure. Deadline elapsed time includes dispatch delay and suspension; WCET active time excludes suspension. A zero-cost logical simulation can check ordering, queues and elapsed waits but cannot certify physical WCET or schedulability.

An implementation binding refines this specification when every trace it produces can be matched by the core relation, with any internal bookkeeping steps hidden, and it satisfies its declared progress/fairness assumptions. A deterministic simulator chooses one such trace. Passing that run does not establish universal safety or liveness across all allowed schedules, nor realize every descriptive case automatically.

The reference simulation profile specifies a concrete, bounded choice of these rules. Reproducible inputs include all model source digests, selected root component, profile version, time horizon, harness releases, transport parameters and resource limits. No theorem of type preservation, kernel compatibility or trace equivalence is claimed without a separately checked proof artifact.

Requirement and presentation extension

The rules in chapter 4 preserve requirements as named obligations with immutable attributes and at most one owned formal doc. Satisfaction references a design identity and requirement identity; verification references a verificationCase identity and requirement identity. Use cases cannot own verification links. Ordinary comments retain source extents and owner identities separately from formal documentation. Cases and links express intent, with no automatic proof status. View include/filter clauses resolve to a stable union of occurrence paths and have no execution steps or store effects. These constructs remain part of the proposed mapping contract; no kernel exporter is implemented.

NORMATIVE / LANGUAGE CONTRACT

SWAML graphical notation

Language: 1.3 · **Notation edition:** 1.3.0 · **Status:** normative

This chapter defines how every SWAML language concern is represented and inspected graphically. The canonical notation catalog pairs each element or property with original graphical and textual notation, contextual constraints, and downloadable SVG/PNG examples. The worked view examples show those ideas in complete architectures. The notation inventory records coverage of the concrete grammar and reserved vocabulary.

The organization separates a concept catalog from construction and interaction rules, informed by the SysML specification review. The drawings and rules here are original SWAML material. They do not claim SysML graphical conformance or introduce language constructs absent from the SWAML grammar. A shipped renderer, editable workbench and kernel exporter remain separate implementation milestones.

1. Scope, authority and coverage

The textual grammar and normative language chapters determine model meaning. A graphical representation MUST preserve that meaning. The catalog supplies canonical depictions of that model; it cannot legalize a missing connector, unsupported state region, free event, cross-component reference or invalid source fragment.

Every grammar production and reserved keyword MUST have an explicit coverage entry in the inventory. Coverage either names a catalog concept or identifies a text-only contribution with a reason. Lexical trivia, precedence productions and grouping rules need no independently editable glyph. Their textual or ownership role must still be accounted for. Every view family also has a canonical projection example.

The classification is precise:

Classification	Graphical contribution	Examples
Node	a bounded selectable semantic element	component, function, calculation, state, interface
Edge	a relationship with typed endpoints	connector, delegation, deployment, case exchange
Compartment	owned content inside a node	parameters, members, entry/exit statements
Label	a value or modifier attached to its actual owner	timing, role, optionality, profile, guard, QoS
Projection	a view of existing identities and relationships	Browser, gv, iv, cv, sv, grv, fv
Text-only coverage	source syntax contributing to an owner without a separate glyph	comments, punctuation, expression precedence and grammar unions

A label sample drawn inside a card is not a new model node. Catalog IDs, concern headings and drawing templates are documentation identifiers, not additional SWAML keywords. Fragments in the textual-notation column require the stated context; only complete model examples can be parsed alone.

The chapter, catalog rules and inventory MUST agree. A conflicting image, label or rule is a notation defect to fix, not permission for tools to invent a third meaning. Normalization and runtime semantics remain those of the language specification.

2. Canonical drawing vocabulary

Canonical SVG and PNG assets use dark strokes/text on a white background. Shape, line style and explicit wording carry meaning. Color is optional presentation data in a workbench or worked example and MUST NOT be the only carrier of type, role, direction, optionality or error state.

All SWAML graphical projections MUST use flat two-dimensional geometry. External deployment targets use ordinary labeled rectangles, like the other allocation and architecture frames; cuboids, isometric faces, perspective, extrusion, bevels, gradients and cast shadows MUST NOT be used. Containment and deployment are expressed by enclosure, labels and edges, never by apparent depth. This rule applies to canonical specimens, worked examples and interactive views.

A semantic node has a header and optional horizontally separated compartments. The header shows its kind and name, with typing where applicable. The project labels «component», «component def» and similar guillemet labels identify SWAML concepts; they do not import a UML stereotype mechanism into the language.

Element family	Required form and identification
Package	namespace frame with package name; a tab may identify its namespace role
Component definition	rectangular frame with component marker and «component def» Name
Component occurrence	corresponding frame with name : Type and occurrence context
Root component	component frame, root occurrence name, optional type and effective profile
Record/enum declaration	contract-style rectangle with kind/name and field/literal compartment
Interface	rectangle with kind, name, version and member signatures
Function	rounded node or owning frame labeled «function», signature and activation source
Calculation	owned calculation compartment with purity indication, complete signature and expression
State	rounded rectangle, state name, optional entry and exit compartments
Requirement	rectangle with kind/name and text or inspectable synopsis
Case	rounded node or case frame with kind/name and selected root context
External deployment target	flat rectangle with «external system» and the exact reference identity; inspectable host-resolution status

Definition and occurrence markings MUST be distinguishable without color. The def indication is mandatory for reusable component types. A root usage has no def; a typed root retains its referenced type. The effective service profile may be shown as service (default) when omitted in source, with source spelling available on inspection. Diagrams MUST NOT add a system frame or authored machine node.

Empty compartments may be omitted. A nonempty collapsed compartment MUST have an omission indicator and a way to inspect its contents. Ordering of fields, parameters, statements and case dependencies follows language rules; visual sorting cannot change ordered semantics. Node size, location, elbow routing, typography and whitespace are layout choices unless a rule below assigns them meaning.

3. Names, ownership, types and values

Names on diagrams resolve through the same package, import and local-scope rules as source. A short name may be displayed only when its fully qualified identity is available and the shortening cannot mislead the reader. An instance view retains both its occurrence path and shared declaration identity. A selected component occurrence never denotes all same-type components' storage at once.

Semantic containment depicts real ownership. A view frame or explicitly labeled context frame identifies the projection’s subject; enclosure in that frame does not reparent elements. In particular, a case’s `of Root` context is a reference to the selected root, while the case remains owned by its package. Component frames may contain child component instances, ports, connections, private variables and functions. A function frame contains its local calculation compartments and statement flow or directly owned states. No performer lane or action-allocation edge is introduced. External software-facing devices must be modeled as components with ports.

A typing reference is explicitly labeled `type` and joins an occurrence to its definition. It is descriptive, does not transport data and does not denote inheritance. A separate `specializes` edge denotes a component definition's single base. A replacement member is marked `redefines` and links to the inherited member. Neither relation is a communication route. The `name : Type` header alone is sufficient when an additional typing edge would clutter the view.

Datatype fields show `field : Type`; enum compartments list their declared literals. Primitive names, fixed-array extents and result types are labels, not autonomous architecture nodes. A private variable may show its declared type and pure initializer. A local value belongs to its lexical function/block scope. Record, enum and array construction are expressions displayed as source text or expression subtrees in that owner.

An expression subtree MUST preserve operand order, precedence, short-circuit conditions and nominal typing. Qualified enum text remains distinguishable from a local field read. Graphical editing MUST use the current expression environment for that distinction. No value edge may expose another component's private state or imply reference-valued payloads. Whole-value assignment and copied payloads retain their source-language meaning.

4. Interfaces, members and ports

An interface node shows one of the four kinds, its nominal name and exact version. Its member compartment distinguishes data, events and function signatures:

Kind	Member display	Permitted role pair
senderReceiver	typed data with sampled/queued mode	sender / receiver
clientServer	ordered input parameters, result and optional idempotent label	client / server
publishSubscribe	event name and ordered payload parameters	publisher / subscriber
service	function signatures and event signatures	requires / provides

A missing function result annotation has effective type void. The diagram may display void for clarity, but MUST expose whether it was explicit or omitted when editing source. Events have no result. An idempotent badge is an asserted contract property; it is not a verified-effects badge. A field-like service property remains separate getter/setter/event members with private storage, never a remotely writable attribute.

A port is a small square touching the boundary of its owning component. Its label identifies `name : Interface` and role. The exact interface version must be visible in an expanded label (`name : Interface @ version`) or available through the identified contract’s inspectable details. Full role text is canonical; a workbench may

abbreviate to `S`, `R`, `C`, `Svr`, `Pub`, `Sub`, `Req`, or `Prov` only with an available legend and full accessible text. Contract kind cannot be inferred from a square alone. In an isolated port specimen, arrows illustrate the role's direction, rather than declare a connector to an unnamed participant. A dashed reverse arrow illustrates RPC replies; service events use solid reverse message arrows in an exchange projection.

Optionality and connectivity are different properties. An optional port MUST carry `optional`; the boundary square remains white like other ports. No separate optionality glyph is required. A required port MUST NOT be inferred merely from dark fill or a drawn line. An unconnected required port in a closed root is visibly invalid, not visually erased. An optional unbound request retains Unavailable semantics; a blank diagram region is not an implicit service.

Port inspection MUST expose its contract members, role-derived directions, handler owner, emitting functions and resolved delegation chain. The diagram may show a local association from a member to a function as a labeled handler/emitter relationship. That association is not a second architectural connector and cannot point directly to a remote function.

5. Architectural edges, direction and communication policy

Every architectural exchange edge MUST terminate at visible port squares or an explicitly elided port endpoint whose full path is inspectable. A line ending at a function box or a component's unlabeled center cannot stand in for a typed communication route. Crossings do not create junctions; each connector keeps its own identity and queue policy.

Relation	Canonical line and endpoint	Required interpretation and label
Sender-receiver connection	solid, filled triangular arrowhead at receiver port	sender → receiver; interface and data relation
Publish-subscribe connection	solid, filled triangular arrowhead at subscriber port	publisher → subscriber; interface and event relation
Client-server connection	solid, filled triangular arrowhead at server port	client → server; RPC request orientation
Service connection	solid, filled triangular arrowhead at provides port	requires → provides; reverse events/replies disclosed
Delegation	solid line without arrowhead between outer and child port	`delegate`, identical role/contract, no queue
Typing reference	dashed, hollow triangular arrowhead at definition	`type`, no exchange or inheritance
Specialization	solid, hollow triangular arrowhead at base definition	`specializes`, single inheritance
Member redefinition	reference link or inspectable inherited-member identity	`redefines`, compatible replacement
Import reference	dashed, open V arrowhead at imported package	`import`, no runtime relationship
Deployment	dashed, open V arrowhead at external target	`deploy`, no communication
Requirement trace	dashed, open V arrowhead at requirement	`satisfy` / `verify` until separate evidence exists

A connector's arrow indicates the declaration's orientation, not a promise that the middleware is one-way. Service notifications and RPC responses travel in the reverse role direction on their existing route. Fanout

draws distinct named connectors, not an unnamed event bus or a junction that merges queue identity. Receiver/client/requires cardinality and duplicate-route checks remain mandatory.

A package-owned reusable policy uses a flat rectangle labeled `«qos def» Name` and a property compartment. A connector selecting it carries `qos Name`; inspection exposes its qualified declaration identity and complete effective fields. This reference is not a communication edge and cannot share queues.

Expanded connector detail MUST show effective delivery, ordering, capacity and overflow. Optional maxLatency/maxAge appear with time units. A compact connector may show an inspectable QoS label or linked compartment, but cannot silently supply omitted mandatory fields. Sampled/queued, atLeastOnce, timeout and idempotent labels do not collapse into a single reliability icon.

Delegation is a forwarding alias. Its line MUST be visually distinguishable from an inter-child connector and labeled `delegate`; no queue, transform, handler or retry icon is added. Routing through a hierarchy preserves each boundary port. Drawing a shortcut to a grandchild without the corresponding chain is invalid.

6. Function signatures, triggers and control flow

A function header identifies its owning component, signature, effective result, trigger and optional timing contract. Its trigger may be start, manual, every with duration, receive with local port member, or call with local port member. Triggers are labels on functions, not free signal nodes. Missing trigger means a local helper subject to the normal invocation restrictions.

A calculation is a pure local expression with explicit non-void result. Its signature and expression occupy the owning function’s calculation compartment; a focused card depicts that compartment without introducing a separate architecture element. A local `call` node references an untriggered same-component flow function and shows its result binding when non-void; a calculation-call expression references a calculation in the enclosing function. These depictions MUST NOT allow arbitrary cross-object calls merely because an edge can be drawn.

fv statement nodes are owned by the displayed function/block. Text-only statements may be shown as a labeled rounded node or source compartment. Their text is authoritative, and they do not become independent action definitions. The minimum required forms are:

Source concern	Required projection
var / assignment	typed declaration or assignment text in an ordered statement node
expression / calculation call	exact expression or inspectable ordered expression subtree
if / else	Boolean decision with true/false branches and a merge of control
bounded while	condition decision, body/back edge and explicit positive bound
counted for	initializer, decision, body, update and back edge; explicit bound and loop scope
forEach	copied-array snapshot, next-element decision and fresh typed element binding; finite array-length bound
do … while	body precedes the first decision; back edge and explicit positive bound
interruptible region	named protected frame, normal exit and labeled typed interrupt edge to its handler
assert	condition and failure outcome, distinct from a prose requirement
local call	function reference, optional typed result binding and continuation
send / publish	verb, local port.member and payload expressions
routed call	local port.member, inputs, result/completion binding, finite timeout and failure branch

Source concern	Required projection
return	completion node with bare or typed-result text
fail	failure terminal with its reason; no implicit event

The first flow marker is a filled circle identifying local activation entry. Ordered flow edges are solid with filled triangular arrowheads; a Boolean decision is a diamond. Return/failure nodes retain explicit terminal labels so neither is mistaken for an interface message. Flow edges sequence local execution and cannot cross an owning component boundary. Decisions MUST show which branch is true/false; an absent else still has a false continuation. Loop depictions MUST show the bound and the bound-violation outcome, even if detailed failure edges are collapsed with an inspectable marker. A back edge is not permission for unbounded execution. Fork/join, parallel regions and arbitrary activity allocations have no notation because they have no SWAML 1.3 syntax.

A request has a success continuation and a terminating otherwise branch. The latter exposes Timeout, Unavailable, Rejected, Transport and RemoteFailure as possible interface outcomes and its local failure binding. The diagram MUST NOT suggest that timeout cancels committed server effects, that the result exists on failure, or that a void completion name is data. A remote request node is still a local function step; the architectural route is inspected through its port reference and iv.

7. Directly owned state behavior

The state projection is the fv of one function. It has a function frame, one initial reference marker, state nodes and transitions. There is no independent machine owner, state definition/usage layer, performer or behavior allocation.

A filled initial circle with an outgoing local-control edge selects the declared initial state. It projects the function’s unique initial reference, rather than adding another authored element. It denotes local initialization, not an interface event. Each state shows its name and optional entry/exit compartments. A final state remains a named state node and carries `final` plus a final marker consisting of an outer circle and filled center; it completes its function after entry. The marker cannot replace the state name or hide its entry behavior. Finality MUST remain apparent when a compartment is collapsed.

State transition edges are solid with filled triangular arrowheads at the destination state. Their labels preserve source and destination, trigger, optional guard, priority and effect. The canonical textual content is `trigger [guard] / effect`, with explicit `priority n` when supplied; absent priority has effective zero. The transition's source name or source identity remains inspectable even when the diagram places text on its edge.

Receive triggers name the owned `port.member` and contract payload. Delay triggers show `after duration`; completion is labeled `completion`. Timer and completion edges are local control, not communication channels. A guard is a Boolean condition, not a new state. Priority resolves competing transitions for one stimulus; it does not globally schedule unrelated inputs.

Transition inspection exposes the exit → effect → state update → entry order. Payload bindings are available only to the guard/effect, not entry/exit. A self-transition visibly returns to the same state and still exits/reenters, resetting timers. A return/failure inside any phase terminates the function and cancels timers; a graphic cannot imply later phases still occur.

Unconditional completion cycles remain invalid; guarded cycles retain the bounded completion-reaction rule. No-enabled-transition consumption, stale-timer discard and undeliverable input to a terminated function may appear in trace overlays. These overlays are observations, not new model messages. Hierarchical/orthogonal regions, history, do-activity compartments and state inheritance are absent.

8. Timing, profiles and deployment

Timing is a label/compartment attached to the function; it lists wcet, deadline and concurrency with exact durations. Every, maxLatency, maxAge and request timeout stay attached to their respective trigger, connector or request. Time units are always shown. A profile label belongs to the root component, not every nested component by visual implication.

The display distinguishes declared, defaulted and measured values. No timing declaration means unspecified physical timing, not zero execution cost. Serial/reentrant applies to function activation roots and their helper context; it does not make a component execute or reenter. Reentrant snapshot semantics and state-reaction non-overlap remain inspectable constraints.

A deployment edge joins an existing component instance to an external system/part identity. Direct and inherited allocations MUST be distinguishable in Properties or grv. A typed root's binding adds deployments only; a drawing editor cannot add behavior or topology there. External existence and suitability are host-checked properties, displayed separately from local path validity.

Runtime trace annotations may show elapsed deadlines, queue counts, timers and failures. They MUST identify the run/profile/source and distinguish observations from source contracts. Logical simulation results cannot be restyled as physical timing evidence.

9. Cases, requirements and sequence exchanges

A requirement node shows its identity, attributes and formal doc compartment. A useCase names intended use and its root. A verificationCase may own verify links to requirements. A satisfy arrow starts at a design element; a verify arrow starts at a verification case. Both point to the requirement. Verified status requires linked evidence with identity, assumptions and result; the source link alone never earns a verification checkmark.

sv lifelines represent component occurrences at the selected case's composition level. Each endpoint names its port and interface member. Exchange labels retain the case step label, kind, named connector and any predecessor. Requests follow client/server or requires/provides orientation; replies reverse their matched request; provider events follow provides/requires direction.

Sequence request/data/event arrows are solid with filled triangular arrowheads and explicit call/send/publish labels. A reply is dashed with an open V arrowhead in the reverse direction. Lifelines are dashed vertical guides; they are not channels. A request and its reply display one shared correlation reference derived from the uniquely matched causal request. A reply line does not create another interface member or independent return channel. An unanswered request remains visible as an incomplete partial trace, rather than receiving an invented reply.

An sv renderer MUST preserve `after` causality. Vertical location is a layout tie-break and MUST NOT impose order on independent steps. Independent groups must be marked as unordered/concurrent or expose the dependency relation explicitly. A reply with ambiguous or missing request ancestry is invalid even if its position looks plausible.

Case diagrams never introduce free actor messages, topic names, hidden connectors or independent action execution. Their routes and member directions are statically checked; realization by function bodies still needs an execution/analysis witness.

10. Views and navigation

Surface	Primary subject	Required content
Browser - Model Browser	workspace or selected subtree	ownership, definition/occurrence identity, references, diagnostics
gv - General View	package or named semantic declaration except another view	declarations, ownership, types, properties and relationships

Surface	Primary subject	Required content
iv - Interconnect View	root/nested component occurrence or component definition	immediate children, boundary ports, connectors and delegation
cv - Case View	case or root component	cases, requirements and claimed coverage
sv - Sequence View	case or case selection from a root	port/member exchanges and explicit dependencies
grv - Grid View	package or named semantic declaration except another view	sortable identity-aware properties, contracts and allocation
fv - Flow View	function	statement/control-flow or directly owned states

These are the complete view families. fv projects statement flow, function-owned states and trace-backed function timing. Interruptible regions are part of flow. Browser is navigation, not a new declaration keyword. A package subject is its own name; nested view subjects retain occurrence context while resolving shared members through definitions.

Interrupt and timing presentation in FV

An interrupt edge leaves a named protected region and targets its handler. Label the edge with the owned input port/member; inspect its interface and existing route through iv. The handler and the normal exit converge on the following statement, unless either branch returns or fails. Enclosure denotes the protected scope, not a second component, performer or activation. The interrupt edge must not imply rollback, server cancellation, pause/resume or physical processor preemption. The plain region frame remains flat and uses explicit labels.

Timing diagrams use a common logical-time axis and distinct lanes for function occurrence and activation identity. Show release, dispatch, ready/running/request-wait phases, completion/failure, interrupts and correlated late responses. Label phases and event kinds independently of color. Show declared WCET, deadline and concurrency separately; never draw WCET as an observed execution duration. Zero-duration execution stays a marker. Incomplete intervals remain visibly open at a horizon or stopped-run boundary. Identify the source model, root, profile and run; layout must not fabricate trace events, completion or scheduling proof.

The Browser follows packages → definitions/root occurrences → child occurrences → ports, variables and functions → calculations, statements or states. Opening a usage keeps its occurrence context. Open declaration selects the shared definition; Up selects the owner; Diagram chooses an applicable view. Selection updates Properties. These are host requirements, not commands already implemented by this repository.

grv minimum columns depend on concern: inventory identity/kind/owner; interface version/member/roles/emitter/handler; connection endpoints/QoS; function signature/trigger/timing; deployment explicit/effective external identity; requirement attributes/doc/satisfaction/verification/evidence status. Sorting and filtering change presentation only. Computed values reveal their source and cannot be edited as independent declarations.

11. Canonical assets and reproduction

Each catalog entry has a stable ID and an SVG, PNG and scene JSON under `docs/notation/`. The SVG is resolution-independent line art; the PNG provides the same drawing for tools that require raster images. Its pixels MUST depict the same geometry, labels and semantic distinctions. PNG compression or metadata differences are not semantic differences; reproduction checks compare decoded pixels.

The scene is a deterministic drawing source containing basic primitives and text, not another semantic model. Inventory entries identify the concept, classification, context, syntax fragment, normative rules and accessible description. The catalog also provides an alphabetical term index. Existing worked figures remain

separately linked to complete validated models and illustrate composed views rather than replace concept coverage.

Run `python3 scripts/build-notation.py` to generate the canonical assets/catalog and `--check` to verify reproducibility. The semantic coverage checker validates exact keyword/production coverage and the declared contexts/links. A passed coverage check establishes documentation completeness, not parser correctness for arbitrary fragments, graphical-editor conformance or execution proof. Every edition also requires visual review of the assets and PDF renditions.

12. Editing, accessibility and graphical conformance

Text and diagrams project one semantic model. Layout sidecars may store view/subject identity, geometry, bends, collapsed regions, filters and styling. They MUST NOT duplicate ports, interfaces, routes, triggers or function bodies as independent semantics.

A graphical mutation resolves semantic identities, prepares a source edit, parses/links/validates the result, reports new errors and applies the edit atomically with undo. Incomplete editing states may be displayed only with visible diagnostics. Moving a node changes layout; reparenting changes ownership and must revalidate all affected scopes, routes and behavior.

Deleting a port reveals affected connectors and handlers. Renaming updates typed references and path-valued references together. Generated AST addresses and source offsets are not stable editor identities. A workbench must preserve a separate identity map or equivalent stable reference strategy.

All drawings and grids require keyboard navigation, visible selection, textual alternatives and non-color error indications. SVG examples contain accessible descriptions; PNG examples accompany those same captions. Labels must remain legible under scaling, and lines must not obscure text or make unrelated edges look joined. Hidden details must remain inspectable.

A graphical-conformance claim requires every applicable projection, identity rule, editing invariant and accessibility obligation in this chapter. Producing the canonical static catalog is useful reference material; it is not a claim that the repository already implements an interactive graphical editor.

Formal documentation and ordinary notes

A formal doc appears once per owner in a labeled doc compartment, or as a doc column in grv. Every ordinary line or block comment is a separate plain rectangular note contained in its owner or connected by a plain dashed line without an arrowhead. Notes have no folded corner, depth or semantic verification status. Collapsing a note preserves its source and ownership. See requirements and documentation.

NORMATIVE / LANGUAGE CONTRACT

Canonical graphical notation catalog

SWAML 1.3 · publication 1.3.0. This generated catalog defines the original monochrome notation for each authored concept and meaningful label group. All specimens derive from the semantic inventory; complete examples supply their validation context.

Each SVG, PNG and vector scene shares the same flat two-dimensional geometry, without perspective, extrusion, gradients or shadows. A graphical specimen is a projection of its textual model; it does not create additional ports, events, state owners or execution semantics. Labels such as QoS values and expression operators appear within their owning element rather than introducing independent nodes.

The following fragments are contextual notation samples, not standalone SWAML files. `def` identifies reusable component types; a root is an instance selected for execution.

Concept index

- Assertion - Function flow.
- Assignment - Function flow.
- Boundary delegation - Interfaces and connections.
- Bounded loop - Function flow.
- Browser - View projections.
- Case causal order - Cases and evidence.
- Case data exchange - Cases and evidence.
- Case event exchange - Cases and evidence.
- Case request - Cases and evidence.
- Case View - View projections.
- Client-server interface - Interfaces and connections.
- Completion transition - Function states.
- Component definition - Structure and values.
- Component specialization - Structure and values.
- Component usage - Structure and values.
- Concurrency mode - Timing and deployment.
- Conditional branch - Function flow.
- Correlated case reply - Cases and evidence.
- Counted for loop - Function flow.
- Data member - Interfaces and connections.
- Data receiver port - Interfaces and connections.
- Data send - Function flow.
- Data sender port - Interfaces and connections.
- Delivery policy - Interfaces and connections.
- Deployment binding - Timing and deployment.
- Duration value - Timing and deployment.
- Enumeration - Structure and values.

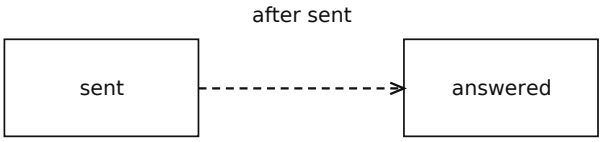
- Enumeration value - Structure and values.
- Event member - Interfaces and connections.
- Event publication - Function flow.
- Event publisher port - Interfaces and connections.
- Event subscriber port - Interfaces and connections.
- Executable function - Function flow.
- Expressions and value paths - Structure and values.
- External deployment target - Timing and deployment.
- Failure - Function flow.
- Filtered view selection - Cases and evidence.
- Final state - Function states.
- Fixed array type and value - Structure and values.
- Fixed-array forEach loop - Function flow.
- Flow View - View projections.
- Formal documentation - Cases and evidence.
- Function return - Function flow.
- Function timing contract - Timing and deployment.
- Function timing diagram - Timing and deployment.
- Function with states - Function states.
- General View - View projections.
- Grid View - View projections.
- Initial state reference - Function states.
- Inline root composition - Structure and values.
- Interconnect View - View projections.
- Interface connector - Interfaces and connections.
- Interface function signature - Interfaces and connections.
- Interruptible flow region - Function flow.
- Local declaration - Function flow.
- Local function call - Function flow.
- Manual activation - Function flow.
- Member redefinition - Structure and values.
- Message freshness bound - Interfaces and connections.
- Named connector policy - Interfaces and connections.
- Occurrence lifeline - Cases and evidence.
- Occurrence typing relationship - Structure and values.
- Optional port - Interfaces and connections.
- Ordinary comment - Cases and evidence.
- Overflow behavior - Interfaces and connections.
- Package - Structure and values.
- Package import - Structure and values.

- Parameter or record field - Structure and values.
- Periodic activation - Function flow.
- Port receive activation - Function flow.
- Post-test do-while loop - Function flow.
- Primitive types - Structure and values.
- Private component variable - Structure and values.
- Publish-subscribe interface - Interfaces and connections.
- Pure calculation - Function flow.
- Queue capacity - Interfaces and connections.
- Queue ordering - Interfaces and connections.
- Record construction - Structure and values.
- Record datatype - Structure and values.
- Requirement - Cases and evidence.
- Requirement attribute - Cases and evidence.
- Residence timer transition - Function states.
- Reusable QoS policy - Interfaces and connections.
- Root component - Structure and values.
- Root execution profile - Timing and deployment.
- Routed function call - Function flow.
- RPC client port - Interfaces and connections.
- RPC handler activation - Function flow.
- RPC server port - Interfaces and connections.
- Satisfaction relation - Cases and evidence.
- Sender-receiver interface - Interfaces and connections.
- Sequence View - View projections.
- Service consumer port - Interfaces and connections.
- Service interface - Interfaces and connections.
- Service provider port - Interfaces and connections.
- Startup activation - Function flow.
- State - Function states.
- State entry behavior - Function states.
- State exit behavior - Function states.
- State transition - Function states.
- Transition guard - Function states.
- Transition priority - Function states.
- Transport latency bound - Interfaces and connections.
- Use case - Cases and evidence.
- Verification case - Cases and evidence.
- Verification relation - Cases and evidence.

Cases and evidence

Notation: case-causal-order

Case causal order

Element	Graphical notation	Textual notation
Case causal order (edge)	**Case dependency** 	`after sent label answered`

Case causal order. Draw an explicitly labeled after dependency between step labels, or show the predecessor in the arrow label.

Context: Dependency fragment of Rpc::Poll’s reply step. **Views:** cv, sv.

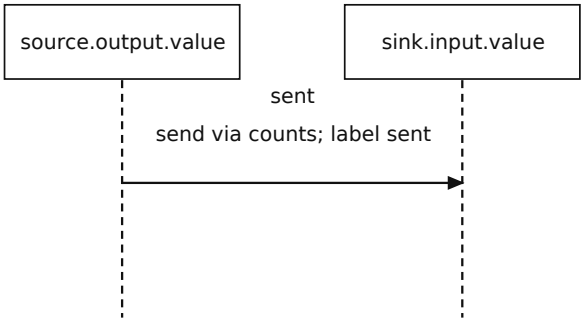
- Draw an explicitly labeled after dependency between step labels, or show the predecessor in the arrow label.
- Unrelated steps remain unordered; source order and vertical placement are layout choices, not happens-before edges.

Complete model · `Rpc::Poll`.

SVG · PNG · Vector scene

Notation: case-data-exchange

Case data exchange

Element	Graphical notation	Textual notation
Case data exchange (edge)		`send source.output.value -> sink.input.value via counts label sent;`

Case data exchange. Draw a sequence arrow between occurrence lifelines labeled send, port.member paths, connector and step label.

Context: Step in Notation::Observe; source and destination are immediate child port/member paths. **Views:** cv, sv.

- Draw a sequence arrow between occurrence lifelines labeled send, port.member paths, connector and step label.

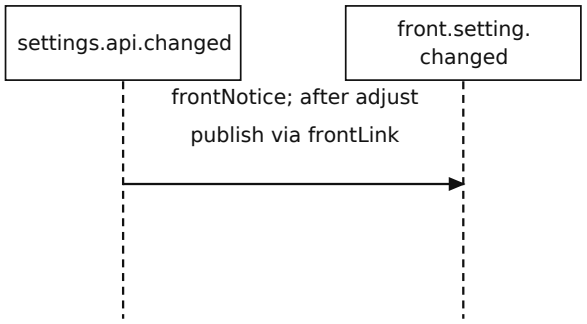
- The arrow projects an existing sender-receiver route; it creates neither a free event nor a connector by itself.

Complete model · `Notation::Observe`.

SVG · PNG · Vector scene

Notation: case-event-exchange

Case event exchange

Element	Graphical notation	Textual notation
Case event exchange (edge)		`publish settings.api.changed -> front.setting.changed via frontLink after adjust label frontNotice;`

Case event exchange. Draw a publication arrow labeled publish and the exact connector/member, retaining its step identity.

Context: Step in Settings::Change, using the reverse event direction of a service route. **Views:** cv, sv.

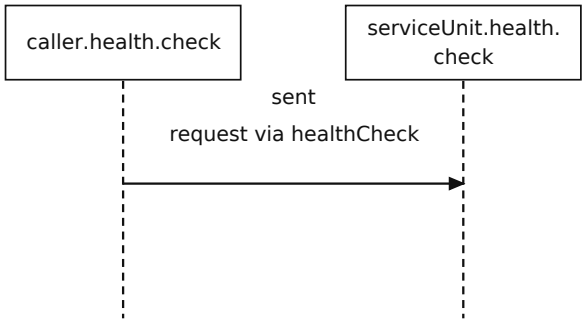
- Draw a publication arrow labeled publish and the exact connector/member, retaining its step identity.
- For service contracts events travel provider-to-consumer; line orientation follows that role even when opposite the connector’s request orientation.

Complete model · `Settings::Change`.

SVG · PNG · Vector scene

Notation: case-request

Case request

Element	Graphical notation	Textual notation
Case request (edge)		`call caller.health.check -> serviceUnit.health.check via healthCheck label sent;`

Case request. Draw a request arrow between occurrence lifelines, labeled the member, connector and step identity.

Context: Routed call step in Rpc::Poll. **Views:** cv, sv.

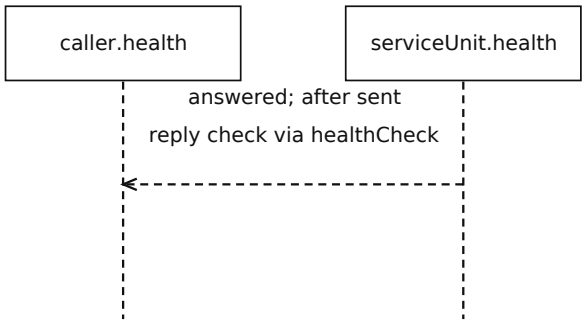
- Draw a request arrow between occurrence lifelines, labeled the member, connector and step identity.
- Its later reply must be uniquely correlated by case ancestry and route/member; vertical spacing does not create that relationship.

Complete model · `Rpc::Poll`.

SVG · PNG · Vector scene

Notation: case-reply

Correlated case reply

Element	Graphical notation	Textual notation
Correlated case reply (edge)		`reply serviceUnit.health.check -> caller.health.check via healthCheck after sent label answered;`

Correlated case reply. Draw a dashed reply arrow reversing the matching request route and label its step identity and correlation.

Context: Reply step in Rpc::Poll; after sent identifies its causal request ancestor. **Views:** cv, sv.

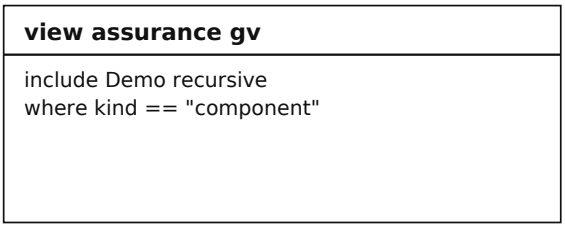
- Draw a dashed reply arrow reversing the matching request route and label its step identity and correlation.
- A reply cannot be matched by visual proximity alone; preserve the unique causal request ancestor.

Complete model · `Rpc::Poll`.

SVG · PNG · Vector scene

Notation: view-selection

Filtered view selection

Element	Graphical notation	Textual notation
Filtered view selection (projection)		`include Demo recursive where kind == "component";`

Show the selected occurrence paths within the named view frame. Combine include clauses as a stable union; filters do not alter model ownership or execution.

Context: Owned source element in the Assurance example. **Views:** gv, grv, cv.

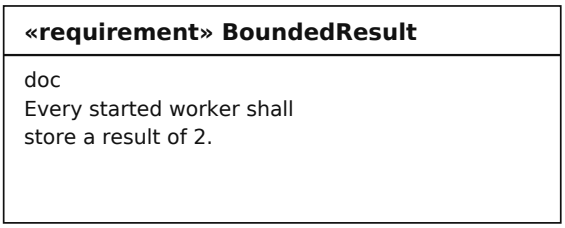
- Show the selected occurrence paths within the named view frame.
- Combine include clauses as a stable union; filters do not alter model ownership or execution.

Complete model · `Assurance::assurance`.

SVG · PNG · Vector scene

Notation: formal-documentation

Formal documentation

Element	Graphical notation	Textual notation
Formal documentation (compartment)		`doc /* Every started worker shall store a result of 2. */`

Draw one labeled doc compartment or one doc table cell for the owner. Keep multiline formal text distinct from ordinary comment notes.

Context: Owned source element in the Assurance example. **Views:** gv, grv, cv.

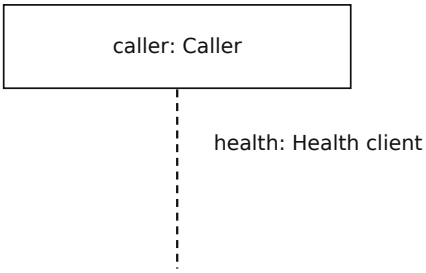
- Draw one labeled doc compartment or one doc table cell for the owner.
- Keep multiline formal text distinct from ordinary comment notes.

Complete model · `Assurance::BoundedResult`.

SVG · PNG · Vector scene

Notation: sequence-lifeline

Occurrence lifeline

Element	Graphical notation	Textual notation
Occurrence lifeline (node)		`component caller: Caller;`

Occurrence lifeline. Draw a lifeline headed by the occurrence name: Type and a dashed vertical line.

Context: Occurrence in Rpc::HealthCheck used as a sequence lifeline. **Views:** cv, sv.

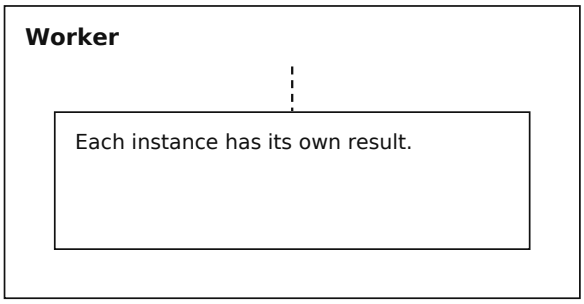
- Draw a lifeline headed by the occurrence name: Type and a dashed vertical line.
- A lifeline identifies an existing component occurrence; an actor/performer glyph MUST NOT replace a modeled communication boundary.

Complete model · `Rpc::HealthCheck.caller`.

SVG · PNG · Vector scene

Notation: ordinary-comment

Ordinary comment

Element	Graphical notation	Textual notation
Ordinary comment (node)		`// Each instance has its own result.`

Draw a plain rectangular note inside its owner or connected by a plain dashed line. Each line comment and each block comment remains separate; there is no comment keyword.

Context: Owned source element in the Assurance example. **Views:** gv, grv, cv.

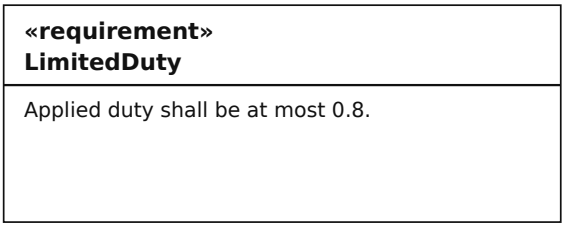
- Draw a plain rectangular note inside its owner or connected by a plain dashed line.
- Each line comment and each block comment remains separate; there is no comment keyword.

Complete model · `Assurance::Worker`.

SVG · PNG · Vector scene

Notation: requirement

Requirement

Element	Graphical notation	Textual notation
Requirement (node)		`requirement LimitedDuty { doc /* Applied duty shall be at most 0.8. */ }`

A named obligation with typed attributes and one optional formal specification.

Context: Requirement declaration in Control. **Views:** gv, cv, grv.

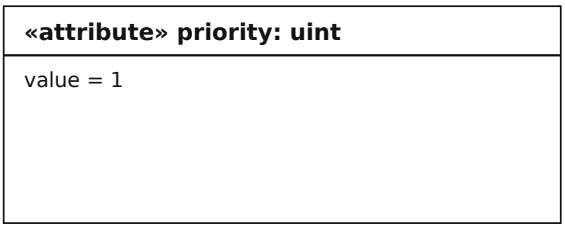
- Draw a flat requirement rectangle with its name, optional attributes and one doc compartment.
- A requirement has no subject; satisfaction and verification are separate relations.

Complete model · `Control::LimitedDuty`.

SVG · PNG · Vector scene

Notation: requirement-attribute

Requirement attribute

Element	Graphical notation	Textual notation
Requirement attribute (compartment)		`attribute priority: uint = 1;`

Display immutable typed metadata in an attribute compartment or table cell. Names are unique within their requirement; metadata is not runtime state.

Context: Owned source element in the Assurance example. **Views:** gv, grv, cv.

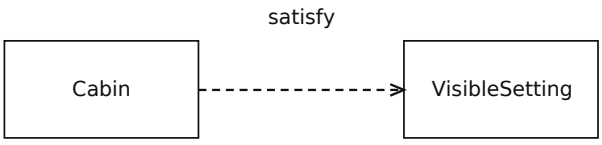
- Display immutable typed metadata in an attribute compartment or table cell.
- Names are unique within their requirement; metadata is not runtime state.

Complete model · `Assurance::BoundedResult`.

SVG · PNG · Vector scene

Notation: satisfies-link

Satisfaction relation

Element	Graphical notation	Textual notation
Satisfaction relation (edge)	**«satisfy»** 	`satisfy VisibleSetting of Cabin;`

Design claims satisfaction of a requirement.

Context: Package-owned design-to-requirement relation. **Views:** cv, sv.

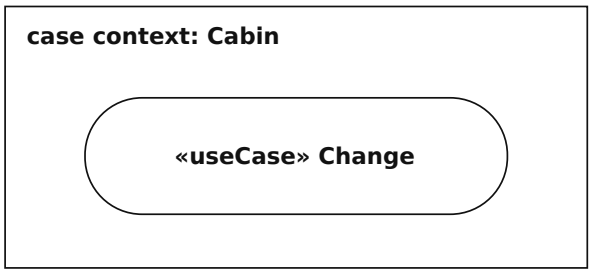
- Draw a dashed open arrow from the design element to the requirement, labeled satisfy.
- A satisfaction claim does not establish verification evidence.

Complete model · `Settings`.

SVG · PNG · Vector scene

Notation: case

Use case

Element	Graphical notation	Textual notation
Use case (node)		`useCase Change of Cabin`

A use case describes intended use and routed exchanges.

Context: Scenario header in Settings; steps reference Cabin’s root-level connectors. **Views:** cv, sv.

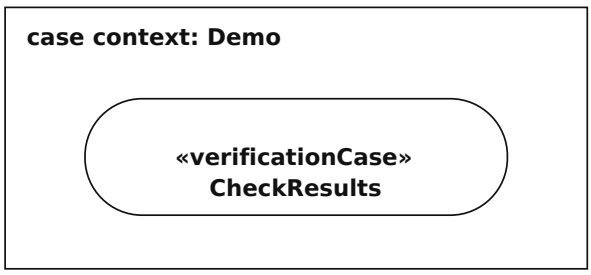
- Draw a case frame labeled useCase with its name and selected root.
- The of relation references a root; enclosure never changes package ownership.

Complete model · `Settings::Change`.

SVG · PNG · Vector scene

Notation: verification-case

Verification case

Element	Graphical notation	Textual notation
Verification case (node)		`verificationCase CheckResults of Demo`

Draw a case frame labeled verificationCase with its name and root context. Documentation describes the procedure; verify links identify requirements without asserting a passed result.

Context: Owned source element in the Assurance example. **Views:** gv, grv, cv.

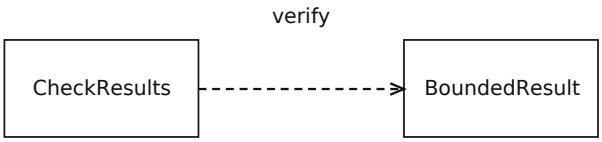
- Draw a case frame labeled verificationCase with its name and root context.
- Documentation describes the procedure; verify links identify requirements without asserting a passed result.

Complete model · `Assurance::CheckResults`.

SVG · PNG · Vector scene

Notation: verification-link

Verification relation

Element	Graphical notation	Textual notation
Verification relation (edge)	**«verify»** 	`verify BoundedResult;`

Draw a dashed open arrow from the verification case to the requirement, labeled verify. A relation expresses verification intent; evidence and outcome are recorded separately.

Context: Owned source element in the Assurance example. **Views:** gv, grv, cv.

- Draw a dashed open arrow from the verification case to the requirement, labeled verify.
- A relation expresses verification intent; evidence and outcome are recorded separately.

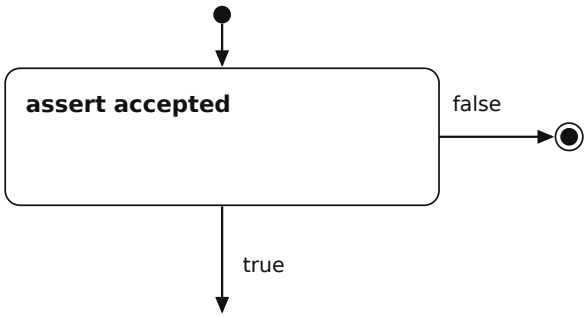
Complete model · `Assurance::CheckResults`.

SVG · PNG · Vector scene

Function flow

Notation: assert

Assertion

Element	Graphical notation	Textual notation
Assertion (node)		`assert accepted;`

Assertion. Draw an owned statement box with the exact statement label and directed control-flow succession.

Context: Statement inside the displayed owning function; the fv node is a projection. **Views:** fv.

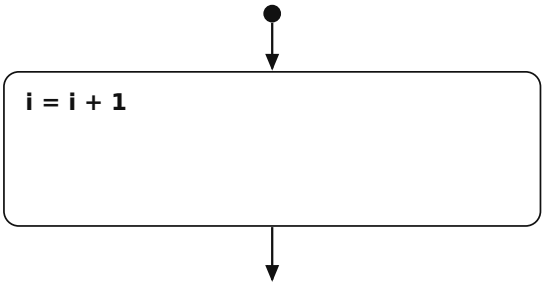
- Draw an owned statement box with the exact statement label and directed control-flow succession.
- False fails the activation; this step is an execution check, not proof of a requirement.

Complete model · `Settings::Panel.adjust`.

SVG · PNG · Vector scene

Notation: assignment

Assignment

Element	Graphical notation	Textual notation
Assignment (node)		`i = i + 1;`

Assignment. Draw an owned statement box with the exact statement label and directed control-flow succession.

Context: Statement inside the displayed owning function; the fv node is a projection. **Views:** fv.

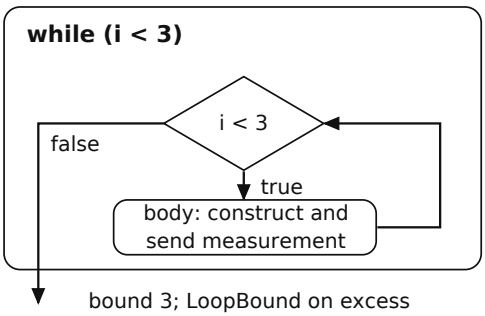
- Draw an owned statement box with the exact statement label and directed control-flow succession.
- Replaces an existing permitted whole value; a field path or another component is not an assignment target.

Complete model · `Values::Producer.prepare`.

SVG · PNG · Vector scene

Notation: bounded-loop

Bounded loop

Element	Graphical notation	Textual notation
Bounded loop (node)		`while (i < 3) bound 3`

Bounded loop. Draw a loop frame with explicit while condition and positive bound label, and a return control edge for iteration.

Context: Loop header in Values::Producer.prepare; the body is rendered as the loop compartment. **Views:** fv.

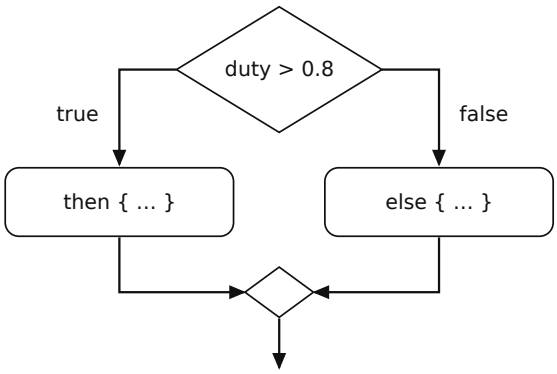
- Draw a loop frame with explicit while condition and positive bound label, and a return control edge for iteration.
- The bound is semantic, not decoration; a still-true condition after the limit leads to failure rather than silent completion.

Complete model · `Values::Producer.prepare`.

SVG · PNG · Vector scene

Notation: conditional

Conditional branch

Element	Graphical notation	Textual notation
Conditional branch (node)		`if (duty > 0.8) { send output.duty(0.8); } else { send output.duty(duty); }`

Conditional branch. Draw a decision diamond with the bool condition and distinct true/false control exits, followed by a merge when control continues.

Context: If/else statement in Control::Regulator.regulate. **Views:** fv.

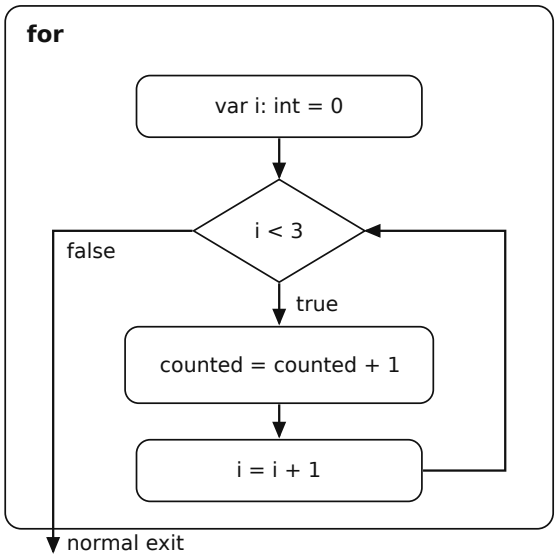
- Draw a decision diamond with the bool condition and distinct true/false control exits, followed by a merge when control continues.
- Exactly one branch executes; the diamond is an if-statement projection, not an independent behavior definition.

Complete model · `Control::Regulator.regulate`.

SVG · PNG · Vector scene

Notation: for-loop

Counted for loop

Element	Graphical notation	Textual notation
Counted for loop (node)	 bound 3; LoopBound before an excess body	`for (var i: int = 0; i < 3; i = i + 1) bound 3`

Counted for loop. A single initializer precedes a Boolean pretest; each completed body is followed by its assignment update and another test.

Context: Header of the loop in ControlFlowNotation::Counter.countedLoop. The initializer, test, body and update are ordered inside one owned loop. **Views:** fv.

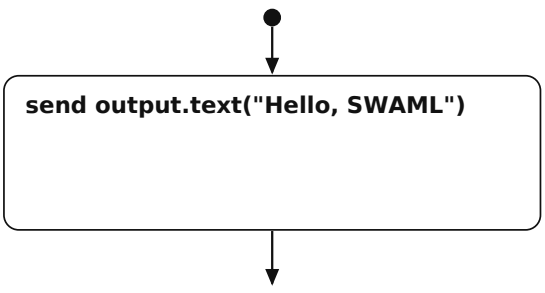
- Draw an owned loop frame with an initializer before the condition, a true edge to the body, an update after the body and an explicit edge back to the test. Label the false normal exit and the positive bound.
- Show a LoopBound failure on an excess requested body. The initializer runs once even if the body runs zero times; the header variable remains inside the loop scope.

Complete model · `ControlFlowNotation::Counter.countedLoop`.

SVG · PNG · Vector scene

Notation: send

Data send

Element	Graphical notation	Textual notation
Data send (node)		`send output.text("Hello, SWAML");`

Data send. Draw an owned statement box with the exact statement label and directed control-flow succession.

Context: Statement inside the displayed owning function; the fv node is a projection. **Views:** fv.

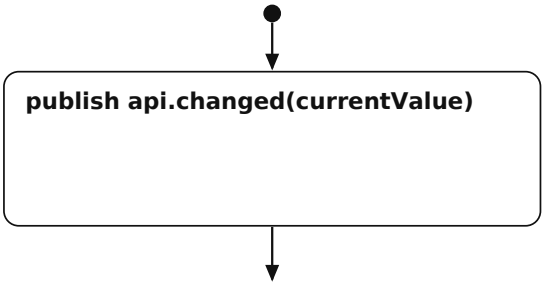
- Draw an owned statement box with the exact statement label and directed control-flow succession.
- The text identifies the emitting function’s owned sender port and data member.

Complete model · `Hello::Speaker.greet`.

SVG · PNG · Vector scene

Notation: publish

Event publication

Element	Graphical notation	Textual notation
Event publication (node)		`publish api.changed(currentValue);`

Event publication. Draw an owned statement box with the exact statement label and directed control-flow succession.

Context: Statement inside the displayed owning function; the fv node is a projection. **Views:** fv.

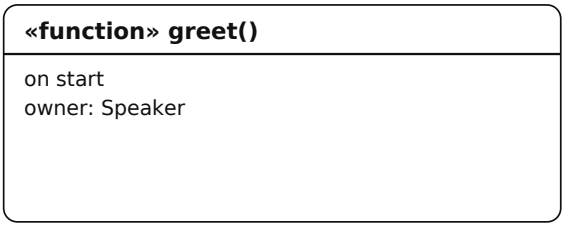
- Draw an owned statement box with the exact statement label and directed control-flow succession.
- The text identifies the emitting function’s owned publisher/provider port and event member.

Complete model · `Settings::SettingStore.update`.

SVG · PNG · Vector scene

Notation: function

Executable function

Element	Graphical notation	Textual notation
Executable function (node)		`function greet() on start`

Executable function. Draw an function frame or compartment with its name, ordered parameters, result and activation label.

Context: Header of Hello::Speaker.greet; its statement body is projected inside the function frame. **Views:** fv.

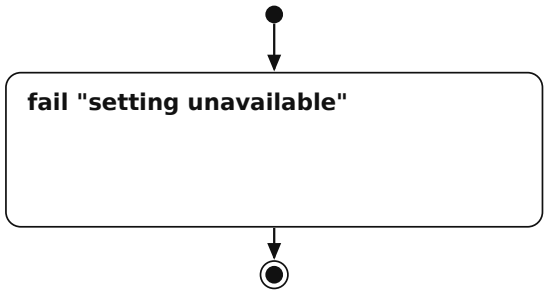
- Draw an function frame or compartment with its name, ordered parameters, result and activation label.
- Every fv step is an owned statement projection within this frame; it MUST NOT create an independently allocated action or performer.

Complete model · `Hello::Speaker.greet`.

SVG · PNG · Vector scene

Notation: fail

Failure

Element	Graphical notation	Textual notation
Failure (node)		`fail "setting unavailable";`

Failure. Draw an owned statement box with the exact statement label and directed control-flow succession.

Context: Statement inside the displayed owning function; the fv node is a projection. **Views:** fv.

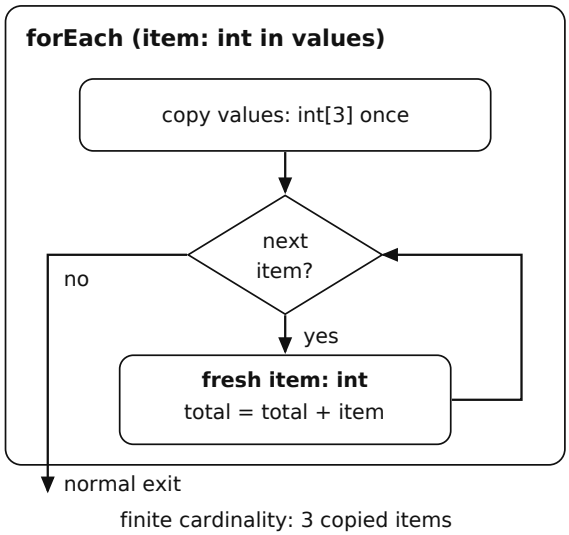
- Draw an owned statement box with the exact statement label and directed control-flow succession.
- Failure unwinds the activation without rollback; it does not imply an unmodeled broadcast.

Complete model · `Settings::Panel.adjust`.

SVG · PNG · Vector scene

Notation: for-each-loop

Fixed-array forEach loop

Element	Graphical notation	Textual notation
Fixed-array forEach loop (node)		`forEach (item: int in values)`

Fixed-array forEach loop. Evaluate and copy the array once, then execute the body for each fresh typed item in array order.

Context: Array loop in ControlFlowNotation::Counter.arrayLoop; values is a fixed int[3] array. **Views:** fv.

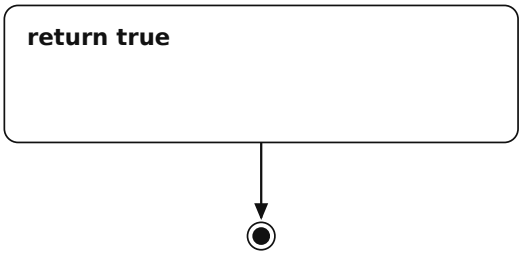
- Draw an array-snapshot step before the loop, an ordered next-item test, a fresh typed item binding and a body edge returning for the next item. Label completion when no item remains.
- Label the copied collection and finite cardinality. The graphic must not imply a live subscription, an extra explicit bound, a shared item alias or dynamic collection growth.

Complete model · `ControlFlowNotation::Counter.arrayLoop`.

SVG · PNG · Vector scene

Notation: return

Function return

Element	Graphical notation	Textual notation
Function return (node)		`return true;`

Function return. Show a labeled return terminal in the owning function flow; retain the returned expression when present.

Context: Return statement in Rpc::Server.checkHealth. **Views:** fv.

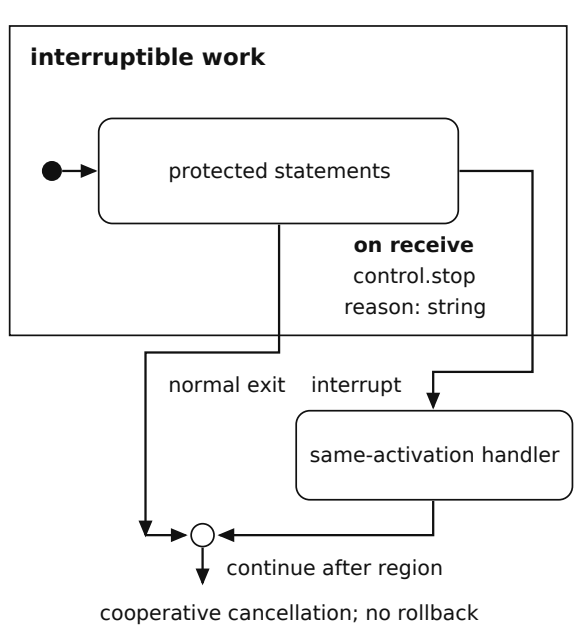
- Show a labeled return terminal in the owning function flow; retain the returned expression when present.
- Return from a helper resumes its local caller; return from a state body terminates the owning stateful function.

Complete model · `Rpc::Server.checkHealth`.

SVG · PNG · Vector scene

Notation: interruptible-region

Interruptible flow region

Element	Graphical notation	Textual notation
Interruptible flow region (node)		`interruptible work`

Interruptible flow region. A named protected region has a normal exit and one routed typed input that cooperatively transfers control to its handler.

Context: Named work region in Interrupts::Worker.run, with a routed Control.stop(reason: string) input and a handler in the same activation. **Views:** fv.

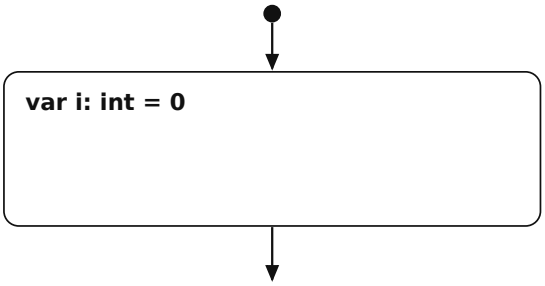
- Draw a named frame around protected statements, a normal exit through the frame boundary and a separate explicitly labeled on receive port.member interruption edge to its handler. Merge continuation after the region unless the handler itself returns or fails.
- Show the incoming typed member and its payload/parameters. An interruption cooperatively abandons protected work and a suspended local RPC correlation, then executes the handler under the existing activation deadline.
- The region belongs only to a triggered serial flow function. It is neither a second handler activation nor hardware preemption, pause/resume, rollback or cancellation of remote effects.

Complete model · `Interrupts::Worker.run.work`.

SVG · PNG · Vector scene

Notation: local-declaration

Local declaration

Element	Graphical notation	Textual notation
Local declaration (node)		`var i: int = 0;`

Local declaration. Draw an owned statement box with the exact statement label and directed control-flow succession.

Context: Statement inside the displayed owning function; the fv node is a projection. **Views:** fv.

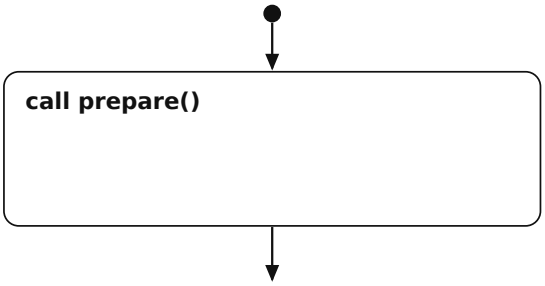
- Draw an owned statement box with the exact statement label and directed control-flow succession.
- Creates a copied typed local value in the enclosing lexical block.

Complete model · `Values::Producer.prepare`.

SVG · PNG · Vector scene

Notation: local-call

Local function call

Element	Graphical notation	Textual notation
Local function call (node)		`call prepare();`

Local function invocation. Draw an owned statement box with the exact statement label and directed control-flow succession.

Context: Statement inside the displayed owning function; the fv node is a projection. **Views:** fv.

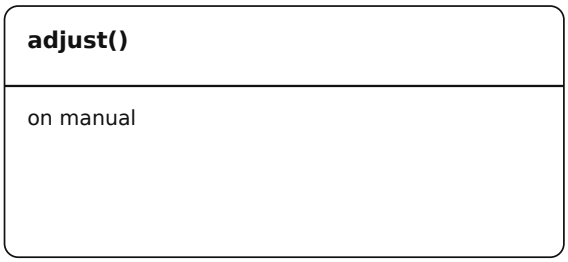
- Draw an owned statement box with the exact statement label and directed control-flow succession.
- Calls an untriggered flow function in the same component, borrowing its activation lock/snapshot. Bind a non-void result with as name; omit the binding for void.

Complete model · `Values::Producer.boot`.

SVG · PNG · Vector scene

Notation: trigger-manual

Manual activation

Element	Graphical notation	Textual notation
Manual activation (label)		`function adjust() on manual`

Manual activation. Attach the activation text to its function header; use no free event symbol or standalone trigger object.

Context: Activation annotation on the displayed function. **Views:** fv.

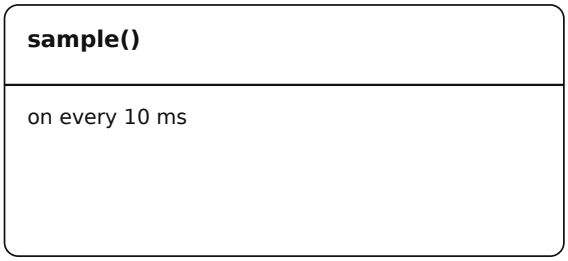
- Attach the activation text to its function header; use no free event symbol or standalone trigger object.
- Harness activation carries no modeled data; external commands still use ports.

Complete model · `Settings::Panel.adjust`.

SVG · PNG · Vector scene

Notation: trigger-periodic

Periodic activation

Element	Graphical notation	Textual notation
Periodic activation (label)		`on every 10 ms`

Periodic activation. Attach the activation text to its function header; use no free event symbol or standalone trigger object.

Context: Activation annotation on the displayed function. **Views:** fv.

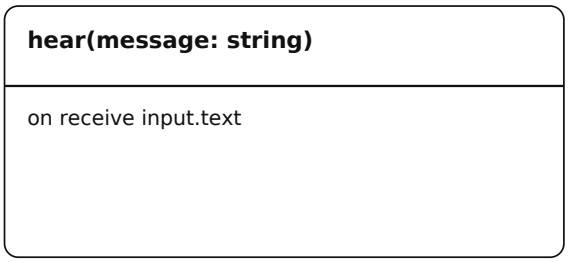
- Attach the activation text to its function header; use no free event symbol or standalone trigger object.
- Release occurs at positive multiples of the declared period.

Complete model · `Control::Sensor.sample`.

SVG · PNG · Vector scene

Notation: trigger-receive

Port receive activation

Element	Graphical notation	Textual notation
Port receive activation (label)		`on receive input.text`

Port receive activation. Attach the activation text to its function header; use no free event symbol or standalone trigger object.

Context: Activation annotation on the displayed function. **Views:** fv.

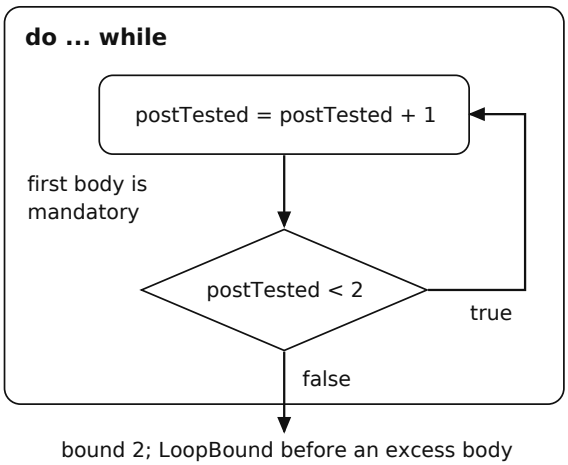
- Attach the activation text to its function header; use no free event symbol or standalone trigger object.
- The port/member label identifies the one incoming handler owner.

Complete model · `Hello::Listener.hear`.

SVG · PNG · Vector scene

Notation: do-while-loop

Post-test do-while loop

Element	Graphical notation	Textual notation
Post-test do-while loop (node)		`do { postTested = postTested + 1;` `} while (postTested < 2) bound 2;`

Post-test do-while loop. Execute the body at least once, then test the condition before requesting another bounded iteration.

Context: Complete post-test statement in ControlFlowNotation::Counter.postTestLoop. **Views:** fv.

- Draw the body before the Boolean test. A true edge returns to the body and the false edge leaves normally. Show the positive bound and its excess-body failure explicitly.

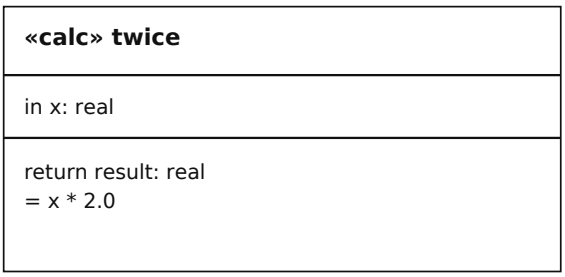
- The first body is mandatory. Condition lookup uses the enclosing scope after the body; body-local declarations are not visible to the test.

Complete model · `ControlFlowNotation::Counter.postTestLoop`.

SVG · PNG · Vector scene

Notation: calculation

Pure calculation

Element	Graphical notation	Textual notation
Pure calculation (compartment)		`calc twice { in x: real; return result: real = x * 2.0; }`

A calculation maps named inputs to one named, typed output expression without mutable state or communication.

Context: Calculation declared at the start of Values::Producer.prepare. **Views:** fv.

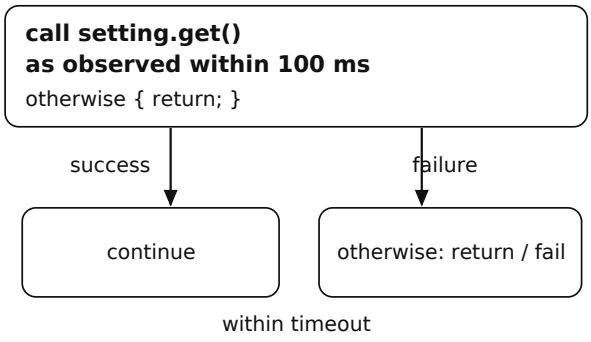
- Draw a flat calculation compartment inside its owning function, headed calc and its name.
- List each input with in and its type; show return, the output name/type and expression in a separate row.
- A calculation call is local expression text such as twice(samples[i]); it is not a message or independent executable function.

Complete model · `Values::Producer.prepare.twice`.

SVG · PNG · Vector scene

Notation: routed-call

Routed function call

Element	Graphical notation	Textual notation
Routed function call (node)		`call setting.get() as observed within 100 ms otherwise { return; };`

RPC request and failure continuation. Draw one request step labeled port.member, result binding and timeout, with distinct success and otherwise continuations.

Context: Statement in Settings::Panel.refresh; successful completion binds observed. **Views:** fv.

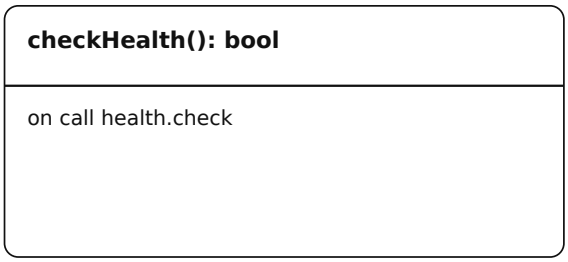
- Draw one request step labeled port.member, result binding and timeout, with distinct success and otherwise continuations.
- The failure continuation remains explicit and terminating; request arrows outside this function must use the corresponding interface route.

Complete model · `Settings::Panel.refresh`.

SVG · PNG · Vector scene

Notation: trigger-call

RPC handler activation

Element	Graphical notation	Textual notation
RPC handler activation (label)		`on call health.check`

RPC handler activation. Attach the activation text to its function header; use no free event symbol or standalone trigger object.

Context: Activation annotation on the displayed function. **Views:** fv.

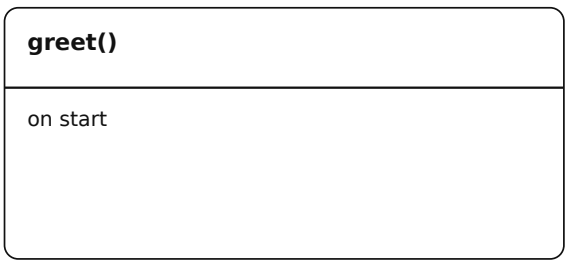
- Attach the activation text to its function header; use no free event symbol or standalone trigger object.
- Requests and correlated results remain on the originating interface route.

Complete model · `Rpc::Server.checkHealth`.

SVG · PNG · Vector scene

Notation: trigger-start

Startup activation

Element	Graphical notation	Textual notation
Startup activation (label)		`function greet() on start`

Startup activation. Attach the activation text to its function header; use no free event symbol or standalone trigger object.

Context: Activation annotation on the displayed function. **Views:** fv.

- Attach the activation text to its function header; use no free event symbol or standalone trigger object.

- Once after component initialization and route readiness.

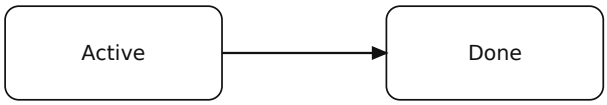
Complete model · `Hello::Speaker.greet`.

SVG · PNG · Vector scene

Function states

Notation: completion-transition

Completion transition

Element	Graphical notation	Textual notation
Completion transition (edge)	**finish** completion [count > 0] on completion when (count > 0) 	`transition finish: Active -> Done on completion when (count > 0) {}`

Completion transition. Label the state edge on completion explicitly, with its guard if present.

Context: Completion transition in Notation::Controller.lifecycle. **Views:** fv.

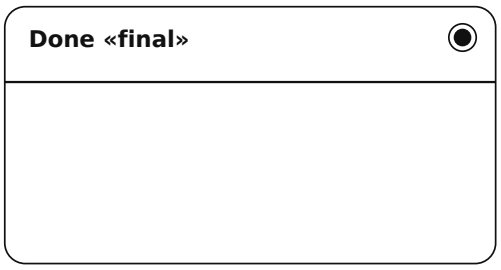
- Label the state edge on completion explicitly, with its guard if present.
- Completion is local to the owning function and MUST NOT be drawn as an emitted global event.

Complete model · `Notation::Controller.lifecycle.finish`.

SVG · PNG · Vector scene

Notation: final-state

Final state

Element	Graphical notation	Textual notation
Final state (node)		`state Done final {}`

Final state. Draw a named final-state symbol with a bullseye marker; preserve the declared name and any entry behavior.

Context: Named final state inside Notation::Controller.lifecycle. **Views:** fv.

- Draw a named final-state symbol with a bullseye marker; preserve the declared name and any entry behavior.

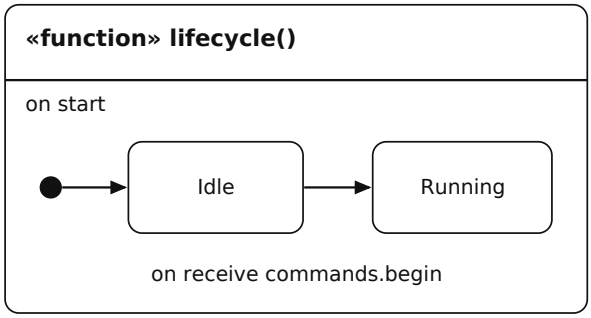
- No outgoing transition is permitted; finality occurs after entry finishes, so the marker MUST NOT hide an entry block.

Complete model · `Notation::Controller.lifecycle.Done`.

SVG · PNG · Vector scene

Notation: stateful-function

Function with states

Element	Graphical notation	Textual notation
Function with states (node)		`function lifecycle() on start`

Function with states. Draw the state graph inside the owning function frame, keeping the function name and on start label visible.

Context: Function header in Lifecycle::Device; its direct body begins with initial. **Views:** fv.

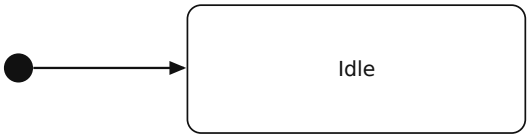
- Draw the state graph inside the owning function frame, keeping the function name and on start label visible.
- There is no machine box, performer, or separate authored state allocation; StateBody contributes only the function’s contents.

Complete model · `Lifecycle::Device.lifecycle`.

SVG · PNG · Vector scene

Notation: initial-state

Initial state reference

Element	Graphical notation	Textual notation
Initial state reference (node)		`initial Idle;`

Initial state reference. Draw a solid initial pseudonode with one outgoing unlabeled initialization edge to the named initial state.

Context: First clause of Lifecycle::Device.lifecycle. **Views:** fv.

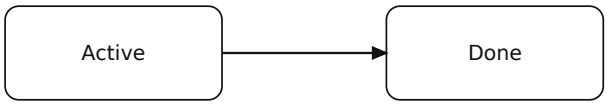
- Draw a solid initial pseudonode with one outgoing unlabeled initialization edge to the named initial state.
- This projects the function’s unique initial reference; it is not a transmitted event or a second start function.

Complete model · `Lifecycle::Device.lifecycle`.

SVG · PNG · Vector scene

Notation: timer-transition

Residence timer transition

Element	Graphical notation	Textual notation
Residence timer transition (edge)	**elapsed** after 1 s on after 1 s 	`transition elapsed: Active -> Done on after 1 s {}`

Residence timer transition. Label the transition on after with an explicit duration and component.

Context: Timer transition in Notation::Controller.lifecycle. **Views:** fv.

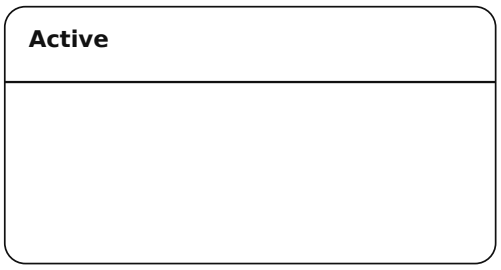
- Label the transition on after with an explicit duration and component.
- The duration starts after state entry completes; leaving the state cancels that entry generation’s timers.

Complete model · `Notation::Controller.lifecycle.elapsed`.

SVG · PNG · Vector scene

Notation: state

State

Element	Graphical notation	Textual notation
State (node)		`state Active {}`

State. Draw a rounded state rectangle bearing the function-local state name, with optional entry/exit compartments.

Context: State declaration in Notation::Controller.lifecycle. **Views:** fv.

- Draw a rounded state rectangle bearing the function-local state name, with optional entry/exit compartments.

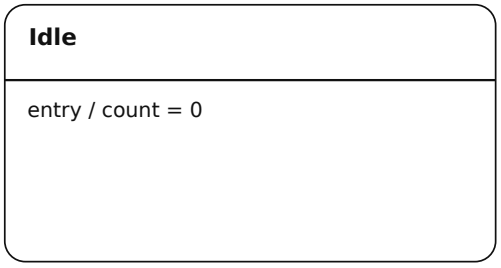
- State rectangles live inside one owning function; position does not allocate them to components or performers.

Complete model · `Notation::Controller.lifecycle.Active`.

SVG · PNG · Vector scene

Notation: state-entry

State entry behavior

Element	Graphical notation	Textual notation
State entry behavior (compartment)		`entry { count = 0; }`

State entry behavior. Show entry and its statements in a compartment inside the owning state.

Context: Entry block on Notation::Controller.lifecycle.Idle. **Views:** fv.

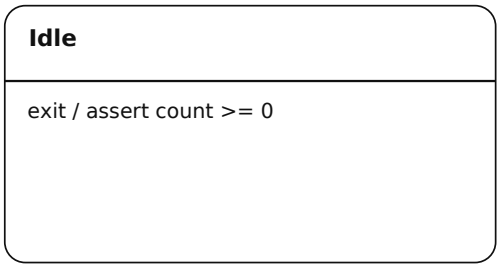
- Show entry and its statements in a compartment inside the owning state.
- Entry has no incoming payload bindings; its completion starts residence timers.

Complete model · `Notation::Controller.lifecycle.Idle`.

SVG · PNG · Vector scene

Notation: state-exit

State exit behavior

Element	Graphical notation	Textual notation
State exit behavior (compartment)		`exit { assert count >= 0; }`

State exit behavior. Show exit and its statements in a compartment inside the owning state.

Context: Exit block on Notation::Controller.lifecycle.Idle. **Views:** fv.

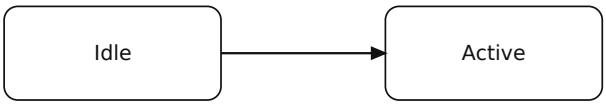
- Show exit and its statements in a compartment inside the owning state.
- Exit precedes transition effect and state update, and does not inherit the incoming payload environment.

Complete model · `Notation::Controller.lifecycle.Idle`.

SVG · PNG · Vector scene

Notation: transition

State transition

Element	Graphical notation	Textual notation
State transition (edge)	**begin** begin: signal.pulse [value > 0] on receive signal.pulse when (value > 0); priority 1 effect: count = value 	`transition begin: Idle -> Active on receive signal.pulse when (value > 0) priority 1 { count = value; }`

State transition. Draw a directed edge between states in the same function; label its identity, stimulus, optional guard/priority and effect.

Context: Transition in Notation::Controller.lifecycle. **Views:** fv.

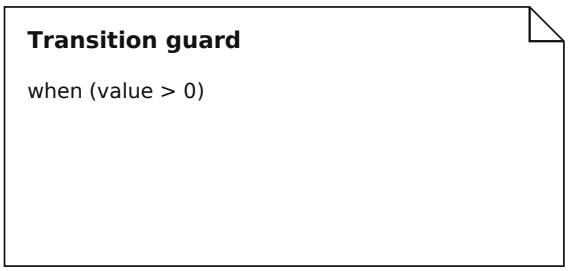
- Draw a directed edge between states in the same function; label its identity, stimulus, optional guard/priority and effect.
- Receive labels MUST include port.member; the transition edge is local control, not the interface connector carrying the message.

Complete model · `Notation::Controller.lifecycle.begin`.

SVG · PNG · Vector scene

Notation: transition-guard

Transition guard

Element	Graphical notation	Textual notation
Transition guard (label)		`when (value > 0)`

Transition guard. Show the bool guard in brackets or with its explicit when keyword on the transition label.

Context: Guard on Notation::Controller.lifecycle.begin. **Views:** fv.

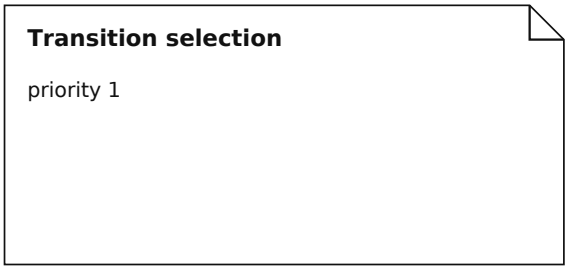
- Show the bool guard in brackets or with its explicit when keyword on the transition label.
- A false guard disables that transition for the current stimulus; guard text is not a separate executable action.

Complete model · `Notation::Controller.lifecycle.begin`.

SVG · PNG · Vector scene

Notation: transition-priority

Transition priority

Element	Graphical notation	Textual notation
Transition priority (label)		`priority 1`

Transition priority. Display priority as an explicit nonnegative integer on a transition label; omitted means zero.

Context: Priority on Notation::Controller.lifecycle.begin. **Views:** fv.

- Display priority as an explicit nonnegative integer on a transition label; omitted means zero.
- Larger wins only among enabled transitions for the same state and stimulus; it is not an function scheduler priority.

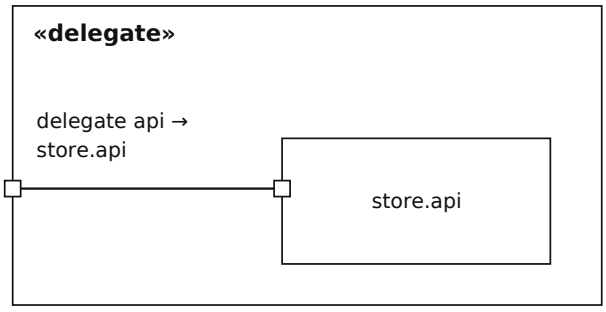
Complete model · `Notation::Controller.lifecycle.begin`.

SVG · PNG · Vector scene

Interfaces and connections

Notation: delegation

Boundary delegation

Element	Graphical notation	Textual notation
Boundary delegation (edge)		`delegate api -> store.api;`

Boundary delegation. Draw a solid unarrowed line labeled delegate from a boundary port to one immediate child port inside its owner.

Context: Boundary-to-immediate-child alias inside Settings::Service. **Views:** iv.

- Draw a solid unarrowed line labeled delegate from a boundary port to one immediate child port inside its owner.
- Both endpoints keep identical interface identity and role; this alias creates no queue, service call, adapter or extra message.

Complete model · `Settings::Service`.

SVG · PNG · Vector scene

Notation: client-server-interface

Client-server interface

Element	Graphical notation	Textual notation
Client-server interface (node)	**«clientServer interface»** **Health** **version "1.0.0"** function check(): bool idempotent	`interface clientServer Health version "1.0.0"`

Client-server interface. Draw an interface classifier rectangle with the exact interface kind, nominal name and version in its heading.

Context: Interface declaration header; member rows occupy the contract body. **Views:** iv.

- Draw an interface classifier rectangle with the exact interface kind, nominal name and version in its heading.
- Keep member categories visible; a contract box is a reusable type and MUST NOT be mistaken for a runtime broker or component.

Complete model · `Rpc::Health`.

SVG · PNG · Vector scene

Notation: data-member

Data member

Element	Graphical notation	Textual notation
Data member (compartment)	**«data» temperature: real** sampled alternative: queued	`data temperature: real sampled;`

Data member. Show data name: Type and sampled or queued on one interface-compartment row.

Context: Member of Control::Samples; queued is the alternative retention mode. **Views:** iv.

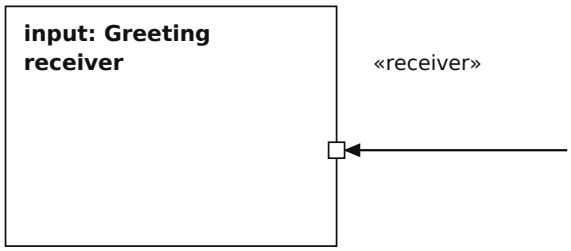
- Show data name: Type and sampled or queued on one interface-compartment row.
- Retention mode is explicit text, not color; sampled uses latest-value buffering while queued retains individual values under QoS.

Complete model · `Control::Samples`.

SVG · PNG · Vector scene

Notation: port-receiver

Data receiver port

Element	Graphical notation	Textual notation
Data receiver port (node)		`port input: Greeting receiver;`

Data receiver port. Draw a small square centered on the owning boundary and label its name: Interface and exact role.

Context: Port declaration on the displayed component boundary. **Views:** iv.

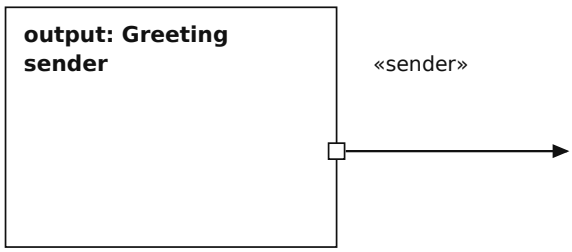
- Draw a small square centered on the owning boundary and label its name: Interface and exact role.
- Data flows inward to the owned receiving function. Do not infer direction only from left/right placement; role text remains mandatory.

Complete model · `Hello::Listener.input`.

SVG · PNG · Vector scene

Notation: port-sender

Data sender port

Element	Graphical notation	Textual notation
Data sender port (node)		`port output: Greeting sender;`

Data sender port. Draw a small square centered on the owning boundary and label its name: Interface and exact role.

Context: Port declaration on the displayed component boundary. **Views:** iv.

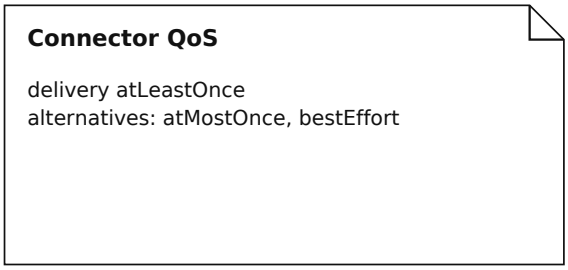
- Draw a small square centered on the owning boundary and label its name: Interface and exact role.
- Data flows outward from the owned port. Do not infer direction only from left/right placement; role text remains mandatory.

Complete model · `Hello::Speaker.output`.

SVG · PNG · Vector scene

Notation: delivery-policy

Delivery policy

Element	Graphical notation	Textual notation
Delivery policy (label)		`delivery atLeastOnce;`

Delivery policy. Attach a QoS property compartment to its connector; write the exact delivery value.

Context: QoS property on Rpc::HealthCheck.healthCheck; the signature is idempotent. **Views:** iv.

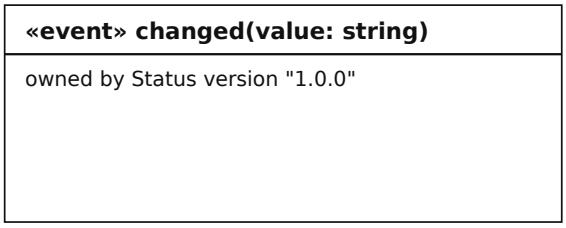
- Attach a QoS property compartment to its connector; write the exact delivery value.
- atLeastOnce, atMostOnce and bestEffort are textual alternatives, not three different edge shapes; atLeastOnce RPC requires idempotent signatures.

Complete model · `Rpc::HealthCheck.healthCheck`.

SVG · PNG · Vector scene

Notation: event-member

Event member

Element	Graphical notation	Textual notation
Event member (compartment)		`event changed(value: string);`

Event member. Show event name(parameters) in its interface compartment; draw no free-floating event definition.

Context: Member of Lifecycle::Status. **Views:** iv.

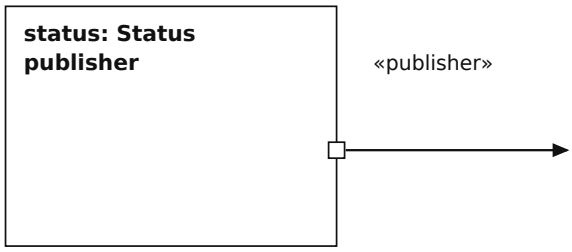
- Show event name(parameters) in its interface compartment; draw no free-floating event definition.
- Every graphical emission or receive label identifies this member through an owned port and function.

Complete model · `Lifecycle::Status`.

SVG · PNG · Vector scene

Notation: port-publisher

Event publisher port

Element	Graphical notation	Textual notation
Event publisher port (node)		`port status: Status publisher;`

Event publisher port. Draw a small square centered on the owning boundary and label its name: Interface and exact role.

Context: Port declaration on the displayed component boundary. **Views:** iv.

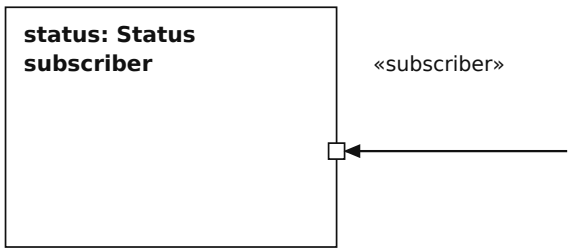
- Draw a small square centered on the owning boundary and label its name: Interface and exact role.
- Events leave this port through explicit subscription connectors. Do not infer direction only from left/right placement; role text remains mandatory.

Complete model · `Lifecycle::Device.status`.

SVG · PNG · Vector scene

Notation: port-subscriber

Event subscriber port

Element	Graphical notation	Textual notation
Event subscriber port (node)		`port status: Status subscriber;`

Event subscriber port. Draw a small square centered on the owning boundary and label its name: Interface and exact role.

Context: Port declaration on the displayed component boundary. **Views:** iv.

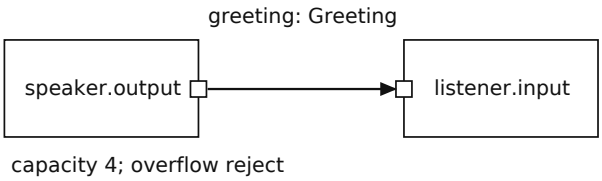
- Draw a small square centered on the owning boundary and label its name: Interface and exact role.
- Events enter the declared handling function through this port. Do not infer direction only from left/right placement; role text remains mandatory.

Complete model · `Lifecycle::Monitor.status`.

SVG · PNG · Vector scene

Notation: connector

Interface connector

Element	Graphical notation	Textual notation
Interface connector (edge)	**greeting: Greeting** 	`connect greeting: speaker.output -> listener.input qos { delivery atMostOnce; ordering fifo; capacity 4; overflow reject; };`

Interface connector. Draw a solid directed connector between port squares, with connector identity and interface name visible.

Context: Connection between immediate child ports of Hello::Conversation. **Views:** iv.

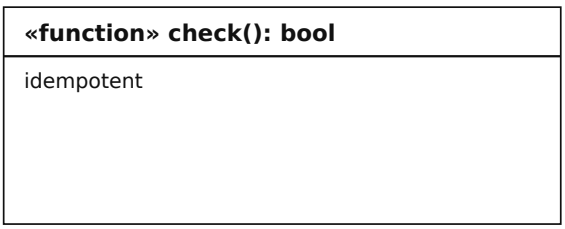
- Draw a solid directed connector between port squares, with connector identity and interface name visible.
- The arrow follows declared source-to-target roles; replies/service events reverse that route rather than creating a second hidden channel.

Complete model · `Hello::Conversation.greeting`.

SVG · PNG · Vector scene

Notation: function-signature

Interface function signature

Element	Graphical notation	Textual notation
Interface function signature (compartment)		`function check(): bool idempotent;`

Interface function signature. Show function name(parameters): Result and idempotent when declared, inside the interface contract.

Context: RPC contract member of Rpc::Health. **Views:** iv.

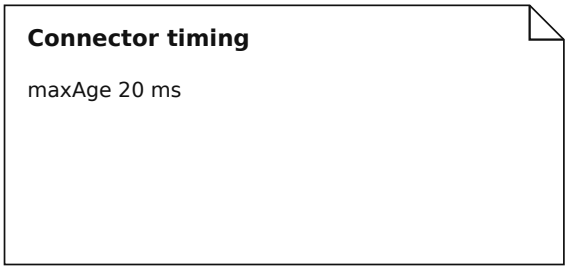
- Show function name(parameters): Result and idempotent when declared, inside the interface contract.
- The signature is distinct from its component implementation; a contract row has no executable body.

Complete model · `Rpc::Health`.

SVG · PNG · Vector scene

Notation: freshness-bound

Message freshness bound

Element	Graphical notation	Textual notation
Message freshness bound (label)		`maxAge 20 ms;`

Message freshness bound. Write maxAge and its explicit component in the connector QoS compartment.

Context: QoS freshness bound on Control::Controller.measured. **Views:** iv.

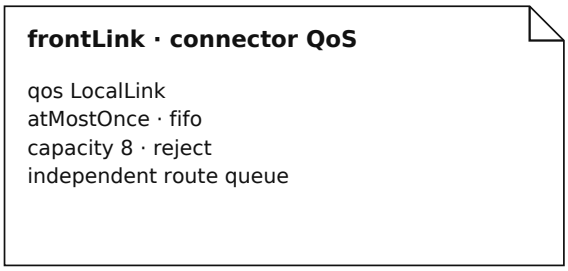
- Write maxAge and its explicit component in the connector QoS compartment.
- Age is measured from original emission, including retries; show expiry as a trace observation only when supported by a run.

Complete model · `Control::Controller.measured`.

SVG · PNG · Vector scene

Notation: qos-policy-reference

Named connector policy

Element	Graphical notation	Textual notation
Named connector policy (label)		`qos LocalLink`

A connector explicitly selects a package-visible QoS declaration while keeping its own route and queue identity.

Context: Explicit policy selection on Settings::Cabin.frontLink. **Views:** iv, grv.

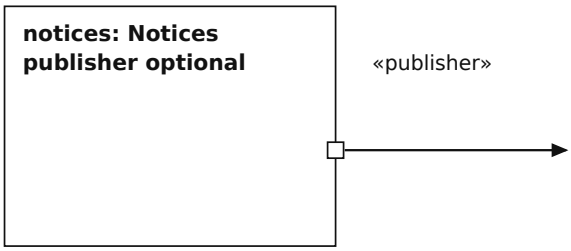
- Attach qos PolicyName to the existing typed connector; the name is a reference, not another communication edge.
- Inspection exposes the qualified policy identity, effective base fields and bounds. Do not merge connector queues or hide invalid policy uses.

Complete model · `Settings::Cabin.frontLink`.

SVG · PNG · Vector scene

Notation: optional-port

Optional port

Element	Graphical notation	Textual notation
Optional port (label)		`port notices: Notices publisher optional;`

Optional port. Use the same square port symbol and append optional to its role label.

Context: Reusable component NotationOptions::Emitter; its outgoing member is tied to an function. **Views:** iv.

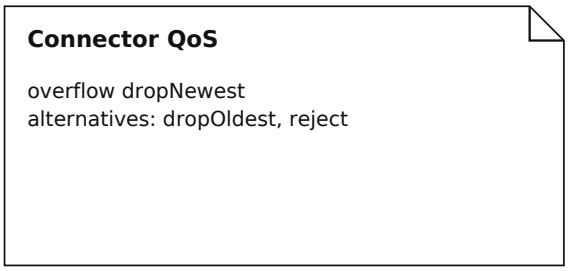
- Use the same square port symbol and append optional to its role label.
- Optionality only permits an unconnected route; it does not remove contract, provenance or receiving-handler obligations.

Complete model · `NotationOptions::Emitter.notices`.

SVG · PNG · Vector scene

Notation: overflow-policy

Overflow behavior

Element	Graphical notation	Textual notation
Overflow behavior (label)		`overflow dropNewest;`

Overflow behavior. Write overflow plus dropNewest, dropOldest or reject in the connector QoS compartment.

Context: QoS property on Notation::Demo.events. **Views:** iv.

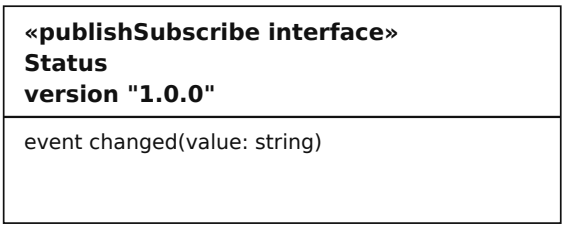
- Write overflow plus dropNewest, dropOldest or reject in the connector QoS compartment.
- The label determines retention/rejection semantics; no style variation may silently substitute another policy.

Complete model · `Notation::Demo.events`.

SVG · PNG · Vector scene

Notation: publish-subscribe-interface

Publish-subscribe interface

Element	Graphical notation	Textual notation
Publish-subscribe interface (node)		`interface publishSubscribe Status version "1.0.0"`

Publish-subscribe interface. Draw an interface classifier rectangle with the exact interface kind, nominal name and version in its heading.

Context: Interface declaration header; member rows occupy the contract body. **Views:** iv.

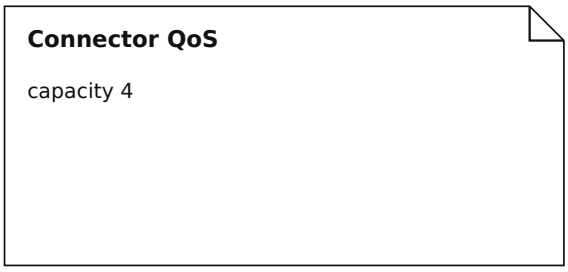
- Draw an interface classifier rectangle with the exact interface kind, nominal name and version in its heading.
- Keep member categories visible; a contract box is a reusable type and MUST NOT be mistaken for a runtime broker or component.

Complete model · `Lifecycle::Status`.

SVG · PNG · Vector scene

Notation: capacity-policy

Queue capacity

Element	Graphical notation	Textual notation
Queue capacity (label)		`capacity 4;`

Queue capacity. Write the positive capacity value in the connector QoS compartment.

Context: QoS property on Hello::Conversation.greeting. **Views:** iv.

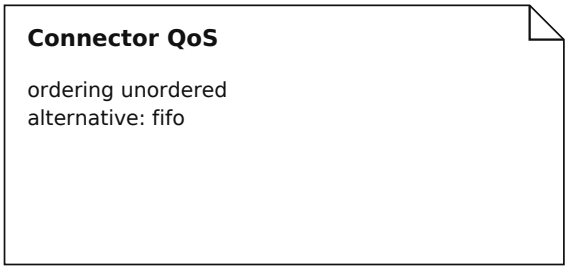
- Write the positive capacity value in the connector QoS compartment.
- Capacity counts pending logical messages for the route/direction/member; do not encode capacity as line thickness.

Complete model · `Hello::Conversation.greeting`.

SVG · PNG · Vector scene

Notation: ordering-policy

Queue ordering

Element	Graphical notation	Textual notation
Queue ordering (label)		`ordering unordered;`

Queue ordering. Write ordering fifo or ordering unordered in the connector QoS compartment.

Context: QoS property on Notation::Demo.events. **Views:** iv.

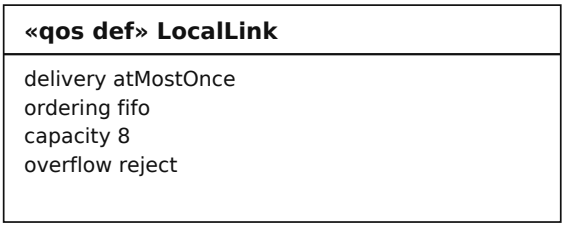
- Write ordering fifo or ordering unordered in the connector QoS compartment.
- Ordering applies per route, direction and member; line position or crossing MUST NOT imply a global order.

Complete model · `Notation::Demo.events`.

SVG · PNG · Vector scene

Notation: qos-policy-definition

Reusable QoS policy

Element	Graphical notation	Textual notation
Reusable QoS policy (node)		`qos def LocalLink { delivery atMostOnce; ordering fifo; capacity 8; overflow reject; }`

Named complete connector policy, shown as a flat declaration card. It has no port, behavior, deployment or runtime queue of its own.

Context: Package-owned policy in Settings; selected independently by both Cabin connector occurrences. **Views:** gv, grv.

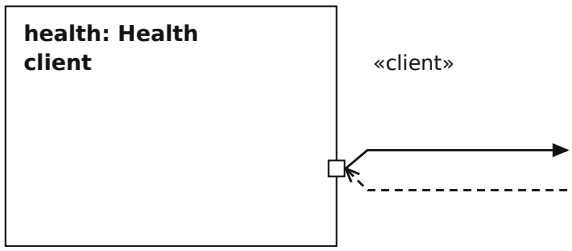
- Draw a flat contract rectangle labeled «qos def» and its name, with all four base fields in a property compartment.
- Expose optional latency/age bounds and declaration identity. Reuse shares values only; queues remain per connector occurrence, direction and member.

Complete model · `Settings::LocalLink`.

SVG · PNG · Vector scene

Notation: port-client

RPC client port

Element	Graphical notation	Textual notation
RPC client port (node)		`port health: Health client;`

RPC client port. Draw a small square centered on the owning boundary and label its name: Interface and exact role.

Context: Port declaration on the displayed component boundary. **Views:** iv.

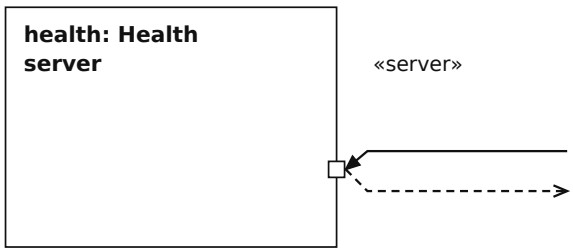
- Draw a small square centered on the owning boundary and label its name: Interface and exact role.
- Requests leave and correlated results return through this same port. Do not infer direction only from left/right placement; role text remains mandatory.

Complete model · `Rpc::Client.health`.

SVG · PNG · Vector scene

Notation: port-server

RPC server port

Element	Graphical notation	Textual notation
RPC server port (node)		`port health: Health server;`

RPC server port. Draw a small square centered on the owning boundary and label its name: Interface and exact role.

Context: Port declaration on the displayed component boundary. **Views:** iv.

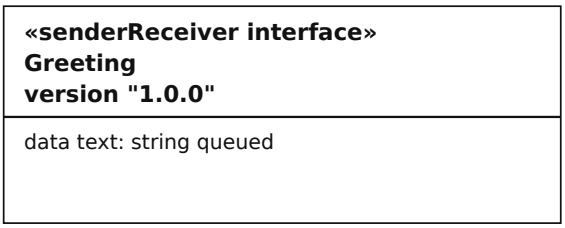
- Draw a small square centered on the owning boundary and label its name: Interface and exact role.
- Requests enter and correlated results leave through this same port. Do not infer direction only from left/right placement; role text remains mandatory.

Complete model · `Rpc::Server.health`.

SVG · PNG · Vector scene

Notation: sender-receiver-interface

Sender-receiver interface

Element	Graphical notation	Textual notation
Sender-receiver interface (node)		`interface senderReceiver Greeting version "1.0.0"`

Sender-receiver interface. Draw an interface classifier rectangle with the exact interface kind, nominal name and version in its heading.

Context: Interface declaration header; member rows occupy the contract body. **Views:** iv.

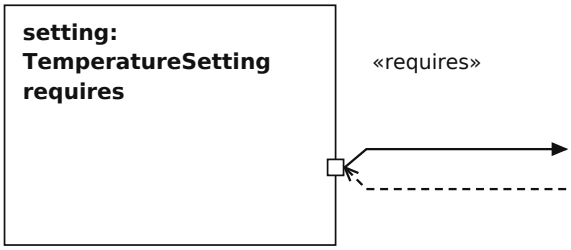
- Draw an interface classifier rectangle with the exact interface kind, nominal name and version in its heading.
- Keep member categories visible; a contract box is a reusable type and MUST NOT be mistaken for a runtime broker or component.

Complete model · `Hello::Greeting`.

SVG · PNG · Vector scene

Notation: port-requires

Service consumer port

Element	Graphical notation	Textual notation
Service consumer port (node)		`port setting: TemperatureSetting requires;`

Service consumer port. Draw a small square centered on the owning boundary and label its name: Interface and exact role.

Context: Port declaration on the displayed component boundary. **Views:** iv.

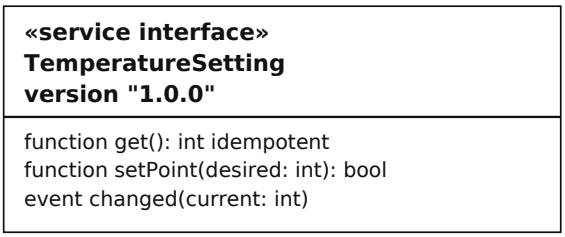
- Draw a small square centered on the owning boundary and label its name: Interface and exact role.
- RPC requests leave; results and provider events enter. Do not infer direction only from left/right placement; role text remains mandatory.

Complete model · `Settings::Panel.setting`.

SVG · PNG · Vector scene

Notation: service-interface

Service interface

Element	Graphical notation	Textual notation
Service interface (node)		`interface service TemperatureSetting version "1.0.0"`

Service interface. Draw an interface classifier rectangle with the exact interface kind, nominal name and version in its heading.

Context: Interface declaration header; member rows occupy the contract body. **Views:** iv.

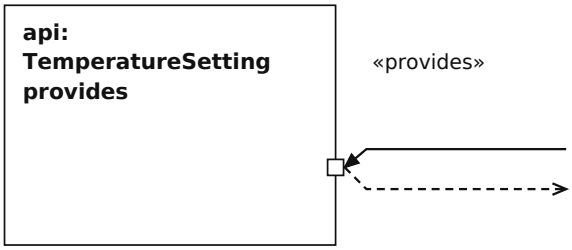
- Draw an interface classifier rectangle with the exact interface kind, nominal name and version in its heading.
- Keep member categories visible; a contract box is a reusable type and MUST NOT be mistaken for a runtime broker or component.

Complete model · `Settings::TemperatureSetting`.

SVG · PNG · Vector scene

Notation: port-provides

Service provider port

Element	Graphical notation	Textual notation
Service provider port (node)		`port api: TemperatureSetting provides;`

Service provider port. Draw a small square centered on the owning boundary and label its name: Interface and exact role.

Context: Port declaration on the displayed component boundary. **Views:** iv.

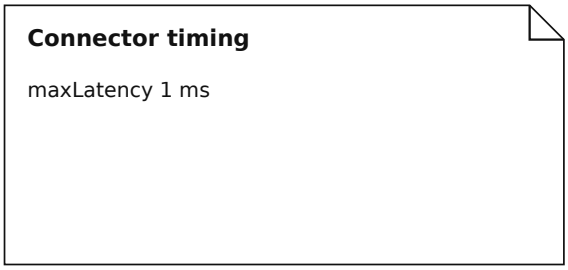
- Draw a small square centered on the owning boundary and label its name: Interface and exact role.
- RPC requests enter; results and provider events leave. Do not infer direction only from left/right placement; role text remains mandatory.

Complete model · `Settings::SettingStore.api`.

SVG · PNG · Vector scene

Notation: latency-bound

Transport latency bound

Element	Graphical notation	Textual notation
Transport latency bound (label)		`maxLatency 1 ms;`

Transport latency bound. Write maxLatency and an explicitly component-bearing duration beside the corresponding connector.

Context: QoS bound on Control::Controller.measured. **Views:** iv.

- Write maxLatency and an explicitly component-bearing duration beside the corresponding connector.
- This bound ends at transport readiness; it MUST NOT be labeled as total execution or dispatch deadline.

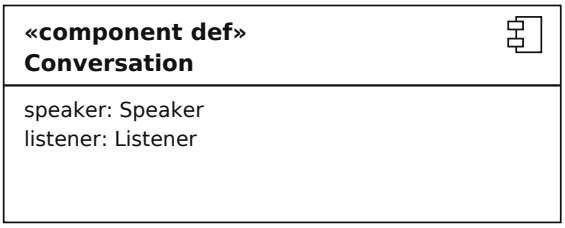
Complete model · `Control::Controller.measured`.

SVG · PNG · Vector scene

Structure and values

Notation: component-definition

Component definition

Element	Graphical notation	Textual notation
Component definition (node)		`component def Conversation`

Component definition. Draw a sharp-corner classifier rectangle with the component double-tab marker and explicit «component def» heading above its type name.

Context: Reusable component type header inside package Hello. **Views:** gv, grv.

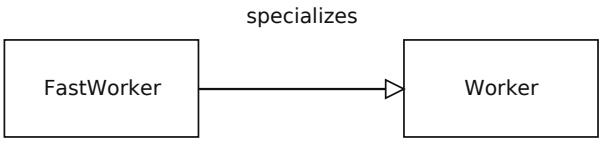
- Draw a sharp-corner classifier rectangle with the component double-tab marker and explicit «component def» heading above its type name.
- Separate the name from ports, child instances, private data and functions; definitions do not allocate singleton state.

Complete model · `Hello::Conversation`.

SVG · PNG · Vector scene

Notation: component-specialization

Component specialization

Element	Graphical notation	Textual notation
Component specialization (edge)	**Component specialization** 	`component def FastWorker specializes Worker`

Single inheritance reuses the base component’s effective members while each instance keeps independent state.

Context: Reusable definition ComponentReuse::FastWorker specializes Worker. **Views:** gv, grv.

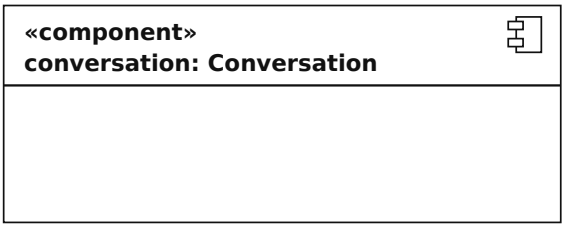
- Draw a solid generalization line from the derived definition to the base, ending in an unfilled triangle at the base and labeled specializes.
- The link connects definitions, not running instances. It does not create an extra component instance or shared private store.
- Inherited members may be replaced only through compatible explicit redefines declarations.

Complete model · `ComponentReuse::FastWorker`.

SVG · PNG · Vector scene

Notation: component-usage

Component usage

Element	Graphical notation	Textual notation
Component usage (node)		`component conversation: Conversation;`

Component usage. Draw a structural rectangle with the component double-tab marker, labeled conversation: Conversation with «component» visible; preserve occurrence and type names.

Context: Contained component occurrence in root Hello::Demo. **Views:** gv, grv.

- Draw a structural rectangle with the component double-tab marker, labeled conversation: Conversation with «component» visible; preserve occurrence and type names.
- Each occurrence has its own path context; two rectangles typed by the same definition MUST NOT merge private storage.

Complete model · `Hello::Demo`.

SVG · PNG · Vector scene

Notation: enumeration

Enumeration

Element	Graphical notation	Textual notation
Enumeration (node)	«enum» **Health** Good Failed	`enum Health { Good, Failed }`

Enumeration. Draw a classifier rectangle headed «enum», with one literal per row in a literals compartment.

Context: Nominal enum declaration inside Values. **Views:** gv, grv.

- Draw a classifier rectangle headed «enum», with one literal per row in a literals compartment.
- The literal order is display order, not an implied integer encoding or state-transition order.

Complete model · `Values::Health`.

SVG · PNG · Vector scene

Notation: enum-value

Enumeration value

Element	Graphical notation	Textual notation
Enumeration value (label)	**Nominal value** Health.Good qualified form: Values::Health.Good	`Health.Good`

Enumeration value. Render the exact nominal value text Type.Literal, or Package::Type.Literal when qualification is needed.

Context: Expression inside Values::Producer.prepare. **Views:** gv, grv.

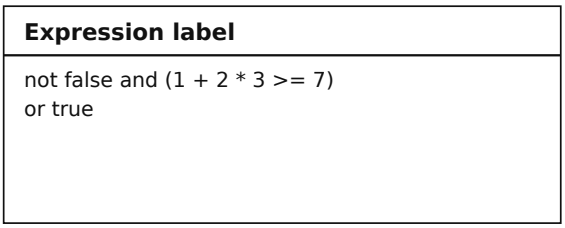
- Render the exact nominal value text Type.Literal, or Package::Type.Literal when qualification is needed.
- An accessible local first segment denotes a value-field read instead; the graphic MUST preserve the resolved identity rather than guessing from the dot.

Complete model · `Values::Producer.prepare`.

SVG · PNG · Vector scene

Notation: expression

Expressions and value paths

Element	Graphical notation	Textual notation
Expressions and value paths (label)		`assert not false and (1 + 2 * 3 >= 7) or true;`

Expressions and value paths. Keep literals, operators, parentheses and value paths as expression text in the owning label or compartment.

Context: Expression inside Notation::Sink.check. **Views:** gv, grv.

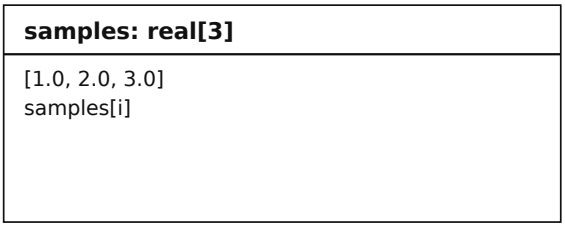
- Keep literals, operators, parentheses and value paths as expression text in the owning label or compartment.
- Expression layout MUST preserve precedence and short-circuit order; it does not add flow nodes for operators unless the fv is explicitly an explanatory expansion.

Complete model · `Notation::Sink.check`.

SVG · PNG · Vector scene

Notation: array-value

Fixed array type and value

Element	Graphical notation	Textual notation
Fixed array type and value (label)		`var samples: real[3] = [1.0, 2.0, 3.0];`

Fixed array type and value. Display fixed extent in the type label T[n] and ordered elements in brackets within the owning expression.

Context: Local declaration in Values::Producer.prepare. **Views:** gv, grv.

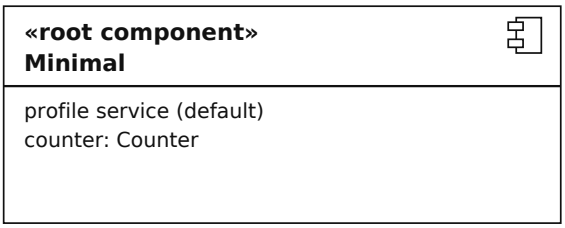
- Display fixed extent in the type label T[n] and ordered elements in brackets within the owning expression.
- Array elements are copied values; no array icon may imply reference-valued payloads or dynamic multiplicity.

Complete model · `Values::Producer.prepare`.

SVG · PNG · Vector scene

Notation: inline-root

Inline root composition

Element	Graphical notation	Textual notation
Inline root composition (node)		`component Minimal { component counter: Counter; }`

An inline top-level component directly owns its architecture and local behavior.

Context: Top-level inline component occurrence in Roots. **Views:** gv, grv.

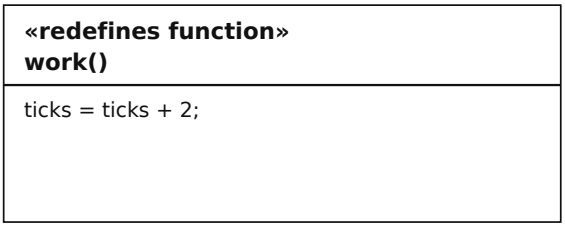
- Draw a root component frame with its owned members: child components, private data, functions and typed routes.
- The root is a closed instance: owned ports participate only in explicit connections or delegation, with no implicit external peer.

Complete model · `Roots::Minimal`.

SVG · PNG · Vector scene

Notation: member-redefinition

Member redefinition

Element	Graphical notation	Textual notation
Member redefinition (label)		`redefines function work() { ticks = ticks + 2; }`

An explicit redefinition replaces an inherited member in the derived component’s effective feature set.

Context: Replacement function owned by ComponentReuse::FastWorker. **Views:** gv, grv, fv.

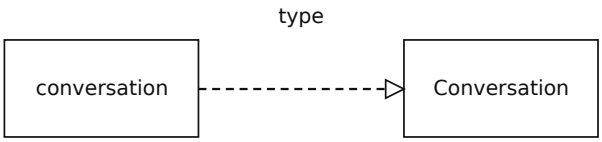
- Prefix the owned member label with redefines and preserve its member kind, name and compatible contract.
- Port, variable, child, function, connector and delegation replacements use their ordinary glyphs with this explicit label.
- Display one effective member for the derived instance; an override does not allocate another behavior or store.

Complete model · `ComponentReuse::FastWorker.work`.

SVG · PNG · Vector scene

Notation: type-relationship

Occurrence typing relationship

Element	Graphical notation	Textual notation
Occurrence typing relationship (edge)	«type» 	`component conversation: Conversation;`

Occurrence typing relationship. A dashed type arrow points from the occurrence to its reusable definition.

Context: Occurrence typing in Hello::Demo; the same type identity is normally shown by the name: Type label. **Views:** gv, grv.

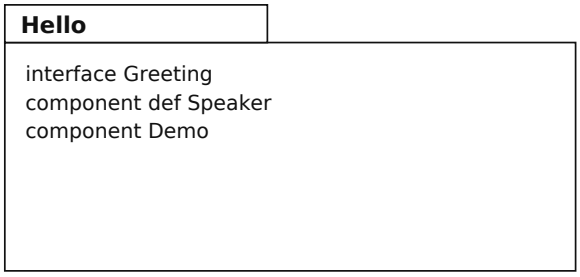
- When typing is shown as a separate relationship, draw a dashed line with a hollow triangular arrowhead pointing from the occurrence to its definition and label it type.
- This is a typing dependency, never inheritance, delegation or a communication route; the name: Type label remains authoritative.

Complete model · `Hello::Demo.conversation`.

SVG · PNG · Vector scene

Notation: package

Package

Element	Graphical notation	Textual notation
Package (node)		`package Hello`

Package. Draw a tabbed rectangular namespace frame; the tab contains the package name.

Context: Package header; declarations occupy its body. **Views:** gv, grv.

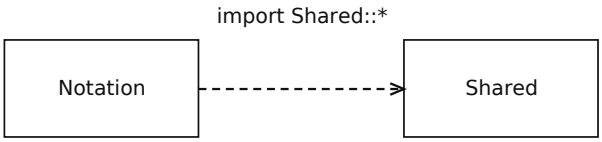
- Draw a tabbed rectangular namespace frame; the tab contains the package name.
- Nested compartments show owned declarations, not runtime instances created by package membership.

Complete model · `Hello`.

SVG · PNG · Vector scene

Notation: import

Package import

Element	Graphical notation	Textual notation
Package import (edge)	**«import»** 	`import Shared::*;`

Package import. Draw a dashed open-arrow dependency from the importing package to the imported package, labeled import Shared::*.

Context: Inside package Notation; the target package is loaded in the same workspace. **Views:** gv, grv.

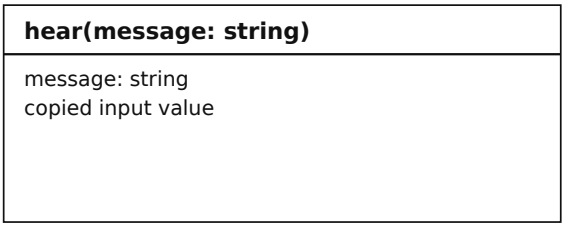
- Draw a dashed open-arrow dependency from the importing package to the imported package, labeled import Shared::*.
- This is namespace visibility only; it MUST NOT resemble or create an interface communication route.

Complete model · `Notation`.

SVG · PNG · Vector scene

Notation: parameter

Parameter or record field

Element	Graphical notation	Textual notation
Parameter or record field (compartment)		`hear(message: string)`

Parameter or record field. Show ordered parameters as name: Type within the function signature; show record fields as name: Type rows in their datatype.

Context: Function signature fragment inside Hello::Listener. **Views:** gv, grv.

- Show ordered parameters as name: Type within the function signature; show record fields as name: Type rows in their datatype.
- Parameter order is semantic; a graphical editor MUST retain it when reordering unrelated compartments.

Complete model · `Hello::Listener.hear`.

SVG · PNG · Vector scene

Notation: primitive-types

Primitive types

Element	Graphical notation	Textual notation
Primitive types (label)	**Type labels** enabled: bool; octet: byte count: uint; delta: int temperature: real; name: string acknowledge(): void	`function acknowledge(): void;`

Basic data types label values and signatures; fixed arrays, nominal records and enums build on them.

Context: void is shown as an interface result; other primitive types label fields/parameters/results. **Views:** gv, grv.

- Display bool, byte, uint, int, real, string or void as textual type labels in the owning compartment.
- byte is 0-255 and uint is a nonnegative safe integer. Integral narrowing checks the destination domain; real does not implicitly narrow.
- void labels function results only; an omitted function result means void.

Complete model · `Notation::Completion`.

SVG · PNG · Vector scene

Notation: private-variable

Private component variable

Element	Graphical notation	Textual notation
Private component variable (compartment)	**«component def» Listener** var last: string = ""	`var last: string = "";`

Private component variable. Show a var name: Type = initializer row inside its component boundary.

Context: Private values compartment of Hello::Listener. **Views:** gv, grv.

- Show a var name: Type = initializer row inside its component boundary.
- Do not draw shared-storage connectors to this row; another component cannot read or write it directly.

Complete model · `Hello::Listener.last`.

SVG · PNG · Vector scene

Notation: record-value

Record construction

Element	Graphical notation	Textual notation
Record construction (label)	**record Measurement** value = twice(samples[i]) health = Health.Good	`record Measurement { value = twice(samples[i]); health = Health.Good; }`

Record construction. Display record construction as expression text attached to its owning statement/field, with the nominal type visible.

Context: Initializer fragment inside Values::Producer.prepare; samples and i are local values. **Views:** gv, grv.

- Display record construction as expression text attached to its owning statement/field, with the nominal type visible.
- All declared fields are present exactly once; a record value MUST NOT be drawn as a mutable component occurrence.

Complete model · `Values::Producer.prepare`.

SVG · PNG · Vector scene

Notation: datatype

Record datatype

Element	Graphical notation	Textual notation
Record datatype (node)	**«datatype»** **Measurement** value: real health: Health	`datatype Measurement { value: real; health: Health; }`

Record datatype. Draw a classifier rectangle headed «datatype» and the nominal type name, followed by a field compartment.

Context: Nominal datatype declaration inside Values. **Views:** gv, grv.

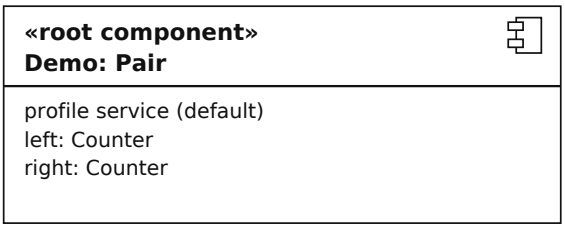
- Draw a classifier rectangle headed «datatype» and the nominal type name, followed by a field compartment.
- Fields retain declared names/types; identical-looking declarations remain distinct nominal types.

Complete model · `Values::Measurement`.

SVG · PNG · Vector scene

Notation: root-component

Root component

Element	Graphical notation	Textual notation
Root component (node)		`component Demo: Pair;`

Root component. Draw a containing component frame labeled «root component» and the selected occurrence name/type.

Context: Top-level named occurrence; its type is Roots::Pair. **Views:** gv, grv.

- Draw a containing component frame labeled «root component» and the selected occurrence name/type.
- Root selection is explicit; the frame is composition context and MUST NOT be drawn as an executable function or another authored system element.

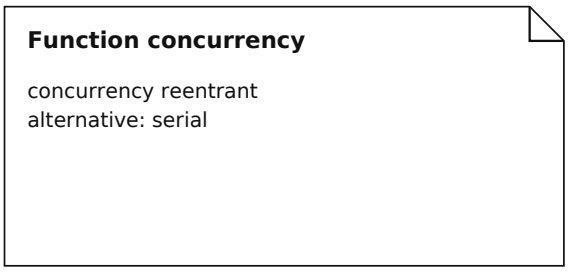
Complete model · `Roots::Demo`.

SVG · PNG · Vector scene

Timing and deployment

Notation: concurrency-mode

Concurrency mode

Element	Graphical notation	Textual notation
Concurrency mode (label)		`concurrency reentrant;`

Concurrency mode. Write serial or reentrant in the function timing compartment; no color or parallel-lane icon substitutes for it.

Context: Timing property on Settings::SettingStore.read. **Views:** gv, grv.

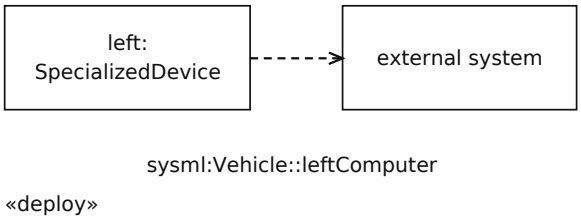
- Write serial or reentrant in the function timing compartment; no color or parallel-lane icon substitutes for it.
- Reentrant activations read dispatch snapshots and cannot mutate component data; helper calls inherit the activation’s lock/snapshot context.

Complete model · `Settings::SettingStore.read`.

SVG · PNG · Vector scene

Notation: deployment

Deployment binding

Element	Graphical notation	Textual notation
Deployment binding (edge)		`deploy left on "sysml:Vehicle::leftComputer";`

A deployment binds a component instance or root to an external system reference.

Context: Deployment of one component instance in ComponentReuse::Demo. **Views:** gv, grv.

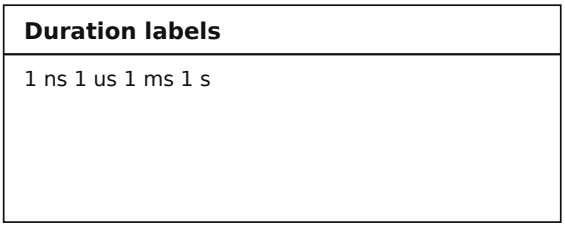
- Draw a dashed open-arrow allocation link labeled deploy from the component instance to a flat external-system reference.
- Show the complete URI-like reference; deployment does not imply that a SysML or infrastructure model was loaded.
- Deployment inherits down the component tree unless a descendant supplies a more specific binding.

Complete model · `ComponentReuse::Demo`.

SVG · PNG · Vector scene

Notation: duration

Duration value

Element	Graphical notation	Textual notation
Duration value (label)		`wcet 1 us;`

Duration value. Render an integer and its explicit time unit together in timing, period, timeout or state-timer labels.

Context: Duration value in Roots::Counter.initialize; ns/us/ms/s are explicit alternatives. **Views:** gv, grv.

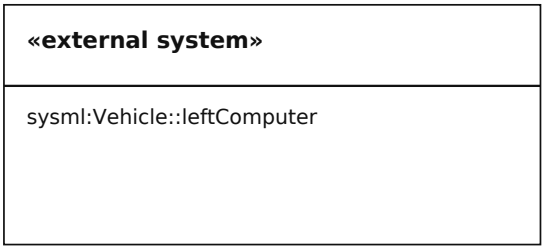
- Render an integer and its explicit time unit together in timing, period, timeout or state-timer labels.
- ns, us, ms and s are label components, not separate symbols; never imply a component from diagram scale.

Complete model · `Roots::Counter.initialize`.

SVG · PNG · Vector scene

Notation: deployment-target

External deployment target

Element	Graphical notation	Textual notation
External deployment target (node)		`deploy left on "sysml:Vehicle::leftComputer";`

An external reference identifies the deployment system without introducing a hardware or execution-node definition into SWAML.

Context: An external system reference in ComponentReuse::Demo. **Views:** gv, grv.

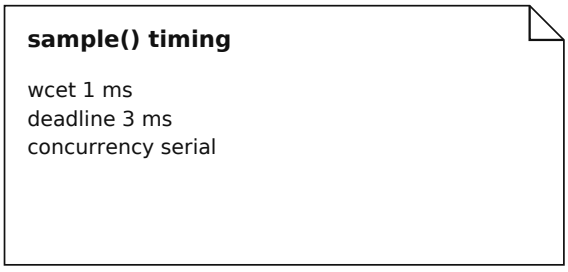
- Draw a flat rectangle headed external system and show the exact reference as text.
- A sysml: reference may identify a separately modeled SysML part; SWAML does not resolve or validate that external model.
- This endpoint is placement metadata and never grants access to another component’s state.

Complete model · `ComponentReuse::Demo`.

SVG · PNG · Vector scene

Notation: timing-contract

Function timing contract

Element	Graphical notation	Textual notation
Function timing contract (label)		`timing { wcet 1 ms; deadline 3 ms; concurrency serial; }`

Function timing contract. Attach a timing compartment to the function, listing wcet, deadline and concurrency explicitly.

Context: Contract on Control::Sensor.sample. **Views:** gv, grv, fv.

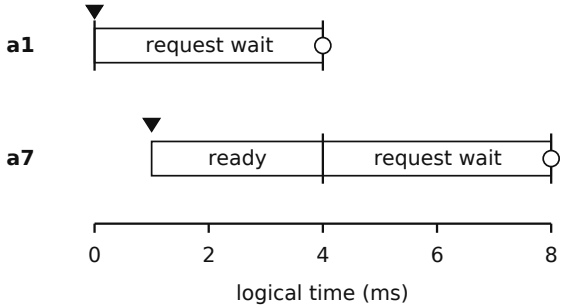
- Attach a timing compartment to the function, listing wcet, deadline and concurrency explicitly.
- WCET counts active execution, deadline counts elapsed release-to-completion; the graphic alone supplies no schedulability proof.

Complete model · `Control::Sensor.sample`.

SVG · PNG · Vector scene

Notation: function-timing-diagram

Function timing diagram

Element	Graphical notation	Textual notation
Function timing diagram (projection)	**fv: first.poll** declared: wcet 1 ms; deadline 30 ms; serial observed: reference logical time, zero CPU cost  ▼ release; | zero-time running; ○ complete display 0–8 ms; quiescent boundary 31 ms declared deadlines 30 / 31 ms lie outside this window	`function poll() on manual timing { wcet 1 ms; deadline 30 ms; concurrency serial; }`

Function timing diagram. An fv projection compares observed activation releases, waits and terminal markers with separately labeled declared timing contracts.

Context: FunctionTiming::Client.poll declares the contract; the success trace of example 17 supplies distinct first.poll activations a1 and a7. **Views:** fv.

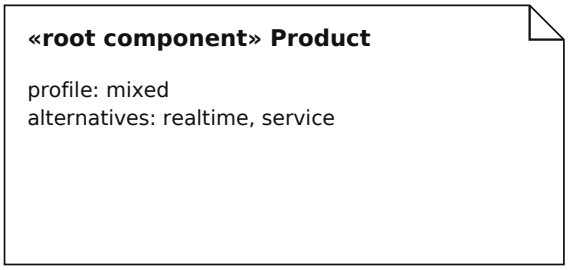
- Project existing function occurrences and activation IDs against a labeled time axis. Label ready, running or zero-time markers, request waits, completion/failure and the observation boundary without relying on color. Preserve trace ordinal when timestamps coincide.
- Keep declared WCET/deadline/concurrency in a separate contract compartment. A deadline is measured from each release; a declared bound outside the displayed time range is identified explicitly.
- Observed logical waits and reference-simulator zero CPU cost are evidence of that run only. The diagram does not turn WCET limits into measured durations or establish physical schedulability. This projection uses fv and introduces no new source keyword or view kind.

Complete model · `FunctionTiming::Client.poll`.

SVG · PNG · Vector scene

Notation: execution-profile

Root execution profile

Element	Graphical notation	Textual notation
Root execution profile (label)		`component Product mixed`

Root execution profile. Write the selected service, realtime or mixed profile in the root-component header or property compartment.

Context: Root header in Gateway; realtime and service are alternative profiles. **Views:** gv, grv.

- Write the selected service, realtime or mixed profile in the root-component header or property compartment.
- An omitted profile means service; suppressing a realtime/mixed label MUST NOT weaken the actual model’s timing obligations.

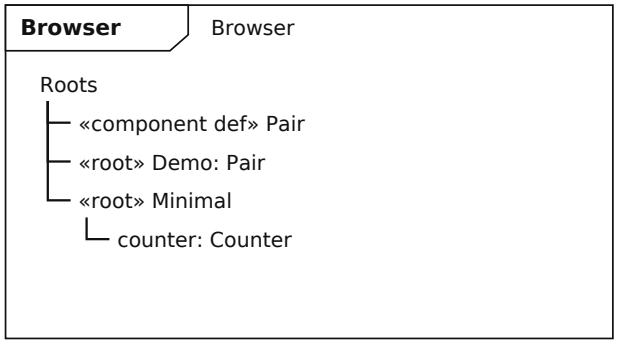
Complete model · `Gateway::Product`.

SVG · PNG · Vector scene

View projections

Notation: view-browser

Browser

Element	Graphical notation	Textual notation
Browser (projection)		`package Roots`

Browser. Draw a containment/navigation tree of existing packages, definitions, occurrences and owned members.

Context: Package header used as Browser navigation context, not a separate Browser declaration. **Views:** Browser.

- Draw a containment/navigation tree of existing packages, definitions, occurrences and owned members.

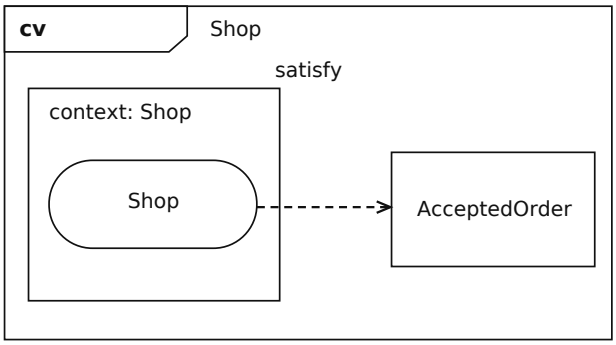
- Browser has no authored view keyword; navigation expansion does not create semantic containment or allocate behavior.

Complete model · `Roots`.

SVG · PNG · Vector scene

Notation: view-cv

Case View

Element	Graphical notation	Textual notation
Case View (projection)		`view cases cv of Shop;`

Case View. Draw a framed projection titled cv with the semantic subject identity visible; use the canonical element symbols within it.

Context: View projection context; syntax refers to an existing semantic subject. **Views:** cv.

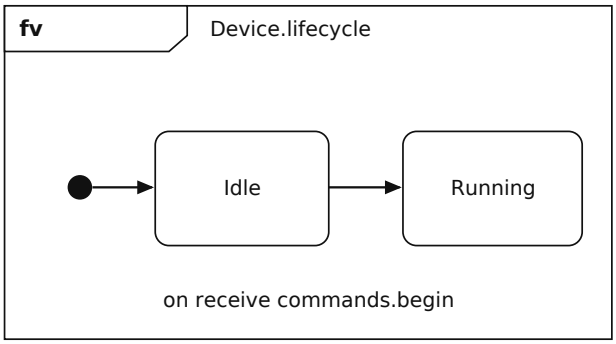
- Draw a framed projection titled cv with the semantic subject identity visible; use the canonical element symbols within it.
- The view adds presentation only. It MUST NOT introduce new routes, functions, performers or execution semantics.

Complete model · `Commerce::cases`.

SVG · PNG · Vector scene

Notation: view-fv

Flow View

Element	Graphical notation	Textual notation
Flow View (projection)		`view lifecycle fv of Device.lifecycle;`

Flow View. Draw a framed projection titled fv with the semantic subject identity visible; use the canonical element symbols within it.

Context: View projection context; syntax refers to an existing semantic subject. **Views:** fv.

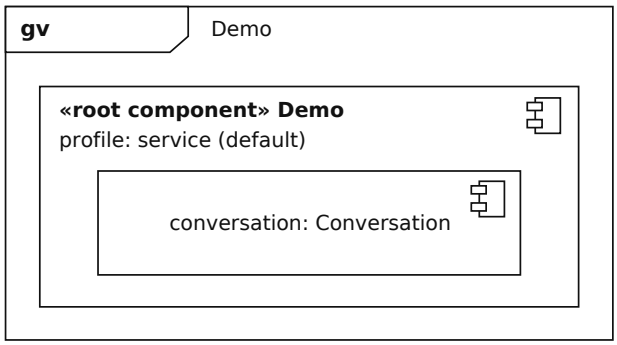
- Draw a framed projection titled fv with the semantic subject identity visible; use the canonical element symbols within it.
- fv supports both statement flow and function-owned states. Keep the owning function frame and never reintroduce separate action/state views or a machine element.

Complete model · `Lifecycle::lifecycle`.

SVG · PNG · Vector scene

Notation: view-gv

General View

Element	Graphical notation	Textual notation
General View (projection)		`view overview gv of Demo;`

General View. Draw a framed projection titled gv with the semantic subject identity visible; use the canonical element symbols within it.

Context: View projection context; syntax refers to an existing semantic subject. **Views:** gv.

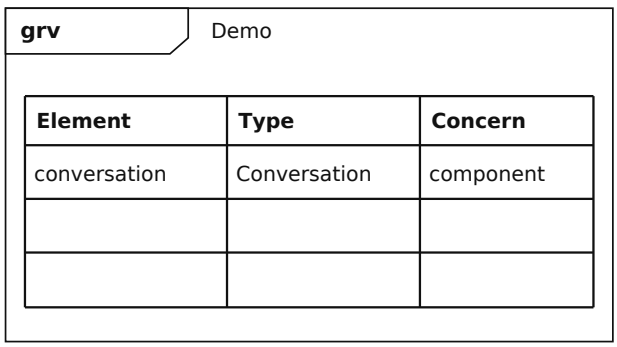
- Draw a framed projection titled gv with the semantic subject identity visible; use the canonical element symbols within it.
- The view adds presentation only. It MUST NOT introduce new routes, functions, performers or execution semantics.

Complete model · `Hello::overview`.

SVG · PNG · Vector scene

Notation: view-grv

Grid View

Element	Graphical notation	Textual notation
Grid View (projection)		`view inventory grv of Demo;`

Grid View. Draw a framed projection titled grv with the semantic subject identity visible; use the canonical element symbols within it.

Context: View projection context; syntax refers to an existing semantic subject. **Views:** grv.

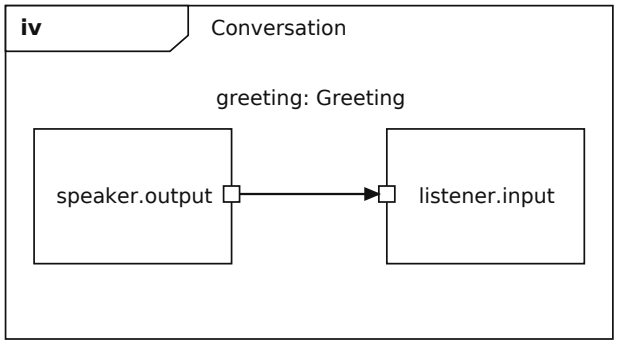
- Draw a framed projection titled grv with the semantic subject identity visible; use the canonical element symbols within it.
- The view adds presentation only. It MUST NOT introduce new routes, functions, performers or execution semantics.

Complete model · `Hello::inventory`.

SVG · PNG · Vector scene

Notation: view-iv

Interconnect View

Element	Graphical notation	Textual notation
Interconnect View (projection)		`view wiring iv of Conversation;`

Interconnect View. Draw a framed projection titled iv with the semantic subject identity visible; use the canonical element symbols within it.

Context: View projection context; syntax refers to an existing semantic subject. **Views:** iv.

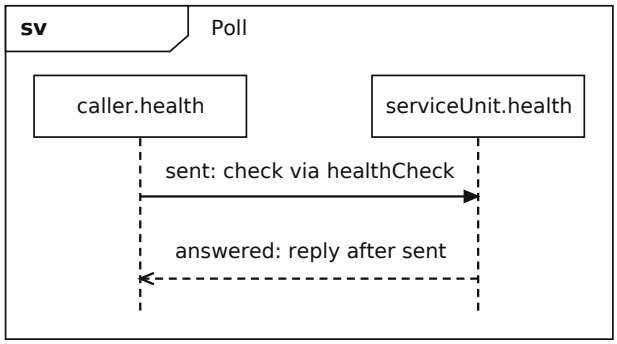
- Draw a framed projection titled iv with the semantic subject identity visible; use the canonical element symbols within it.
- The view adds presentation only. It MUST NOT introduce new routes, functions, performers or execution semantics.

Complete model · `Hello::wiring`.

SVG · PNG · Vector scene

Notation: view-sv

Sequence View

Element	Graphical notation	Textual notation
Sequence View (projection)		`view interaction sv of Poll;`

Sequence View. Draw a framed projection titled sv with the semantic subject identity visible; use the canonical element symbols within it.

Context: View projection context; syntax refers to an existing semantic subject. **Views:** sv.

- Draw a framed projection titled sv with the semantic subject identity visible; use the canonical element symbols within it.
- The view adds presentation only. It MUST NOT introduce new routes, functions, performers or execution semantics.

Complete model · `Rpc::interaction`.

SVG · PNG · Vector scene

Coverage and interpretation

The inventory explicitly assigns every current grammar production and keyword to a graphical concept or a justified text-only role. This is a notation coverage claim, not an assertion that an editor or layout engine is implemented. Run `node scripts/check-notation.mjs` for semantic coverage and context checks, and `python3 scripts/build-notation.py --check` for reproducible artifacts.

Keyword coverage

Term	Representation	Concept and rationale
`after`	glyph	Residence timer transition, Case causal order. Residence timer transition; Case causal order.
`and`	label	Expressions and value paths. Expressions and value paths.
`as`	glyph	Routed function call. Routed function call.
`assert`	glyph	Assertion. Assertion.
`atLeastOnce`	label	Delivery policy. Delivery policy.
`atMostOnce`	label	Delivery policy. Delivery policy.
`attribute`	label	Requirement attribute. Requirement attribute.

Term	Representation	Concept and rationale
bestEffort	label	Delivery policy. Delivery policy.
bool	label	Primitive types. Primitive types.
bound	glyph	Bounded loop, Counted for loop, Post-test do-while loop. Bounded loop; Counted for loop; Post-test do-while loop.
byte	label	Primitive types. Primitive types.
calc	label	Pure calculation. Pure calculation.
call	glyph	RPC handler activation, Local function call, Routed function call, Case request. RPC handler activation; Local function call; Routed function call; Case request.
capacity	label	Queue capacity. Queue capacity.
client	glyph	RPC client port. RPC client port.
clientServer	glyph	Client-server interface. Client-server interface.
completion	glyph	Completion transition. Completion transition.
component	glyph	Component definition, Component usage, Occurrence typing relationship, Root component, Inline root composition. Component definition; Component usage; Occurrence typing relationship; Root component; Inline root composition.
concurrency	label	Concurrency mode. Concurrency mode.
connect	glyph	Interface connector. Interface connector.
cv	glyph	Case View. Case View.
data	label	Data member. Data member.
datatype	glyph	Record datatype. Record datatype.
deadline	glyph	Function timing contract, Function timing diagram. Function timing contract; Function timing diagram.
def	glyph	Component definition, Reusable QoS policy. Component definition; Reusable QoS policy.
delegate	glyph	Boundary delegation. Boundary delegation.
delivery	label	Delivery policy. Delivery policy.
deploy	glyph	Deployment binding. Deployment binding.
do	glyph	Post-test do-while loop. Post-test do-while loop.
doc	label	Formal documentation. Formal documentation.
dropNewest	label	Overflow behavior. Overflow behavior.
dropOldest	label	Overflow behavior. Overflow behavior.
else	glyph	Conditional branch. Conditional branch.

Term	Representation	Concept and rationale
`entry`	label	State entry behavior. State entry behavior.
`enum`	glyph	Enumeration. Enumeration.
`event`	label	Event member. Event member.
`every`	label	Periodic activation. Periodic activation.
`exit`	label	State exit behavior. State exit behavior.
`fail`	glyph	Failure. Failure.
`false`	label	Expressions and value paths. Expressions and value paths.
`fifo`	label	Queue ordering. Queue ordering.
`final`	glyph	Final state. Final state.
`for`	glyph	Counted for loop. Counted for loop.
`forEach`	glyph	Fixed-array forEach loop. Fixed-array forEach loop.
`function`	glyph	Interface function signature, Executable function, Function with states. Interface function signature; Executable function; Function with states.
`fv`	glyph	Flow View, Function timing diagram. Flow View; Function timing diagram.
`grv`	glyph	Grid View. Grid View.
`gv`	glyph	General View. General View.
`idempotent`	label	Interface function signature. Interface function signature.
`if`	glyph	Conditional branch. Conditional branch.
`import`	glyph	Package import. Package import.
`in`	glyph	Pure calculation, Fixed-array forEach loop. Pure calculation; Fixed-array forEach loop.
`include`	glyph	Filtered view selection. Filtered view selection.
`initial`	glyph	Function with states, Initial state reference. Function with states; Initial state reference.
`int`	label	Primitive types. Primitive types.
`interface`	glyph	Sender-receiver interface, Client-server interface, Publish-subscribe interface, Service interface. Sender-receiver interface; Client-server interface; Publish-subscribe interface; Service interface.
`interruptible`	glyph	Interruptible flow region. Interruptible flow region.
`iv`	glyph	Interconnect View. Interconnect View.
`label`	glyph	Case data exchange, Case event exchange, Case request, Correlated case reply, Case causal order. Case data exchange; Case event exchange; Case request; Correlated case reply; Case causal order.

Term	Representation	Concept and rationale
`manual`	label	Manual activation. Manual activation.
`maxAge`	label	Message freshness bound. Message freshness bound.
`maxLatency`	label	Transport latency bound. Transport latency bound.
`mixed`	label	Root execution profile. Root execution profile.
`ms`	label	Duration value. Duration value.
`not`	label	Expressions and value paths. Expressions and value paths.
`ns`	label	Duration value. Duration value.
`of`	glyph	Use case, Satisfaction relation, General View, Interconnect View, Case View, Sequence View, Grid View, Flow View, Verification case. Use case; Satisfaction relation; General View; Interconnect View; Case View; Sequence View; Grid View; Flow View; Verification case.
`on`	glyph	Startup activation, Manual activation, Periodic activation, Port receive activation, RPC handler activation, State transition, Deployment binding, Interruptible flow region. Startup activation; Manual activation; Periodic activation; Port receive activation; RPC handler activation; State transition; Deployment binding; Interruptible flow region.
`optional`	label	Optional port. Optional port.
`or`	label	Expressions and value paths. Expressions and value paths.
`ordering`	label	Queue ordering. Queue ordering.
`otherwise`	glyph	Routed function call. Routed function call.
`overflow`	label	Overflow behavior. Overflow behavior.
`package`	glyph	Package. Package.
`port`	glyph	Data sender port, Data receiver port, RPC client port, RPC server port, Event publisher port, Event subscriber port, Service provider port, Service consumer port. Data sender port; Data receiver port; RPC client port; RPC server port; Event publisher port; Event subscriber port; Service provider port; Service consumer port.
`priority`	label	Transition priority. Transition priority.
`provides`	glyph	Service provider port. Service provider port.
`publish`	glyph	Event publication, Case event exchange. Event publication; Case event exchange.
`publisher`	glyph	Event publisher port. Event publisher port.
`publishSubscribe`	glyph	Publish-subscribe interface. Publish-subscribe interface.
`qos`	glyph	Delivery policy, Reusable QoS policy, Named connector policy. Delivery policy; Reusable QoS policy; Named connector policy.
`queued`	label	Data member. Data member.
`real`	label	Primitive types. Primitive types.

Term	Representation	Concept and rationale
realtime	label	Root execution profile. Root execution profile.
receive	glyph	Port receive activation, State transition, Interruptible flow region. Port receive activation; State transition; Interruptible flow region.
receiver	glyph	Data receiver port. Data receiver port.
record	label	Record construction. Record construction.
recursive	glyph	Filtered view selection. Filtered view selection.
redefines	label	Member redefinition. Member redefinition.
reentrant	label	Concurrency mode. Concurrency mode.
reject	label	Overflow behavior. Overflow behavior.
reply	glyph	Correlated case reply. Correlated case reply.
requirement	glyph	Requirement. Requirement.
requires	glyph	Service consumer port. Service consumer port.
return	glyph	Pure calculation, Function return. Pure calculation; Function return.
s	label	Duration value. Duration value.
sampled	label	Data member. Data member.
satisfy	glyph	Satisfaction relation. Satisfaction relation.
send	glyph	Data send, Case data exchange. Data send; Case data exchange.
sender	glyph	Data sender port. Data sender port.
senderReceiver	glyph	Sender-receiver interface. Sender-receiver interface.
serial	label	Concurrency mode. Concurrency mode.
server	glyph	RPC server port. RPC server port.
service	glyph	Service interface, Root execution profile. Service interface; Root execution profile.
specializes	glyph	Component specialization. Component specialization.
start	label	Startup activation. Startup activation.
state	glyph	State, Final state. State; Final state.
string	label	Primitive types. Primitive types.
subscriber	glyph	Event subscriber port. Event subscriber port.
sv	glyph	Sequence View. Sequence View.
timing	glyph	Function timing contract, Function timing diagram. Function timing contract; Function timing diagram.

Term	Representation	Concept and rationale
`transition`	glyph	State transition. State transition.
`true`	label	Expressions and value paths. Expressions and value paths.
`uint`	label	Primitive types. Primitive types.
`unordered`	label	Queue ordering. Queue ordering.
`us`	label	Duration value. Duration value.
`useCase`	glyph	Use case. Use case.
`var`	glyph	Private component variable, Local declaration. Private component variable; Local declaration.
`verificationCase`	glyph	Verification case. Verification case.
`verify`	glyph	Verification relation. Verification relation.
`version`	glyph	Sender-receiver interface, Client-server interface, Publish-subscribe interface, Service interface. Sender-receiver interface; Client-server interface; Publish-subscribe interface; Service interface.
`via`	glyph	Case data exchange, Case event exchange, Case request, Correlated case reply. Case data exchange; Case event exchange; Case request; Correlated case reply.
`view`	glyph	General View, Interconnect View, Case View, Sequence View, Grid View, Flow View. General View; Interconnect View; Case View; Sequence View; Grid View; Flow View.
`void`	label	Primitive types. Primitive types.
`wcet`	glyph	Function timing contract, Function timing diagram. Function timing contract; Function timing diagram.
`when`	label	Transition guard. Transition guard.
`where`	glyph	Filtered view selection. Filtered view selection.
`while`	glyph	Bounded loop, Post-test do-while loop. Bounded loop; Post-test do-while loop.
`within`	glyph	Routed function call. Routed function call.

Grammar production coverage

Term	Representation	Concept and rationale
`AdditiveExpression`	text-only	Precedence grouping only; arithmetic text belongs to the expression label.
`AndExpression`	text-only	Precedence grouping only; operator text belongs to the expression label.
`Argument`	label	Parameter or record field. Parameter or record field.
`ArrayLiteral`	label	Fixed array type and value. Fixed array type and value.

Term	Representation	Concept and rationale
Assert	glyph	Assertion. Assertion.
Assign	glyph	Assignment. Assignment.
Attribute	label	Requirement attribute. Requirement attribute.
Block	glyph	Executable function, State entry behavior, State exit behavior. Executable function; State entry behavior; State exit behavior.
Calculation	label	Pure calculation. Pure calculation.
CalculationCall	label	Pure calculation. Pure calculation.
Component	glyph	Component definition, Component specialization. Component definition; Component specialization.
ComponentMember	label	Member redefinition. Member redefinition.
Connect	glyph	Interface connector. Interface connector.
DataMember	label	Data member. Data member.
DataType	glyph	Record datatype. Record datatype.
Delegate	glyph	Boundary delegation. Boundary delegation.
Deployment	glyph	External deployment target, Deployment binding. External deployment target; Deployment binding.
DOC_COMMENT	label	Formal documentation. Formal documentation.
Documentation	label	Formal documentation. Formal documentation.
DoWhile	glyph	Post-test do-while loop. Post-test do-while loop.
Duration	glyph	Transport latency bound, Message freshness bound, Residence timer transition, Duration value. Transport latency bound; Message freshness bound; Residence timer transition; Duration value.
Element	text-only	Grammar union selecting a top-level element; each concrete alternative has its own notation.
EnumType	glyph	Enumeration. Enumeration.
EqualityExpression	text-only	Precedence grouping only; equality text belongs to the expression label.
EventMember	label	Event member. Event member.
Expression	label	Expressions and value paths. Expressions and value paths.
Fail	glyph	Failure. Failure.
FieldValue	label	Record construction. Record construction.
For	glyph	Counted for loop. Counted for loop.
ForEach	glyph	Fixed-array forEach loop. Fixed-array forEach loop.
ForUpdate	glyph	Counted for loop. Counted for loop.

Term	Representation	Concept and rationale
FunctionDef	glyph	Executable function, Function with states. Executable function; Function with states.
FunctionFlow	glyph	Executable function. Executable function.
ID	text-only	Identifier token supplies names on shapes and labels; no independent glyph.
If	glyph	Conditional branch. Conditional branch.
Import	glyph	Package import. Package import.
Instance	glyph	Component usage, Occurrence typing relationship, Occurrence lifeline. Component usage; Occurrence typing relationship; Occurrence lifeline.
INT	text-only	Integer literal token is rendered as exact expression/property text; no independent glyph.
Interface	glyph	Sender-receiver interface, Client-server interface, Publish-subscribe interface, Service interface. Sender-receiver interface; Client-server interface; Publish-subscribe interface; Service interface.
InterfaceMember	label	Data member, Event member, Interface function signature. Data member; Event member; Interface function signature.
Interruptible	glyph	Interruptible flow region. Interruptible flow region.
Literal	label	Expressions and value paths. Expressions and value paths.
LocalCall	glyph	Local function call. Local function call.
LocalVariable	glyph	Local declaration. Local declaration.
MemberPath	glyph	Routed function call. Routed function call.
ML_COMMENT	glyph	Ordinary comment. Ordinary comment.
Model	text-only	A source/workspace container, shown by Browser/package context; it has no additional authored glyph.
MultiplicativeExpression	text-only	Precedence grouping only; arithmetic text belongs to the expression label.
NUMBER	text-only	real literal token is rendered as exact expression text; no independent glyph.
Numeric	text-only	Number conversion/lexical grouping; render the resulting value text in its owning label.
OrExpression	text-only	Precedence grouping only; operator text belongs to the expression label.
Package	glyph	Package, Browser. Package; Browser.
Parameter	label	Parameter or record field. Parameter or record field.
Path	text-only	Containment/member path text identifies diagram subjects or endpoints, not an independent shape.

Term	Representation	Concept and rationale
Port	glyph	Data sender port, Data receiver port, RPC client port, RPC server port, Event publisher port, Event subscriber port, Service provider port, Service consumer port, Optional port. Data sender port; Data receiver port; RPC client port; RPC server port; Event publisher port; Event subscriber port; Service provider port; Service consumer port; Optional port.
PrimaryExpression	text-only	Grammar grouping of concrete expression forms; no separate authored semantic element.
Qos	label	Delivery policy, Queue ordering, Queue capacity, Overflow behavior, Transport latency bound, Message freshness bound, Named connector policy. Delivery policy; Queue ordering; Queue capacity; Overflow behavior; Transport latency bound; Message freshness bound; Named connector policy.
QosBody	glyph	Delivery policy, Queue ordering, Queue capacity, Overflow behavior, Transport latency bound, Message freshness bound, Reusable QoS policy. Delivery policy; Queue ordering; Queue capacity; Overflow behavior; Transport latency bound; Message freshness bound; Reusable QoS policy.
QosPolicy	glyph	Reusable QoS policy. Reusable QoS policy.
QualifiedName	text-only	Package-qualified declaration identity is text in the owning label.
RecordLiteral	label	Record construction. Record construction.
ReferenceExpression	label	Enumeration value, Expressions and value paths. Enumeration value; Expressions and value paths.
RelationalExpression	text-only	Precedence grouping only; comparison text belongs to the expression label.
RemoteCall	glyph	Routed function call. Routed function call.
Requirement	glyph	Requirement. Requirement.
Return	glyph	Function return. Function return.
RootComponent	glyph	Occurrence typing relationship, Root component, Inline root composition, Root execution profile. Occurrence typing relationship; Root component; Inline root composition; Root execution profile.
RootMember	glyph	Root component, Inline root composition. Root component; Inline root composition.
Satisfaction	glyph	Satisfaction relation. Satisfaction relation.
Scenario	glyph	Use case, Verification case. Use case; Verification case.
ScenarioStep	glyph	Case data exchange, Case event exchange, Case request, Correlated case reply, Case causal order. Case data exchange; Case event exchange; Case request; Correlated case reply; Case causal order.
Send	glyph	Data send, Event publication. Data send; Event publication.
Signature	label	Interface function signature. Interface function signature.
SL_COMMENT	glyph	Ordinary comment. Ordinary comment.
State	glyph	State, Final state, State entry behavior, State exit behavior. State; Final state; State entry behavior; State exit behavior.

Term	Representation	Concept and rationale
StateBody	glyph	Function with states, Initial state reference. Function with states; Initial state reference.
Statement	glyph	Executable function. Executable function.
STRING	text-only	string literal token supplies values/requirement text; preserve escaping in source notation.
Timing	glyph	Function timing contract, Concurrency mode, Function timing diagram. Function timing contract; Concurrency mode; Function timing diagram.
Transition	glyph	State transition, Transition guard, Transition priority, Completion transition, Residence timer transition. State transition; Transition guard; Transition priority; Completion transition; Residence timer transition.
Trigger	label	Startup activation, Manual activation, Periodic activation, Port receive activation, RPC handler activation. Startup activation; Manual activation; Periodic activation; Port receive activation; RPC handler activation.
TypeRef	label	Primitive types, Fixed array type and value. Primitive types; Fixed array type and value.
UnaryExpression	label	Expressions and value paths. Expressions and value paths.
ValuePath	label	Enumeration value. Enumeration value.
ValueType	glyph	Record datatype, Enumeration. Record datatype; Enumeration.
Variable	label	Private component variable. Private component variable.
Verification	glyph	Verification relation. Verification relation.
View	glyph	General View, Interconnect View, Case View, Sequence View, Grid View, Flow View, Function timing diagram. General View; Interconnect View; Case View; Sequence View; Grid View; Flow View; Function timing diagram.
ViewSelection	glyph	Filtered view selection. Filtered view selection.
While	glyph	Bounded loop. Bounded loop.
WS	text-only	Whitespace affects textual layout only; no semantic graphical representation.

NORMATIVE APPENDIX

A. Complete concrete grammar

Authoritative Langium grammar, reproduced in full. Line wraps are typographic continuations; only numbered rows begin original source lines.

LANGIUM GRAMMAR

```
1  grammar Swaml
2
3  entry Model: packages+=Package*;
4  Package: 'package' name=ID '{' documentation=Documentation? (imports+=Import | elements+=Element |
   satisfactions+=Satisfaction)* '}';
5  Import: 'import' namespace=QualifiedName '::' '*' ';';
6  Element: DataType | EnumType | Interface | Component | RootComponent | Requirement | Scenario | View |
   QosPolicy;
7  DataType: 'datatype' name=ID '{' documentation=Documentation? fields+=Parameter* '}';
8  EnumType: 'enum' name=ID '{' documentation=Documentation? literals+=ID (',' literals+=ID)* '}';
9  Parameter: name=ID ':' type=TypeRef documentation=Documentation? ';';
10 TypeRef: (primitive=('bool'|'byte'|'uint'|'int'|'real'|'string'|'void') |
   type=[ValueType:QualifiedName]) ('[' size=INT ']')?;
11 type ValueType = DataType | EnumType;
12 Interface: 'interface' kind=('senderReceiver'|'clientServer'|'publishSubscribe'|'service') name=ID
   'version' version=STRING '{' documentation=Documentation? members+=InterfaceMember* '}';
13 InterfaceMember: DataMember | EventMember | Signature;
14 DataMember: 'data' name=ID ':' type=TypeRef mode=('sampled'|'queued') documentation=Documentation? ';';
15 EventMember: 'event' name=ID '(' (parameters+=Argument (',' parameters+=Argument)*)? ')'
   documentation=Documentation? ';';
16 Signature: 'function' name=ID '(' (parameters+=Argument (',' parameters+=Argument)*)? ')' (':'
   result=TypeRef)? (idempotent?='idempotent')? documentation=Documentation? ';';
17 Argument: name=ID ':' type=TypeRef;
18 Component: 'component' 'def' name=ID ('specializes' specializes=[Component:QualifiedName])? '{'
   documentation=Documentation? members+=ComponentMember* '}';
19 ComponentMember: Port | Instance | Connect | Delegate | Variable | FunctionDef;
20 Instance: (redefines?='redefines')? kind='component' name=ID ':' definition=[Component:QualifiedName]
   documentation=Documentation? ';';
21 Port: (redefines?='redefines')? 'port' name=ID ':' interface=[Interface:QualifiedName]
   role=('sender'|'receiver'|'client'|'server'|'publisher'|'subscriber'|'provides'|'requires')
   (optional?='optional')? documentation=Documentation? ';';
22 Variable: (redefines?='redefines')? 'var' name=ID ':' type=TypeRef '=' value=Expression
   documentation=Documentation? ';';
23 RootComponent: 'component' name=ID (':' definition=[Component:QualifiedName]
   (profile=('realtime'|'service'|'mixed'))? (';' | '{' documentation=Documentation? members+=RootMember*
   '}') | (profile=('realtime'|'service'|'mixed'))? '{' documentation=Documentation? members+=RootMember*
   '}');
24 RootMember: ComponentMember | Deployment;
25 Deployment: 'deploy' target=Path 'on' system=STRING documentation=Documentation? ';';
26 Connect: (redefines?='redefines')? 'connect' name=ID ':' source=Path '->' target=Path qos=Qos?
   documentation=Documentation? ';';
27 Delegate: (redefines?='redefines')? 'delegate' outer=[Port:ID] '->' target=Path
   documentation=Documentation? ';';
28 QosPolicy: 'qos' 'def' name=ID documentation=Documentation? body=QosBody;
29 Qos: 'qos' (body=QosBody | policy=[QosPolicy:QualifiedName]);
30 QosBody: '{' 'delivery' delivery=('bestEffort'|'atMostOnce'|'atLeastOnce') ';' 'ordering'
   ordering=('fifo'|'unordered') ';' 'capacity' capacity=INT ';' 'overflow'
   overflow=('dropOldest'|'dropNewest'|'reject') ';' ('maxLatency' maxLatency=Duration ';')? ('maxAge'
   maxAge=Duration ';')? '}';
31 FunctionDef: (redefines?='redefines')? 'function' name=ID '(' (parameters+=Argument (','
   parameters+=Argument)*)? ')' (':' result=TypeRef)? trigger=Trigger? timing=Timing? '{'
   documentation=Documentation? calculations+=Calculation* (body=FunctionFlow | stateBody=StateBody)? '}';
32 FunctionFlow infers Block: statements+=Statement+;
33 Trigger: 'on' (kind=('start'|'manual') | kind='every' period=Duration | kind=('receive'|'call')
   endpoint=Path);
34 Timing: 'timing' '{' 'wcet' wcet=Duration ';' 'deadline' deadline=Duration ';' 'concurrency'
   concurrency=('serial'|'reentrant') ';' '}';
35 Calculation: 'calc' name=ID '{' documentation=Documentation? ('in' parameters+=Parameter)* 'return'
```

LANGIUM GRAMMAR / CONTINUED

```
    outputName=ID ':' result=TypeRef '=' value=Expression ';' '}';
36  Block: '{' statements+=Statement* '}';
37  Statement: LocalVariable | Assign | Send | RemoteCall | LocalCall | Return | If | While | For | ForEach
    | DoWhile | Interruptible | Fail | Assert;
38  LocalVariable: 'var' name=ID ':' type=TypeRef '=' value=Expression ';';
39  Assign: target=ID '=' value=Expression ';';
40  Send: kind=('send'|'publish') endpoint=Path '(' (args+=Expression (',' args+=Expression)*)? ')' ';';
41  RemoteCall: 'call' endpoint=MemberPath '(' (args+=Expression (',' args+=Expression)*)? ')' 'as' name=ID
    'within' timeout=Duration 'otherwise' failure=Block ';';
42  LocalCall: 'call' function=[FunctionDef:ID] '(' (args+=Expression (',' args+=Expression)*)? ')' ('as'
    name=ID)? ';';
43  Return: 'return' value=Expression? ';';
44  If: 'if' '(' condition=Expression ')' then=Block ('else' otherwise=Block)?;
45  While: 'while' '(' condition=Expression ')' 'bound' bound=INT body=Block;
46  For: 'for' '(' initializer=LocalVariable condition=Expression ';' update=ForUpdate ')' 'bound' bound=INT
     body=Block;
47  ForUpdate infers Assign: target=ID '=' value=Expression;
48  ForEach: 'forEach' '(' variable=Argument 'in' collection=Expression ')' body=Block;
49  DoWhile: 'do' body=Block 'while' '(' condition=Expression ')' 'bound' bound=INT ';';
50  Interruptible: 'interruptible' name=ID documentation=Documentation? body=Block 'on' 'receive'
    endpoint=Path handler=Block;
51  Fail: 'fail' reason=STRING ';';
52  Assert: 'assert' condition=Expression ';';
53  StateBody: 'initial' initial=[State:ID] ';' states+=State+ transitions+=Transition*;
54  State: 'state' name=ID (final?='final')? '{' documentation=Documentation? ('entry' entry=Block)? ('exit'
     exit=Block)? '}';
55  Transition: documentation=Documentation? 'transition' name=ID ':' from=[State:ID] '->' to=[State:ID]
    'on' (kind='completion' | kind='after' delay=Duration | kind='receive' endpoint=Path) ('when' '('
    guard=Expression ')')? ('priority' priority=INT)? effect=Block;
56  Requirement: 'requirement' name=ID '{' documentation=Documentation? attributes+=Attribute* '}';
57  Attribute: 'attribute' name=(ID|'priority') ':' type=TypeRef '=' value=Expression
    documentation=Documentation? ';';
58  Documentation: text=DOC_COMMENT;
59  Satisfaction: 'satisfy' requirement=[Requirement:QualifiedName] 'of' source=Path ';';
60  Verification: 'verify' requirement=[Requirement:QualifiedName] ';';
61  Scenario: kind=('useCase'|'verificationCase') name=ID 'of' root=[RootComponent:QualifiedName] '{'
    documentation=Documentation? verifications+=Verification* steps+=ScenarioStep* '}';
62  ScenarioStep: kind=('send'|'publish'|'call'|'reply') source=Path '->' target=Path 'via' connector=ID
    ('after' predecessor=ID)? 'label' name=ID ';';
63  View: 'view' name=ID kind=('gv'|'iv'|'cv'|'sv'|'grv'|'fv') ('of' subject=Path
    documentation=Documentation? ';' | '{' documentation=Documentation? selections+=ViewSelection+ '}');
64  ViewSelection: 'include' subject=Path (recursive?='recursive')? ('where' filter=Expression)? ';';
65  Duration: value=INT unit=('ns'|'us'|'ms'|'s');
66  MemberPath returns string: ID '.' ID;
67  Path returns string: ID ('.' ID)*;
68  QualifiedName returns string: ID ('::' ID)*;
69  ValuePath returns string: QualifiedName ('.' ID)*;
70  type Expression = BinaryExpression | UnaryExpression | Literal | CalculationCall | ReferenceExpression |
     RecordLiteral | ArrayLiteral;
71  Expression returns Expression: OrExpression;
72  OrExpression returns Expression: AndExpression ({infer BinaryExpression.left=current} operator='or'
    right=AndExpression)*;
73  AndExpression returns Expression: EqualityExpression ({infer BinaryExpression.left=current}
    operator='and' right=EqualityExpression)*;
74  EqualityExpression returns Expression: RelationalExpression ({infer BinaryExpression.left=current}
    operator=('=='|'!=') right=RelationalExpression)*;
75  RelationalExpression returns Expression: AdditiveExpression ({infer BinaryExpression.left=current}
    operator=('<'|'<='|'>'|'>=') right=AdditiveExpression)*;
76  AdditiveExpression returns Expression: MultiplicativeExpression ({infer BinaryExpression.left=current}
    operator=('+'|'-') right=MultiplicativeExpression)*;
77  MultiplicativeExpression returns Expression: UnaryExpression ({infer BinaryExpression.left=current}
    operator=('*'|'/'|'%') right=UnaryExpression)*;
78  UnaryExpression returns Expression: {infer UnaryExpression} operator=('not'|'-') operand=UnaryExpression
     | PrimaryExpression;
79  PrimaryExpression returns Expression: Literal | CalculationCall | ReferenceExpression | RecordLiteral |
    ArrayLiteral | '(' Expression ')';
80  Literal: {infer Literal} (number=Numeric | text=STRING | boolean=('true'|'false'));
```

LANGIUM GRAMMAR / CONTINUED

```
81 CalculationCall: calculation=[Calculation:ID] '(' (args+=Expression (',' args+=Expression)*)? ')';
82 ReferenceExpression: path=ValuePath ('[' index=Expression ']')?;
83 RecordLiteral: 'record' type=[DataType:QualifiedName] '{' fields+=FieldValue* '}';
84 FieldValue: name=ID '=' value=Expression ';';
85 ArrayLiteral: '[' (values+=Expression (',' values+=Expression)*)? ']';
86 Numeric returns number: NUMBER | INT;
87 hidden terminal WS: /\s+/;
88 terminal DOC_COMMENT: /doc\s*\/\*[\s\S]*?\*\//;
89 terminal ID: /[_a-zA-Z][\w]*/;
90 terminal NUMBER returns number: /[0-9]+\.[0-9]+/;
91 terminal INT returns number: /[0-9]+/;
92 terminal STRING: /"([^"\\\x00-\x1F]|\\["\\nrt])*"/;
93 hidden terminal ML_COMMENT: /\/\*[\s\S]*?\*\//;
94 hidden terminal SL_COMMENT: /\/\/[^\n\r]*/;
```

NORMATIVE DESIGN INVENTORY

B. Grammar-to-semantics inventory

Every grammar rule/type name is represented. Proposed kernel bases identify the design target; they do not constitute an implemented or validated exporter. Empty bases indicate syntax normalized through its owner.

Rule / kind	Proposed kernel base	Meaning to preserve
Model element	Package	Workspace source container under a host-pinned 1.3 language edition; no file version header.
Package element	Package	Flat unique declaration namespace with ordered owned members.
Import element	NamespaceImport	Direct wildcard visibility edge; no transitive import or implicit copying.
Element grouping	Owner representation	Grammar union; normalize the selected concrete alternative exactly once.
DataType element	DataType	Finite nominal value domain: ordered record fields or unique enum literals.
EnumType element	DataType	Finite nominal value domain: ordered record fields or unique enum literals.
Parameter element	Feature	Typed field/input feature; preserve declared position and copied values.
TypeRef reference	Feature	Primitive bool/byte/uint/int/real/string/void or nominal value type and optional fixed extent; copied value domains and checked integral assignments.
ValueType grouping	Owner representation	Grammar union; normalize the selected concrete alternative exactly once.
Interface element	Classifier	Nominal versioned member contract; legal member set depends on kind.
InterfaceMember grouping	Owner representation	Grammar union; normalize the selected concrete alternative exactly once.
DataMember element	Feature	Typed payload contract with sampled/queued delivery interpretation.
EventMember element	Behavior	Contract-only payload signature; only port-projected function emission can instantiate an exchange.
Signature element	Behavior	Interface-owned function signature with copied input/result types, optional void result and idempotency declaration; distinct from implementing component behavior.
Argument element	Feature	Typed field/input feature; preserve declared position and copied values.
Component element	Structure	Reusable software structure owning child instances, ports, private values and functions; single specialization with compatible member redefinition.

Rule / kind	Proposed kernel base	Meaning to preserve
ComponentMember grouping	Owner representation	Owned structural or behavioral member of a component; effective members include inherited definitions and compatible explicit replacements.
Instance element	Feature	Exactly-one composite component usage typed by a definition; distinct instance path, private store and activation state; compatible redefinition may select a specialization.
Port element	Feature	Component-owned nominal interface endpoint with role/optionality; same-contract compatible redefinition. Effective behavior may be local or uniquely delegated.
Variable element	Feature, FeatureValue	Private variable with pure initializer and per-component-occurrence storage.
RootComponent element	Feature	Closed concrete component occurrence with inline structural/behavioral members or a referenced component definition; service default; typed body is deployment-only.
RootMember grouping	Owner representation	Inline root structural/behavioral member or external deployment binding; typed roots accept deployments only.
Deployment element	MetadataFeature	Existing component-instance path and exact external system/part identity string; nearest-ancestor inheritance. A host resolves the external identity without implying SysML import or conformance.
Connect element	Connector	Explicit route between local or immediate-child typed ports; independent occurrence/direction/member queues and complete effective policy. Redefinition preserves endpoints.
Delegate element	Connector	Port route or queue-free forwarding alias; preserve hierarchy and nominal interface identity.
Qos reference	MetadataFeature	Explicit inline body or package-visible named policy selection; preserve reference and use provenance and elaborate effective values independently per connector occurrence.
QosPolicy element	MetadataFeature	Package-owned named complete queue policy; same namespace/import ambiguity rules as other declarations. No inheritance, overrides, queue identity or runtime storage.
QosBody grouping	Owner representation	Complete ordered delivery, ordering, capacity, overflow and optional latency/age fields; checked at declaration and every connector use. No independent executable identity.
FunctionDef element	Behavior	Executable function plus synthetic component-owned executable feature; copied parameters, activation/trigger, timing and one statement or state body. Local references use the effective component context.
Trigger element	MetadataFeature	Local release rule or port-member handler binding; not a free event.
Timing element	MetadataFeature	WCET, elapsed deadline and concurrency contract; activation root and local helper interpretation.
Calculation element	Function	Function-owned pure calculation with copied explicit inputs and one named non-void output expression. Lexical ownership prevents capture of mutable or function-local values and access from sibling functions.

Rule / kind	Proposed kernel base	Meaning to preserve
Block element	Step, Succession	Ordered steps, lexical scope, success/failure exits and source trace.
Statement grouping	Owner representation	Grammar union; normalize the selected concrete alternative exactly once.
LocalVariable element	Step, Feature	Evaluate then create/replace one copied value in the proper lexical/private environment.
Assign element	Step, Feature	Evaluate then create/replace one copied value in the proper lexical/private environment.
Send element	Step	SWAML library emission behavior with function/port/member provenance and independent route offers.
RemoteCall element	Step	SWAML library awaited RPC behavior with correlation, bounded timeout and exclusive success/failure.
LocalCall element	Step	Synchronous untriggered flow function call in the same component, resolved against effective inherited functions. Borrows activation lock/snapshot; copied non-void result binds a fresh caller local.
Return element	Step	Frame completion or failure unwinding; no rollback or implicit event.
If element	Step, Succession	Guarded structured control with branch exclusivity, loop count and bound failure.
While element	Step, Succession	Guarded structured control with branch exclusivity, loop count and bound failure.
Fail element	Step	Frame completion or failure unwinding; no rollback or implicit event.
Assert element	Step	Pure Boolean check with explicit activation failure continuation.
StateBody grouping	Owner representation	Initial-first syntax grouping contributing directly to its function: states, transitions, current-state, timer generations and ordered reactions. No independently authored/executable element.
State element	Step	State identity, entry/exit groups and finality in one function context.
Transition element	Step, Succession	Source/target, stimulus matcher, guard, priority and exit/effect/update/entry sequence.
Requirement element	MetadataFeature	Named obligation owning immutable typed attributes and optional formal doc, without a subject.
Scenario element	Behavior	Use or verification case with root context and partial-order exchanges; only verification cases own verify links.
ScenarioStep element	Step, Succession	Port-member exchange with connector, explicit causal predecessor and inferred unique reply correlation.
View element	MetadataFeature	Named projection of the stable union of selected occurrence paths; no executable effects.

Rule / kind	Proposed kernel base	Meaning to preserve
Duration value	LiteralInteger	Exact positive integer nanoseconds with retained authoring time unit.
Path reference	Owner representation	Resolve according to reference context; preserve authoring spelling and resolved identities.
QualifiedName reference	Owner representation	Resolve according to reference context; preserve authoring spelling and resolved identities.
ValuePath reference	Owner representation	Value-reference syntax permitting local field paths and explicit package-qualified enum literals. An accessible value first segment wins over an unqualified enum with no field-failure fallback; cross-component private lookup remains forbidden.
Expression lowering	Expression	Typed expression tree; grouping/precedence, operand order and conditional short-circuit evaluation.
OrExpression lowering	Expression	Typed expression tree; grouping/precedence, operand order and conditional short-circuit evaluation.
AndExpression lowering	Expression	Typed expression tree; grouping/precedence, operand order and conditional short-circuit evaluation.
EqualityExpression lowering	Expression	Typed expression tree; grouping/precedence, operand order and conditional short-circuit evaluation.
RelationalExpression lowering	Expression	Typed expression tree; grouping/precedence, operand order and conditional short-circuit evaluation.
AdditiveExpression lowering	Expression	Typed expression tree; grouping/precedence, operand order and conditional short-circuit evaluation.
MultiplicativeExpression lowering	Expression	Typed expression tree; grouping/precedence, operand order and conditional short-circuit evaluation.
UnaryExpression lowering	Expression	Typed expression tree; grouping/precedence, operand order and conditional short-circuit evaluation.
PrimaryExpression lowering	Expression	Typed expression tree; grouping/precedence, operand order and conditional short-circuit evaluation.
Literal value	Expression	Concrete typed literal expression: bool, bounded int, finite real or portable string.
CalculationCall lowering	Expression	Copied argument evaluation and acyclic expression evaluation resolved only within the enclosing function calculation namespace.
ReferenceExpression lowering	Expression	Local/copied/private value lookup with record traversal and checked final indexing; alternatively a nominal enum value resolved by lexical-first then package/import rules. Qualified enum identity is retained.
RecordLiteral value	Expression	Nominal construction, declared field/literal identities, fixed ordered array values.
FieldValue value	Expression	Nominal construction, declared field/literal identities, fixed ordered array values.

Rule / kind	Proposed kernel base	Meaning to preserve
ArrayLiteral value	Expression	Nominal construction, declared field/literal identities, fixed ordered array values.
Numeric lexical	Owner representation	Unsigned decimal value; unary minus is an expression; finite/safe literal range checks.
WS lexical	Owner representation	Insignificant source trivia; retain only for authoring round trips.
ID lexical	Owner representation	Case-sensitive ASCII identifier; grammar keywords are reserved.
NUMBER lexical	Owner representation	Unsigned decimal value; unary minus is an expression; finite/safe literal range checks.
INT lexical	Owner representation	Unsigned decimal value; unary minus is an expression; finite/safe literal range checks.
STRING lexical	Owner representation	Unicode text with only quote, backslash, n, r and t escapes; no raw controls.
ML_COMMENT lexical	Owner representation	Insignificant source trivia; retain only for authoring round trips.
SL_COMMENT lexical	Owner representation	Insignificant source trivia; retain only for authoring round trips.
FunctionFlow grouping	Step	Nonempty statement sequence after function-local calculation declarations; normalizes to Block. An absent sequence is an empty void flow.
MemberPath reference	Feature	Exactly port.member for a routed call; distinguishes it from a bare local function identifier.
Attribute element	MetadataFeature	Immutable typed requirement metadata with no runtime store.
Documentation element	MetadataFeature	One owned raw formal specification; preserve text and ownership without executable effects.
Satisfaction element	MetadataFeature	Directed design-identity to requirement-identity claim; no automatic evidence.
Verification element	MetadataFeature	Directed verificationCase to requirement claim; source kind and uniqueness are checked.
ViewSelection reference	MetadataFeature	Occurrence-path selection, optional recursive expansion and pure Boolean filtering; stable union.
DOC_COMMENT lexical	Owner representation	Visible lexical token containing doc and its non-nested raw block; distinct from hidden comments.
For element	Step, Succession	Structured counted loop with one header-local initializer, typed Boolean test, assignment update, fresh body scopes and a positive finite execution bound. Failure is taken before an excess body; the header binding does not escape.
ForUpdate grouping	Owner representation	Header spelling of Assign without a terminating semicolon. Normalize to the same typed assignment step and preserve its source location and effects; it sees the header scope, never body-local declarations.

Rule / kind	Proposed kernel base	Meaning to preserve
ForEach element	Step, Succession, Feature	Ordered finite iteration over one copied fixed-array snapshot. Each item is a fresh typed copied local feature; cardinality bounds iterations and source/item mutation cannot alter the snapshot.
DoWhile element	Step, Succession	Post-test loop with at least one body, a Boolean condition in the enclosing scope, fresh body scopes and positive finite bound. A true condition after the final permitted body takes a bound-failure exit.
Interruptible element	Step, Succession, Feature	Named protected region and explicit typed data/event handler within one triggered serial flow activation. Cooperative interruption abandons protected work and pending local RPC correlation, retains prior/remote effects and runs the handler under the same deadline; it does not create a second activation or hardware interrupt.

INFORMATIVE / COMPLETE MODELS

C. Model examples

01 hello

SWAML / 01-hello.swaml

```
package Hello {
    interface senderReceiver Greeting version "1.0.0" {
        data text: string queued;
    }
    component def Speaker {
        port output: Greeting sender;
        function greet() on start {
            send output.text("Hello, SWAML");
            return;
        }
    }
    component def Listener {
        port input: Greeting receiver;
        var last: string = "";
        function hear(message: string) on receive input.text {
            last = message;
            return;
        }
    }
    component def Conversation {
        component speaker: Speaker;
        component listener: Listener;
        connect greeting: speaker.output -> listener.input qos {
            delivery atMostOnce; ordering fifo; capacity 4; overflow reject;
        };
    }
    component Demo { component conversation: Conversation; }
    view overview gv of Demo;
    view wiring iv of Conversation;
    view greetingFlow fv of Speaker.greet;
    view inventory grv of Demo;
}
```

02 realtime controller

SWAML / 02-realtime-controller.swaml

```
package Control {
    interface senderReceiver Samples version "1.0.0" { data temperature: real sampled; }
    interface senderReceiver Commands version "1.0.0" { data duty: real sampled; }
    component def Sensor {
        port output: Samples sender;
        function sample() on every 10 ms timing { wcet 1 ms; deadline 3 ms; concurrency serial; } {
            send output.temperature(23.5);
            return;
        }
    }
    component def Regulator {
        port input: Samples receiver;
        port output: Commands sender;
        function regulate(temperature: real) on receive input.temperature timing { wcet 1 ms; deadline 3 ms; concurrency serial; } {
            calc clamp { in x: real; return result: real = x / 100.0; }
            var duty: real = clamp(temperature);
            if (duty > 0.8) { send output.duty(0.8); }
            else { send output.duty(duty); }
            return;
        }
    }
    component def Actuator {
        port input: Commands receiver;
        var applied: real = 0.0;
        function apply(duty: real) on receive input.duty timing { wcet 1 ms; deadline 2 ms; concurrency serial; } {
            applied = duty;
            assert applied <= 0.8;
            return;
        }
    }
    component def Controller {
        component sensor: Sensor;
        component regulator: Regulator;
        component actuator: Actuator;
        connect measured: sensor.output -> regulator.input qos {
            delivery atMostOnce; ordering fifo; capacity 1; overflow dropOldest; maxLatency 1 ms; maxAge 20 ms;
        };
        connect commanded: regulator.output -> actuator.input qos {
            delivery atMostOnce; ordering fifo; capacity 1; overflow dropOldest; maxLatency 1 ms; maxAge 20 ms;
        };
    }
    component Heating realtime {
        component controller: Controller;
        deploy controller on "system:ecu";
    }
    requirement LimitedDuty { doc /* Applied duty shall be at most 0.8. */ }
    view overview gv of Heating;
    view wiring iv of Controller;
    view regulating fv of Regulator.regulate;
    view deployment grv of Heating;
}
```

03 service pubsub

SWAML / 03-service-pubsub.swaml

```
package Commerce {
    interface service Orders version "1.0.0" {
        // Submitting publishes effects, so this contract does not claim idempotency.
        function submit(orderId: string): bool;
        event accepted(orderId: string);
    }
    interface publishSubscribe Audit version "1.0.0" { event recorded(orderId: string); }
    component def OrderService {
        port api: Orders provides;
        port audit: Audit publisher;
        function submit(orderId: string): bool on call api.submit {
            publish api.accepted(orderId);
            publish audit.recorded(orderId);
            return true;
        }
    }
    component def Frontend {
        port orders: Orders requires;
        var lastAccepted: string = "";
        function checkout() on manual {
            call orders.submit("order-42") as accepted within 2 s otherwise { fail "checkout
unavailable"; };
            assert accepted;
            return;
        }
        function confirmation(orderId: string) on receive orders.accepted { lastAccepted = orderId;
return; }
    }
    component def AuditStore {
        port input: Audit subscriber;
        var last: string = "";
        function storeRecord(orderId: string) on receive input.recorded { last = orderId; return; }
    }
    component def Backend {
        port api: Orders provides;
        port audit: Audit publisher;
        component serviceUnit: OrderService;
        delegate api -> serviceUnit.api;
        delegate audit -> serviceUnit.audit;
    }
    component def Web {
        port orders: Orders requires;
        component frontend: Frontend;
        delegate orders -> frontend.orders;
    }
    component def Archive {
        port input: Audit subscriber;
        component store: AuditStore;
        delegate input -> store.input;
    }
    component Shop {
        component web: Web;
        component backend: Backend;
        component archive: Archive;
        connect orderLink: web.orders -> backend.api qos { delivery atMostOnce; ordering fifo; capacity
32; overflow reject; };
        connect auditing: backend.audit -> archive.input qos { delivery atLeastOnce; ordering fifo;
capacity 128; overflow dropOldest; };
        deploy web on "system:edge";
        deploy backend on "system:application";
        deploy archive on "system:storage";
    }
    requirement AcceptedOrder { doc /* Accepted orders shall be published to audit subscribers. */ }
    satisfy AcceptedOrder of Shop; useCase Checkout of Shop {
        call web.orders.submit -> backend.api.submit via orderLink label submitOrder;
```

SWAML / 03-service-pubsub.swaml / CONTINUED

```
        publish backend.api.accepted -> web.orders.accepted via orderLink after submitOrder label notification;
        publish backend.audit.recorded -> archive.input.recorded via auditing after submitOrder label auditRecord;
        reply backend.api.submit -> web.orders.submit via orderLink after notification label response;
    }
    view overview gv of Shop;
    view wiring iv of Shop;
    view cases cv of Shop;
    view checkout sv of Checkout;
    view services grv of Shop;
    view submitFlow fv of OrderService.submit;
}
```

04 stateful function

SWAML / 04-stateful-function.swaml

```
package Lifecycle {
    interface publishSubscribe Commands version "1.0.0" { event begin(); event stop(); }
    interface publishSubscribe Status version "1.0.0" { event changed(value: string); }
    component def Operator {
        port commands: Commands publisher;
        function startDevice() on manual { publish commands.begin(); return; }
        function stopDevice() on manual { publish commands.stop(); return; }
    }
    component def Device {
        port commands: Commands subscriber;
        port status: Status publisher;
        var cycles: int = 0;
        function lifecycle() on start {
            initial Idle;
            state Idle { entry { publish status.changed("idle"); } }
            state Running { entry { cycles = cycles + 1; publish status.changed("running"); } }
            state Stopped final { entry { publish status.changed("stopped"); } }
            transition beginRun: Idle -> Running on receive commands.begin {}
            transition stopRun: Running -> Stopped on receive commands.stop {}
            transition expire: Running -> Stopped on after 30 s { publish status.changed("timeout"); }
        }
    }
    component def Monitor {
        port status: Status subscriber;
        var current: string = "unknown";
        function observe(value: string) on receive status.changed { current = value; return; }
    }
    component def Assembly {
        component operatorUnit: Operator;
        component device: Device;
        component monitor: Monitor;
        connect commands: operatorUnit.commands -> device.commands qos { delivery atMostOnce; ordering
fifo; capacity 8; overflow reject; };
        connect status: device.status -> monitor.status qos { delivery atMostOnce; ordering fifo;
capacity 8; overflow dropOldest; };
    }
    component Product { component assembly: Assembly; }
    view lifecycle fv of Device.lifecycle;
    view wiring iv of Assembly;
}
```

05 values and flow

SWAML / 05-values-and-flow.swaml

```
package Values {
    enum Health { Good, Failed }
    datatype Measurement { value: real; health: Health; }
    interface senderReceiver Measurements version "1.0.0" { data sample: Measurement queued; }
    component def Producer {
        port output: Measurements sender;
        function prepare() {
            calc twice { in x: real; return result: real = x * 2.0; }

            var samples: real[3] = [1.0, 2.0, 3.0];
            var i: int = 0;
            while (i < 3) bound 3 {
                var measurement: Measurement = record Measurement {
                    value = twice(samples[i]);
                    health = Health.Good;
                };
                send output.sample(measurement);
                i = i + 1;
            }
            return;
        }
        function boot() on start { call prepare(); return; }
    }
    component def Consumer {
        port input: Measurements receiver;
        var latest: real = 0.0;
        function update(measurement: Measurement) on receive input.sample {
            if (measurement.health == Health.Good) { latest = measurement.value; }
            return;
        }
    }
    component def Pipeline {
        component producer: Producer;
        component consumer: Consumer;
        connect samples: producer.output -> consumer.input qos { delivery atMostOnce; ordering fifo;
capacity 3; overflow reject; };
    }
    component Demo { component pipeline: Pipeline; }
    view preparation fv of Producer.prepare;
}
```

06 mixed gateway

SWAML / 06-mixed-gateway.swaml

```
package Gateway {
    interface senderReceiver Samples version "1.0.0" { data value: real sampled; }
    interface service Telemetry version "1.0.0" {
        function read(): real idempotent;
        event observed(value: real);
    }
    component def Sensor {
        port output: Samples sender;
        function sample() on every 10 ms timing { wcet 1 ms; deadline 2 ms; concurrency serial; } {
            send output.value(21.0); return;
        }
    }
    component def Adapter {
        port input: Samples receiver;
        port api: Telemetry provides;
        var latest: real = 0.0;
        function receiveSample(value: real) on receive input.value timing { wcet 1 ms; deadline 3 ms;
concurrency serial; } {
            latest = value;
            publish api.observed(value);
            return;
        }
        function readLatest(): real on call api.read timing { wcet 1 ms; deadline 3 ms; concurrency
serial; } { return latest; }
    }
    component def Dashboard {
        port telemetry: Telemetry requires;
        var displayed: real = 0.0;
        function refresh() on manual timing { wcet 1 ms; deadline 30 ms; concurrency serial; } {
            call telemetry.read() as measured within 25 ms otherwise { return; };
            displayed = measured; return;
        }
        function update(value: real) on receive telemetry.observed timing { wcet 1 ms; deadline 5 ms;
concurrency serial; } {
            displayed = value; return;
        }
    }
    component def Acquisition {
        port output: Samples sender;
        component sensor: Sensor;
        delegate output -> sensor.output;
    }
    component def Bridge {
        port input: Samples receiver;
        port api: Telemetry provides;
        component adapter: Adapter;
        delegate input -> adapter.input;
        delegate api -> adapter.api;
    }
    component def Display {
        port telemetry: Telemetry requires;
        component dashboard: Dashboard;
        delegate telemetry -> dashboard.telemetry;
    }
    component Product mixed {
        component acquisition: Acquisition;
        component bridge: Bridge;
        component display: Display;
        connect sensing: acquisition.output -> bridge.input qos {
            delivery atMostOnce; ordering fifo; capacity 1; overflow dropOldest; maxLatency 1 ms; maxAge
 20 ms;
        };
        connect observing: display.telemetry -> bridge.api qos {
            delivery atMostOnce; ordering fifo; capacity 8; overflow reject; maxLatency 2 ms;
        };
```

SWAML / 06-mixed-gateway.swaml / CONTINUED

```
        deploy acquisition on "system:controller";
        deploy bridge on "system:edge";
        deploy display on "system:edge";
    }
    view overview gv of Product;
    view connections iv of Product;
    view conversion fv of Adapter.receiveSample;
    view allocation grv of Product;
}
```

07 client server

SWAML / 07-client-server.swaml

```
package Rpc {
    interface clientServer Health version "1.0.0" {
        function check(): bool idempotent;
    }
    component def Server {
        port health: Health server;
        function checkHealth(): bool on call health.check { return true; }
    }
    component def Client {
        port health: Health client;
        var healthy: bool = false;
        function poll() on every 1 s {
            call health.check() as statusValue within 100 ms otherwise { healthy = false; return; };
            healthy = statusValue;
            return;
        }
    }
    component def Service {
        port health: Health server;
        component serverUnit: Server;
        delegate health -> serverUnit.health;
    }
    component def Caller {
        port health: Health client;
        component clientUnit: Client;
        delegate health -> clientUnit.health;
    }
    component HealthCheck {
        component caller: Caller;
        component serviceUnit: Service;
        connect healthCheck: caller.health -> serviceUnit.health qos {
            delivery atLeastOnce; ordering fifo; capacity 4; overflow reject;
        };
    }
    useCase Poll of HealthCheck {
        call caller.health.check -> serviceUnit.health.check via healthCheck label sent;
        reply serviceUnit.health.check -> caller.health.check via healthCheck after sent label answered;
    }
    view interaction sv of Poll;
    view requestFlow fv of Client.poll;
}
```

08 service field

SWAML / 08-service-field.swaml

```
package Settings {
    // Reuse policy values while each connector retains independent queues.
    qos def LocalLink {
        delivery atMostOnce; ordering fifo; capacity 8; overflow reject;
    }
    // A remotely visible property is an explicit getter, setter and event contract.
    interface service TemperatureSetting version "1.0.0" {
        function get(): int idempotent;
        function setPoint(desired: int): bool;
        event changed(current: int);
    }
    component def SettingStore {
        port api: TemperatureSetting provides;
        var currentValue: int = 20;
        // Each reentrant query sees its dispatch-time snapshot.
        function read(): int on call api.get timing {
            wcet 1 ms; deadline 5 ms; concurrency reentrant;
        } { return currentValue; }
        function update(desired: int): bool on call api.setPoint {
            if (desired < 10 or desired > 30) { return false; }
            currentValue = desired;
            publish api.changed(currentValue);
            return true;
        }
    }
    component def Panel {
        port setting: TemperatureSetting requires;
        var displayed: int = 20;
        function adjust() on manual {
            call setting.setPoint(22) as accepted within 100 ms otherwise { fail "setting unavailable";
};
            assert accepted;
        }
        function refresh() on manual {
            call setting.get() as observed within 100 ms otherwise { return; };
            displayed = observed;
        }
        function display(current: int) on receive setting.changed { displayed = current; }
    }
    component def Service {
        port api: TemperatureSetting provides;
        component store: SettingStore;
        delegate api -> store.api;
    }
    component def Client {
        port setting: TemperatureSetting requires;
        component panel: Panel;
        delegate setting -> panel.setting;
    }
    component Cabin {
        component settings: Service;
        component front: Client;
        component rear: Client;
        connect frontLink: front.setting -> settings.api qos LocalLink;
        connect rearLink: rear.setting -> settings.api qos LocalLink;
        deploy settings on "system:controller";
        deploy front on "system:displays";
        deploy rear on "system:displays";
    }
    requirement VisibleSetting { doc /* Both panels can observe accepted temperature setting updates. */
 }
    satisfy VisibleSetting of Cabin; useCase Change of Cabin {
        call front.setting.setPoint -> settings.api.setPoint via frontLink label adjust;
        publish settings.api.changed -> front.setting.changed via frontLink after adjust label
frontNotice;
```

SWAML / 08-service-field.swaml / CONTINUED

```
        publish settings.api.changed -> rear.setting.changed via rearLink after adjust label rearNotice;
        reply settings.api.setPoint -> front.setting.setPoint via frontLink after frontNotice label
accepted;
    }
    view catalog grv of Settings;
    view overview gv of Cabin;
    view wiring iv of Cabin;
    view coverage cv of Cabin;
    view exchanges sv of Change;
    view updateFlow fv of Cabin.settings.store.update;
}
```

09 component roots

SWAML / 09-component-roots.swaml

```
package Roots {
    component def Counter {
        var count: int = 0;
        function initialize() on start
            timing { wcet 1 us; deadline 1 ms; concurrency serial; }
        {
            count = count + 1;
        }
    }

    // A reusable type defines topology. A typed root creates a concrete occurrence.
    component def Pair {
        component left: Counter;
        component right: Counter;
    }
    component Demo: Pair;
    component Embedded: Pair realtime {
        deploy left on "system:cpu";
        deploy right on "system:cpu";
    }

    // An inline root may contain a component directly, without a wrapper definition.
    component Minimal { component counter: Counter; }

    view reusable gv of Pair;
    view deployed iv of Embedded;
    view initializeFlow fv of Minimal.counter.initialize;
    view inventory grv of Demo;
}
```

10 reusable qos

SWAML / 10-reusable-qos.swaml

```
package ReusablePolicies {
    // Policy reuse shares values, while each connector keeps independent queues.
    qos def LocalEvents {
        delivery atMostOnce;
        ordering fifo;
        capacity 2;
        overflow reject;
    }
    interface publishSubscribe Updates version "1.0.0" {
        event changed(value: int);
    }
    component def Producer {
        port output: Updates publisher;
        function announce() on start { publish output.changed(7); }
    }
    component def Consumer {
        port input: Updates subscriber;
        var latest: int = 0;
        function observe(value: int) on receive input.changed { latest = value; }
    }
    component def Pair {
        component producer: Producer;
        component first: Consumer;
        component second: Consumer;
        connect firstUpdate: producer.output -> first.input qos LocalEvents;
        connect secondUpdate: producer.output -> second.input qos LocalEvents;
    }
    component Demo: Pair;
    view overview gv of Demo;
    view wiring iv of Demo;
    view policy grv of LocalEvents;
}
```

11 autosar classic

SWAML / 11-autosar-classic.swaml

```
package ClassicClimate {
    // Logical Classic SW-C architecture; platform-specific RTE/OS bindings are separate.
    interface senderReceiver CabinTemperature version "1.0.0" {
        data measured: real sampled;
    }
    interface senderReceiver FanCommand version "1.0.0" {
        data duty: real sampled;
    }
    interface clientServer Setpoint version "1.0.0" {
        function read(): real idempotent;
    }
    qos def LatestSample {
        delivery atMostOnce; ordering fifo; capacity 1; overflow dropOldest;
        maxLatency 1 ms; maxAge 20 ms;
    }
    qos def CalibrationCall {
        delivery atMostOnce; ordering fifo; capacity 4; overflow reject;
        maxLatency 1 ms;
    }
    component def TemperatureSensor {
        port output: CabinTemperature sender;
        function sample() on every 10 ms timing {
            wcet 1 ms; deadline 2 ms; concurrency serial;
        } { send output.measured(28.0); }
    }
    component def ClimateLogic {
        port temperature: CabinTemperature receiver;
        port fan: FanCommand sender;
        port calibration: Setpoint client;
        var lastTemperature: real = 0.0;
        var commandedDuty: real = 0.0;
        function regulate(value: real) on receive temperature.measured timing { wcet 1 ms; deadline 8
ms; concurrency serial; } {
            calc proportional { in error: real; return result: real = error / 10.0; }
            lastTemperature = value;
            call calibration.read() as target within 5 ms otherwise {
                commandedDuty = 0.0;
                send fan.duty(commandedDuty);
                return;
            };
            var calculated: real = proportional(value - target);
            if (calculated < 0.0) { commandedDuty = 0.0; }
            else {
                if (calculated > 1.0) { commandedDuty = 1.0; }
                else { commandedDuty = calculated; }
            }
            send fan.duty(commandedDuty);
        }
    }
    component def SetpointCalibration {
        port api: Setpoint server;
        var desiredTemperature: real = 22.0;
        function readSetpoint(): real on call api.read timing {
            wcet 1 ms; deadline 2 ms; concurrency reentrant;
        } { return desiredTemperature; }
    }
    component def FanActuator {
        port input: FanCommand receiver;
        var appliedDuty: real = 0.0;
        function apply(value: real) on receive input.duty timing {
            wcet 1 ms; deadline 2 ms; concurrency serial;
        } {
            assert value >= 0.0 and value <= 1.0;
            appliedDuty = value;
        }
```

SWAML / 11-autosar-classic.swaml / CONTINUED

```
    }
    component def ClimateController {
        port temperature: CabinTemperature receiver;
        port fan: FanCommand sender;
        port calibration: Setpoint client;
        component control: ClimateLogic;
        delegate temperature -> control.temperature;
        delegate fan -> control.fan;
        delegate calibration -> control.calibration;
    }
    component Vehicle realtime {
        component sensor: TemperatureSensor;
        component controller: ClimateController;
        component calibration: SetpointCalibration;
        component actuator: FanActuator;
        connect measured: sensor.output -> controller.temperature qos LatestSample;
        connect calibrated: controller.calibration -> calibration.api qos CalibrationCall;
        connect commanded: controller.fan -> actuator.input qos LatestSample;
        deploy sensor on "system:bodyEcu";
        deploy controller on "system:bodyEcu";
        deploy calibration on "system:bodyEcu";
        deploy actuator on "system:bodyEcu";
    }
    requirement BoundedFan { doc /* The climate SW-C commands a duty in [0, 1] using the current setpoint. */ }
    satisfy BoundedFan of Vehicle; useCase RegulateCabin of Vehicle {
        send sensor.output.measured -> controller.temperature.measured via measured label measurement;
        call controller.calibration.read -> calibration.api.read via calibrated after measurement label readTarget;
        reply calibration.api.read -> controller.calibration.read via calibrated after readTarget label target;
        send controller.fan.duty -> actuator.input.duty via commanded after target label command;
    }
    view overview gv of Vehicle;
    view contracts iv of Vehicle;
    view requirements cv of Vehicle;
    view regulation sv of RegulateCabin;
    view allocation grv of Vehicle;
    view controlFlow fv of Vehicle.controller.control.regulate;
}
```

12 autosar adaptive

SWAML / 12-autosar-adaptive.swaml

```
package AdaptiveDiagnostics {
    // A resolved ara::com-style service topology with explicit methods and notification.
    datatype HealthReport { code: int; healthy: bool; }
    interface service VehicleHealth version "1.0.0" {
        function getSummary(): HealthReport idempotent;
        function runCheck(): bool;
        event changed(report: HealthReport);
    }
    interface publishSubscribe DiagnosticAudit version "1.0.0" {
        event checked(code: int);
    }
    qos def ServiceCall {
        delivery atMostOnce; ordering fifo; capacity 8; overflow reject;
    }
    qos def AuditDelivery {
        delivery atMostOnce; ordering fifo; capacity 16; overflow reject;
    }
    component def HealthMonitor {
        port api: VehicleHealth provides;
        port audit: DiagnosticAudit publisher;
        var report: HealthReport = record HealthReport { code = 0; healthy = false; };
        var completedChecks: int = 0;
        function summary(): HealthReport on call api.getSummary {
            return report;
        }
        function check(): bool on call api.runCheck {
            report = record HealthReport { code = 200; healthy = true; };
            completedChecks = completedChecks + 1;
            publish api.changed(report);
            publish audit.checked(report.code);
            return true;
        }
    }
    component def DiagnosticClient {
        port health: VehicleHealth requires;
        var observedHealthy: bool = false;
        var lastCode: int = 0;
        var notifications: int = 0;
        function inspect() on start {
            call health.runCheck() as accepted within 100 ms otherwise {
                fail "health check unavailable";
            };
            assert accepted;
            call health.getSummary() as current within 100 ms otherwise {
                fail "health summary unavailable";
            };
            observedHealthy = current.healthy;
            lastCode = current.code;
        }
        function changed(report: HealthReport) on receive health.changed {
            observedHealthy = report.healthy;
            lastCode = report.code;
            notifications = notifications + 1;
        }
    }
    component def DiagnosticLog {
        port input: DiagnosticAudit subscriber;
        var lastCode: int = 0;
        var entries: int = 0;
        function append(code: int) on receive input.checked {
            lastCode = code;
            entries = entries + 1;
        }
    }
    component def DiagnosticApplication {
```

SWAML / 12-autosar-adaptive.swaml / CONTINUED

```
        port health: VehicleHealth requires;
        component diagnostic: DiagnosticClient;
        delegate health -> diagnostic.health;
    }
    component def HealthService {
        port api: VehicleHealth provides;
        port audit: DiagnosticAudit publisher;
        component monitor: HealthMonitor;
        delegate api -> monitor.api;
        delegate audit -> monitor.audit;
    }
    component Vehicle {
        component application: DiagnosticApplication;
        component health: HealthService;
        component audit: DiagnosticLog;
        connect healthLink: application.health -> health.api qos ServiceCall;
        connect auditLink: health.audit -> audit.input qos AuditDelivery;
        deploy application on "system:applicationProcess";
        deploy health on "system:healthProcess";
        deploy audit on "system:auditProcess";
    }
    requirement ObservableHealth { doc /* A completed health check exposes its result to the application and audit log. */ }
    satisfy ObservableHealth of Vehicle; useCase InspectVehicle of Vehicle {
        call application.health.runCheck -> health.api.runCheck via healthLink label inspect;
        publish health.api.changed -> application.health.changed via healthLink after inspect label changed;
        publish health.audit.checked -> audit.input.checked via auditLink after changed label audited;
        reply health.api.runCheck -> application.health.runCheck via healthLink after audited label complete;
        call application.health.getSummary -> health.api.getSummary via healthLink after complete label readSummary;
        reply health.api.getSummary -> application.health.getSummary via healthLink after readSummary label summary;
    }
    view overview gv of Vehicle;
    view contracts iv of Vehicle;
    view requirements cv of Vehicle;
    view inspection sv of InspectVehicle;
    view allocation grv of Vehicle;
    view inspectionFlow fv of Vehicle.application.diagnostic.inspect;
}
```

13 web application

SWAML / 13-web-application.swaml

```
package WebShop {
    interface service Checkout version "1.0.0" {
        function submit(orderId: string, quantity: int): bool;
        event accepted(orderId: string);
    }
    interface clientServer Inventory version "1.0.0" {
        function reserve(quantity: int): bool;
    }
    interface publishSubscribe Receipts version "1.0.0" {
        event created(orderId: string);
    }
    qos def RequestQueue {
        delivery atMostOnce; ordering fifo; capacity 16; overflow reject;
    }
    qos def ReceiptQueue {
        delivery atMostOnce; ordering fifo; capacity 32; overflow reject;
    }
    component def BrowserClient {
        port checkout: Checkout requires;
        var lastOrder: string = "";
        var accepted: bool = false;
        var rejectedOversize: bool = false;
        function buy() on start {
            call checkout.submit("order-1042", 2) as result within 1 s otherwise {
                fail "checkout unavailable";
            };
            accepted = result;
        }
        function overOrder() on manual {
            call checkout.submit("order-1043", 99) as result within 1 s otherwise {
                fail "checkout unavailable";
            };
            rejectedOversize = not result;
            assert rejectedOversize;
        }
        function confirmation(orderId: string) on receive checkout.accepted {
            lastOrder = orderId;
        }
    }
    component def CheckoutService {
        port api: Checkout provides;
        port inventory: Inventory client;
        port receipts: Receipts publisher;
        var acceptedOrders: int = 0;
        function submit(orderId: string, quantity: int): bool on call api.submit {
            call inventory.reserve(quantity) as reserved within 200 ms otherwise {
                return false;
            };
            if (not reserved) { return false; }
            acceptedOrders = acceptedOrders + 1;
            publish api.accepted(orderId);
            publish receipts.created(orderId);
            return true;
        }
    }
    component def StockStore {
        port api: Inventory server;
        var available: int = 10;
        function reserve(quantity: int): bool on call api.reserve {
            if (quantity <= 0 or quantity > available) { return false; }
            available = available - quantity;
            return true;
        }
    }
    component def ReceiptWorker {
```

SWAML / 13-web-application.swaml / CONTINUED

```
        port input: Receipts subscriber;
        var lastOrder: string = "";
        var sent: int = 0;
        function prepareReceipt(orderId: string) on receive input.created {
            lastOrder = orderId;
            sent = sent + 1;
        }
    }
    component def WebApplication {
        port api: Checkout provides;
        port inventory: Inventory client;
        port receipts: Receipts publisher;
        component checkout: CheckoutService;
        delegate api -> checkout.api;
        delegate inventory -> checkout.inventory;
        delegate receipts -> checkout.receipts;
    }
    component def InventoryService {
        port api: Inventory server;
        component stock: StockStore;
        delegate api -> stock.api;
    }
    component Production {
        component browser: BrowserClient;
        component application: WebApplication;
        component inventory: InventoryService;
        component mailer: ReceiptWorker;
        connect checkoutLink: browser.checkout -> application.api qos RequestQueue;
        connect inventoryLink: application.inventory -> inventory.api qos RequestQueue;
        connect receiptLink: application.receipts -> mailer.input qos ReceiptQueue;
        deploy browser on "system:clientProcess";
        deploy application on "system:applicationContainer";
        deploy inventory on "system:inventoryContainer";
        deploy mailer on "system:workerContainer";
    }
    requirement ReserveBeforeAccept { doc /* An accepted order reserves stock before its confirmation
and receipt are published. */ }
    satisfy ReserveBeforeAccept of Production; useCase Purchase of Production {
        call browser.checkout.submit -> application.api.submit via checkoutLink label checkout;
        call application.inventory.reserve -> inventory.api.reserve via inventoryLink after checkout
label reserve;
        reply inventory.api.reserve -> application.inventory.reserve via inventoryLink after reserve
label stock;
        publish application.api.accepted -> browser.checkout.accepted via checkoutLink after stock label
 confirmation;
        publish application.receipts.created -> mailer.input.created via receiptLink after confirmation
label receipt;
        reply application.api.submit -> browser.checkout.submit via checkoutLink after receipt label
result;
    }
    view overview gv of Production;
    view contracts iv of Production;
    view requirements cv of Production;
    view purchase sv of Purchase;
    view allocation grv of Production;
    view checkoutFlow fv of Production.application.checkout.submit;
    verificationCase CheckPurchase of Production {
        doc /* Run the purchase simulation; inspect reserved stock, confirmation and receipt. */
        verify ReserveBeforeAccept;
    }
}
```

14 cloud service

SWAML / 14-cloud-service.swaml

```
package CloudTelemetry {
    datatype Sample { deviceId: string; sequence: int; temperature: real; }
    interface service TelemetryIngest version "1.0.0" {
        function submit(sample: Sample): bool;
    }
    interface publishSubscribe AcceptedSamples version "1.0.0" {
        event accepted(sample: Sample);
    }
    interface publishSubscribe Measurements version "1.0.0" {
        event recorded(sample: Sample);
    }
    qos def IngressQueue {
        delivery atMostOnce; ordering fifo; capacity 8; overflow reject;
    }
    qos def EventQueue {
        delivery atLeastOnce; ordering fifo; capacity 32; overflow reject;
    }
    component def DeviceClient {
        port ingest: TelemetryIngest requires;
        var uploaded: int = 0;
        var retryAccepted: bool = false;
        var invalidRejected: bool = false;
        function upload() on start {
            call ingest.submit(record Sample { deviceId = "sensor-01"; sequence = 1; temperature = 27.0; })
                as first within 1 s otherwise { fail "ingestion unavailable"; };
            assert first;
            uploaded = uploaded + 1;
            call ingest.submit(record Sample { deviceId = "sensor-01"; sequence = 2; temperature = 28.5; })
                as second within 1 s otherwise { fail "ingestion unavailable"; };
            assert second;
            uploaded = uploaded + 1;
        }
        function retry() on manual {
            call ingest.submit(record Sample { deviceId = "sensor-01"; sequence = 2; temperature = 28.5; })
                as result within 1 s otherwise { fail "retry unavailable"; };
            retryAccepted = result;
            assert retryAccepted;
        }
        function sendInvalid() on manual {
            call ingest.submit(record Sample { deviceId = "sensor-01"; sequence = 3; temperature = 200.0; })
                as result within 1 s otherwise { fail "ingestion unavailable"; };
            invalidRejected = not result;
            assert invalidRejected;
        }
    }
    component def IngestionGateway {
        port api: TelemetryIngest provides;
        port accepted: AcceptedSamples publisher;
        var lastSequence: int = 0;
        var acceptedCount: int = 0;
        function ingest(sample: Sample): bool on call api.submit {
            if (sample.deviceId != "sensor-01" or sample.sequence <= 0 or
                sample.temperature < -50.0 or sample.temperature > 100.0) { return false; }
            // This example admits one ordered device stream, with an in-memory high watermark.
            if (sample.sequence <= lastSequence) { return true; }
            publish accepted.accepted(sample);
            lastSequence = sample.sequence;
            acceptedCount = acceptedCount + 1;
            return true;
        }
    }
```

SWAML / 14-cloud-service.swaml / CONTINUED

```
    component def MessageBroker {
        port input: AcceptedSamples subscriber;
        port output: Measurements publisher;
        var lastSequence: int = 0;
        var forwarded: int = 0;
        function route(sample: Sample) on receive input.accepted {
            if (sample.sequence <= lastSequence) { return; }
            publish output.recorded(sample);
            lastSequence = sample.sequence;
            forwarded = forwarded + 1;
        }
    }
    component def AnalyticsWorker {
        port input: Measurements subscriber;
        var lastSequence: int = 0;
        var processed: int = 0;
        var latestTemperature: real = 0.0;
        function aggregate(sample: Sample) on receive input.recorded {
            if (sample.sequence <= lastSequence) { return; }
            lastSequence = sample.sequence;
            processed = processed + 1;
            latestTemperature = sample.temperature;
        }
    }
    component def ArchiveWriter {
        port input: Measurements subscriber;
        var lastSequence: int = 0;
        var records: int = 0;
        var lastDevice: string = "";
        function append(sample: Sample) on receive input.recorded {
            if (sample.sequence <= lastSequence) { return; }
            lastSequence = sample.sequence;
            records = records + 1;
            lastDevice = sample.deviceId;
        }
    }
    component def IngestionService {
        port api: TelemetryIngest provides;
        port accepted: AcceptedSamples publisher;
        component gateway: IngestionGateway;
        delegate api -> gateway.api;
        delegate accepted -> gateway.accepted;
    }
    component Production {
        component device: DeviceClient;
        component ingestion: IngestionService;
        component broker: MessageBroker;
        component analytics: AnalyticsWorker;
        component archive: ArchiveWriter;
        connect ingestionLink: device.ingest -> ingestion.api qos IngressQueue;
        connect brokerLink: ingestion.accepted -> broker.input qos EventQueue;
        connect analyticsLink: broker.output -> analytics.input qos EventQueue;
        connect archiveLink: broker.output -> archive.input qos EventQueue;
        deploy device on "system:edgeProcess";
        deploy ingestion on "system:apiContainer";
        deploy broker on "system:brokerContainer";
        deploy analytics on "system:analyticsContainer";
        deploy archive on "system:archiveContainer";
    }
    requirement ValidatedTelemetry { doc /* Accepted samples from the ordered device stream reach analytics and archive without duplicate local processing. */ }
    satisfy ValidatedTelemetry of Production; useCase IngestSample of Production {
        call device.ingest.submit -> ingestion.api.submit via ingestionLink label upload;
        publish ingestion.accepted.accepted -> broker.input.accepted via brokerLink after upload label admitted;
        publish broker.output.recorded -> analytics.input.recorded via analyticsLink after admitted label analytics;
```

SWAML / 14-cloud-service.swaml / CONTINUED

```
        publish broker.output.recorded -> archive.input.recorded via archiveLink after admitted label archived;
        reply ingestion.api.submit -> device.ingest.submit via ingestionLink after admitted label acknowledged;
    }
    view overview gv of Production;
    view contracts iv of Production;
    view requirements cv of Production;
    view ingestion sv of IngestSample;
    view allocation grv of Production;
    view validationFlow fv of Production.ingestion.gateway.ingest;
}
```

15 component specialization

SWAML / 15-component-specialization.swaml

```
package ComponentReuse {
    component def Worker {
        var ticks: uint = 0;
        function boot() on start { call work(); }
        function work() { ticks = ticks + 1; }
    }
    component def FastWorker specializes Worker {
        redefines function work() { ticks = ticks + 2; }
    }

    component def Device {
        var count: uint = 0;
        var sample: byte = 0;
        component worker: Worker;
        function encode(value: uint): byte {
            calc offset { in inputValue: uint; return octet: byte = inputValue + 1; }
            return offset(value);
        }
        function step() { count = count + 1; call encode(count) as encoded; sample = encoded; }
        function boot() on start { call step(); }
        function advance() on manual { call step(); }
    }
    component def SpecializedDevice specializes Device {
        redefines var sample: byte = 10;
        redefines component worker: FastWorker;
        redefines function encode(value: uint): byte {
            calc offset { in inputValue: uint; return octet: byte = inputValue + 10; }
            return offset(value);
        }
        redefines function step() { count = count + 2; call encode(count) as encoded; sample = encoded;
}
    }

    component Demo {
        var running: bool = false;
        function initialize() on start { running = true; }
        component base: Device;
        component left: SpecializedDevice;
        component right: SpecializedDevice;
        deploy Demo on "system:ReferenceHost";
        deploy left on "sysml:Vehicle::leftComputer";
        deploy right on "sysml:Vehicle::rightComputer";
    }
    view overview gv of Demo;
    view inventory grv of Demo;
    view inheritedStartup fv of Demo.left.boot;
    view specializedBehavior fv of Demo.left.step;
}
```

16 requirements and views

SWAML / 16-requirements-and-views.swaml

```
package Assurance {
    doc /* A compact architecture with explicit requirements and verification intent. */
    requirement BoundedResult {
        doc /* Every started worker shall store a result of 2. */
        attribute id: string = "REQ-001";
        attribute priority: uint = 1;
        attribute expected: int = 2;
    }
    component def Worker {
        doc /* Keep the result private to each worker instance. */
        // Each instance has its own result.
        var result: int = 0;
        function startWork() on start {
            doc /* Calculate twice the input and store the copied result. */
            calc twice {
                in input: int;
                return output: int = input * 2;
            }
            result = twice(1);
            assert result == 2;
        }
    }
    component Demo {
        doc /* Start two independent workers. */
        component first: Worker;
        component second: Worker;
    }
    satisfy BoundedResult of Demo.first;
    satisfy BoundedResult of Demo.second;
    useCase StartWorkers of Demo {
        doc /* A user starts the application and both workers initialize. */
    }
    verificationCase CheckResults of Demo {
        doc /* Run the reference simulation and inspect both final result values.
        Expected: first.result = 2 and second.result = 2. */
        verify BoundedResult;
    }
    view assurance gv {
        doc /* Requirements, design and verification intent in one projection. */
        include BoundedResult;
        include Demo recursive where kind == "component";
        include StartWorkers;
        include CheckResults;
    }
    view functions fv {
        include Demo recursive where kind == "function" and
            (name == "startWork" or path == "Demo.second.startWork");
    }
    view requirements grv {
        include Assurance recursive where kind == "requirement";
    }
}
```

17 function timing

SWAML / 17-function-timing.swaml

```
package FunctionTiming {
    interface clientServer Health version "1.0.0" {
        function check(): bool idempotent;
    }
    component def Client {
        port health: Health client;
        var healthy: bool = false;
        function poll() on manual timing {
            wcet 1 ms; deadline 30 ms; concurrency serial;
        } {
            call health.check() as answer within 20 ms otherwise { fail "request failed"; };
            healthy = answer;
        }
    }
    component def Server {
        port health: Health server;
        function checkHealth(): bool on call health.check timing {
            wcet 1 ms; deadline 10 ms; concurrency serial;
        } { return true; }
    }
    component Demo {
        component first: Client;
        component second: Client;
        component firstServer: Server;
        component secondServer: Server;
        connect firstLink: first.health -> firstServer.health qos {
            delivery atMostOnce; ordering fifo; capacity 4; overflow reject;
        };
        connect secondLink: second.health -> secondServer.health qos {
            delivery atMostOnce; ordering fifo; capacity 4; overflow reject;
        };
    }
    view functionTiming fv {
        include Demo recursive where kind == "function";
    }
}
```

18 control loops

SWAML / 18-control-loops.swaml

```
package ControlLoops {
    component Demo {
        doc /* Bounded control flow with lexical loop variables and copied array iteration. */
        var counted: int = 0;
        var total: int = 0;
        var postTested: int = 0;
        var nested: int = 0;
        var values: int[3] = [1, 2, 3];

        function run() on start {
            for (var i: int = 0; i < 3; i = i + 1) bound 3 {
                counted = counted + 1;
            }

            // Replacing the source does not change the copied iteration sequence.
            forEach (item: int in values) {
                total = total + item;
                values = [9, 9, 9];
            }

            do {
                postTested = postTested + 1;
            } while (postTested < 2) bound 2;

            for (var i: int = 0; i < 2; i = i + 1) bound 2 {
                forEach (item: int in [1, 2]) {
                    var j: int = 0;
                    while (j < 2) bound 2 {
                        nested = nested + item;
                        j = j + 1;
                    }
                }
            }

            assert counted == 3;
            assert total == 6;
            assert postTested == 2;
            assert nested == 12;
        }
    }
    view execution fv of Demo.run;
}
```

19 interruptible function

SWAML / 19-interruptible-function.swaml

```
package Interrupts {
    interface clientServer Job version "1.0.0" {
        function perform(): int;
    }
    interface publishSubscribe Control version "1.0.0" {
        event stop(reason: string);
    }
    component def Worker {
        port job: Job client;
        port control: Control subscriber;
        var status: int = 0;
        var why: string = "";
        function run() on start {
            interruptible work doc /* Stop waiting and handle cancellation on this typed route. */ {
                status = 1;
                call job.perform() as value within 50 ms otherwise { fail "Job failed"; };
                status = value;
            } on receive control.stop {
                status = 2;
                why = reason;
            }
            assert status == 2;
        }
    }
    component def Service {
        port api: Job server;
        var committed: int = 0;
        function perform(): int on call api.perform {
            committed = committed + 1;
            return 10;
        }
    }
    component def Controller {
        port control: Control publisher;
        function stop() on manual { publish control.stop("operator stop"); }
    }
    component App {
        component worker: Worker;
        component backend: Service;
        component controller: Controller;
        connect workRoute: worker.job -> backend.api qos {
            delivery atMostOnce; ordering fifo; capacity 4; overflow reject;
        };
        connect stopRoute: controller.control -> worker.control qos {
            delivery atMostOnce; ordering fifo; capacity 4; overflow reject;
        };
    }
    useCase StopWork of App {
        call worker.job.perform -> backend.api.perform via workRoute label started;
        publish controller.control.stop -> worker.control.stop via stopRoute after started label
cancelled;
    }
    view interrupted fv of App.worker.run;
}
```

catalog

SWAML / catalog.swaml

```
// Additional valid contexts for the canonical notation catalog.
package Shared {
    enum Mode { Idle, Running }
}
package Notation {
    import Shared::*;
    interface publishSubscribe Signals version "1.0.0" { event pulse(value: int); }
    interface senderReceiver Counts version "1.0.0" { data value: int queued; }
    interface clientServer Completion version "1.0.0" { function acknowledge(): void; }
    component def Source {
        port signal: Signals publisher;
        port output: Counts sender;
        function emit() on start {
            publish signal.pulse(1);
            send output.value(1);
        }
    }
    component def Controller {
        port signal: Signals subscriber;
        var count: int = 0;
        var mode: Mode = Shared::Mode.Idle;
        function lifecycle() on start {
            initial Idle;
            state Idle {
                entry { count = 0; }
                exit { assert count >= 0; }
            }
            state Active {}
            state Done final {}
            transition begin: Idle -> Active on receive signal.pulse when (value > 0) priority 1 { count
 = value; }
            transition skip: Idle -> Done on receive signal.pulse priority 0 {}
            transition finish: Active -> Done on completion when (count > 0) {}
            transition elapsed: Active -> Done on after 1 s {}
        }
    }
    component def Sink {
        port input: Counts receiver;
        var latest: int = 0;
        function accept(value: int) on receive input.value { latest = value; }
        function check() on manual {
            assert not false and (1 + 2 * 3 >= 7) or true;
            assert (5 % 2 == 1) and (4 - 2 != 0) and (8 / 2 <= 4.0);
        }
    }
    component Demo {
        component source: Source;
        component controller: Controller;
        component sink: Sink;
        connect events: source.signal -> controller.signal qos {
            delivery bestEffort; ordering unordered; capacity 2; overflow dropNewest;
            maxLatency 10 us; maxAge 1 ms;
        };
        connect counts: source.output -> sink.input qos {
            delivery atMostOnce; ordering fifo; capacity 4; overflow reject;
        };
    }
    requirement CountObserved { doc /* A sent count can reach the receiving function. */ }
    satisfy CountObserved of Demo; useCase Observe of Demo {
        send source.output.value -> sink.input.value via counts label sent;
        publish source.signal.pulse -> controller.signal.pulse via events after sent label notified;
    }
    view inventory grv of Notation;
    view overview gv of Demo;
    view interconnect iv of Demo;
```

SWAML / catalog.swaml / CONTINUED

```
    view cases cv of Demo;
    view sequence sv of Observe;
    view lifecycle fv of Controller.lifecycle;
}
package NotationOptions {
    interface publishSubscribe Notices version "1.0.0" { event changed(); }
    component def Emitter {
        port notices: Notices publisher optional;
        function emit() on manual { publish notices.changed(); }
    }
}
```

control flow

SWAML / control-flow.swaml

```
package ControlFlowNotation {
    component Counter {
        var counted: int = 0;
        var total: int = 0;
        var postTested: int = 0;
        var values: int[3] = [1, 2, 3];

        function countedLoop() {
            for (var i: int = 0; i < 3; i = i + 1) bound 3 {
                counted = counted + 1;
            }
        }

        function arrayLoop() {
            forEach (item: int in values) {
                total = total + item;
            }
        }

        function postTestLoop() {
            do {
                postTested = postTested + 1;
            } while (postTested < 2) bound 2;
        }

        function run() on start {
            call countedLoop();
            call arrayLoop();
            call postTestLoop();
        }
    }
    view counted fv of Counter.countedLoop;
    view elements fv of Counter.arrayLoop;
    view postTest fv of Counter.postTestLoop;
}
```

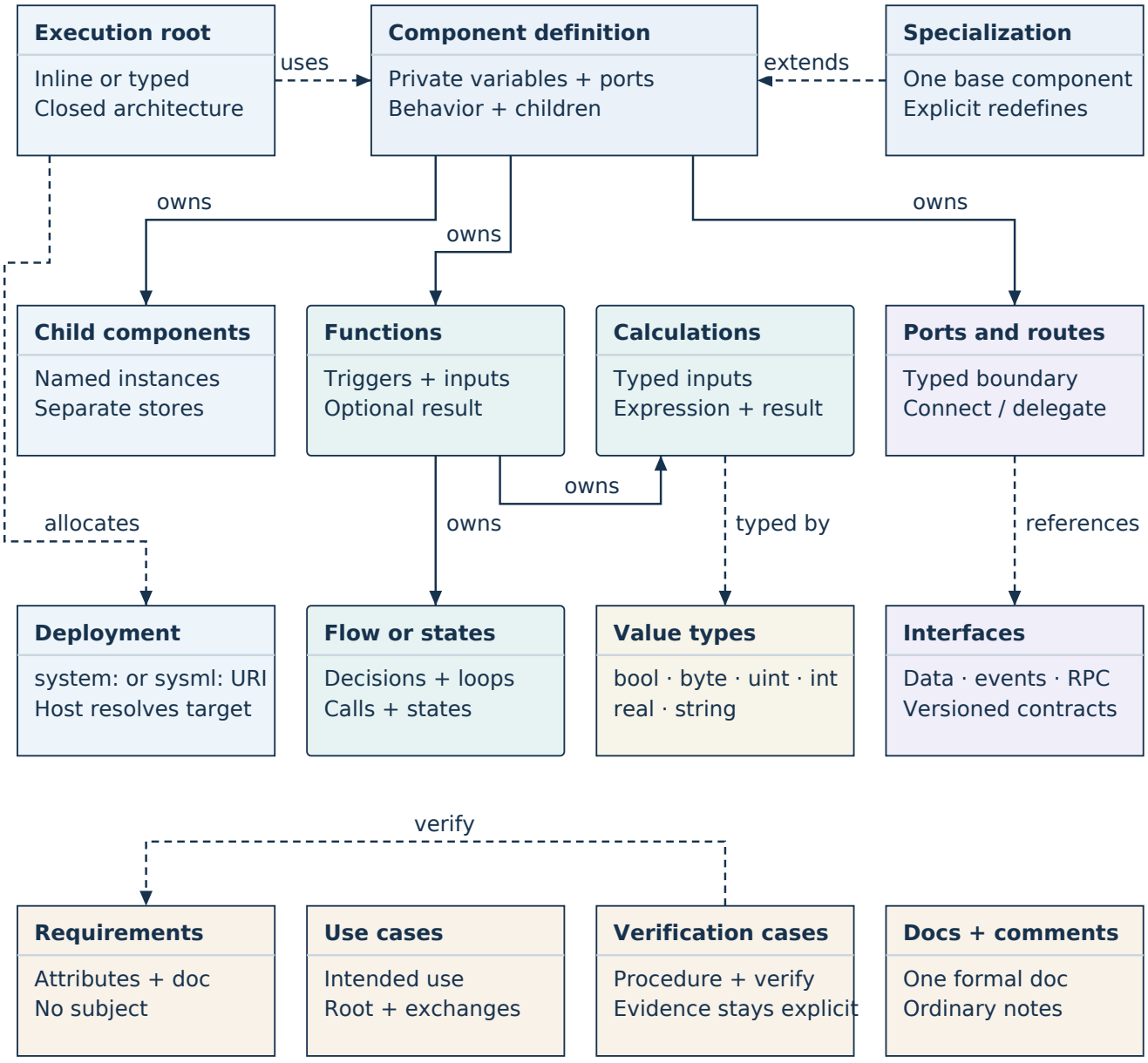
INFORMATIVE / GUIDANCE AND PROJECT STATUS

SWAML at a glance

SWAML models software composition, typed communication, function behavior, timing, deployment and traceable requirements. Follow one example from a buyer’s goal to the final software structure below. The interactive landing page presents the same graphs with selectable source details.

The graphical ontology

Definitions describe reusable types; occurrences identify their use in a concrete assembly. Components own child components, private state and functions. Functions own calculations and their flow or states; calculations turn typed inputs into a named pure result. Every cross-component exchange follows a typed port and explicit route.



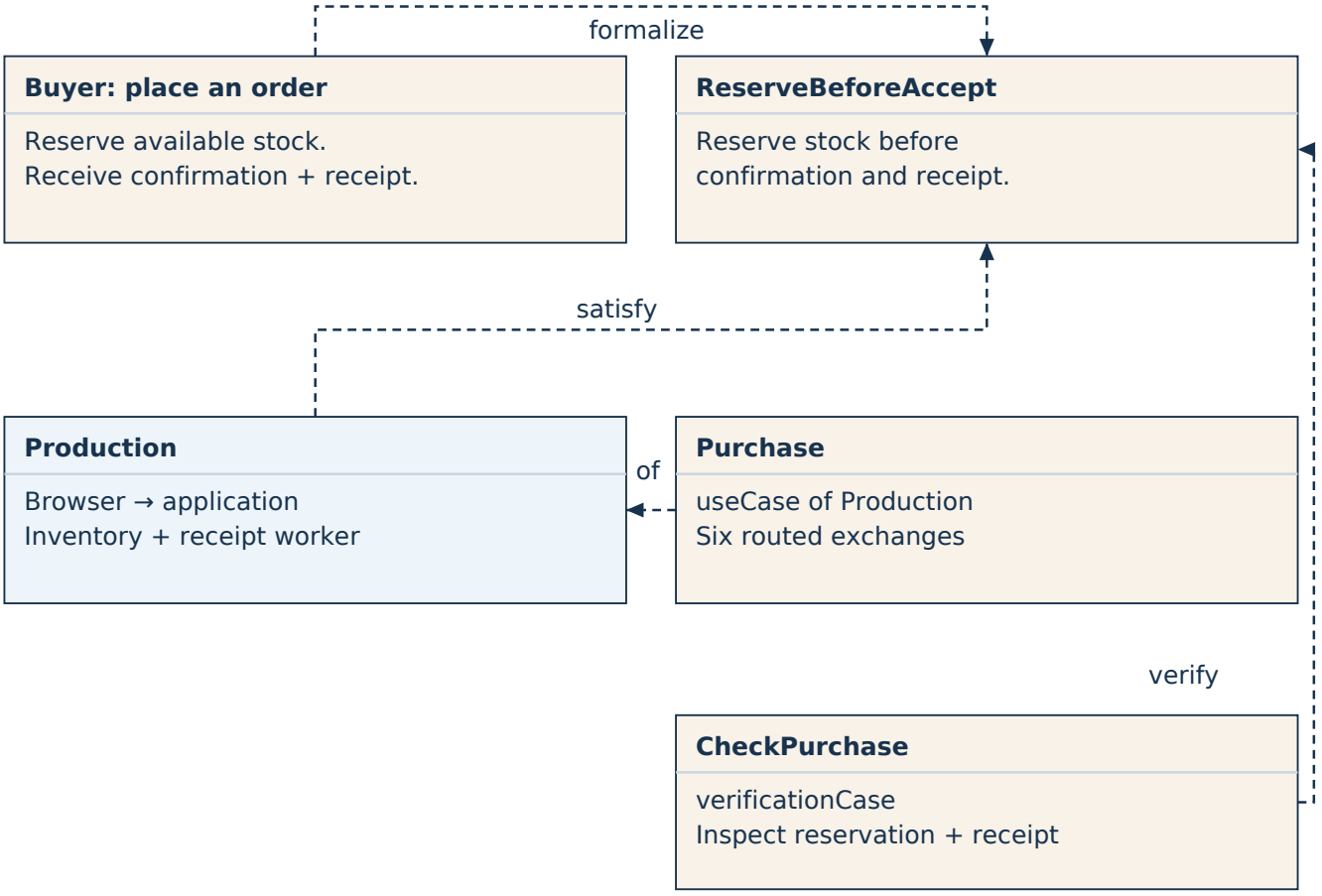
SWAML graphical ontology: definitions, occurrences, owned behavior, values, contracts, routes and traceable views

Solid ontology arrows indicate ownership or expansion; dashed arrows indicate references. The cards group related concepts for a quick introduction. The complete ontology and view guide and canonical notation catalog provide the full inventory.

Modeling question	SWAML elements	Useful view
What software exists, and what owns it?	component definitions and occurrences	Browser, gv
What crosses a boundary?	interfaces, ports, connectors, delegation, QoS	iv
Which outcome and scenario matter?	requirements, cases and routed exchange steps	cv, sv
What does the software do?	component-owned functions, statement flow or owned states	fv
What runs where, and under which constraints?	external system/part references, timing and concurrency	gv, grv

1. Begin with the use case

Buyer goal: place an order, reserve available stock, receive confirmation and a receipt. This user story motivates the named requirement `ReserveBeforeAccept`. The `Purchase` case references the `Production` root and claims coverage of that requirement.

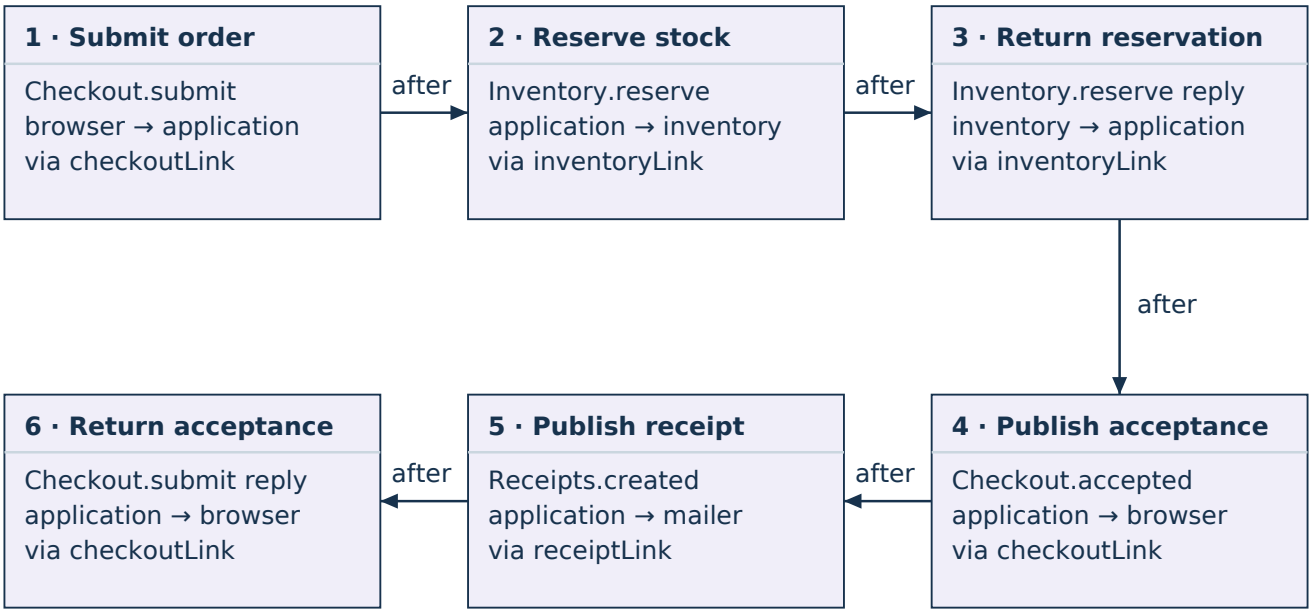


Buyer goal, ReserveBeforeAccept requirement, Purchase case and Production root

The buyer is explanatory context. `browser : BrowserClient` models the communicating client through an owned port. The `satisfy` relation records intent; supporting evidence is assessed separately.

2. Describe the message flow

The successful `Purchase` scenario requests checkout, reserves stock, receives the reservation result, publishes two events and returns the checkout result. Each step names its interface member and route; `after` labels define the intended order.

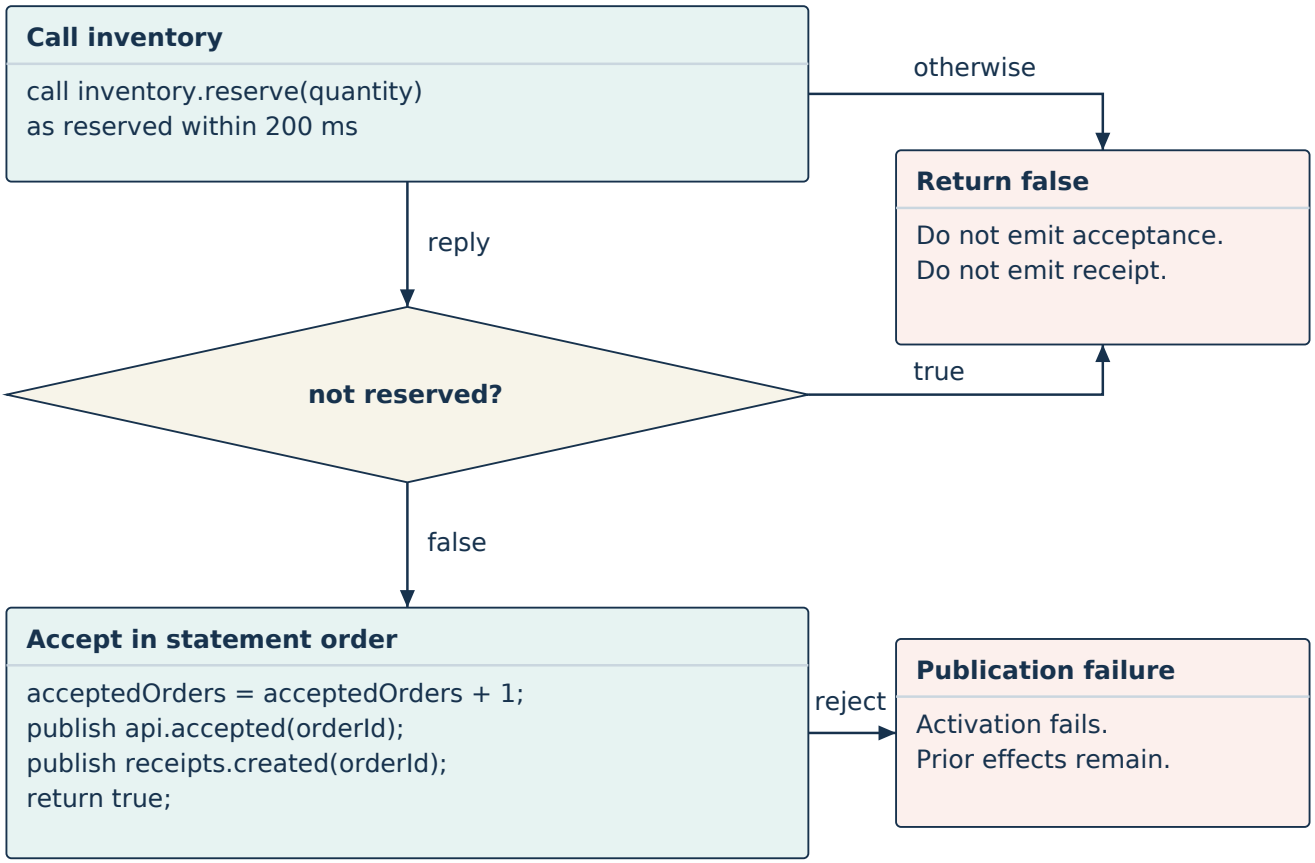


Six Purchase exchanges connected by their explicit after dependencies

These arrows connect scenario steps. The architectural connectors are `checkoutLink`, `inventoryLink` and `receiptLink`. Replies stay correlated with their requests; receipt and confirmation have their own receiving functions.

3. Open the function flow

`Production.application.checkout.submit` implements checkout. It requests inventory with a 200 ms timeout and returns `false` on request failure or refused stock. On acceptance, it updates its private counter, publishes acceptance, publishes a receipt and returns `true`.

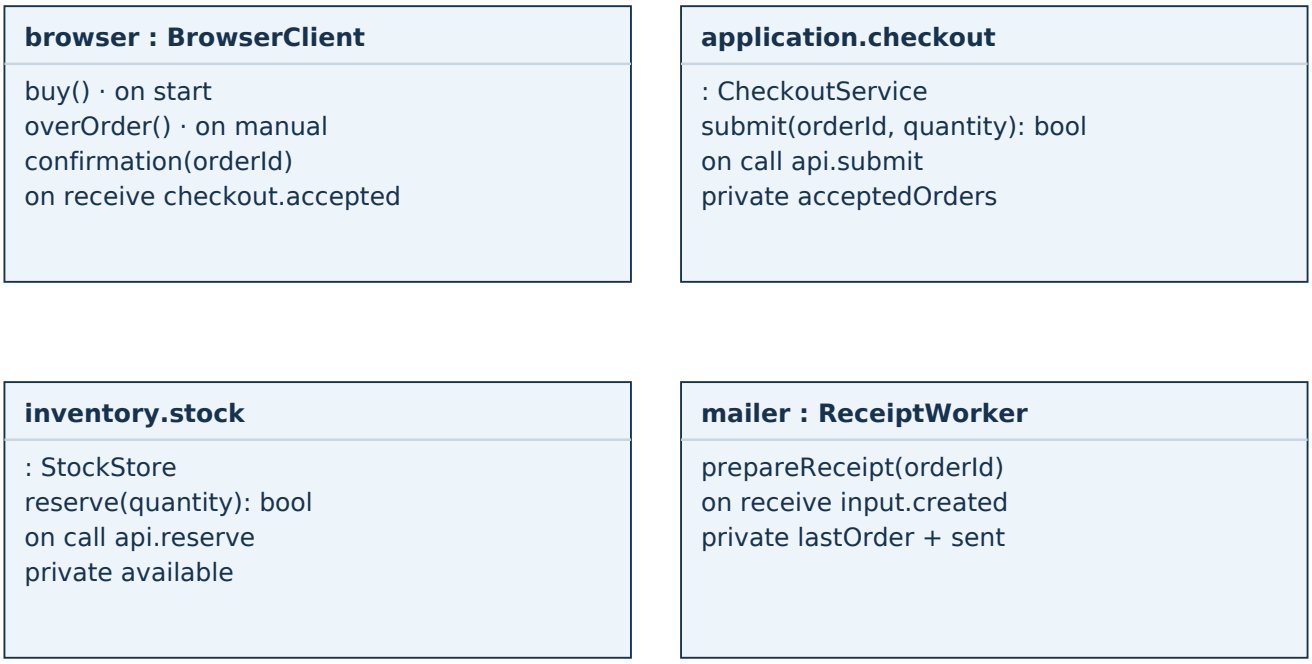


Checkout function control flow with reservation refusal, request failure and publication failure

Publication admission can fail after earlier effects have committed. The example has no distributed rollback. A timed-out inventory request also does not cancel stock changes that may already have occurred.

4. Assign clear software ownership

Four behavior-owning component instances contain six functions. `application` and `inventory` are component boundaries around their respective `checkout` and `stock` components; their boundary ports delegate to those components.



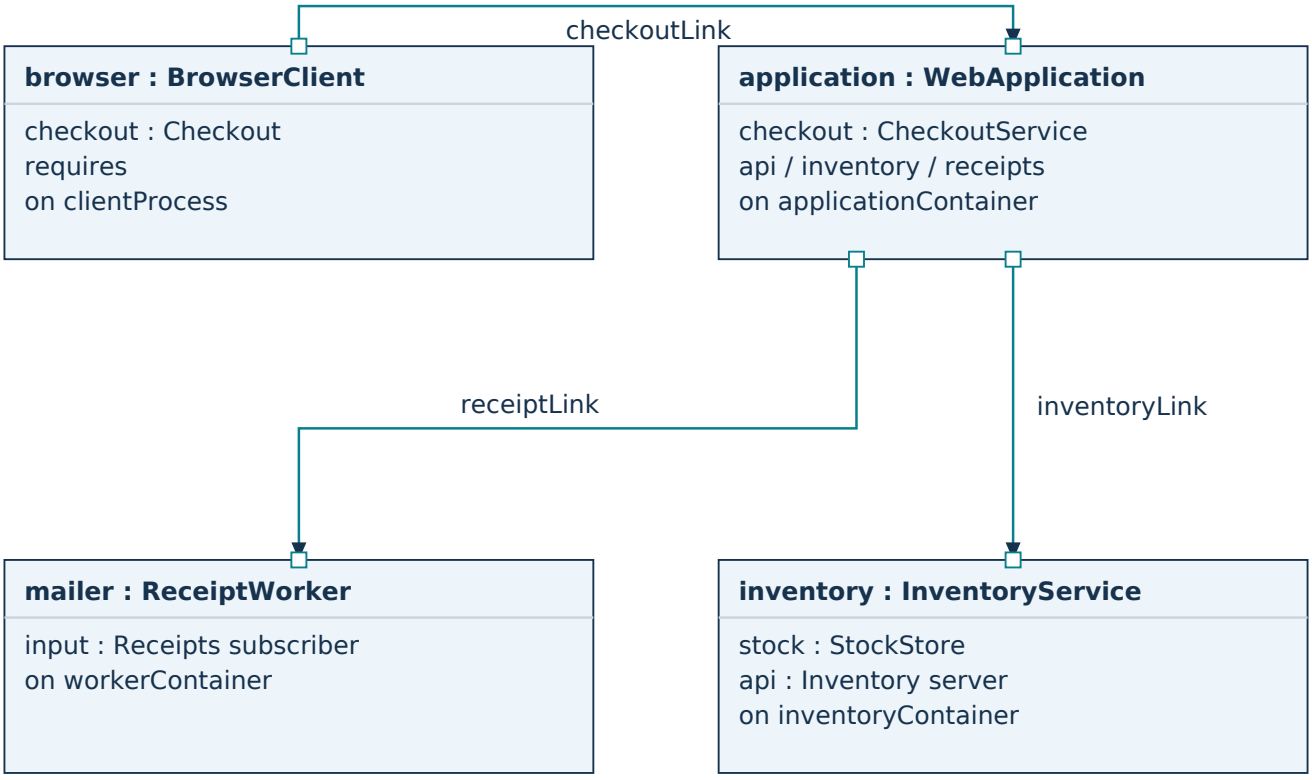
Component occurrence compartments listing owned functions, triggers and private data

Scenario concern	Actual behavior owner
Start the purchase	`Production.browser.buy`
Decide acceptance and emit both events	`Production.application.checkout.submit`
Check and decrement stock	`Production.inventory.stock.reserve`
Record the acceptance event	`Production.browser.confirmation`
Process the receipt event	`Production.mailer.prepareReceipt`
Exercise an oversized-order rejection	`Production.browser.overOrder`

The case references exchanges; it does not call these functions. Their declared triggers and routed messages activate them. For example, the browser's serial `buy` function can delay its `confirmation` handler until the checkout request completes.

5. Inspect the final structure

`WebShop::Production` contains four immediate child components. The application and inventory components each contain one further component, for six descendants in total. Three named connectors and four external deployment references complete the architecture.



Production software structure with browser, application and checkout, inventory and stock, receipt worker and three connectors

Connector	Port route	Contract and policy
`checkoutLink`	`browser.checkout` → `application.api`	`Checkout` service; `RequestQueue`
`inventoryLink`	`application.inventory` → `inventory.api`	`Inventory` client-server; `RequestQueue`
`receiptLink`	`application.receipts` → `mailer.input`	`Receipts` publish-subscribe; `ReceiptQueue`

Both policies use at-most-once delivery, FIFO ordering and rejecting overflow; capacities are 16 and 32 respectively. Selecting the same `RequestQueue` values leaves the two connectors' queues independent. Service events and RPC replies travel back over their existing route.

Try the complete model

Open the complete WebShop source, simulation configuration and expected final stores. The configured run accepts two items from stock 10, rejects an order for 99, records one accepted order and one receipt, and leaves stock at 8. This is bounded reference-simulation evidence; the descriptive case is not automatically a verified execution trace.

The same modeling steps apply to the Classic, Adaptive and cloud examples. Use the quick start for authoring and validation commands. These original flat graphs share one source inventory with the interactive presentation and reproducible SVG/PDF scenes.

Requirements and formal documentation

Requirements own typed attributes and one optional `doc /* ... */` specification. Design elements link through `satisfy`; verification cases link through `verify`. Use cases describe intent. Ordinary comments remain independent notes. View kinds use lower camel case, and view blocks combine multiple selections and Boolean filters. See the guide and complete model.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

SWAML in five minutes

Model a contract, give each participant a typed port, then connect their occurrences in a root component. Functions own behavior and each component occurrence owns its private values. You can use the same workflow for embedded software, service applications and cloud systems.

1. Define the exchanged contract

Start with the information crossing a boundary. A `senderReceiver` interface carries data, `clientServer` carries requests and results, `publishSubscribe` carries events, and `service` combines functions with events. Every port names one versioned interface and a compatible role.

2. Give behavior an owner

A `component def` owns ports, private variables and functions. The sender's function emits through its own port; the receiver's function names that port member as its trigger. Declare `calc` blocks first inside a function for pure input-to-output expressions.

3. Compose and connect

A `component def` can combine local behavior with reusable child composition. A top-level `component` is a concrete root. A connector joins complementary ports and selects complete QoS values. Define a shared policy with `qos def` when several routes use the same values. Each route retains its own queues.

Save this complete model as `hello.swaml`:

SWAML

```
package QuickStart {
    interface senderReceiver Greeting version "1.0.0" {
        data text: string queued;
    }
    component def Speaker {
        port output: Greeting sender;
        function greet() on start {
            send output.text("Hello, SWAML");
        }
    }
    component def Listener {
        port input: Greeting receiver;
        var last: string = "";
        function hear(message: string) on receive input.text {
            last = message;
        }
    }
    qos def LocalQueue {
        delivery atMostOnce; ordering fifo; capacity 4; overflow reject;
    }
    component Demo {
        component speaker: Speaker;
        component listener: Listener;
        connect greeting: speaker.output -> listener.input qos LocalQueue;
    }
    view structure gv of Demo;
    view wiring iv of Demo;
    view behavior fv of Speaker.greet;
}
```

`Speaker` and `Listener` are definitions; `Demo.speaker` and `Demo.listener` are occurrences. `last` belongs to the listener occurrence. The connector permits the exchange; the view declarations select existing model subjects.

4. Validate the model

From a clone of this repository, with Node.js 22+ and npm 10.2.3+ installed:

SH

```sh
npm ci
npm run build
node out/src/cli/main.js --language-version 1.3 hello.swaml
```

The validator checks syntax, contracts, ownership, routing, closure and applicable timing rules. Supply related files together when packages import declarations from other files. A successful validation establishes static acceptance; use the reference simulator for bounded execution evidence.

5. Review through views

Functions can reuse local behavior with `call`, and calculations stay inside their owning function:

SWAML

```
package LocalBehavior {
    component Demo {
        var latest: int = 0;
        function doubled(input: int): int {
            calc scale { in value: int; return result: int = value * 2; }
            return scale(input);
        }
        function boot() on start {
            call doubled(21) as answer;
            latest = answer;
        }
    }
}
```

Use `var` for both private component data and mutable block locals. Local void calls use `call helper();`. Routed calls use `call api.member(...) as result within 1 s otherwise { return; };` through an owned port and an explicit connector. Only calculations use expression-call syntax.

The graphical modeling overview connects a use case to its interaction flow, function logic, function owners and final software structure. Its ontology diagram shows the relationships between the concepts used below.

Use gv for composition and deployment, iv for ports and connections, cv for cases and requirement claims, sv for exchange dependencies, grv for properties, and fv for function flow or states. Browser navigation follows declarations and occurrence paths.

Run the interactive showcase with `npm --prefix showcase ci` and `npm --prefix showcase run dev`. It contains a seven-view walkthrough, an interface reference and four domain architectures. Read the full model source alongside its diagram.

Your architecture	Start with
AUTOSAR Classic software component	Climate controller: sampled inputs, client/server calibration, periodic acquisition and MCU allocation
AUTOSAR Adaptive application	Adaptive diagnostics: service functions/events and process allocation
Service-based web application	Web shop: browser, backend, inventory and explicit event consumers

Your architecture	Start with
Cloud service	Cloud telemetry: ingestion, explicit broker, analytics and archive consumers

These examples model logical software architecture. Read their architecture patterns and platform mappings. Continue with the user guide, UML transition guide, or complete specification.

Requirements and formal documentation

Requirements own typed attributes and one optional `doc /* ... */` specification. Design elements link through `satisfy`; verification cases link through `verify`. Use cases describe intent. Ordinary comments remain independent notes. View kinds use lower camel case, and view blocks combine multiple selections and Boolean filters. See the guide and complete model.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Architecture patterns

SWAML 1.3 uses the same architecture model for embedded control, Adaptive applications, service-oriented systems and cloud services: reusable definitions, owned ports, typed contracts, explicit connectors, local behavior and deployment occurrences. Start from the example closest to your system, then replace its contracts and behavior with your own.

These models are executable logical examples. Their platform names describe the intended architecture context. A target binding supplies middleware, operating-system and infrastructure configuration; SWAML does not generate those artifacts. See execution bindings for the contract and implementation status for delivered capabilities.

Architecture	Complete model	Root	Main interactions
AUTOSAR Classic climate SW-C	11-autosar-classic.swaml	`ClassicClimate::Vehicle`	Sampled temperature, client/server calibration, sampled actuator command
AUTOSAR Adaptive diagnostic application	12-autosar-adaptive.swaml	`AdaptiveDiagnostics::Vehicle`	Service methods, changed notification, audit publication
Service-based web application	13-web-application.swaml	`WebShop::Production`	Checkout request, inventory reservation, receipt publication
Cloud telemetry service	14-cloud-service.swaml	`CloudTelemetry::Production`	Ingestion, explicit broker, independent analytics/archive subscriptions

Each model declares gv for composition, iv for contracts and wiring, cv for intended requirement coverage, sv for a representative interaction, grv for deployment and fv for a function. Browser navigation follows the same definition and occurrence ownership. The interactive showcase presents these four architectures alongside the interface reference.

AUTOSAR Classic: a climate software component

AUTOSAR Classic separates application software, the runtime environment and basic software on a microcontroller. SWAML can describe the application responsibilities and their communication before a target integration supplies the RTE and basic-software configuration. AUTOSAR Classic overview.

The example gives the climate controller three explicit boundary ports. A sensor emits 28 °C every 10 ms; the controller requests a 22 °C setpoint, calculates a bounded fan duty and sends 0.6 to the actuator. A missing calibration response selects duty 0.0 through an explicit timeout branch. The root's components are allocated to the external identity `system:bodyEcu`; a SysML integration can bind an equivalent `sysml:` part identity.

SWAML subject	Architecture responsibility
`controller: ClimateController`	SW-C boundary with temperature, calibration and fan ports
`controller.control: ClimateLogic`	Internal behavior and private controller values
`regulate` function	Runnable-like reaction to received temperature

SWAML subject	Architecture responsibility
`CabinTemperature`, `FanCommand`	Sender/receiver data contracts
`Setpoint`	Client/server calibration query
`measured`, `calibrated`, `commanded`	Explicit assembly communication routes

This is a modeling correspondence. AUTOSAR's Software Component Template defines its own component, port, internal-behavior and runnable metamodel. A SWAML component is useful for grouping owned data and runnable-like functions; it is not an additional AUTOSAR component forced into the implementation. Preserve that grouping in an explicit mapping. AUTOSAR Software Component Template, R24-11.

The `realtime` profile requires timing contracts and connector bounds. In this example each sampled route has capacity 1, `dropOldest`, `maxLatency 1 ms` and `maxAge 20 ms`. The logical simulation checks the declared timeouts and transfers. RTE generation, ECU extract, OS task/event assignment, runnable scheduling, E2E protection and physical WCET evidence remain target integration work. A SWAML sampled receive activates a handler with a copied value; an AUTOSAR integration must choose a compatible read/activation strategy explicitly.

AUTOSAR Adaptive: an ARA application

The Adaptive Platform provides the AUTOSAR Runtime for Adaptive Applications (ARA), with platform services and APIs. Its communication management uses service providers and consumers, including discovery during runtime. AUTOSAR Adaptive overview.

The diagnostic application requires the `VehicleHealth` service. It invokes `runCheck`, obtains a correlated bool response, and queries `getSummary` for a `HealthReport`. The provider publishes `changed(report)` on the same service contract and independently publishes `DiagnosticAudit.checked` to an audit subscriber. Each application/service has its own external deployment identity, resolved by the target binding.

SWAML subject	ARA-oriented interpretation
`application: DiagnosticApplication`	Consumer application boundary
`application.health` with role `requires`	Consumer-side service contract, suitable for a proxy binding
`health.api` with role `provides`	Provider-side service contract, suitable for a skeleton binding
`getSummary`, `runCheck`	Service methods
`changed`	Provider-to-consumer notification
`healthLink`	One selected service-instance relationship

These are application-level correspondences to the proxy/skeleton, method and event concepts in AUTOSAR Communication Management, R25-11. A field can be represented explicitly by getter, optional setter and changed-event members; the setting-service example demonstrates that pattern.

The example records the topology after service selection. Model known provider variants as separate roots or explicit adapters. Dynamic discovery, service-offer/subscription lifecycle, execution and service manifests, SOME/IP or other network bindings, and generated `ara::com` C++ APIs belong in the target binding. The name `Application` carries architecture intent and does not imply a generated executable or manifest.

Service-oriented web application: checkout and inventory

The web example separates a browser client, application service, inventory service and receipt worker. The browser is a modeled component so that its requests obey the same typed-boundary rules as server-to-server communication.

`Checkout.submit` first requests `Inventory.reserve`. The stock service validates quantity and updates its own private `available` value. Only a successful reservation lets checkout publish acceptance and a receipt. The response is correlated to its request; notifications are separate events on the declared routes. The example rejects an order for 99 items after accepting an order for 2 items from initial stock 10.

Contract	Choice and reason
`Checkout` - service	Combines a request/result with provider notifications
`Inventory` - clientServer	Reservation uses a request/result without subscription traffic
`Receipts` - publishSubscribe	Receipt processing has independent activation and owned data

HTTP routes, status codes, cookies, authorization, serialization and browser delivery mechanisms are deployment decisions. For example, a target may bind the request to HTTP and the service event to server-sent events while retaining the same modeled contract. Choose an adapter when that target uses different nominal contracts. The sample keeps its stores in memory; persistence and atomic order/reservation guarantees require an explicit storage/transaction design.

Cloud service: ingestion and event distribution

The telemetry example accepts a typed `Sample` from one ordered device stream. Ingestion checks the device identity, sequence and temperature range before publishing accepted data. An explicit broker receives that contract and forwards a `Measurements` contract to two independent subscribers. Analytics owns its aggregate values; archive owns its record count and latest device identity. There is one edge process and four container allocations.

The three event connectors select the same `EventQueue` policy with `atLeastOnce`, FIFO ordering and capacity 32. Each connector still has its own queue. The broker and consumers retain a private sequence high watermark, so repeated delivery of an already processed sequence creates no repeated local effect in this single-stream example. The simulator tests duplicate delivery independently on all three event routes.

The source sends sequences 1 and 2, retries sequence 2, then submits an invalid temperature for sequence 3. Expected results are two accepted samples, two broker forwards, two analytics updates and two archive records. The retry succeeds without another publication; the invalid sample returns false.

An ingestion result of true confirms admission at ingestion; it does not acknowledge downstream processing. If a broker publication fails on one route, other routes may already have accepted the sample. Their effects remain committed, and the failed broker handler is not automatically retried. The adverse test injects a transport failure on the first publication to analytics: ingestion still accepts both samples and archive records both, while analytics processes only the second and the trace reports the broker failure. Recovery requires an explicit acknowledgment/replay protocol.

This policy is intentionally scoped to one monotonically ordered device stream. A multi-device system needs per-device state; an out-of-order stream needs a different deduplication strategy. The sample's high watermarks and effects are in memory. Recovery, durable broker acknowledgements, transactional publication, replay and exactly-once business effects require a separate persistence/failure design. Container declarations identify locations; an infrastructure binding maps explicitly modeled replica occurrences to resources and supplies service addresses, topics, storage and orchestration configuration.

Validate and run the models

After `npm ci` and `npm run build`, validate all four examples:

```sh
node out/src/cli/main.js examples/11-autosar-classic.swaml examples/12-autosar-adaptive.swaml examples/13-web-application.swaml examples/14-cloud-service.swaml
```

Run each logical deployment with its checked-in options:

```sh
npm run simulate -- ClassicClimate::Vehicle examples/simulation/11-autosar-classic.json examples/11-autosar-classic.swaml
npm run simulate -- AdaptiveDiagnostics::Vehicle examples/simulation/12-autosar-adaptive.json examples/12-autosar-adaptive.swaml
npm run simulate -- WebShop::Production examples/simulation/13-web-application.json examples/13-web-application.swaml
npm run simulate -- CloudTelemetry::Production examples/simulation/14-cloud-service.json examples/14-cloud-service.swaml
```

Root	Evidence checked by the run catalog
`ClassicClimate::Vehicle`	Received temperature 28; commanded/applied duty 0.6; calibration 22
`AdaptiveDiagnostics::Vehicle`	One completed check, one notification, one audit entry; observed healthy/code 200
`WebShop::Production`	One accepted order and receipt; remaining stock 8; oversized order rejected
`CloudTelemetry::Production`	Two effects per stage; retry accepted; invalid sample rejected; latest temperature 28.5

`node scripts/check-simulations.mjs` verifies those final stores and repeatability. Architecture-pattern regressions additionally inspect timeout, transport failure, partial fanout and duplicate-delivery traces. The `case` declarations describe intended exchange dependencies; simulation results supply bounded evidence for function behavior, and are not automatic proofs that every case or target execution is satisfied.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

SWAML for UML architects

SWAML 1.3 supports familiar structural and behavioral modeling with explicit software boundaries. Start with components and their contracts, compose named parts, then inspect interaction and behavior. The reference for UML terminology is OMG UML 2.5.1. The correspondences below are modeling guidance, not a UML import format or conformance claim.

Translate the modeling concepts

Familiar UML concept	SWAML representation	Boundary to keep in mind
Package	`package` and explicit imports	Package declarations form namespaces; imports are not transitive.
Component type	`component def`	Owns child composition, ports, private values and functions; functions own calculations.
Part / typed occurrence	`component name: Type;`	Each expanded component occurrence owns separate private state.
Generalization / redefinition	`specializes` and `redefines`	Single component specialization with explicit compatible member replacement.
Interface	Versioned interface with member signatures	Exact nominal identity; structural similarity does not establish compatibility.
Provided / required interface	Service ports with `provides` / `requires`	Other interface kinds use sender/receiver, client/server or publisher/subscriber.
Assembly connector	Named `connect` between child ports	Select a complete QoS policy; the arrow identifies the contract orientation.
Delegation connector	`delegate outer -> child.port`	Same role and contract; no conversion, additional queue or independent behavior.
Operation implementation	Component function bound with `on call port.member`	Signature declaration and implementation have distinct owners.
Signal reception	Interface event and `on receive port.member`	Events are contract members; there are no free signals or hidden event buses.
Activity	Function statement body, shown in fv	Structured conditionals and bounded loops; no fork/join tokens or activity allocation.
State machine	States directly inside a function, shown in fv	Flat states, entry/exit, guarded transitions; no orthogonal or hierarchical regions.
Interaction / sequence diagram	`case` exchanges shown in sv	Explicit dependency order and routes; a case is not automatically executed as a test.
Use case	Case View (cv) is the nearest navigation surface	A SWAML case specifies exchanges for one root; actors, include and extend are absent.
Datatype / enumeration	`datatype` / `enum`	Copied nominal values; neither classes nor references to remote objects.

Familiar UML concept	SWAML representation	Boundary to keep in mind
Deployment	External system/part references in `deploy` bindings	Allocation metadata does not prove physical schedulability.

The diagram labels `«component def»`, `«component»` and `«interface»` name SWAML concepts. They do not declare UML stereotypes. Rectangular compartments, typed names, square boundary ports, explicit connectors, sequence lifelines, decisions and states provide a familiar reading vocabulary. All drawings are flat. The notation chapter defines the exact vocabulary.

Choose the interface by interaction

Architectural need	Kind and roles	Example
Retain individual accepted values	senderReceiver: sender → receiver; queued member	Greeting
Retain the latest pending measurement	senderReceiver: sender → receiver; sampled member	Controller
Call a function and await a result	clientServer: client → server	Health check
Publish typed events to explicit subscribers	publishSubscribe: publisher → subscriber	Audit publication
Expose functions and notifications together	service: requires → provides	Temperature setting

The interactive interface reference displays all five patterns from these exact sources. A source/target arrow is not a statement that all traffic is one-way: RPC replies return on the correlated route, and service events travel from provides to requires. Sampled data is a queued value transfer with capacity one and dropOldest, not shared-memory access. The full direction and cardinality rules are in chapter 2.

Separate contracts when parts need different membership, policies or lifecycles. An interface containing sampled and queued members applies the same capacity-one policy to both. A consumer of a service must implement all of its incoming events; there is no implicit partial implementation. Use an adapter component to translate between contracts and versions.

Build the model incrementally

1. Declare value types and interface signatures. Contract-only packages are valid reusable libraries.
2. Define component boundaries and implement their handlers and emitters. Every incoming member needs one handler owner; an empty implementation stub is not evidence of behavior.
3. Compose components into reusable component definitions. Expose boundaries through same-role delegation. A library definition can leave ports awaiting its eventual context.
4. Instantiate an inline or typed root. Closure is checked for each concrete root, including expanded library parts. Every nonoptional port must be connected; optionality does not waive handlers or provenance.
5. Select a named complete QoS policy for each connector. Choose delivery and overflow deliberately; reliability does not establish handler success or exactly-once effects.
6. Add cases and views to explain the architecture. Add deployment and timing when the target question requires them; distinguish declared bounds from execution evidence.

Begin with the complete greeting model, then compare reusable policy syntax. Headerless files require a host explicitly pinned to 1.2. The user guide contains validation commands.

Read the seven surfaces

Surface	Architectural question	UML reading aid
Browser	What is declared, owned or instantiated?	Model explorer; definition and occurrence paths are separate.
gv	Which parts compose this architecture?	Structural overview and containment.
iv	Which contracts connect the parts?	Ports, assembly and delegation.
cv	Which cases claim requirement coverage?	Scenario navigation; narrower than use-case modeling.
sv	Which exchanges depend on which others?	Lifelines and typed messages; visual height alone adds no order.
grv	What are the exact properties and bindings?	Model table and property inspection.
fv	How does this function execute?	Structured flow or a flat state machine owned by the function.

Deliberate language limits

SWAML 1.3 supports single component specialization and compatible explicit member redefinition. It has no separate class construct, association classes, arbitrary multiplicities, dynamic creation, generics or cross-component attribute access. A part declaration denotes one occurrence; multiple instances are explicit declarations. These omissions keep ownership, closure and queue identity checkable. Do not emulate missing constructs by drawing untyped association lines or treating the same definition as shared state.

The core uses components for both composition and local behavior, functions for executable flow/states, and calculations for pure expressions. Complete-value `qos def` reuse, default service profiles and omitted void results reduce repetition. Additional aliases such as class, signal or machine would need semantics and migration rules; they are not introduced merely for familiar spelling. The implementation matrix separates implemented validation and simulation from future graphical authoring, KerML mapping and production execution.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

SWAML 1.2 architecture usability review

Review scope: simpler software architecture modeling, flat graphics, complete ontology/views, an interactive React Flow showcase, GitHub presentation, and familiarity for UML architects. This is an engineering review of the local edition, not an independent conformance certification.

Findings and changes

Finding	Assessment	Resolution
Repeated connector policy text	Named complete policies reduce real authoring repetition without changing route semantics.	Retain `qos def` and effective-use validation.
3D execution-node graphics	Conflicted with the flat-only requirement.	Flat canonical frames and reproducibility checks already implemented.
Showcase demonstrated only service communication	Did not explain the full interface vocabulary.	Add five selectable source-backed patterns covering all four interface kinds, including sampled/queued data.
Promotional headings and decorative presentation	Too informal for architecture review.	Use restrained typography, structured contract compartments, exact names and reference-oriented copy.
Familiar names concealed semantic differences	cv is not a UML use-case model; service arrows also carry reverse traffic.	Add a UML transition guide, traffic/cardinality table and explicit projection explanations.
Manual diagram metadata could drift	Build success alone does not prove agreement with the language model.	Parse fixture sources and check catalog contract kinds, port roles, connector endpoints and effective policies in the language gate.
Generic diagnostics for incorrect roles or missing QoS	Rejection did not explain the repair in familiar architectural terms.	Include legal roles/orientation and inline or named policy choices, preserving diagnostic codes.
Tool-specific planning in public guidance	Coupled the language reference to an unrelated implementation project.	Keep parser, CLI, LSP and browser-host guidance independent; check maintained sources and PDF content, navigation and attachments for regressions.
Platform examples were too general	Architects could not follow a complete Classic, Adaptive, web or cloud design through validation and execution.	Add four complete models, configured simulations, adverse-path tests and inspectable source-backed architecture diagrams.
Profile, deployment and platform mapping could be confused	A service profile is a timing choice; placement alone supplies no middleware or platform guarantees.	Clarify those distinctions in chapters 4 and 6, with explicit activation, value-conversion, transport and lifecycle mapping obligations.
Entry guidance assumed familiarity with the language	New users needed a short route from a contract to a running example.	Add a landing-page quick start and linked architecture-pattern guide.

Language decision

The 1.2.0 release uses a unified component boundary and component-owned functions. Pure calculations declare typed inputs and a named output. Single specialization and compatible redefinition support reuse; deployment refers to external systems or parts. ADR 0007 records this revision and supersedes the earlier split ownership model.

Retain explicit interface kind/version and port role: they identify interaction semantics. Retain explicit QoS selection: omitted policies would hide queue and failure decisions. Retain mandatory handler ownership and emitter provenance: weakening them would let an incomplete implementation pass as a closed architecture. Contract-only libraries provide an existing first step without pretending an unimplemented root is executable.

Multiple inheritance, abstract executable stubs, collections of parts, hierarchical states, true UML use-case relations and graphical source editing need separate proposals with semantics and evidence. They are future design topics, not silently accepted 1.2 capabilities. Familiarity is delivered through concept correspondence, notation and workflow; it does not imply UML interchange or conformance.

Review evidence

The executable acceptance gates are `npm run check`, `npm --prefix showcase run build`, both figure generator check modes, and the PDF workflow. The language gate includes `scripts/check-showcase.mjs`, which resolves every displayed pattern against the actual parser/validator. Manual browser checks must cover all pattern selectors, endpoint inspection, full source expansion, all seven existing surfaces and narrow layouts. The release record records completed verification and publication status.

The platform review preserves the fixed instance topology, copied payloads, typed routes, private stores and explicit failure behavior. It does not add platform-specific keywords. The architecture guide records which decisions belong in the model and which require a target binding; reference simulations check bounded logical behavior under their supplied options.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

SWAML ontology and flat views

SWAML 1.3 models software with **components, functions and calculations**. A component may contain other components and own private data and executable functions. Functions expose statement flow or states; calculations produce pure values from explicit inputs. Start with the graphical overview, then use this inventory alongside the normative specification.

Ownership and reuse

TEXT

```
Component instance
├─ Child component instances, recursively
├─ Ports, connectors and boundary delegation
├─ Private values
└─ Functions
   ├─ Statement flow OR states and transitions
   └─ Calculations
      └─ Typed inputs → pure expression → named output
```

A package organizes names; it is not an extra software layer. `component def` declares a reusable type. `component child: Type;` creates an instance with separate state. A top-level component selects the root architecture. Definitions may specialize one base, with compatible inherited replacements marked `redefines`. Specialization is reuse of declarations, not shared runtime storage.

Concept	Owns or contains	Meaning
Package	Imports and declarations	Namespace for definitions, contracts and views.
Component definition	Children, ports, private data, functions and connections	Reusable software boundary, optionally specializing one base.
Component instance	Effective members and separate private state	One use of a definition with a distinct path. Leaf and composite instances follow the same rules.
Root component	Inline members or one definition instance; deployment bindings	Closed architecture selected for execution. Omitted profile means service.
Function	Inputs/result, trigger, timing, local calculations and statements or states	Executable component-owned behavior. Omitted result means void.
Calculation	Typed inputs and one named non-void output expression	Pure function-local computation; cannot read mutable values or communicate.
State / transition	Entry/exit blocks / guard, trigger, priority and effect	Control owned directly by one function.
Private variable	Type and pure initializer	Mutable copied-value storage inside one component instance.
Value type	Primitive domain, record fields, enum literals or fixed array extent	bool, byte, uint, int, real, string, result-only void and structured values.
Interface	Typed data, events or function signatures	Nominal versioned communication contract. Signatures and implementations have distinct owners.
Port	Contract, role and optionality	An owned boundary endpoint.

Concept	Owns or contains	Meaning
Connector	Typed endpoints and complete policy selection	Route between owned or immediate-child ports; its queues are independent.
Delegation	Outer and immediate-child port references	Boundary alias with the same contract and role; introduces no queue or handler.
QoS policy	Complete delivery, ordering, capacity, overflow and optional bounds	Reusable values selected explicitly on each connector.
Deployment	Instance path and external identity	Link to a host-resolved system or SysML part, with nearest-ancestor inheritance.
Requirement / case	Obligation text / exchange dependencies and coverage links	Intended outcomes and representative traces; verification requires evidence.
View	Kind and subject	Projection of existing model identities.

The same component may implement an external contract and contain collaborating children. Local behavior cannot bypass those children's interfaces. An inherited function resolves local names against its specialized component's effective members, while another instance keeps its own state. The specialization example demonstrates this distinction.

Communication contracts

Interface kind	Members	Role pair	Direction
senderReceiver	Sampled/queued data	sender / receiver	Data travels sender → receiver.
publishSubscribe	Events with typed payloads	publisher / subscriber	Events travel publisher → subscriber.
clientServer	Function signatures	client / server	Request client → server; correlated result/failure returns.
service	Function signatures and events	requires / provides	Request requires → provides; result/failure and provider events return.

`Settings::Panel` is a definition. `Settings::Cabin.front.panel` and `Settings::Cabin.rear.panel` are two instances with distinct `displayed` values and function activations. `Cabin.frontLink` and `Cabin.rearLink` may select the same LocalLink policy while retaining separate queues. The service setting model supplies these exact identities.

How the views look

Browser is the navigation surface. gv, iv, cv, sv, grv and fv are declared views. Every drawing uses flat geometry, explicit labels and stable source identities. Color supports grouping; it never carries the only semantic distinction.

Surface	Subject	Flat appearance and purpose
Browser	Workspace or selected subtree	Indented ownership tree with distinct definitions, inherited members and instance paths.

Surface	Subject	Flat appearance and purpose
gv - General	Package or named semantic element	Component frames with child, data and function compartments, with calculations nested inside functions; labeled type, specialization and deployment references.
iv - Interconnect	Root, component definition or component instance	Owner/child cards, square ports, named typed connectors and boundary delegation. Locally handled ports remain visible.
cv - Case	Case or root	Design and verification case cards connect to requirements through `satisfy` / `verify` links.
sv - Sequence	Case or root case selection	Participant headers, lifelines and typed messages with explicit dependencies and correlated replies. Vertical layout adds no ordering.
grv - Grid	Package or named semantic element	Filterable rows for identities, owners, contracts, functions, policies, external destinations and evidence.
fv - Flow	Function	Owned statement flow with decisions, bounded loops and calls, or directly owned states with guarded/timed transitions.

External deployment targets are labeled reference cards, not software children. A `sysml:` string remains an unresolved external reference until the integration host supplies validation. The graphical notation, canonical catalog and worked examples define the detailed symbols.

Start small

Begin with an inline root and the components that represent useful private-state boundaries. Add reusable definitions and specialization when they help express a real family of architectures. Keep a calculation local when it only transforms values. Add a typed port and route whenever work crosses a component boundary.

Use gv/iv for responsibility and communication, cv/sv for intended interactions, grv for contract review and fv for function detail. The showcase supports inspection and layout; arbitrary graphical source editing, production timing proof and cross-language export require the additional implementations listed in implementation status.

Requirements and formal documentation

Requirements own typed attributes and one optional `doc /* ... */` specification. Design elements link through `satisfy`; verification cases link through `verify`. Use cases describe intent. Ordinary comments remain independent notes. View kinds use lower camel case, and view blocks combine multiple selections and Boolean filters. See the guide and complete model.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

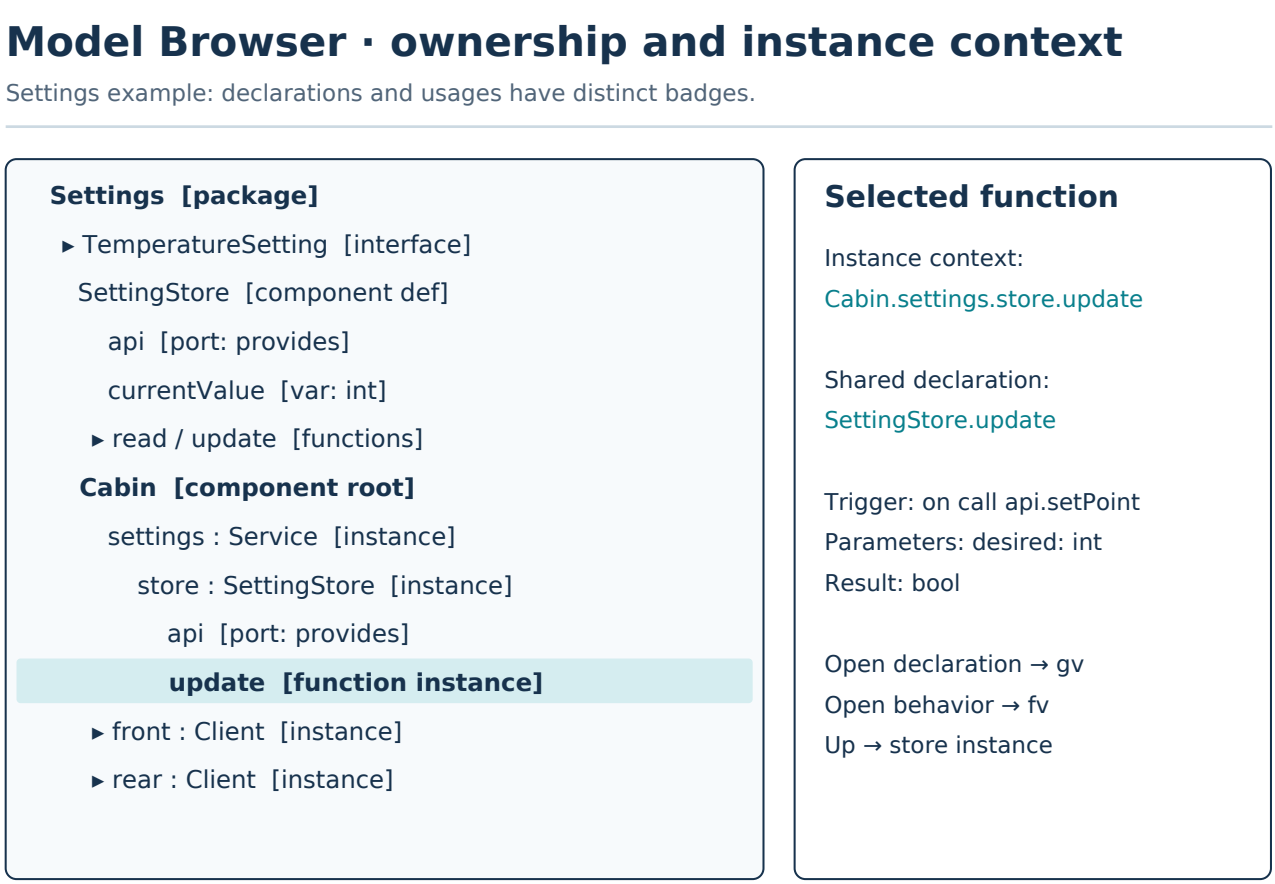
Graphical notation examples

These rendered figures illustrate the normative view contract. The same vector scene sources are rendered into the PDF so labels and lines remain sharp; PNG copies support tools that require raster images. Every figure has accessible text and a specific model source; the source model remains authoritative.

For individual language concepts, use the canonical notation catalog, with paired SVG/PNG downloads and textual counterparts. The notation inventory covers every grammar production and keyword, distinguishing standalone glyphs from compartments, labels and text-only syntax. The figures below are worked compositions of those rules, not the exhaustive symbol index.

The first seven figures use the service setting model. A provider owns the setting, two client components use its RPC/events, and private storage remains behind component ports. The state figure uses the lifecycle model. The figure manifest links each illustration to its model and semantic subject.

Model Browser: actual ownership

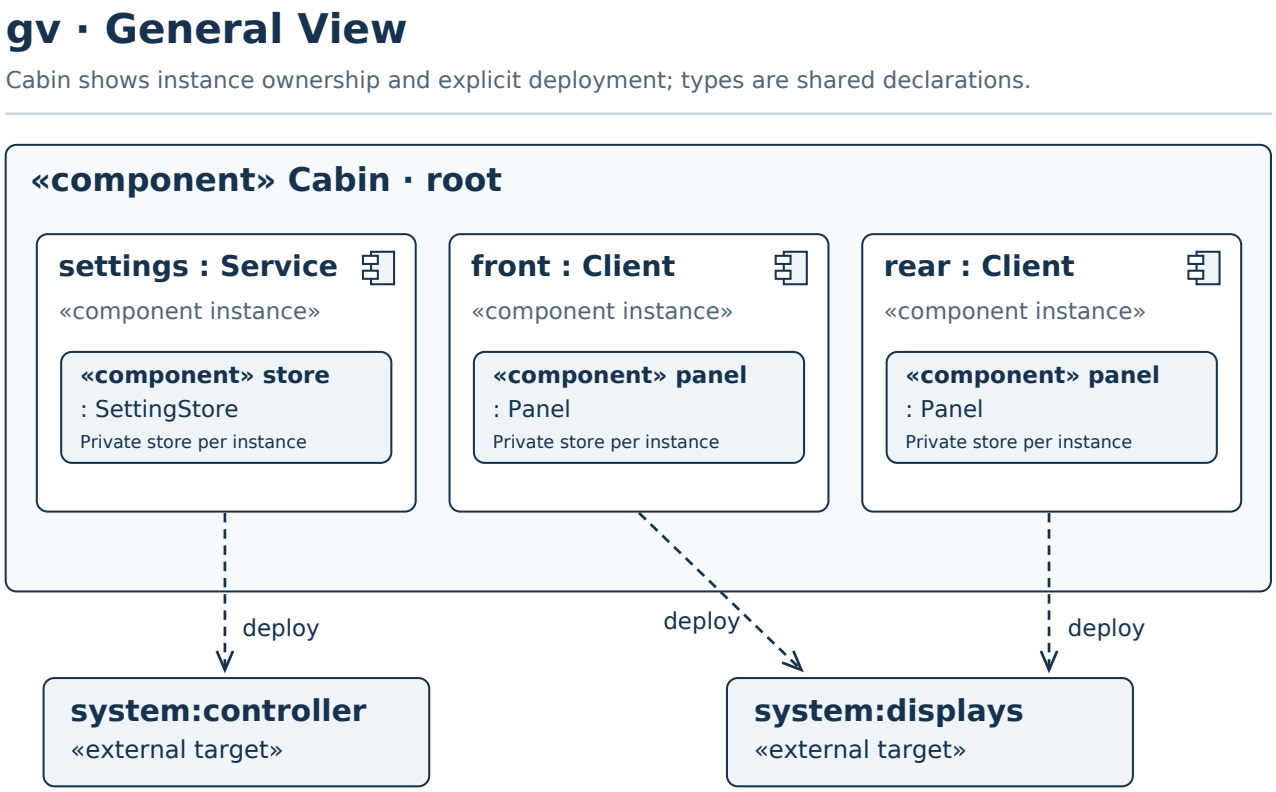


Model Browser showing Settings declarations, Cabin occurrences and selected settings.store.update function

SVG · PNG

The tree distinguishes `SettingStore` as a component definition from `Cabin.settings.store` as an occurrence. Selection of `update` retains its instance context. A declaration command navigates to the shared definition; an fv command opens the function's body. There is no performer or separate action allocation node. Collapsed rows are presentation choices, not omitted semantics.

gv: general structure and allocation

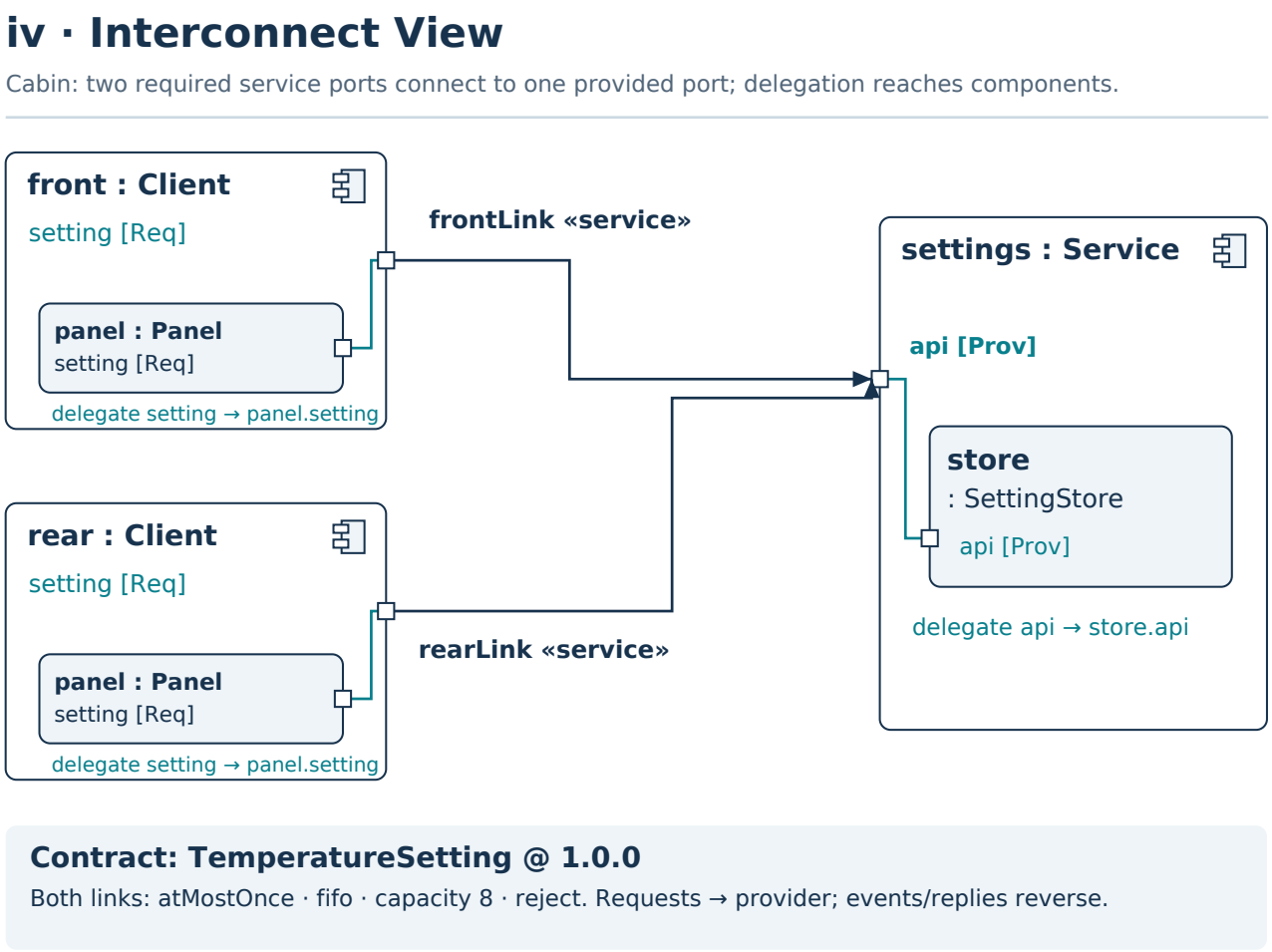


General View of Cabin with settings, front and rear component occurrences and inherited deployments

SVG · PNG

`Cabin` is the root component occurrence and its profile is service. `settings`, `front` and `rear` are component occurrences. Each owns a child component instance. Dashed `deploy` edges assign settings to controller and the panels to displays; descendant components inherit those locations. These are allocation edges, not communication lines. The two Panel occurrences have distinct private stores despite sharing one definition.

iv: explicit interconnection and delegation

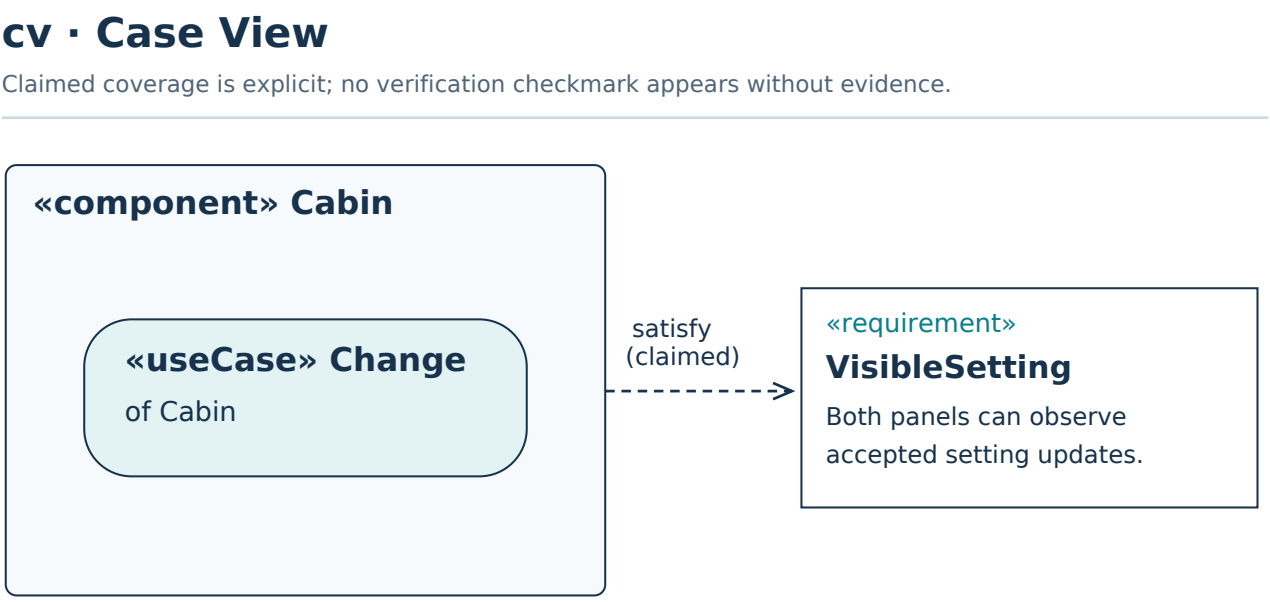


Interconnect View showing required panel ports connected to the provided TemperatureSetting service and delegated component ports

SVG · PNG

Both connectors are oriented from `requires` to `provides`. Each visible boundary port is a square and delegates to a child port with the same contract/role. The shared provided port can serve both clients. The arrows indicate connector orientation; member inspection exposes reverse provider-event and response directions. Every route has its own capacity-8 FIFO queue policy per member/direction. There is no line directly coupling panel data to store data.

cv: cases and claimed requirement coverage

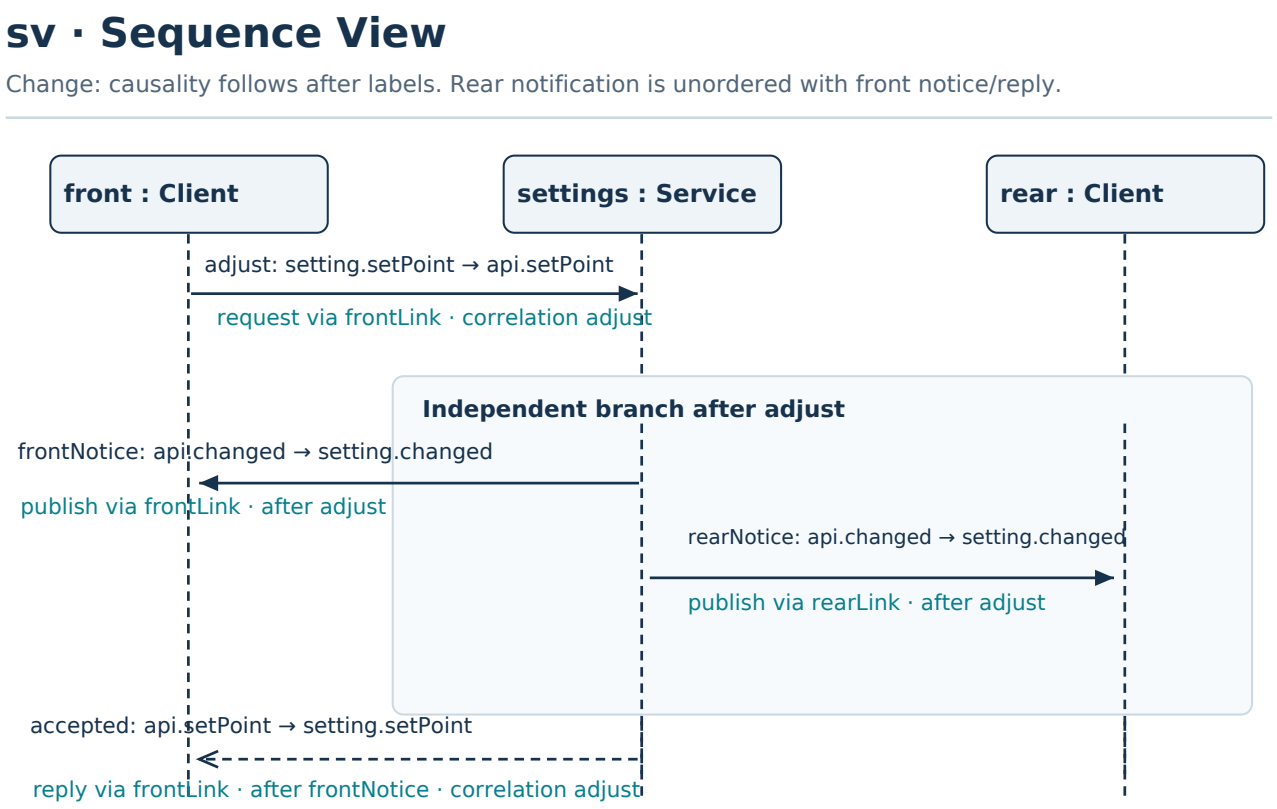


Case View of Change in Cabin linked to VisibleSetting with satisfies claimed

SVG · PNG

The dashed `satisfy` / `verify` link records intended coverage. It does not assert that the case is executable or verified. A future tool may attach actual analysis/test evidence; it must retain evidence identity and assumptions before showing verified status.

sv: interface exchanges and partial order



Sequence View of Change showing a setPoint request, two changed notifications and one correlated reply with explicit causal dependencies

SVG · PNG

Lifelines are component occurrences; every message names its exposed port/member and connector. `adjust` precedes both notifications; `frontNotice` precedes `accepted`. `rearNotice` is unordered relative to `frontNotice` and `accepted`, even though the drawing places its row between them. The pale branch and explicit causal summary prevent vertical placement from inventing a total order. The reply retains the `adjust` correlation. This diagram depicts the case's partial order, not a simulator's chosen trace.

grv: inspect properties without changing semantics

grv · Grid View

Function inventory from Cabin; blank timing means no declared contract in service profile.

Instance	Function	Trigger	Result	WCET	Deadline
settings.store	read()	call api.get	int	1 ms	5 ms
settings.store	update(desired)	call api.setPoint	bool	-	-
front.panel	adjust()	manual	void	-	-
front.panel	refresh()	manual	void	-	-
front.panel	display(current)	receive setting.changed	void	-	-
rear.panel	(same functions)	distinct instance	void	-	-

read is reentrant; other functions default to serial. Sorting this grid changes no execution order.

Grid View listing service and panel function signatures, triggers, results and declared timing

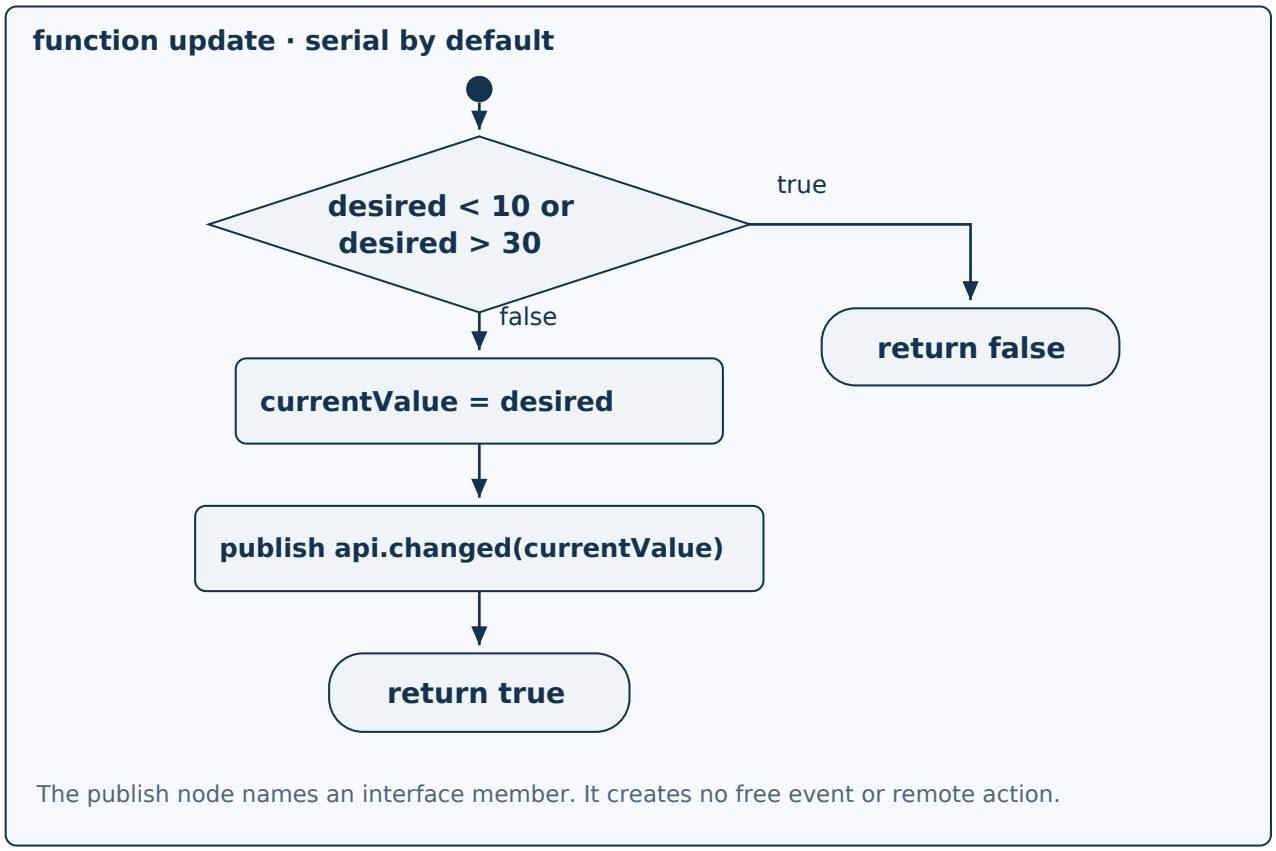
SVG · PNG

The grid shows effective occurrence paths and source-declared contracts. A dash means the service-profile function has no timing declaration; it never means zero physical execution time. `read` is reentrant; other activation roots use the serial default. The rear row summarizes the same three declarations in a different occurrence context. Sorting a grid does not reorder source statements, queues or case dependencies.

fv: function statement flow

fv · Function flow

SettingStore.update(desired: int): bool · on call api.setPoint · directly owned by the component.

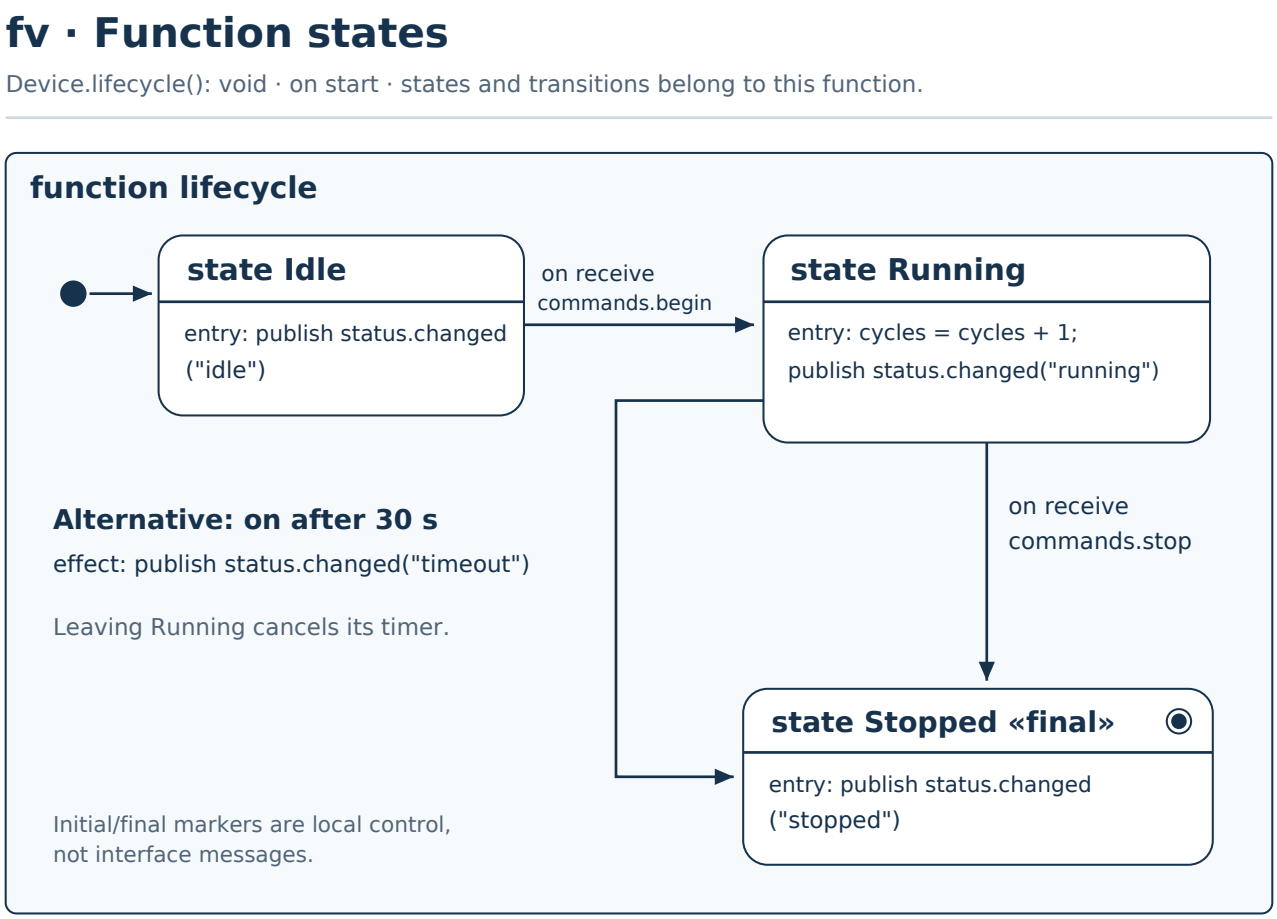


Flow View of SettingStore.update with bounds check, false return, private assignment, interface publication and true return

SVG · PNG

The frame belongs directly to `SettingStore.update`. The decision tests the actual source condition. A valid value updates private state, emits `api.changed(currentValue)` through the provider port, then returns true to the correlated request. An invalid value returns false without mutation/publication. A connector is not drawn from this local flow node to another function: the iv and port path describe the architectural exchange.

fv: function-owned states



Flow View of Device.lifecycle with Idle, Running, Stopped final, interface event transitions and a 30-second timer

SVG · PNG

`Idle`, `Running` and `Stopped` belong directly to `Device.lifecycle`. Initial entry publishes idle status. The interface event `commands.begin` enters Running; entry increments cycles and publishes running. Either `commands.stop` or the local 30-second timer enters the final state. The timer edge first publishes timeout in its effect. Final entry publishes stopped and terminates its owning function. Leaving Running cancels the old timer generation. fv projects both state behavior and statement flow from their owning functions.

Reproduction and semantic checks

Run `python3 scripts/build-graphical-examples.py` to regenerate the eight SVG/scene pairs, and use `--check` to detect stale figures. Run `python3 scripts/build-notation.py` to regenerate their PNG copies together with the canonical catalog; its `--check` mode verifies decoded pixels. `node scripts/check-graphical-examples.mjs` resolves every manifest subject against its validated source model and verifies that all view families have an illustration. The semantic test establishes subject/reference validity; it is not a graphical editor conformance certification. Publication QA additionally renders and visually inspects the figures in the PDF.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Reference simulation profile 1.3

This document specifies the **swaml-reference/1.3** execution binding. This profile executes unified components, inherited effective members, executable functions and pure calculations; each connector retains its independent effective policy and queues. The portable TypeScript interpreter and value evaluator execute valid SWAML 1.3 root components. The profile implements logical behavior with explicit resource bounds. It does not model instruction duration, operating-system scheduling, hardware or network deployment.

Run a model

Build with `npm ci` and `npm run build`. Then run:

```sh
npm run simulate -- Commerce::Shop examples/simulation/03-service-pubsub.json
examples/03-service-pubsub.swaml
npm run simulate -- Lifecycle::Product examples/simulation/04-stateful-function.json
examples/04-stateful-function.swaml
node scripts/check-simulations.mjs
```

The command takes a qualified root component name, a JSON options file and all workspace model files. It validates the entire input before executing. JSON output contains the trace, final stores, machine states, effective deployments, exact source text and per-source SHA-256 digests. Redirect stdout to a file when collecting evidence. The direct `node out/src/cli/simulate.js ...` command emits only JSON to stdout on a valid invocation; `npm run` additionally prints its script banner.

The selected root may compose components inline or instantiate a component definition. A typed root's deployment body does not extend its topology. Instance paths, stores, manual release paths and deployment keys remain relative to the selected root without a root-name prefix. Root-owned functions use their local name, such as `initialize`, without a root prefix. Store/deployment output includes every expanded component, with the root represented by the empty-string path. Source deployment may explicitly name the root to establish its default location. The output names the selected architecture `root`, and identifies the binding as `swaml-reference/1.3`.

Exit status is 0 when no failure was observed, 1 for invalid models, a failed activation, a contract violation or a resource limit, and 2 for an invocation/configuration error. A bounded horizon with no observed failure is not proof of eventual success. Descriptive `case` declarations are not a second executable script and are not automatically proven by this command.

The run catalog names the published roots, options and expected final values. The hello trace is a complete reproducible observation artifact, including its exact model source. CI reruns every catalog entry, compares final values, checks repeatability and detects a stale published trace.

Binding parameters

All times and counts are safe integers. Times are nanoseconds. Unknown options and invalid numeric bounds are rejected.

JSON property	Default / domain	Meaning
`untilNs`	required; ≥ 0	Process events at times ≤ this horizon. Start epoch is zero.
`maxSteps`	100000; ≥ 1	Fuel for expansion, events, statements, expressions and transition work. Stops explicitly on exhaustion; it is a resource count, not elapsed CPU time.

JSON property	Default / domain	Meaning
`maxPending`	1024; ≥ 1	Global bound on pending lifecycle/manual/periodic/timer work, excluding active frames and connector queues.
`maxReentrant`	64; ≥ 1	Maximum simultaneous reentrant activation roots per component. Serial activation roots have their separate one-lock rule.
`latencyNs`	0; ≥ 0	Constant one-way transport delay for accepted request, response, data and event offers.
`manual`	empty list	Parameterless harness releases: `{ "atNs": 0, "function": "web.frontend.checkout" }`. Path is relative to selected root component. Only on `manual` functions may be selected.
`faults`	empty list	Fault schedule keyed by the one-based `offer` ordinal shown in the trace. Each offer has at most one entry.

Example options:

JSON

```
{
  "untilNs": 3000000000,
  "latencyNs": 1000,
  "maxSteps": 100000,
  "maxPending": 1024,
  "maxReentrant": 64,
  "manual": [{ "atNs": 0, "function": "web.frontend.checkout" }],
  "faults": [{ "offer": 1, "kind": "drop" }]
}
```

Manual releases at the same time retain their list order. A release carries no model payload. Use modeled adapter functions and their typed ports to represent external commands or data. The simulator never allows a harness to call a remote handler directly.

Deterministic scheduling and time

The event ordering key is `(timeNs, phase, insertionOrdinal)`. At one timestamp, phase 0 processes request expiries; phase 1 processes lifecycle/timer releases and transport readiness; phase 2 advances execution; phase 3 checks deadlines/latency. Thus timeout wins an exactly simultaneous response, while an activation completing exactly at its deadline satisfies the bound. Each event is followed by a dispatch pass: pending lifecycle work in release order, then connector queues in first-offer order. Enabled work may acquire a lock during that pass; newly released work does not preempt an already held serial lock.

Execution advances between completed statements, local function steps, loop iterations and state transitions. Pure expressions and individual assignments are atomic. Root activations interleave through the stable event ordinal. FIFO delivery is maintained per connector/direction/member. `unordered` permits this FIFO choice too; the simulator does not randomize it. Independent queues have a deterministic tie-break for reproducibility, not a source-language happens-before relation.

All local CPU costs are abstracted to zero. Transport delay, timeout, period, timer residence, suspension and queue delay advance logical time. Deadline observation begins at lifecycle/timer release or input readiness, including a message awaiting a busy component. Helpers receive their own elapsed deadline watchdog at invocation. A detected deadline or latency violation stops this run and remains in the trace. Zero-cost execution cannot establish a physical WCET bound; external deployment destinations retain their identities and do not become simulated processors.

Serial activation roots retain their lock while awaiting RPC. Reentrant activation roots read a deep dispatch-time snapshot; their helpers share that snapshot and mode. Reentrant activation roots may coexist with the serial activation root, subject to the reentrant limit. A machine allows one reaction at a time. Input stays in its connector queue until the relevant dispatch limit/lock permits execution. Pending lifecycle capacity is checked before adding another release; overflow stops with `ActivationOverflow`.

The finite-fuel policy also bounds non-progressing or Zeno model executions. No fairness or liveness result beyond the observed finite trace is claimed. There is no randomness, wall-clock dependency or automatic restart.

Transport and fault interpretation

Each connector occurrence, direction and member owns a separate queue. Delegation resolves boundary aliases to locally handled component ports without adding queues. Capacity includes messages waiting for transport readiness; dispatched handlers are excluded. A send/publish attempts every matching route before reporting rejection. Offers that succeeded remain committed.

Fault `kind`	**Meaning**
`drop`	For bestEffort/atMostOnce, discard the admitted message observably. For atLeastOnce, lose the first attempt, retain the logical identity/timestamp and retry once after an additional `max(1, delayNs)`; the retry succeeds unless overflow, age or availability terminates its obligation.
`duplicate`	Only legal on atLeastOnce. Dispatch two attempts of the same logical message/correlation. The retained logical queue entry reserves capacity until both attempts have been dispatched, with both preceding the next FIFO message. Response duplicates are observed and ignored after terminal completion.
`transport`	Detect a failed route immediately at emission. Report `Transport` to the relevant producer/correlation. This is an interface transport outcome, not a free event.

An entry may set `delayNs` to override the normal delay for that offer. Default transport is reliable with one attempt for every admitted offer; this is a permitted trace of each delivery policy. The finite injected loss/retry model exercises failure rules but does not simulate arbitrary networks or prove eventual delivery. An entry targeting an offer beyond the chosen horizon remains unused; inspect `emission.offer` to verify that the intended fault was exercised.

Queue reject fails a data/event producer after fanout. RPC admission rejection enters `otherwise` with `Rejected`. Drop policies do not fail the producer. Age is measured from original emission, inclusive at the declared maximum, with expiry on the next nanosecond. A timed-out request does not cancel server effects. Remote handler failure returns `RemoteFailure`; late/duplicate replies cannot resume a caller again. An optional unbound RPC reports `Unavailable`; an unbound optional data/event send succeeds without offers.

Values, flow and state coverage

Every expression and statement in the grammar is interpreted: primitives, enums, nominal records, fixed arrays, field/index reads, calculations, local functions, copied parameters, lexical scopes, assignment, branches, bounded loops, assertions, returns, explicit failure, send, publish and awaited request. int arithmetic uses exact intermediate integer operations before range checks. real uses binary64. Scalar byte/uint/int/real widening preserves values. Checked assignment into uint or byte rejects values outside the destination domain. Payloads, parameters and results are copied values.

State initialization, entry/exit, event payloads, guards, priority, timer generations, self transitions, completion reactions and finality are interpreted. Entry/exit lack event payload scope. Stale timers and inputs to a terminated machine are observable discards. A return or failure terminates the machine without completing later transition phases. A guarded completion cycle fails before taking edge 1,025.

These rules have regression tests for both success and failure, including callback blocking, snapshots across suspension, fanout partial rejection, freshness boundaries, retry identity, duplicate RPC, exact-time timeout races, arithmetic/bounds failures, timer cancellation, backlog limits and queued-input deadlines. They exercise observable semantics rather than only mirroring parser implementation.

Evidence and limitations

`status` reports why scheduling stopped: `quiescent` (no scheduled events), `horizon` (future events remain), `violation` (root contract/resource failure), or `limit` (fuel exhausted). `outcome` is `failed` when a failure/violation occurred and `no-failure-observed` otherwise. A failed ordinary activation need not stop unrelated components, so a quiescent run may still have a failed outcome. `timeNs` is the last processed event time, including inert expiry/deadline bookkeeping; it is not necessarily the horizon or last visible effect.

Trace records have monotonically increasing `ordinal`, nondecreasing `timeNs`, `kind`, and `subject`. Activation (`a`), correlation (`r`) and logical message (`m`) IDs are unique within a run; pair them with the evidence artifact's run identity when combining runs. Emissions carry producing activation, leaf endpoints, member, route, offer number and copied payload; source text preserves the boundary delegation provenance. Deliveries carry attempts and correlation where relevant. Statement records include source lines. Final stores and machine states remain available after limits and violations.

The CLI supplies SHA-256 per source; the portable library returns exact sources and leaves hashing/storage to its host. Keep those sources, digests, selected root, complete options, profile version and output together. Absolute source URIs may identify the local workspace; the published hello artifact uses stable repository-relative file URIs.

The implementation provides **reference simulation evidence with an explicit logical-time abstraction**. It is not a blanket E certification, model checker, production runtime, KerML exporter, AUTOSAR generator or graphical editor. Physical WCET/schedulability, distributed clock error, real serialization, middleware outages and independent conformance assessment still require separate evidence. A host can call the portable API after building and validating the entire referenced workspace.

Extended control flow and interruption

The 1.3 profile executes while, counted for, copied-array forEach and post-test do loops with their documented scopes and bounds. An armed interruptible region receives typed data/events through its connector queue, transfers control in its current serial activation, cancels a pending caller continuation and preserves earlier effects. Available input outside the armed interval is discarded explicitly. Region and cancellation events retain activation/correlation identities. See the control-flow guide for examples and the operational contract for ties and late responses.

`projectFunctionTiming` derives observed ready/running/request-wait phases and markers from these traces. Declared timing values remain separate. Open horizon/stop intervals are retained, and zero-cost execution is never reported as physical CPU duration. Reproduce the four showcase traces with `node scripts/build-timing-demo.mjs`; the normal gate checks their exact generated projection.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Migrating from SWAML 1.0 to 1.1

SWAML **1.1.0** is the compact authoring edition. It retains explicit `def` type declarations, the component → unit → operation ownership model and every interface, behavior, timing and graphical requirement. It intentionally changes source syntax despite the minor version number. Do not treat 1.1 as a drop-in parser for 1.0 models.

Published 0.1 and 1.0 tags, PDFs and references remain historical artifacts. Keep an original model copy and its pinned tool version when preserving old simulation evidence. Interface versions such as `version "1.0.0"` are independent contract identities; syntax migration does not change them.

Select the language edition in the host

Remove `swaml "1.0";` from source files. A 1.1 file begins with `package`, or is empty during editing. Use the 1.1 language-core package/CLI; an editor host must pin the supported edition before loading its workspace. No project configuration file format is invented by the model language, and the current CLI does not search for one. The 1.1 parser rejects old headers rather than silently accepting or guessing a version.

Keep definitions explicit; make roots component occurrences

`component def Assembly` and `unit def Controller` remain unchanged. `def` is required for a reusable type. A type declaration never starts a singleton or selects a simulation root.

Replace a 1.0 assembly wrapper with a named root component while preserving its child names and connections:

TEXT

```
// 1.0
system Product realtime {
    component controller: Controller;
    node ecu mcu;
    deploy controller on ecu;
}

// 1.1  -  same child path, routes and allocation
component Product realtime {
    component controller: Controller;
    node ecu mcu;
    deploy controller on ecu;
}
```

The profile can be `service`, `realtime` or `mixed`. Omitting it means service. Do not remove a realtime/mixed profile: doing so would weaken the model's required timing contracts.

An inline root can contain units directly. This avoids introducing a definition used only to hold one unit. Alternatively a root can instantiate a reusable assembly:

TEXT

```
component def Assembly {
    unit controller: Controller;
}
component Product: Assembly;
component DeployedProduct: Assembly {
    node application process;
    deploy controller on application;
}
```

The typed root's body is a deployment binding, containing only `node` and `deploy`. It cannot add units, components, ports, connectors or delegation. Its architecture comes entirely from `Assembly`. Binding names must not collide with members inherited from that definition. An inline root has its own composition, but no outward ports/delegation. Required boundary ports on a reused definition retain their closure obligations.

Selecting `Package::Product` executes that root occurrence. Simulation, deployment and case paths are relative to it: a child is `controller`, not `Product.controller`. Replacing a wrapper with `component Product: ExistingType;` may shorten the old child paths; make that structural refactoring separately and update every case, view, deployment, manual release and expected store path. Merely changing `system` to `component` preserves paths.

Put states directly in their operation

Remove the `machine` marker. Keep the `initial` declaration first; states follow, then transitions. The operation still has no parameters, has a Void result, and starts with its unit.

TEXT

```
// 1.0
operation lifecycle(): Void on start machine {
    initial Idle;
    state Idle {}
}

// 1.1
operation lifecycle() on start {
    initial Idle;
    state Idle {}
}
```

This is still stateful behavior with the same entry/exit, guard, priority, timer, completion and finality rules. `StateBody` is a grammar grouping, not a second authored executable element. Runtime documentation may use the mathematical term “state machine” without reintroducing a source declaration or performer.

Trim expressions and no-result annotations

1.0 source	1.1 source	Meaning retained
`operation stop(): Void`	`operation stop()`	missing result means Void; explicit `: Void` remains valid
`set count = count + 1;`	`count = count + 1;`	whole-value assignment to an existing permitted cell
`compute clamp(value)`	`clamp(value)`	pure function in the same unit
`array [1, 2, 3]`	`[1, 2, 3]`	nonempty fixed array value
`enumValue Status.Ready`	`Status.Ready`	nominal enum value, subject to lexical resolution
`enumValue Contracts::Status.Ready`	`Contracts::Status.Ready`	explicit package-qualified enum value

Only operation implementations and interface signatures may omit result annotations. Function results remain explicit and non-Void. Variable, parameter, field and local types remain explicit. No arithmetic, copying, scope or communication behavior is inferred from these shorter spellings.

A local or unit value named `Status` that is accessible in the current expression environment makes `Status.Ready` a field read, even if an enum has that name. A missing field is an error; the resolver does not

fall back to the enum. Qualify `Contracts::Status.Ready` when the enum is intended. Unqualified nominal names retain the usual local-package/direct-import ambiguity checks across declaration kinds. Pure functions expose only parameters, and variable initializers expose no mutable values; inaccessible unit variables do not shadow enums in those contexts. Qualified enum syntax never grants access to another unit's private values.

The removed words `swaml`, `system`, `machine`, `set`, `compute`, `array` and `enumValue` are no longer reserved. The current grammar has 122 reserved keywords, including type names, units, roles and policy values. Remaining words have defined modeling work: `record` identifies nominal construction, `invoke` distinguishes imperative local calls, and `request`/`send`/`publish` retain communication semantics.

Preserve architecture and check evidence

No 1.1 simplification removes interface versions, typed ports, handler ownership, emission provenance, explicit connectors/delegation, mandatory QoS, RPC timeout/failure branches or bounded loops. All exchange between distinct units/components still follows interfaces. States and operations retain direct ownership. Browser, GV, IV, CV, SV, GRV and FV remain.

Run `npm run check` after migrating. It checks current examples, executable documentation, reserved-word coverage, grammar mapping, simulation values and trace freshness. The simulation binding is now `swaml-reference/1.1`; its selector and result field are named `root`, replacing `system`. Rebuild any host using generated AST types: `RootComponent`/`RootMember` replace `System`/`SystemMember`, `StateBody` replaces `Machine`, and enum values resolve from `ReferenceExpression` using `ValuePath` rather than an `EnumLiteral` node.

A successful migration should preserve expected stores and observable effects after accounting for intentional occurrence-path edits. Exact trace source lines and source digests change when text changes. Do not overwrite historical evidence to imply it was generated by the new parser. Logical-time simulation continues to require a finite horizon and explicit resource bounds; it is not a physical scheduling proof or a KerML-conformance claim.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Migrating to SWAML 1.2

The 1.2 language uses **components → functions → statements or states**, with **calculations** for pure expressions. This is a deliberate syntax change from 1.1 and earlier unpublished 1.2 drafts. Historical publications remain unchanged. ADR 0007 records the ownership, reuse and integration decisions.

Migrate 1.2.0 to 1.2.1

Version 1.2.1 makes subject-assignment syntax consistent. Replace case `for` and satisfaction `by` with `of`. Compact views already use `of`. These contextual edits preserve root selection, satisfaction direction, case traces and execution behavior. Do not replace ordinary prose, string contents, `on` triggers/deployment destinations or `as` result bindings.

1.2.0 fragment	1.2.1 fragment
`useCase Start for App {}`	`useCase Start of App {}`
`verificationCase Check for App { verify Ready; }`	`verificationCase Check of App { verify Ready; }`
`satisfy Ready by App;`	`satisfy Ready of App;`

The current parser rejects the old relation spellings. Hosts select the 1.2 family; the installed package revision determines this concrete grammar. Pin the package/release as well as the family when reproducing an older model. `for` has no relation meaning in 1.2.1; bounded `while (...) bound n` remains the supported loop syntax. See ADR 0010.

Convert the model

Earlier source	Current 1.2 source	Meaning
`unit def Worker`	`component def Worker`	One software boundary may own both behavior and children.
`unit worker: Worker;`	`component worker: Worker;`	Each instance retains distinct private state.
`operation handle(...)`	`function handle(...)`	Apply to executable bodies and interface signatures.
`function twice(x: real): real = x * 2.0;`	`calc twice { in x: real; return result: real = x * 2.0; }`	Move each calculation into its owning function. Calls remain `twice(x)`.
`node ecu mcu; deploy controller on ecu;`	`deploy controller on "sysml:Vehicle::ecu";`	Resolve the external system/part identity in the host binding.
Manual input key `operation`	Manual input key `function`	Use reference simulation profile `swaml-reference/1.2`.
Component-owned `calc`	Declare it first inside each owning function	Calculations are function-local, including states and guards. Reuse across functions through a value-returning function.
`invoke helper();`	`call helper();`	A local void call. Use `as result` for a non-void return.

Earlier source	Current 1.2 source	Meaning
`request api.read() as value within …`	`call api.read() as value within …`	Keep the explicit route, timeout and failure branch. Case RPC steps also use `call`.
`let value: int = 0;`	`var value: int = 0;`	The enclosing block determines local storage.

These are contextual fragments. Preserve each function's body, trigger, route, timeout and timing when renaming it. Convert old pure functions before renaming operations so effectful and pure behavior remain distinct. Obsolete spellings are rejected; the parser does not silently reinterpret them.

Components that previously only grouped child units can remain as meaningful boundaries. Both composite and leaf components can now own private state and functions. Functions own calculations. A parent function communicates with a child through a connector between an owned port and an immediate child port; private child access remains invalid. Remove a wrapper only after deliberately updating its paths, ports, delegation, cases, views and deployment references.

Reuse a component type

`component def Tuned specializes Controller { … }` inherits one base. Prefix compatible inherited-member replacements with `redefines`; adding a distinct new member needs no prefix. Inherited-name collisions without the prefix are errors. Specialization and effective containment are acyclic, and every instance has its own inherited values and function state. The specialization example demonstrates the complete source form.

A typed root remains an instance with a deployment-only binding body. Use a specialized definition when changing reusable behavior or topology, then instantiate it. An inline root can own its own functions and variables as well as children. Deploying the root name establishes the default destination for its entire subtree.

Keep data and communication explicit

bool, int, real, string, result-only void, records, enums and fixed arrays remain. uint adds nonnegative safe integers; byte adds values 0 through 255. Integral assignments into these narrower domains are checked. Scalar numeric widening preserves values; payloads remain copied values.

Named complete policies reduce repetition without merging queues:

```text
qos def LocalLink {
    delivery atMostOnce; ordering fifo; capacity 8; overflow reject;
}
connect frontLink: front.setting -> settings.api qos LocalLink;
connect rearLink: rear.setting -> settings.api qos LocalLink;
```

Every policy is complete, and every effective connector use is validated for its interface and timing obligations. Selecting the same policy never shares queues, correlations or private state. See the complete policy model.

Update hosts and check fixtures

Pin the parser to language **1.2** before loading headerless source. The CLI uses its compiled 1.2 pin and rejects unsupported requested editions:

```sh
npm ci
npm run build
node out/src/cli/main.js --language-version 1.2 examples/15-component-specialization.swaml
```

Regenerate the AST. Executable functions use `FunctionDef`; pure calculations use `Calculation` and `CalculationCall`. The old Unit, UnitMember, Definition and Node rules are removed. ComponentMember includes structural members and executable functions. FunctionDef owns its calculations; FunctionFlow lowers to Block, with an absent flow meaning an empty body. LocalVariable replaces Let, LocalCall replaces Invoke, and RemoteCall replaces Request. Resolve effective inherited members before building diagrams, instance stores or path services. Deployment holds an external `system` string, and the host owns its resolution.

Update simulation input keys and collected evidence to `swaml-reference/1.2`; rerun fixtures rather than relabeling historical traces. Keep named-policy elaboration through the effective-policy helper. All diagrams remain flat and project the same checked semantic subjects. Historical 1.0 syntax first needs the 1.1 migration, followed by the changes above.

Final 1.2 authoring vocabulary

Change uppercase primitive/view keywords to lower camel case: `int`, `string`, `void`, `iv`, `fv` and their peers. Replace generic case declarations with `useCase` or `verificationCase`. Replace string-only requirements with bodies containing `doc /* ... */` and optional typed attributes. Replace case satisfies clauses with package-level `satisfy Requirement of Design.path;`; add `verify Requirement;` only to a verificationCase. A use case is not a satisfaction source. View blocks can now combine include clauses and filters. See the requirements guide.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Migrating to SWAML 1.3

Version 1.3 adds loops, interruptible flow and function timing diagrams. Pin the host/parser to **1.3** and use the `swaml-reference/1.3` simulation profile. The current parser rejects requests for older or inferred editions; use an archived 1.2 release to reproduce its exact grammar.

Source changes

Existing 1.2.1 constructs retain their meaning, including `of` relations, component ownership, copied values and explicit communication routes. New reserved words are `for`, `forEach`, `do` and `interruptible`; rename earlier identifiers with those spellings. Files remain headerless. The new loops are optional; bounded while loops continue to work.

Use `forEach`, not two separate words. Counted for headers require one var declaration, Boolean condition and assignment update. while, for and do loops require an explicit bound. forEach uses its array length and does not accept an additional bound. The complete syntax and scope rules are in the control-flow guide.

Interrupts and traces

An interruptible region adds no free event or communication route. Bind its handler to an incoming member of an owned typed port and keep exactly one receiving owner. Functions that declare regions must be triggered serial flow functions. Regions cannot be introduced into reentrant helpers or state actions as a substitute for their existing behavior rules.

Interrupted pending requests gain a terminal caller-side cancellation observation. Earlier server effects remain committed, and late responses remain correlated and discarded. Trace consumers must recognize regionEnter, regionExit, interrupt and requestCancelled events. Do not reinterpret old profile traces as if they contained these observations.

Timing presentation

Timing diagrams are an FV mode; there is no new view-kind keyword. Compact and block FV selections still target executable function occurrences. Trace-backed timing diagrams must keep declared contracts separate from observed phases and preserve unfinished activations at a horizon or stop. Existing flow/state views remain available.

The published 1.2.0 and 1.2.1 consolidated references, PDFs, manifests, checksums and release assets retain their versioned filenames. Current generation writes the separate 1.3.0 edition.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Loops, interrupts and function timing

SWAML 1.3 adds three loop forms and interruptible regions. FV presents flow, states, timing and an interrupt example through the same component-owned behavior model.

Choose a loop

Form	Test and bound	Scope
`while (condition) bound n { … }`	Test first; zero or more iterations; explicit bound.	Fresh body locals each iteration.
`for (var i: int = 0; i < 3; i = i + 1) bound 3 { … }`	Initialize once; test first; update after body; explicit bound.	Header variable remains inside the loop.
`forEach (item: int in values) { … }`	Visit each element of one copied fixed-size array.	Fresh typed element and body locals each iteration.
`do { … } while (condition) bound n;`	Execute at least once, then test; explicit bound.	Body locals are not visible in the condition.

The spelling is `forEach`, following lower camel case keywords. Conditional bounds must be positive safe-integer literals. If a condition is still true after the permitted number of body executions, execution fails before an extra iteration. A return or failure exits the enclosing function/frame. There are no break, continue, range shorthand or unbounded loops.

SWAML

```
package LoopGuide {
    component Demo {
        var total: int = 0;
        function run() on start {
            for (var i: int = 0; i < 3; i = i + 1) bound 3 {
                total = total + i;
            }
            forEach (value: int in [1, 2, 3]) {
                total = total + value;
            }
            do { total = total + 1; } while (total < 10) bound 2;
            assert total == 10;
        }
    }
}
```

The complete loop model also demonstrates nested loops and mutation of the original array during snapshot iteration.

Interrupt a protected region

Use an owned input port member to interrupt a named region:

TEXT

```
function run() on start {
    interruptible work {
        call job.perform() as value within 50 ms
            otherwise { fail "Job failed"; };
```

TEXT / CONTINUED

```
        status = value;
    } on receive control.stop {
        status = 2;
        why = reason;
    }
    // Both normal completion and the interrupt handler continue here.
}
```

Here control.stop is an event with a string parameter named reason. A sender reaches it through the ordinary typed connector; an external actor label cannot deliver it directly. The complete interrupt model includes the contracts, sender, server, routes and a simulation configuration.

The handler replaces the remaining protected work. It retains the same activation and deadline; it does not run concurrently with the protected body. Previous changes remain committed. If a request is pending, its continuation is cancelled and its later reply cannot resume the function. The server may still finish and retain its effects. Input arriving before entry or after exit is explicitly discarded.

The reference example starts a request at 0 ms, commits the server effect at 5 ms, receives the stop event at 7 ms, completes the interrupt handler at 7 ms and discards the late reply at 10 ms. The final worker status is 2 and the server's committed count remains 1.

Interruptible regions require a triggered serial flow function. Receive transitions remain the stateful-function mechanism. Nested regions are supported with distinct interrupt members; an outer interrupt unwinds the active inner region. A protected region can call helpers, but helpers cannot declare their own interrupt endpoint. Handler payload bindings cannot shadow enclosing values.

Read an FV timing diagram

Select **Timing** in the showcase's function view. The timing model provides two independent clients and servers, including repeated activations. Four trace scenarios show delayed success, timeout with a late reply, an observation horizon and a deadline stop. Select a function occurrence or activation to inspect its events and source.

Drawing	Meaning
Release and dispatch markers	Activation readiness and the start of its dispatch.
Ready interval	Waiting before dispatch, including serial backlog.
Running marker or interval	Observed active phase; zero-cost steps retain zero duration.
Request-wait interval	Suspension while the caller awaits its correlation.
Completion, failure or interrupt marker	The observed lifecycle event for that activation.
Open interval at the boundary	Still unfinished when the trace stops or its horizon is reached.
Declared deadline marker	Absolute release-plus-deadline contract boundary.

WCET, deadline and concurrency are shown separately as declared values. Waiting time is not CPU execution time. A bounded logical-time trace supplies observations, not physical WCET measurement, production schedulability or a universal verification result. Trace identity, profile, selected root and source digest remain inspectable.

The portable `projectFunctionTiming` API consumes trace evidence and optional occurrence contracts. It retains source ordinals, activation/correlation identity, cancellation and late responses without mutating the

supplied evidence. The checked-in showcase projection is regenerated with `node scripts/build-timing-demo.mjs` and verified by the normal language gate.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

ADR 0011 - Loops, interruptible flow and function timing

Status: accepted for SWAML 1.3.0. Date: 2026-10-18.

Control flow

Add counted `for`, copied-array `forEach` and post-test `do … while` alongside `while`. Conditional loops require an explicit positive safe-integer literal bound. `forEach` is bounded by its fixed array length and accepts no second bound. These forms preserve copied values, lexical scope, range checks and statement-boundary scheduling.

The counted header contains one `var` initializer, one Boolean condition and one assignment update. The initializer executes once; the condition executes before each iteration; the update executes after a normally completed body. Header variables exist only in the header and body. Each body iteration has a fresh child scope. A `forEach` evaluates and copies its array once, then supplies a fresh copied, typed element to each iteration. Changes to the source array or element local do not alter that snapshot. A do loop executes its body before its first test; body locals are not visible in its condition.

Interruptible regions

`interruptible work { … } on receive control.stop { … }` defines a named protected region and one interrupt handler. The endpoint is an incoming data/event member of a typed port owned by the same component. The function owns the receiving behavior; delivery uses the existing explicit route and queue. There is no global interrupt or new architectural participant.

Regions are limited to triggered serial flow functions. Stateful functions use their receive transitions; reentrant functions and untriggered helpers cannot declare regions. Helpers called from a protected body can be unwound. Region names are unique in their function, and an interrupt member cannot also be an activation trigger or another region's interrupt member. Existing single-handler and delegation rules still apply.

A region is armed from entry to normal exit or interruption. Available input outside that interval is discarded with an explicit undeliverable-interrupt observation. Delivery selects the active region for its member, disarms it, and transfers control at a statement boundary to the handler. Nested regions unwind when an outer region is interrupted. The handler sees enclosing values and copied payload bindings, but cannot see protected-body locals. After normal handler completion, execution continues after the region.

Interruption retains the activation identity, serial lock, snapshot and deadline. It cancels an outstanding caller-side request continuation, including its otherwise branch, and makes a still-pending correlation terminal with an Interrupted reason. It does not cancel server execution or roll back effects. A late reply is discarded. Existing terminal outcomes remain immutable: timeout wins its correlation race at expiry, even if a later interrupt transfers control before the queued failure continuation runs. Already scheduled continuations cannot resume the unwound stack.

Return and failure still terminate their function/frame and disarm regions. A region does not catch ordinary runtime failures. Interrupt dispatch bypasses ordinary activation dispatch only to resume its existing serial activation; it creates no parallel handler activation. This is cooperative model-level cancellation, with no claim of processor-interrupt latency or physical preemption.

Function timing diagrams

Timing is a further FV projection, alongside statement flow and function-owned states. Keep the existing view kinds. A timing projection shows selected function occurrences, activation identities, release/dispatch, ready/running/request-wait phases, completion/failure, interruption and correlated late responses on a common logical-time axis.

Declared WCET, deadline and concurrency remain distinct contract values. Observed bars come from an identified trace with model/run provenance. Zero-cost simulation execution remains a zero-duration marker; it must not become a measured CPU duration. Horizon or stopped runs retain open intervals, and missing completion is not invented. Reused definitions and concurrent activations keep distinct lanes. Layout cannot create scheduling or verification evidence.

Edition and evidence

The new keywords can conflict with earlier identifiers. The current parser and simulator therefore select language/profile 1.3 explicitly; archived 1.2 releases retain their grammar and evidence. Source migration is described in the 1.3 guide.

Acceptance covers positive and malformed parsing, scope/type/bound failures, observable loop stores, interruption of suspended and nested execution, timeout ties, duplicate/inactive input, independent occurrence identities and reproducible timing projections. The canonical catalog and interactive showcase use source-backed contexts. Target bindings still supply physical scheduling and cancellation-latency evidence.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

SWAML 1.3 user guide

Architects familiar with UML: start with the concept correspondence and modeling workflow.

SWAML describes what software contains, how it communicates, and how functions and state evolve. The model remains independent of C++, Rust, an RTOS, a cloud runtime, or a particular network protocol.

1. Install and check a model

From the repository root:

```sh
npm ci
npm run build
node out/src/cli/main.js examples/01-hello.swaml
```

A successful result is `Checked 1 model(s): 0 error(s).` This checks the implemented rules; it does not execute the software or prove schedulability. `npm run check` also runs the repository tests. Open the implementation matrix before interpreting a passed check as a stronger assurance claim.

A file begins directly with named packages. Use the 1.2 language core or pin SWAML 1.3 in the editor host; there is no per-file version header. The 1.2 parser rejects historical headers rather than guessing a version. Names are case sensitive. Keywords cannot be identifiers. Use `orderLink`, for example, rather than the reserved word `ordering`.

2. Write the contract first

Choose the communication model by the meaning of the exchange:

Need	Interface	Roles	Function statement
A current sensor value	`senderReceiver`, sampled data	sender → receiver	`send p.member(value);`
Every queued reading	`senderReceiver`, queued data	sender → receiver	`send p.member(value);`
Request and correlated response	`clientServer`	client → server	`call p.member(...) as result within … otherwise …;`
One event to many subscribers	`publishSubscribe`	publisher → subscriber	`publish p.member(...);`
RPC plus server notifications	`service`	requires → provides	call and publish

Each connector uses one nominal interface definition. Equal-looking interfaces are not automatically interchangeable. Create an adapter component when their meaning, payload, or version differs.

This is a complete small model; save it as `example.swaml`:

```swaml
package First {
    interface senderReceiver Reading version "1.0.0" {
        data value: real sampled;
    }
```

SWAML / CONTINUED

```
    component def Source {
        port output: Reading sender;
        function sample() on start {
            send output.value(21.5);
            return;
        }
    }
    component def Sink {
        port input: Reading receiver;
        var last: real = 0.0;
        function update(value: real) on receive input.value {
            last = value;
            return;
        }
    }
    component def Pair {
        component source: Source;
        component sink: Sink;
        connect readings: source.output -> sink.input qos {
            delivery atMostOnce; ordering fifo; capacity 1; overflow dropOldest;
        };
    }
    component Demo: Pair;
    view overview gv of Demo;
    view wiring iv of Pair;
    view updateFlow fv of Sink.update;
}
```

`component def Pair` is a reusable type; `component Demo: Pair;` selects one root occurrence for execution. Its omitted profile means service. `Reading` defines the payload. Both ports use it with complementary roles. `sample` sends through its own port, and `update` declares that same port-member contract as its activation source. The connector allows the transfer. The variable `last` belongs to each `Sink` instance, not to every instance of its definition.

Reuse a communication policy

When several connectors have the same QoS, declare a complete policy once at package scope:

TEXT

```
qos def LocalLink {
    delivery atMostOnce; ordering fifo; capacity 8; overflow reject;
}
// Inside a component, explicitly select it on each route:
connect frontLink: front.setting -> settings.api qos LocalLink;
connect rearLink: rear.setting -> settings.api qos LocalLink;
```

The complete service-setting model uses this form. Inline blocks remain valid. A named policy shares values, while every connector keeps its own queues, directions and member identities. Qualified names and direct imports follow normal namespace rules. The validator checks the actual interface and root at every use, so a generic policy cannot bypass sampled buffering, idempotency or realtime bounds. The 1.2 migration guide explains the AST update.

3. Choose component and component boundaries

Use a **component** for a cohesive software boundary: an embedded controller, a service backend, or a subsystem. It may own functions, private data and child components. A component need not be a thread, file, class, or process; deployment is separate.

A component definition owns component instances, ports and connectors. `def` makes this type declaration explicit, following the familiar SysML definition/usage distinction. A root is an occurrence: use `component`

`Demo: Pair;` to instantiate a definition, or `component Demo { component source: Source; … }` to assemble components directly. Statements belong to functions; a root can own functions and private variables. This keeps executable ownership unambiguous.

A function can call a local helper with `call helper(...);`. The helper is an untriggered flow function in the same component. For a value result, write `call helper(...) as result;`. Pure calculations use expression syntax such as `scaled(...)` only inside their owning function. These local calls do not need ports because they do not cross the agreed component boundary.

Never write `otherComponent.function()` or `otherUnit.privateValue`. The syntax and validator reject that access. To reach another component, declare a port, choose an interface, connect it in their common parent, and use `call`, `send` or `publish`.

For a nested component, expose a boundary port and delegate it to one immediate child port. The service example demonstrates `Backend.api` delegating to `serviceUnit.api`. Delegation preserves the port's role and interface; it does not reverse them or convert data.

4. Describe function control flow

Functions contain ordered statements: typed local declarations, assignments, assertions, conditionals, bounded loops, local calls, interface exchanges, returns and failures. Read Values and flow as a worked example.

Functions and interface signatures may omit `: void`; a missing result annotation means exactly void. Calculations are expressions with typed parameters and an explicit non-void result. They cannot read mutable component data, issue requests, or emit events. Pass their inputs explicitly. Records, enums and arrays are values, so sending them does not expose a remote reference to local mutable data. Write `count = count + 1;` for whole-value assignment, `clamp(value)` for a pure local calculation, `[1, 2, 3]` for an array, and `Status.Ready` for an enum value. Use `Contracts::Status.Ready` when a local value named `Status` would otherwise hide that enum.

A `while` includes a maximum iteration count. Exceeding that count at runtime is a failure, not permission to silently stop the loop. This makes the bound useful to a later analysis or code generator.

Use `assert` for a model-level assumption that must hold at that point. A requirement is a named prose obligation. A case can link to requirements using `satisfies`; that trace link is not proof that the requirement is met.

5. Model a service call and its failure

Read Services and pub/sub. The service declares `submit` in its interface; the server component implements it with `on call api.submit`. The client's `call` includes a timeout and an `otherwise` block.

A request binds its result only on success. The failure block has a local `failure: string` value and must return from or fail the current function. This prevents subsequent statements from reading an uninitialized result. A timeout says the caller stopped waiting; it does not prove the server did nothing. Later responses to that timed-out request are ignored.

For application-specific errors, use an enum or result record in the function's declared return type. Transport failures and application results are different concepts. Do not retry a request unless its idempotency and retry budget are established. The core rejects `atLeastOnce` RPC when the function is not declared idempotent; proving that declaration is an implementation obligation.

6. Use publish/subscribe without a hidden event bus

An event is a member of a `publishSubscribe` or `service` interface. Only a function can publish it, through one of its own outgoing ports. Subscribers receive it through their own typed ports. The normal handler uses `on receive input.eventName` with the event's parameter types in order.

Fan-out is multiple explicit connectors from a publisher port to subscriber ports. Each subscription has its own queue and QoS. A slow subscriber does not silently change the queue of another subscriber. Multiple publishers may connect to one subscriber; ordering is per connector, not global.

In a combined service, the connector is drawn from `requires` to `provides` for requests. Service events travel back from provider to consumer, and replies return on the originating request's correlation. Both remain members of the same declared interface.

Model a broker as a component with incoming and outgoing interfaces if broker behavior, failure, buffering or deployment matters. A connector alone represents a logical subscription, without selecting MQTT, DDS, Kafka or SOME/IP. Dynamic discovery and unsubscribe functions are reserved for future language versions.

7. Put states inside a function

The stateful function example owns `Idle`, `Running`, and `Stopped` inside `Device.lifecycle`. It starts once with its component. No performer or external behavior allocation is required.

A stateful function has no parameters, a void result (usually omitted), and `on start { initial Idle; … }`. The initial reference follows any function-local calculation declarations and precedes all states. There is no extra behavior wrapper. It declares an initial state, states with optional entry/exit blocks, and transitions. Triggers are an incoming port member, a positive delay after entering a state, or completion. A receiving transition makes event parameter names available in its guard and effect; sender-receiver data uses `payload`.

This complete model places the states directly where they execute:

SWAML

```
package TimerExample {
    component def Device {
        var cycles: int = 0;
        function lifecycle() on start {
            initial Waiting;
            state Waiting {
                entry { cycles = cycles + 1; }
            }
            state Finished final {}
            transition complete: Waiting -> Finished on after 10 ms {}
        }
    }
    component Demo { component device: Device; }
    view lifecycle fv of Demo.device.lifecycle;
}
```

The root contains its component directly. The delay is local to that function; it is not a free event or cross-component channel. The interface-triggered lifecycle model adds commands and status ports.

On a transition: run the source exit block, the transition effect, then the destination entry block. A final state completes the stateful function. A received event with no enabled transition in the current state is consumed and ignored. Add an explicit transition if ignoring it is unacceptable.

Use one stateful function as the receiver for a given member, or one ordinary function. Do not bind the same member to both: that would create ambiguous delivery.

8. Model realtime constraints

The controller example uses `on every 10 ms`, execution-time bounds and deadlines. A root with a realtime or mixed profile requires timing contracts for every used function and maximum connector latency. Sampled realtime data also needs `maxAge`.

The validator checks positivity, `wcet <= deadline`, and `deadline <= period` for a periodic function. These are necessary checks, not a scheduling proof. Document the actual RTOS policy, interrupt load, CPU allocation, priorities, clock assumptions, transport timing and measured/analyzed WCET in the implementation project. A production backend must establish them before claiming hard realtime execution.

`serial` is the default component-local execution policy. `reentrant` requires independent activations and must not mutate shared component state, including through helpers. A function waiting for a service response consumes elapsed deadline time even when it consumes no CPU execution time.

9. Separate deployment from architecture

Write `deploy instance.path on "sysml:Vehicle::ecu";` to refer to an external system or part. A general host identity such as `"system:production::checkout"` is also supported. SWAML checks the component path and reference syntax; the integration host must resolve the destination and validate its suitability.

A typed root may have a deployment-only body: `component Product: Controller realtime { deploy sensor on "sysml:Vehicle::ecu"; }`. Its `sensor` must already exist in `Controller`; the binding cannot add children, functions or connections. An inline root may combine composition, local behavior and deployments. Deploying the root name establishes its own location and the default for its children. More specific descendant bindings override that default; each explicit path may occur only once.

The component roots example shows reusable assemblies and concrete bindings. The software model retains its typed communication boundaries wherever components are placed. A SysML reference identifies an integration point; it does not import a SysML model or certify cross-language mapping.

Specialize a reusable component

`component def Tuned specializes Controller { … }` reuses one base definition. Prefix an inherited replacement with `redefines`; an unmarked name collision is invalid. Replacement ports and functions preserve their contracts, while bodies, pure expressions and initial values can vary. Child instances may use the original type or a specialization of it. Instances remain separate stores and connected peers. See the complete specialization example and ownership decision.

Add a pure calculation

Use `calc scaled { in value: real; in gain: real; return result: real = value * gain; }` at the start of a function body, then write `scaled(measured, 0.5)` in an expression. Inputs are copied values and the named output is a single pure result. Functions carry effects and control flow; calculations can appear in that function’s guards, local initializers and other expressions.

The basic types are `bool`, `int`, `uint`, `byte`, `real`, `string` and result-only `void`. Add nominal records, enums and fixed arrays such as `byte[8]` for structured values. uint and byte assignments check their ranges instead of truncating or wrapping.

10. Organize a larger model

Packages are flat namespaces in 1.2. Write `import Contracts::*;` to use another package's declarations unqualified, or name a declaration `Contracts::Reading`. All relevant files must be loaded together. Two imports exposing the same short name make it ambiguous; use the qualified name.

The editor host should load the whole model workspace. The CLI deliberately loads the files you supply and does not search unrelated directories. Do not put secrets in model payloads, examples or deployment notes.

11. Navigate through views

Surface	Purpose
Browser	Navigate packages → definitions → instances → ports/functions → states
gv - General View	Inspect structure, declarations and their relationships
iv - Interconnect View	Inspect connected ports and component delegation

Surface	Purpose
cv - Case View	Inspect cases and linked requirements
sv - Sequence View	Inspect the partial ordering of interface exchanges in a case
grv - Grid View	Inspect sortable model, interface, timing and deployment tables
fv - Flow View	Inspect a function's control flow or owned state transitions

These are specified editor behaviors. The current package supplies the model syntax and language server, not a diagram renderer. View declarations can already be parsed and checked. See graphical notation for the drawing and editing contract.

12. Resolve diagnostics

Diagnostic	Typical fix
`SW030`	Connect immediate child ports; expose/delegate a port across each component boundary
`SW031` / `SW032`	Use the same interface definition and complementary port roles
`SW034`	Connect the port in a closed root component, or mark an intentionally unbound port `optional`
`SW050` / `SW051`	Use a local port member and the correct communication verb/direction
`SW052` / `SW055`	Match handler parameters/results and give each incoming member one owner
`SW057`	Make every failure path return or fail
`SW062`	Give competing transitions distinct priorities

A parser error usually means a reserved identifier, missing delimiter, or malformed block. Resolve syntax errors first; semantic checks are skipped on syntactically incomplete documents to avoid misleading secondary errors.

Service fields, repeated consumers and checked traces

Service field models one temperature setting with explicit `get`, `setPoint` and `changed` interface members. The service stores the value privately. Its serial setter publishes through its provided port, which fans out to two required ports on separate panel instances. Each panel owns a distinct displayed value and its own receiving function.

The getter is reentrant and reads the private-state snapshot taken when it is dispatched. It can safely overlap the serial setter because it does not mutate storage, directly or through a helper. Snapshot implementation is an execution-backend obligation; parsing this example does not simulate it.

A case reply must follow a unique unanswered matching request through its `after` ancestry. Two outstanding requests on the same route/member cannot be disambiguated by drawing one reply lower on the page. Order the pairs or use separate cases. Requests may remain unanswered in a partial trace. A route-valid case still needs a function-level witness before it counts as verified behavior.

Use the owning package's name as a gv/grv subject for an inventory. iv takes a component definition/root component; fv takes a function, including an occurrence path such as `Cabin.settings.store.update`. cv/sv take a case or root component. A view cannot use another view as its semantic subject.

If a routed call returns void, its required `as` name marks completion but cannot be read as a value. Reserve `failure` for the implicit error-path binding. Write string line breaks with `\n`; raw line breaks, tabs and unsupported escapes inside strings are rejected.

The operational contract defines execution edge cases, and the conformance obligations distinguish validator acceptance from runtime and analysis evidence.

Find a construct or run an example

Use the alphabetical keyword reference for exact words and context, the concern guide for architectural questions, or the complete Markdown book for a single user-facing document. The maintained examples have executable simulation configurations; follow the reference simulation guide to run them and inspect their traces.

Requirements and formal documentation

Requirements own typed attributes and one optional `doc /* ... */` specification. Design elements link through `satisfy`; verification cases link through `verify`. Use cases describe intent. Ordinary comments remain independent notes. View kinds use lower camel case, and view blocks combine multiple selections and Boolean filters. See the guide and complete model.

Extended loops, interrupts and timing

Use counted `for`, copied-array `forEach` or post-test `do` when their scope and test order fit the behavior. Conditional loops retain explicit bounds. `interruptible` regions bind cancellation to an owned typed input route and preserve the activation deadline and committed effects. FV offers trace-backed timing and interrupt examples alongside flow and states. See the complete guide and 1.3 migration.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Software architecture rationale

This is an original comparison of modeling choices, not a compatibility claim. AUTOSAR sources below provide a conceptual baseline; target generation requires a separate mapping and evidence.

Concern	SWAML 1.3 choice	Reason
Software ownership	Components containing components and owning private values/functions	One concept covers atomic and composite software boundaries.
Executable behavior	Functions with statements or directly owned states	Decisions, loops, local calls and stateful reactions retain one behavior owner.
Pure values	Calculations with typed inputs and one named output expression	Guards and expressions can reuse calculations without side effects.
Architectural reuse	Definitions, instances, single specialization and explicit compatible redefinition	Related designs share declarations while retaining distinct instance state.
Communication	Versioned interfaces, owned ports and explicit routes	Every cross-component exchange remains traceable and typed.
Placement	External system/part identities	The software model can connect to a system model without duplicating its hardware ontology.
Data	bool, byte, uint, int, real, string, records, enums and fixed arrays	Copied values with explicit bounds and nominal contracts.

AUTOSAR Classic and Adaptive

Classic separates application software, runtime integration and basic software. Its software-component template defines sender-receiver and client-server interfaces, ports and runnable activation. SWAML preserves these architectural distinctions through components, contracts and triggered functions. Mapping a function to a runnable still requires an activation, parameter, task and scheduling binding. Classic overview, Software Component Template R21-11.

Adaptive Communication Management distinguishes methods, events and fields. SWAML's service contracts combine function signatures and events; an explicit getter/setter/change-event protocol represents a field. The language does not infer discovery, service-instance identifiers or ara::com configuration from a connector. Communication Management R21-11.

The complete architecture examples demonstrate a Classic climate component and an Adaptive diagnostic application. Their target bindings must preserve copied values, routes, response correlation, queue behavior, concurrency and timing. Model validation is not ARXML conformance or an RTE implementation.

UML and object-oriented implementation

UML provides components, classes, interfaces, operations, activities, state machines and interactions. SWAML chooses a smaller software architecture vocabulary: components define boundaries; functions define behavior; ports/interfaces define collaboration. A backend may implement a component as a class, process, service or group of objects, provided it preserves those boundaries. SWAML therefore does not require a separate class construct to describe object-oriented implementations. OMG UML 2.5.1.

Component specialization is deliberately bounded: one base, explicit compatible replacement, fixed instances and no reference-valued payloads. It does not imply UML generalization semantics or dynamic object creation. See the UML transition guide for correspondences and limits.

SysML 2.0 and KerML

SysML uses definition/usage modeling and includes calculations with inputs and a return. SWAML adopts a similar readable input/output form while restricting a calculation to one pure expression in a component-local context. This is an original subset choice, not a SysML importer or grammar alias. SysML 2.0 Language, clause 7.19.

An external `sysml:` deployment identity is an integration hook for a part or system described elsewhere. A host must resolve that identity and validate the allocation. Merely carrying a string does not load or validate a SysML model. The software hierarchy and the external system hierarchy keep distinct identities.

SWAML also defines a proposed KerML mapping contract. The Langium AST and coverage inventory are not a kernel export or conformance result. A real mapper needs a versioned library, kernel validation, source traces and round-trip tests. KerML 1.0.

Analysis and runtime boundaries

Architectural timing languages such as AADL emphasize explicit performance and deployment assumptions. SWAML includes timing contracts, but a deadline or external deployment reference alone supplies no scheduler or timing proof. SEI introduction to AADL.

For service and event-driven designs, SWAML makes queue capacity, overflow, duplicate delivery, partial fanout and timeout uncertainty explicit. A broker is modeled when its behavior matters. Storage, middleware and lifecycle guarantees remain part of a target binding. The execution contract and implementation matrix identify what acceptance and bounded simulation establish.

ADR 0007 records the 1.2 simplification. Primary-source links support the comparison; no external standard content or proprietary templates are redistributed.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Language tooling and integration

Package contract

The package supplies a reusable language core, command-line tools and a language-server adapter. It is marked `private` to prevent accidental npm publication. Consumers can build it locally and use the documented APIs with an explicitly selected language version.

This edition parses headerless **SWAML 1.3** only. A host must pin its language-core dependency and identify 1.2 in project/workspace settings before source loading; these settings belong to the host, not to an undocumented model grammar. The repository CLI is itself pinned to 1.2. Unsupported requested editions must be rejected explicitly; a host must never silently upgrade models to the latest parser. Read the migration guide before consuming the named-policy AST and host-pin update.

The source of concrete syntax is swaml.langium. Langium generates AST types, grammar metadata, reflection and TextMate syntax. Generated files and `out/` are ignored; rebuild them after checkout.

The implementation follows Langium's grammar → generated AST → references → validation workflow. Langium grammar reference, reference-resolution guide.

Reproducible commands

SH

```sh
npm ci
npm run build
npm test
npm run validate:examples
npm run check
node out/src/cli/main.js examples/01-hello.swaml
node out/src/server/main.js --stdio
```

Node 22 and 24 are the CI targets; Node 22+ is required. Dependencies are pinned in `package-lock.json`. `npm run build` regenerates the grammar before compiling. Never hand-edit generated files.

The package exports `createSwamlServices` from `out/src/language/swaml-module.js`, and AST declarations from `out/src/language/generated/ast.js`. Example usage after a build:

TS

```ts
import { EmptyFileSystem } from 'langium';
import { parseHelper } from 'langium/test';
import { createSwamlServices } from './out/src/language/swaml-module.js';

const { Swaml } = createSwamlServices(EmptyFileSystem);
const parse = parseHelper(Swaml);
const document = await parse(source, { validation: true });
console.log(document.diagnostics);
```

For multiple files, create all documents first and build them together so that imported declarations are available. CLI arguments are the complete loaded workspace. Definitions in unloaded files cannot resolve.

Services and source ownership

File	Responsibility
swaml-module.ts	Dependency injection and registration
swaml-document-builder.ts	Conservative workspace revalidation for paths and negative lookups
swaml-scope.ts	Local-owner scopes, imported/qualified declarations, ambiguity
semantic.ts	Instance/port/member paths, type identities, time conversion
swaml-validator.ts	Model invariants and typed behavior validation
CLI	File loading, diagnostics, exit status
LSP server	Standard language-server transport and startup

Cross-references use Langium references for declarations, base component types, calculations, local functions and states. Composite endpoint/value/view paths are strings with explicit SWAML resolution; deployment target identities are external strings. This keeps encapsulation checks explicit, but **generic rename/find-references/navigation does not cover those strings**. A host must supply path-aware providers before advertising full refactoring.

The current scope provider scans loaded documents for clarity. Every changed/added/deleted document conservatively relinks and revalidates all loaded SWAML models, including string paths and previously absent declarations. This favors correct editor diagnostics over incremental performance. Large-model dependency indexing and cancellation-aware expensive checks remain optimization work. The validator skips semantic checks after parser/lexer errors; built-in diagnostics still report malformed input.

Language-server integration

The server implements the Language Server Protocol over a standard transport. Start it with `node out/src/server/main.js --stdio`; reserve stdout for protocol messages and send diagnostic logging to stderr. Clients supply document URIs, text versions, changes and the complete loaded workspace. They consume diagnostics with source ranges and stable diagnostic codes.

Language hosts must select the supported SWAML edition before loading a document, preserve source ranges, and rebuild affected documents after changes. A formatter or graphical host must apply semantic changes to source and validate the result. File associations, transport startup and presentation are host responsibilities.

Browser hosts

The language core has no Node filesystem dependency. Browser applications can create services with an in-memory filesystem and supply document text directly. The Node CLI and language-server adapter own their filesystem access. A worker-based browser host needs its own transport adapter; test document loading, dependency bundling and import resolution in that environment.

Graphical architecture

Implement a semantic projection layer between Langium and diagrams. It should produce stable node/edge IDs, instance-aware paths, source ranges, declaration references, roles, signatures, diagnostics and presentation kinds. View renderers consume that projection, never parse source text independently.

All edits go through a source-edit service. Layout-only edits update a presentation sidecar. Semantic edits update `.swaml`, then parse/link/validate. Reject silent creation of unmodeled exchanges. Graphical contract.

Kernel backend boundary

Build a separate normalized model from the linked/validated authoring AST, then lower through a versioned SWAML semantic library onto KerML. Keep kernel IDs and authoring IDs traceable. The mapping chapter defines obligations; the JSON inventory is design metadata, not serialized KerML.

Validation and release gates

Before a host consumes a new language-core version, run `npm ci && npm run check`, inspect the spec/grammar changes and update versioned fixtures. A grammar change, even if additive, can reserve an identifier previously used in source. Release notes must explain migration.

Tests cover valid fixtures, illegal interface access, scoping, type errors, QoS, state/timing rules and the LSP transport lifecycle. Broader conformance claims require the additional tests in implementation status.

Simulation API for a separate host

After linking and validating every referenced document, call `simulate(models, "Package::Root", options)` from the package's `./simulation` export. It has no Node filesystem dependency. The host supplies a finite horizon and may supply manual releases, transport fault schedule and resource limits. The profile contract defines the exact scheduler, observability and limits. The CLI adapter adds file loading and SHA-256 source digests.

Present `outcome` and `status` together: a run can quiesce after a failed activation, and a bounded horizon is not a proof of liveness. Hosts can connect trace source lines and occurrence paths to fv/iv/grv selections. They must preserve the simulation contract and validate every semantic edit.

Component-model integration

Use the generated `FunctionDef` AST for executable `function` bodies and `Calculation`/`CalculationCall` for pure `calc` expressions. Build semantic projections from effective inherited members, retaining declaration and instance identity. `specializes` and `redefines` are validated source relationships; a generic AST reference is insufficient for context-sensitive inherited paths. Deployment destinations are external identity strings; hosts own resolution and platform validation. The portable language core must not acquire a filesystem or SysML download dependency to resolve them.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Implementation status - 1.3.0

The normative specification defines the bounded 1.3 language. The repository implements the Langium parser, CLI, language-server foundation and broad static checking. It also implements a bounded logical-time reference simulator; it does not implement a production runtime, KerML exporter or graphical editor. See the 1.2 migration, historical 1.1 migration, historical 1.0 evidence and historical 0.1 audit and conformance obligations.

Implemented and tested

Version 1.3 adds counted for, copied-array forEach and post-test do loops; typed interruptible regions in triggered serial flow functions; and a portable trace-to-timing projection with FV timing/interrupt showcase modes. Regression coverage checks scope, bounds, cancellation races, nested unwind, inactive input and source-backed diagram metadata. Graphical source editing and physical interrupt/timing guarantees remain outside the implementation.

Capability	Evidence
Headerless 1.3 grammar, portable string escapes, generated AST/TextMate syntax	generator, all examples, lexical regressions
Unified components, local behavior in composite/root components, explicit definitions/instances, deployment-only typed bindings	component-model parser and simulation regressions
Single specialization and compatible explicit redefinition, with distinct inherited stores	reuse, cycle, contract and isolation regressions
Executable functions with direct states or statements; typed calc inputs and named pure output	parser, purity, shadowing and simulation regressions
Function-local calculation scope, value-returning local functions, unified call and var syntax	lexical isolation, state actions/guards, inherited results, copying/ranges, failures and malformed-input regressions
uint and byte domains with checked integral conversion	static constant-range and runtime range regressions
Imports, qualified names, ambiguity across declaration kinds and owner-scoped references	multi-document and namespace tests
Interface member/role/signature checks; no cross-component function/data bypass	contract and isolation tests
Composition, delegation, required boundary/leaf closure, route uniqueness and cardinality	topology tests
Emitting-function provenance and unique handler ownership	SW050/SW055 tests

Capability	Evidence
Value typing, records/enums/arrays, expressions, calculations, void completion restrictions	values/flow and audit tests
Local calls, acyclic call/containment graphs, transitive reentrant write analysis	call-cycle and helper-effect tests
Named complete QoS policies and inline selections; imported/qualified resolution, effective-use validation and queue independence	1.2 policy regressions
Explicit QoS, sampled buffering and idempotency declaration checks	QoS tests
Positive safe bounds, WCET/deadline/period consistency and realtime completeness	timing and numeric tests
Stateful signature, transition priorities, graph reachability, finality and unconditional-cycle checks	state tests
External system deployment identities, case direction/dependencies and unique causal reply correlation	allocation/case regressions
All six view subject categories, package and occurrence contexts	view regressions
Conservative workspace relink/revalidation after change/addition/deletion	editor update tests including newly ambiguous names
LSP initialize/diagnostics/shutdown and CLI exit behavior	integration tests
Reference execution	all statements/expressions, startup/periodic/manual activation, machines, queue policies, RPC, snapshots, fault injection and bounded trace evidence
Classic, Adaptive, web and cloud architecture examples	four complete models and configured simulations; six timeout, transport, duplicate-delivery and partial-fanout regressions
Graphical ontology and end-to-end modeling walkthrough	shared diagram inventory, reproducible flat SVG/vector scenes, parsed subject and source-excerpt checks, interactive stage navigation
Production-to-spec/normalized mapping inventory	automatic coverage for every current grammar rule/type name

`npm run check` rebuilds, runs tests, validates every example, checks documentation links and executable snippets, and detects missing/duplicate grammar mappings. Mapping coverage is a documentation consistency check, not a KerML validation result.

The canonical graphical catalog and its SVG/PNG exports define the visual language, with inventory coverage and semantic example checks. The current notation is entirely flat and reproducibly exported.

`scripts/check-showcase.mjs` checks five interface patterns and four domain architectures against parsed definitions, occurrences, ports, routes and deployment allocations. The quick start, architecture patterns and UML transition guide provide authoring guidance. The React Flow showcase implements selection, navigation and layout interactions over curated checked-in models. It is an interactive presentation, not arbitrary graphical source editing; it makes no G conformance claim.

Limits of static acceptance

The reference core checks the static groups listed in chapter 8; it remains labeled **S, partial V** until an independent conformance suite audits every normative clause and boundary case. It is not a model checker. Acceptance does not establish useful emissions, guard feasibility, case realization, idempotency of implemented effects, termination without runtime failure, schedulability, absence of request cycles, or snapshot support on a particular target.

General arithmetic ranges and computed array bounds are runtime checks. Guarded completion cycles are protected by a specified runtime limit. Activation backlog is bounded by a required execution binding. These are specified execution/analysis obligations, not unimplemented syntax disguised as successful static proof.

string-valued paths now receive conservative dependency invalidation. Path-aware rename, navigation, reference search and persisted editor IDs still need dedicated IDE providers. The all-document rebuild is correct for the loaded workspace but may be expensive on large projects.

Specified, not implemented

Capability	Remaining implementation
KerML K backend	versioned SWAML library, normalized mapper, kernel validation, reverse traces and round-trip tests
Production E backend	physical CPU/scheduling, real middleware/serialization, platform clock/resource evidence and independent assessment; logical reference simulation is implemented
Timing/behavioral analysis	target scheduling model, case witnesses, idempotency and safety/liveness evidence
Browser and graphical G workbench	projections, layout sidecars, Properties, validated source edits and undo
Complete IDE refactoring	path-aware rename/find references/navigation and stable identity map
Platform exports	target bindings and serialization such as AUTOSAR ARXML
External system integration	resolve and validate deployment destinations, including SysML part identities; core checks local paths and reference syntax only

No blanket K, E or G certification is claimed. The reference simulation profile documents its executable scope and time abstraction. These implementation milestones have complete language-side contracts; specification coverage does not establish implementation conformance.

Intentionally outside 1.3

Multiple inheritance, generics, dynamic object creation, unbounded arrays, reference-valued payloads, hierarchical/orthogonal state regions, fork/join syntax, host-language code blocks, dynamic service discovery, runtime subscribe/unsubscribe, topic wildcards, exactly-once delivery, full requirements algebra and geometry require a versioned language proposal. They cannot be introduced as undocumented backend behavior.

Requirements and selected views in 1.2.1

Implemented: typed requirement attributes; one raw formal documentation block per grammar owner; lexical-comment projection with owner/source extents; separate useCase and verificationCase; satisfy/verify target and duplicate checks; lower camel case primitive/view keywords; occurrence-preserving multi-subject views and typed Boolean predicates. The exported documentationText, modelComments, viewCandidates and selectView helpers are portable. Requirement links are claims; verification cases are not an automatic test runner. The showcase includes source-checked diagram/table projections.

Relation syntax in 1.2.1

Cases and satisfaction claims use `of`, matching compact view subjects. Parsing rejects the former case `for` and satisfaction `by` spellings; qualified/imported targets and unchanged link orientation have regression coverage. Version 1.3 adds bounded for/do loops, finite copied-array iteration and serial interruptible flow regions. The current host selects 1.3 explicitly; use an archived 1.2 release for its exact grammar and profile.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

SWAML 1.3.0 release notes

Release date: 2026-10-18. Language family and reference simulation profile: 1.3.

The 1.3 edition adds counted `for`, copied-array `forEach` and bounded post-test `do … while` loops. Named interruptible regions receive typed input through existing routes and transfer control to a handler within the same serial activation. FV adds function timing and interrupt examples alongside flow and states.

The parser and simulator explicitly select the 1.3 family/profile. New keywords require contextual identifier migration; existing 1.2.1 semantics and component boundaries remain intact. See the migration guide, control-flow guide and design decision.

Validation includes loop parsing/type/scope/bound and observable-store regressions; typed interrupt routing, cancellation, nested unwind, inactive/duplicate input and timeout ties; portable timing projection tests; source-backed showcase metadata; complete example simulations; and exhaustive keyword, mapping and graphical coverage.

The reference simulator remains a bounded logical-time model. Interrupts are cooperative cancellation at statement boundaries. Timing diagrams distinguish declared limits from observed phases and retain open intervals at trace boundaries. Neither supplies processor preemption guarantees, measured physical WCET or production scheduling proof.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

SysML-style specification review

The review used the official **OMG SysML 2.0, Part 1: Language Specification**, document formal/2026-03-02, March 2026. Section 7 presents representative element/graphical/textual notation tables; section 8.2.3 defines graphical notation in more detail. These serve complementary reading purposes. Official language PDF.

Reviewed clause	Presentation concern	SWAML treatment
8.2.3.5	Namespaces and packages	namespace frame and explicit import reference
8.2.3.6	Definition and usage	mandatory def distinction and occurrence typing
8.2.3.12-14	Ports, connections, interfaces	boundary squares, nominal contracts, explicit routes
8.2.3.17	Actions	owned function flow only
8.2.3.18	States	flat states directly owned by a function
8.2.3.21	Requirements	prose obligations and claimed coverage
8.2.3.26	Views and viewpoints	existing SWAML view families

Visual inspection included printed pages 20, 199, 210 and 232. It confirmed the separation of representative tables from graphical productions and the use of explicit headers and compartments. This is a review of organization, not evidence of language compatibility.

SWAML's normative notation and canonical catalog are original designs. Their coverage includes every grammar production and keyword, while distinguishing glyphs, compartments, labels and text-only syntax. Each concept has reproducible SVG/PNG examples and contextual constraints. No OMG figure or text is copied into those assets. SWAML does not add performers, free actions/events, hierarchical states or other unsupported constructs to resemble another language's diagrams.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

ADR 0001 - Ownership and communication boundaries

Status: accepted for SWAML 0.1. Date: 2026-10-14.

The ownership/authoring decisions in this historical record are superseded for the published 1.2 language by ADR 0007.

The repository owner confirmed that units and components communicate only through interfaces, while local calls and variables can remain internal to a unit. The authoring hierarchy is component → unit → operation, with states directly inside stateful operations. Components may compose other components. Pure functions support local value computation.

An operation is the executable primitive. A contract operation signature and its implementation are separate because the interface must remain reusable across implementations. The handler declaration binds them explicitly. There is no performer or independently allocated action object in the user language.

Events belong to interfaces; publishing statements identify a unit-owned port and member. Each outgoing event/data member has a syntactic emitting operation. Each incoming member has exactly one handler owner. This supplies traceable provenance without pretending to prove runtime reachability.

Consequences: more ports are required when decomposing a component into several units. Shared-memory shortcuts cannot be introduced as “internal implementation” across a modeled unit boundary. Keep tightly coupled helpers inside one unit, or model the shared-state service through a contract.

The view families are General, Interconnect, Case, Sequence, Grid and Flow. Browser provides ownership navigation. Flow View covers operation statement flow and owned-state behavior.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Decision 0002: Complete semantics before claiming complete tooling

Status: accepted for the 0.1.1 draft revision, 2026-10-14.

The specification audit found implementation gaps and ambiguous operational details. The language remains a bounded 0.1 design with direct ownership and interface-only boundaries. This revision keeps the syntax/header and clarifies interpretation while tightening previously incomplete validation.

- A reentrant root and all local helpers use one dispatch-time private-state snapshot and cannot write unit state. Static analysis follows helper calls.
- Initialization installs all handlers/routes before startup dispatch. Lifecycle backlog has an explicit bounded target contract; overflow is observable and stops execution.
- All fanout offers are attempted; a rejected offer cannot undo successful ones. Request timeout wins a response observed at the exact expiry instant.
- State timers start at entry completion and belong to a state-entry generation. Completion is processed before the next external stimulus, with a defined microstep bound.
- Replies infer correlation from one unique unanswered request ancestor. The current grammar has no explicit correlation selector; ambiguous partial traces must be split or ordered rather than guessed.
- Required port status survives every delegation level. Normalized routes reject duplicate leaf pairs.
- Every grammar production has a normative home and a mapping-design entry. These entries do not stand in for an implemented KerML library/exporter.
- Static validity, runtime failures, analysis evidence and IDE work have separate conformance obligations. The tool continues to claim S and partial V honestly.

The stricter lexer rejects raw controls/unknown escapes; Void completions cannot be read; imported names are ambiguous even across target kinds. Models accepted accidentally by 0.1.0 may now receive errors. No valid example from the initial draft requires semantic migration. Keeping the 0.1 header is a draft clarification policy, not permission for silent breaking changes after stabilization.

See issues 1-5 and evidence, the operational contract and normalized model.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Decision 0003: stabilize the language with reproducible execution evidence

Status: accepted for SWAML 1.0.0, 2026-10-14.

The repository is the public language authority. Version 1.0 retains the bounded 0.1 construct set and promotes its clarified rules into explicit typing and operational judgments. The header changes to 1.0 so consumers cannot silently treat this edition as an old draft. Specification and package versions are aligned at 1.0.0; interface versions remain independent.

A small deterministic interpreter makes the language testable through observed behavior. It interprets all supported values, statements and state machines; it resolves the same interface routes checked statically. The execution binding deliberately uses zero local CPU cost and explicit transport delay, resource limits, timeout ties and fault schedules. These assumptions produce useful reproducible traces without claiming hardware scheduling proof or covering every possible schedule.

The full user-facing Markdown/PDF references provide two access paths: an alphabetical concern guide and all grammar keywords in alphabetical order. A machine-checked inventory prevents a new keyword from entering the grammar without a user-facing explanation. Maintained chapters, exact grammar and complete examples are assembled into the edition; generated files are checked for drift.

Browser and diagrams retain the established view vocabulary. Eight rendered examples demonstrate definition/occurrence identity, port-bound exchange, case partial order, grid properties and both FV forms. Their semantic subjects are resolved against real models; static illustrations do not imply that an editor has been implemented.

KerML mapping remains an explicit normative design contract requiring a separately implemented library/exporter and kernel validation before K conformance. The reference execution binding is independently testable without pretending that its Langium AST is a KerML serialization. Future production or editor backends must preserve the defined meaning rather than importing undocumented behavior from their frameworks.

Publication preserves the historical v0.1.0 tag/assets. Each 1.0 work item has a focused issue, resolution commit and acceptance evidence. A release-ready tag and draft package can be reviewed before a separate public release action. Once published, assets must not be silently replaced to change language meaning.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

ADR 0004 - Compact component language

Status: accepted for SWAML 1.1. Date: 2026-10-14.

The ownership/authoring decisions in this historical record are superseded for the published 1.2 language by ADR 0007.

Version 1.1.0 simplifies authoring by retaining `def` for reusable types, removing redundant `system`/`machine` layers, and preserving architecture and simulation requirements. This is an intentional incompatible syntax migration, despite using a minor version; no backward-compatible parser claim is made.

Decision

A component usage can be the selected execution root. `component Demo { … }` supplies inline composition; `component Demo: Assembly;` instantiates a reusable component definition. Roots may contain units directly. Omitted profile means service. A typed root may add nodes/deployment in a binding body, but cannot extend topology. Definition identity, root occurrence identity and child occurrence paths remain distinct.

Operations own states directly. A state body starts with `initial`, followed by states and transitions, so parsing can distinguish it from a statement block. `StateBody` is syntax grouping and normalization contributes to the owning operation. No independent machine, action or performer allocation is introduced.

The language header is removed. The CLI/parser is pinned to 1.1; external hosts must pin an explicit supported edition before loading source. A file does not silently choose the latest parser, and no unimplemented configuration format is presented as language syntax.

No-result operation/signature annotations may be omitted. Assignment, local pure function calls, arrays and enum values use familiar punctuation without `set`, `compute`, `array` or `enumValue`. Enum resolution is lexical-first; qualification selects the nominal package enum explicitly. Functions retain explicit non-Void results, and reusable component/unit declarations retain `def`.

Consequences

The reserved vocabulary decreases from 129 to 122 words. This count includes primitive types, time units, role names, view kinds and policy values; it is not the number of independent modeling concepts. Further removal needs evidence of a redundant meaning, not only a lower token count. Explicit QoS, timeouts, interface identity, port ownership and boundary routing continue to express necessary architectural choices.

The specification, grammar, validators, normalized mapping, simulator, examples, keyword index and graphical illustrations must describe the same 1.1 edition. Old release assets remain immutable. The migration guide documents changes to source and host API. The specification defines every supported construct and its simulation contract; graphical tooling and KerML export have their own implementation and conformance obligations.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

ADR 0005 – Canonical graphical notation catalog

Status: accepted for notation edition 1.1.1; language version 1.1 unchanged. Date: 2026-10-14.

The graphical contract previously defined the seven view families and supplied eight worked illustrations. A complete language specification also needs concept-level reference notation: an author or implementer must be able to find how any supported declaration, relationship, behavior or property is drawn and how it relates to source.

Decision

Use an original concept catalog organized by concern, with element/graphical/textual notation tables and an alphabetical term index. A machine-readable inventory explicitly covers every grammar production and keyword. Each concept has canonical SVG, PNG and scene artifacts, context, rules and an accessible description.

Distinguish semantic nodes and edges from compartments, labels, projections and text-only source contributions. A duration unit or policy value does not require a new independent glyph. All canonical drawings use monochrome line art with explicit names and labels. Reproduction checks compare SVG/scene content and decoded PNG pixels.

The normative chapter defines construction, direction, ownership, identity, editing and accessibility requirements; the inventory and catalog specialize those rules into concrete examples. Worked architectures remain available separately. Future graphical hosts must preserve one semantic model with source, rather than treating the image catalog as an alternate executable representation.

Consequences

The notation edition is a publication refinement. It changes no language grammar, model semantics, parser version or simulation profile. Historical 1.1.0 artifacts remain available. SysML's specification organization informs the presentation, while SWAML drawings, text and software-specific constraints remain independently authored. No graphical conformance or shipped-editor claim follows from producing these static assets.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

ADR 0006 - Reusable QoS and flat architecture views

Status: accepted for SWAML 1.2. Date: 2026-10-16.

Repeated connector policies obscure small software architectures. Reusable policies simplify authoring; flat graphics and an interactive React Flow showcase make the architecture easier to inspect. The showcase supports GitHub Pages deployment from the language repository.

Decision

Add package-owned `qos def Name { ... }` declarations. A connector explicitly selects either `qos Name` or its existing inline `qos { ... }` block. Policy names follow the ordinary package/import/qualified-name and ambiguity rules. Every policy body includes delivery, ordering, positive capacity and overflow in the existing order, with optional latency and age bounds. Policies cannot extend other policies, override fields or provide implicit defaults.

Resolution elaborates the selected body into each connector's effective policy. Sharing a policy never shares queues: queue identity remains connector occurrence, direction and interface member. Sampled buffering, idempotent RPC and realtime completeness are checked at each use. Declaration values and durations are checked even when a policy is unused. An unresolved policy is an error, never a fallback to another policy or defaults.

The headerless parser/host now pins language 1.2 and rejects requests for unsupported editions. Existing valid 1.1 source remains valid under 1.2; hosts must deliberately update the pin. The reference simulation profile remains `swaml-reference/1.1`, because this authoring elaboration does not change scheduling or delivery semantics. The normalized model preserves policy declaration identity, reference provenance and effective per-connector values.

All maintained graphics use flat two-dimensional geometry. Execution locations use front-facing frames instead of cuboids. Browser, gv, iv, cv, sv, grv and operation-owned flow/state fv remain the complete surfaces. Layout may change; semantic ownership and typed routes may not.

The static React Flow showcase is hosted from `showcase/` and demonstrates checked-in models. Selection, filtering and dragging change presentation only. It is not a graphical source editor or execution backend. GitHub Pages deployment is prepared through a repository workflow; public publication remains a separately authorized action.

Consequences

The vocabulary remains unchanged: `qos` and `def` are existing keywords. The concrete grammar adds `QosPolicy` and `QosBody`; inventories, normalization, validation, simulation, tests and reference material account for both. Inline models need no source migration. See migration to 1.2 and release evidence.

Historical Markdown/PDF editions remain unchanged. The new edition includes regenerated flat assets and independently checked publication artifacts. The core still does not claim production timing proof, a KerML exporter or graphical editing conformance.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

ADR 0007 - Components, functions and calculations

Status: accepted for SWAML 1.2.0. Date: 2026-10-16.

ADR 0010 records the 1.2.1 relation-syntax correction and its migration.

The software vocabulary uses nested components, functions for executable behavior, calculations for pure values and deployment references to an external system model. This decision supersedes the component/unit split and operation terminology in ADR 0001 and the corresponding restrictions in ADR 0004. Historical editions retain their original syntax and semantics.

Ownership

A component is the software boundary for private state, typed ports and functions. It may also contain child components, connectors and delegation. Leaf and composite components use the same rules. Each instance has independent state and activation locks; containment does not permit direct access to a child's private values or functions. Local calls stay inside the owning component, while every cross-component exchange follows a typed port/interface/member route.

An executable `function` owns statements or directly owned states. Interface `function` signatures remain separate contracts whose handlers bind explicitly. As refined by ADR 0008, calculations belong only to functions. A `calc` declares typed `in` inputs and one named non-void `return` expression. Calculations use copied inputs and other local calculations; they cannot read mutable state, communicate or call executable functions. The separation makes guards and value expressions effect-free without requiring an architectural participant for every calculation.

The input/result presentation draws on the calculation concepts in SysML 2.0, clause 7.19. SWAML's calculation is a deliberately bounded, original language rule; the similar vocabulary does not establish SysML syntax compatibility or kernel conformance.

Reuse

`component def` defines a reusable type, and `component name: Type;` creates one concrete instance. `specializes Base` permits one base definition. `redefines` explicitly replaces an inherited member with a compatible member of the same category. Specialization and effective containment must remain acyclic. Local references in inherited behavior resolve against the effective specialized component, while every instance receives distinct storage.

Compatibility preserves ports, private value types, function signatures, triggers, concurrency and body category. Function timing cannot be weakened. A redefined child may use its original type or a specialization. Connectors and delegations retain endpoint paths. Unmarked inherited-name collisions are errors. This is controlled architectural reuse; a separate class construct, dynamic creation/destruction, reference payloads and multiple inheritance are outside the contract.

Deployment and types

`deploy controller on "sysml:Vehicle::ecu";` links a component instance to an external system/part identity. General `system:` identities are also supported. SWAML validates the local path and reference form; a host resolves the external model and establishes platform suitability. The core does not define hardware nodes or claim an implemented SysML importer/exporter.

The value vocabulary is bool, int, uint, byte, real, string, result-only void, nominal records/enums and fixed arrays. byte and uint have checked bounded integral domains. Numeric conversion must preserve values or report a range failure; it must not wrap or truncate silently.

Migration and evidence

This change intentionally breaks earlier 1.2 candidate and 1.1 source syntax. Update source, model fixtures, public guides, diagrams, keyword/mapping inventories and the reference simulation profile together. The selected language edition remains 1.2 because this is an unpublished candidate revision, not a silent change to a published edition. Profile `swaml-reference/1.2` records the new execution representation and function-based harness inputs. The migration guide supplies the conversion steps.

Acceptance requires parser and semantic negative cases, inherited behavior and independent-store traces, calculation purity, numeric bounds, external deployment checks, reproducible figures and current documentation fixtures. Source reuse, a bounded simulator run and an external identity string make no claim of production schedulability, universal verification or cross-language conformance.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

ADR 0008 - Function-local calculations and one call syntax

Status: accepted for SWAML 1.2.0.

Calculations are local to functions. Local and routed calls share `call`, while private and local mutable values share `var`. This refines ADR 0007.

Components own private data, ports, children and executable functions. Each function declares its calculations first, followed by flow or states. A calculation resolves only in its enclosing function, including state guards and action blocks. Its expression sees only explicit copied inputs, constants and sibling calculations. It cannot capture mutable data or call executable behavior. Redefining a function replaces the complete body and calculation scope; calculations have no independent specialization or redefinition.

Local flow functions may return values. `call helper(args) as result;` binds a copied, typechecked value to a fresh caller local. void local calls omit the binding. The helper shares the caller's activation lock and snapshot, and a failure unwinds the local call stack without undoing earlier effects. Triggered and stateful functions cannot be called locally. Inherited calls select the effective function on the concrete component instance; calculation references remain lexical to the selected function.

`call port.member(args) as result within D otherwise { … };` preserves explicit typed RPC routes, correlation, timeout and failure handling. Case RPC steps also use `call`. Local and routed calls share one keyword while retaining distinct AST nodes and execution rules. Executable functions cannot appear in expressions.

`var` declares both private component storage and mutable block locals. Ownership determines lifetime and visibility. Live-value shadowing remains forbidden. The retired `invoke`, `request` and `let` spellings require migration and receive no compatibility fallback.

The reference parser, validator and simulator must cover result copying/ranges, lexical calculation isolation, inherited dispatch, state guards, malformed inputs and unchanged routing/failure behavior. This change adds no dynamic object construction, implicit cross-component access, production scheduling proof or external-language conformance.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

ADR 0010 - Consistent subject relations

Status: accepted for SWAML 1.2.1. Date: 2026-10-18.

Subject-assignment relations consistently use `of`. The 1.2.0 grammar used case `for`, satisfaction `by` and compact-view `of` for analogous references. This patch corrects those spellings without changing their semantic targets, ownership or execution. Any future use of `for` is limited to loop syntax.

Decision

Use `useCase Name of Root { ... }`, `verificationCase Name of Root { ... }`, `satisfy Requirement of Design.path;` and `view Name iv of Subject;`. View blocks continue to use `include` to select multiple subjects. A case's root context is a reference; the package still owns the case. Satisfaction still points from the design element to the requirement. Imports, qualified targets, duplicate checks and occurrence identity are unchanged.

The remaining relation vocabulary denotes different concepts: `on` selects an activation stimulus or deployment location; `as` binds a call result or completion; `:` denotes typing/header structure; `=` initializes or assigns values; arrows denote direction; `via` selects a route; `after` denotes a causal predecessor or timer. These constructs do not receive a general-purpose `of` alias.

The only current loop is bounded `while (...) bound n`. There is no `for` loop production in 1.2.1. The word `for` has no relation meaning; any future language use must be a separately specified loop construct. Removing `for` and `by` from current keyword coverage reflects the actual grammar, not an implementation of new loop syntax.

Migration and version selection

Version 1.2.1 intentionally changes source compatibility after the 1.2.0 publication. The model meaning is preserved, but the parser rejects old case `for` and satisfaction `by` forms. Migration is contextual: change those delimiters to `of`, while preserving strings, ordinary prose, triggers, deployment locations and result bindings.

Hosts currently select the 1.2 language family. The package/release revision determines its concrete parser, so reproducible consumers must pin both. Use the archived 1.2.0 release to reproduce its earlier grammar. Published 1.2.0 Markdown, PDF, manifest, checksum, tag and release assets remain preserved. New artifacts have 1.2.1 filenames.

Audit and evidence

The audit covers grammar keywords, semantic reference roles, keyword and notation inventories, current examples, source-backed showcase projections and maintained guides. It also corrects outdated descriptions of requirements, root-owned case endpoints, view selection unions and the published release status. Regressions cover positive `of` references, imported/qualified roots and requirements, alias duplicate satisfaction, rejected old spellings and malformed input. Existing simulations check that execution traces and stores remain unchanged.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Requirements, documentation and views

Capture an obligation as a requirement. Add only the attributes you need, write its formal specification in `doc`, and connect the design and verification intent explicitly.

SWAML

```
package Example {
    requirement Ready {
        doc /* The application shall initialize its result to 2. */
        attribute id: string = "REQ-001";
        attribute priority: uint = 1;
    }
    component App {
        doc /* Initialize application state. */
        // This value belongs to App.
        var result: int = 0;
        function initialize() on start {
            result = 2;
            assert result == 2;
        }
    }
    satisfy Ready of App;
    useCase Start of App {
        doc /* A user starts the application. */
    }
    verificationCase CheckStartup of App {
        doc /* Run startup and inspect the final result: it must equal 2. */
        verify Ready;
    }
    view coverage gv {
        include Ready;
        include App;
        include Start;
        include CheckStartup;
    }
    view behavior fv {
        include App recursive where kind == "function" and name == "initialize";
    }
}
```

Concept	What it records	How it is displayed
requirement	An obligation with typed attributes and optional formal text	Flat rectangle with attributes and doc compartments; one row in a requirements table
doc	One formal specification owned by a declaration	A labeled doc compartment or a doc table column
ordinary comment	One line or block of informal source explanation	A plain rectangular note inside its owner, or joined to it by a plain dashed line
satisfy	A design element claims to fulfill a requirement	Dashed arrow from the design element to the requirement, labeled satisfy
useCase	Intended use in a selected root context	Case frame labeled useCase, with optional routed exchanges

Concept	What it records	How it is displayed
verificationCase	A verification procedure or scenario	Case frame labeled verificationCase; verify arrows point to requirements

`doc /* ... */` preserves multiline text literally; string escape sequences are not interpreted. Write it first inside a declaration body. For a leaf such as a port, put it before the semicolon: `port input: Values receiver doc /* Receives the sensor sample. */;`. A declaration can have one doc and any number of ordinary comments. Each `//` line and each `/* ... */` block is separate. Blocks are not nested. A word `doc` inside an ordinary comment or string stays ordinary text.

Attributes are immutable metadata, not variables. They use the same value types as the software model, including records, enums and fixed arrays. Their expressions cannot read private data, call functions or capture other attributes. Requirements have no mandatory subject, identifier attribute, priority or verification method. `priority` is allowed as an attribute name even though it is also a transition keyword.

Use `satisfy` for the design and `verify` for verification intent. A use case cannot verify a requirement. A verification case may describe a manual inspection in its doc, a route-valid sequence of exchanges, or both. These declarations do not run a test or prove the requirement. Record actual simulation, test, analysis or review results separately with the model and tool version that produced them.

A compact view remains valid: `view wiring iv of App;`. A view block combines `include` entries. Each entry selects its subject, optionally its descendants with `recursive`, and optionally filters those candidates with `where`. Predicates use `name`, `kind` or `path`, string/Boolean literals, equality/inequality and Boolean combinations. Filters do not prune traversal: excluding a parent by name does not hide matching descendants. Repeated selections are deduplicated by occurrence path, in first-selection order. An empty result is valid; an empty block is not.

Common kinds are `component`, `function`, `requirement`, `useCase`, `verificationCase`, `port`, `interface`, `variable`, `calculation` and `state`. Names and paths are exact and case-sensitive. All selections resolve in the view's package. Each selected element must fit the view kind. A view never selects another view.

See the complete example, normative rules, graphical catalog and interactive showcase.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

ADR 0009 - Requirements, documentation and selected views

Status: accepted for SWAML 1.2.0. Date: 2026-10-16.

The relation spellings in this historical decision are superseded in 1.2.1 by ADR 0010: current satisfaction and case contexts use `of`.

Requirements have no subject. Declarations support one formal documentation block, separate lexical comments, two case kinds and composable view selections. This decision replaces the earlier string-only requirement, generic case and case-to-requirement satisfies clause.

`requirement` owns immutable typed `attribute` values and optional `doc /* ... */` text. It has no subject, behavior or private runtime store. `satisfy Requirement by Design.path;` records a design claim. `verificationCase` owns `verify Requirement;` links; `useCase` describes intended use. Neither relation establishes evidence by itself. Both case kinds share the existing routed, partially ordered exchange semantics.

Formal documentation is raw multiline text, attached to its declaration, with at most one block per owner. Ordinary line and block comments remain separate lexical annotations. They have no declared identity or executable effect. The source projection retains every comment and its owner; graphical notes and formal documentation compartments remain distinct.

The lexer consumes `doc` and its block as one DOC_COMMENT token. This makes multiline documentation unambiguous with hidden ordinary comments and preserves text exactly. The keyword inventory explicitly includes this lexical keyword. Unterminated blocks are syntax errors. The grammar identifies every permitted owner and position; body-bearing declarations put documentation first, leaf declarations put it before the terminating semicolon, and transitions use a prefix.

All reserved words use lower camel case. Primitive type words are `bool`, `byte`, `uint`, `int`, `real`, `string`, `void`; view kinds are `gv`, `iv`, `cv`, `sv`, `grv`, `fv`. Identifiers remain case-sensitive and may use capitals. Browser is a navigation surface, not a source keyword.

View blocks contain one or more `include` selections. Optional `recursive` expansion preserves occurrence paths; optional `where` predicates combine typed `name`, `kind` and `path` comparisons with `and`, `or`, `not` and parentheses. Selections form a stable union. Filters select model elements; they cannot execute model behavior or introduce communication routes.

The conceptual comparison is with the official SysML 2.0 language. SWAML deliberately defines its own smaller syntax and semantics. This release does not add SysML interchange or kernel validation.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

PDF specification and release publication

The complete PDF contains the normative chapters, flat notation, grammar, mapping inventory, complete example models, the notation fixture and project guidance. It includes clickable contents, bookmarks, links, vector figures, selectable numbered code, repeated table headings and embedded sources. This edition also includes the ontology and view guide, migration notes, release notes and control-flow and timing decision.

Version identifiers

This edition is specification/package **1.3.0**, with a pinned **1.3** parser and headerless source. The full Markdown reference and PDF are generated from maintained chapters. The PDF additionally includes project guidance, a complete mapping inventory and embedded source/figure files. Keyword navigation covers every current reserved word; the concern guide is alphabetical.

The v1.0.0, v1.1.0 and v1.1.1 Markdown/PDF editions retain their historical syntax, semantics and inventories. The copies distributed in this repository received an editorial correction on 2026-10-16 to make tooling guidance platform-neutral and describe views directly. Their regenerated manifests identify that correction and the previous PDF digest; source attachments match each corrected historical edition. Existing Git tags and remotely published release assets are unchanged. Current publication checks target 1.3 sources and write new artifacts under a versioned filename.

Historical v0.1.0 packages draft/core revision 0.1.1 with header 0.1 and has the same editorial correction to its guidance. The current reference simulation profile is **1.3**: unified components, executable functions, pure calculations and effective inherited members are interpreted with separate instance stores. Historical embedded traces retain their original profile identity. Physical timing proof, KerML export and a source-editing graphical workbench remain separate implementations.

The published v1.2.0 and v1.2.1 consolidated Markdown, PDF, manifest, checksum and release assets retain their original case and satisfaction spellings. Current generators write the separate 1.3.0 edition.

Rebuild the book

Use Python 3.10+ and the pinned dependencies below. Install DejaVu fonts (`fonts-dejavu-core` on Debian/Ubuntu), or supply a directory containing DejaVu Sans regular/bold/oblique/bold-oblique and Sans Mono TTF files with `--font-dir` or `SWAML_PDF_FONT_DIR`. The two regular/bold fonts in `resources/fonts` pin notation rendering; a full PDF build additionally requires the oblique and monospace faces. All source and scene files are read as UTF-8 on every platform.

SH

```sh
python3 -m venv .venv-pdf
. .venv-pdf/bin/activate
python3 -m pip install -r scripts/pdf-requirements.txt
python3 scripts/build-spec-pdf.py --font-dir /usr/share/fonts/truetype/dejavu
python3 scripts/check-spec-pdf.py
```

On Windows, activate `.venv-pdf\Scripts\Activate.ps1` in PowerShell. If the supplied Python runtime cannot create a virtual environment, install into a repository-local target and set `PYTHONPATH` for the build:

POWERSHELL

```powershell
New-Item -ItemType Directory -Force .venv-pdf/tmp | Out-Null
$env:TEMP = (Resolve-Path .venv-pdf/tmp).Path
$env:TMP = $env:TEMP
```

POWERSHELL / CONTINUED

```
python -m pip install --no-cache-dir --target .venv-pdf/packages -r scripts/pdf-requirements.txt
$env:PYTHONPATH = (Resolve-Path .venv-pdf/packages).Path
python scripts/build-spec-pdf.py --font-dir path/to/dejavu/ttf
python scripts/check-spec-pdf.py
```

First run `python3 scripts/build-graphical-examples.py`, `python3 scripts/build-notation.py`, `node scripts/build-traceability-demo.mjs` and `npm run docs:build` after changing source chapters or figures. The explicit source list is built directly; unsupported Markdown fails the build. Mermaid illustrations become vector figures. The concept catalog and eight worked examples share SVG/vector-scene sources across Markdown/PDF. Canonical PNGs are exported from the same geometry with Pillow and the pinned DejaVu Sans files in `resources/fonts` (with their separate license); check exports with `scripts/build-notation.py --check`. Poppler is used for final PDF visual inspection. Keyword links jump to individual entries. Every grammar line is preserved; wrapped continuations have no new line number. Attachments preserve exact source bytes.

The graphical ontology and modeling walkthrough use `spec/modeling-overview.json` for both the website and static diagrams. Regenerate their SVG/vector scenes with `python3 scripts/build-modeling-overview.py`, and verify reproducibility with its `--check` option. The PDF includes the overview guide, diagram inventory and exact scene/SVG attachments.

Outputs are the PDF, a JSON source/hash manifest and a SHA-256 checksum under `output/pdf/`. The checksum identifies the generated artifact; PDF metadata may vary between builds. The manifest allows readers to verify exactly which source content an edition contains.

Quality and release gate

Run `npm run check`, build the PDF, and run the PDF checker. Render all pages with Poppler and inspect them for clipped text, overlapping elements, unreadable tables, broken figures, orphan headings and excessive blank pages. Check dense tables and code at full resolution as well as page contact sheets. Automated extraction cannot replace that visual review.

Create the downloadable SVG/PNG bundle with `python3 scripts/package-notation.py --output /tmp/SWAML-1.3.0-Graphical-Assets.zip`. It contains all canonical and worked assets, the notation inventory, license and per-file checksums.

Commit the reviewed source, builder and final artifacts together. The release must target the reviewed commit, with the PDF, source manifest and checksum attached. Use `--notes-file` for release notes and verify the published tag and assets after upload. Preserve release assets once published; issue a new documented edition for corrections.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Contributing to SWAML

SWAML is developed as a language: specification, grammar, semantics, diagnostics and examples must evolve together. The 1.3 syntax keeps explicit `def` declarations and distinguishes them from concrete component usages; named QoS policies reuse complete values while connector queues stay independent. Source files are headerless; a host must pin language 1.3. Read the 1.3 migration when updating hosts and the historical 1.1 migration when converting 1.0 models.

Start with the language specification, ownership and reuse decision and implementation status.

Development setup

Use Node.js 22+ and npm 10.2.3+:

```sh
npm ci
npm run check
```

This generates the AST/TextMate files, compiles TypeScript, runs the test suite, validates the examples and checks documentation. Generated files, dependencies and compiled output are not committed. The dependency lockfile is committed.

Propose a language change

Describe the actual modeling problem and a small before/after model. Identify the affected grammar productions, normative clauses, KerML representation, validation rules, view projections and migration behavior. Distinguish a new language feature from a new target-platform binding.

A proposal that adds cross-component communication must identify the port/interface/member route. A proposal for an event must identify its declaration, emitting function, receiving owner, delivery policy and failure behavior. A proposal for state or control flow must define ordering, concurrency and termination. Avoid accepting syntax before its semantics exists.

Published syntax changes require an explicit release revision and migration notes. The 1.2.1 correction replaces case for and satisfaction by with of; it deliberately rejects the old spellings, as recorded in ADR 0010 and the migration guide. The current host selects language 1.3; pin its package revision and follow the 1.3 migration guide. Published editions remain archived. Additive keywords can still break existing identifiers; document that impact. Record substantial decisions under `docs/decisions/`.

Implement and test

1. Update the normative prose and grammar together.
2. Update the grammar mapping inventory and classify each obligation as static, runtime, analysis, kernel or graphical. Every rule/type needs a normative home. Regenerate with `npm run langium:generate`; update scoping and validators as needed.
3. Add a valid model and a negative regression that would have caught the original issue. Cross-file changes need imported/qualified and ambiguous-name cases.
4. Update the keyword inventory, user guide, affected examples, simulation configurations, graphical projections, implementation matrix and changelog. Regenerate the complete reference with `npm run docs:build`.
5. Run `npm run check` and report the result in the pull request.

Tests must verify observable language behavior, not copy the implementation's internal algorithm. Preserve malformed-input recovery: editor diagnostics must not crash the language server. New communication features need tests that attempts to bypass interfaces are rejected.

Pull requests and review

Use a focused branch and describe the user-visible modeling behavior. Include a concise syntax/semantic example, the affected specification clauses, validation evidence and any remaining conformance gap. Do not describe a proposed feature as implemented merely because the grammar accepts it.

Reviewers should check that definitions and instances remain distinct, private data cannot escape through paths, all exchanges remain typed, and diagrams do not invent semantics. Compatibility or migration decisions should be explicit before merging.

Documentation and standards

State behavior, changes and technical rationale directly. Cite relevant tracked tickets by number; omit personal attribution for decisions.

Link to primary sources for external standards. Write original explanations; do not copy standard PDFs or proprietary specification text into the repository. Preserve the existing MIT license for original code and documentation and retain any applicable third-party notices.

Executable `swaml` code fences must be complete, valid models: the documentation checker parses and validates them. Use inline snippets or `text` fences for deliberate fragments, and put intentionally invalid models under `examples/invalid/` with their expected diagnostic.

Resolve specification issues

Track a concrete contradiction, missing semantic choice or validator defect as a GitHub issue with acceptance criteria. Resolve it with matching normative text, implementation where in scope, examples and regression evidence. Close the issue only when its stated criteria are met; do not close runtime/editor implementation work because a specification paragraph exists. Link the resolution commit or merged pull request and record any remaining conformance limitation.

Report a defect

Include the language/core version, a minimal model, exact CLI command or editor action, observed diagnostics and expected rule. For graphical work include the view ID and semantic subject path. For timing/delivery concerns state the target assumptions; a compiler acceptance result alone is not execution evidence.

Do not include credentials, private service URLs or production payloads in examples. Replace them with minimal synthetic values.

Graphical notation changes

Update the graphical contract and `spec/notation.json` together. A concept must identify its semantic owner, required labels, shape or edge grammar, textual example and applicable views. Classify syntax that has no independent glyph explicitly. Generate original SVG and PNG representations from the shared scene geometry with `scripts/build-notation.py`; verify coverage and exports before editing the full publication. Read the presentation review for the adopted document style.

Publication changes

The source chapters and `spec/keywords.json` are maintained inputs. The complete Markdown and keyword reference are generated. Run `npm run docs:build` after editing those inputs. Figure changes require `python3 scripts/build-graphical-examples.py` and semantic subject checks; commit SVG and scene pairs together. Do not silently edit one rendition.

For an edition, follow the PDF workflow: build, verify exact source attachments and hashes, render all pages, and visually inspect figures, code, tables and navigation. Attach the full Markdown, PDF, manifest and checksum to the prepared release. A public release is a separate explicit publication action; a release-ready commit/tag or draft preserves reviewability.

Simulator changes must retain the profile contract or introduce a new named profile/version. Include a trace-level regression and record configuration/source identity. Review timeout ties, partial fanout, stale timers, queue limits and snapshot behavior whenever communication or dispatch changes. Never equate a bounded successful run with physical schedulability or a proof of all possible executions.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Instructions for agents working on SWAML

Read first

- `docs/LANGUAGE_SPECIFICATION.md` and the normative chapter affected by the task.
- `docs/IMPLEMENTATION_STATUS.md` to distinguish specified and implemented behavior.
- `docs/spec/06-execution-contract.md`, `07-normalized-model.md`, `08-conformance-obligations.md` and `09-formal-semantics.md` for execution, mapping and evidence boundaries.
- `docs/decisions/0007-component-functions-and-calculations.md` for the architecture choices.
- `CONTRIBUTING.md` for contribution and validation expectations.

Preserve the language invariants

- Components own child components, private data and executable functions. Functions own calculations and statement flow or states directly. Calculations are declared first and resolve only inside their enclosing function. Both leaf and composite components may own behavior.
- No performers, separate action allocations, global mutable state or free events.
- Named QoS policies reuse complete values only; preserve per-connector occurrence queues and validate every effective use.
- Every exchange between distinct components uses an owned typed port, interface member and explicit connector/delegation route.
- Local calls/data may stay inside a component. Never resolve a function from another component or a calculation from another function as a convenience fallback. Use call for local and routed functions, and var for private and local mutable values.
- Outgoing events/data must be tied to emitting functions; incoming members must have one handler owner. Service responses remain correlated through their interface route.
- Keep Browser, gv, iv, cv, sv, grv and fv. fv covers statement flow and function-owned states.
- Distinguish definition identity from instance paths. Never make same-type instances share private state.

SWAML 1.3 authoring contract

Keep `component def` for reusable types, `specializes` for acyclic single inheritance and `redefines` for compatible explicit inherited-member replacement. A top-level component usage is an execution root; it may contain inline topology or instantiate a component definition with optional deployment bindings. Typed roots cannot extend topology. Every concrete root is checked for closure, independently of reusable library definitions. Omitted root profile means service; omitted function/signature result means void. Calculations use `calc` with typed `in` inputs and exactly one named, explicitly typed non-void `return` expression. They are never component members and cannot read mutable data or call executable functions. Deployment targets are external system/part identity strings resolved by the host; keep system modeling outside the software core.

Files are headerless and interpreted by the explicitly selected 1.3 parser/host. Do not restore `swaml`, `system`, `machine`, `set`, `compute`, `array` or `enumValue` as required markers. States are directly authored inside a function's body; the StateBody AST grouping has no separate executable identity. Preserve the documented ambiguity rules for enum values and local records. Historical tagged syntax requires migration, not silent fallback.

Source and architecture

The normative concrete grammar is `src/language/swaml.langium`. Prose specifies validity and semantics. Update both when meaning changes. Generated files in `src/language/generated/`, TextMate output and `out/` must not be hand-edited or committed.

Keep the language core free of Node filesystem dependencies; CLI/server adapters own platform I/O. Composite paths currently use explicit semantic resolution and need path-aware IDE services. Do not claim generic Langium rename covers them.

KerML mapping is a proposed contract until a real exporter and kernel validation exist. Do not label Langium AST generation as KerML conformance. Likewise do not call an accepted model schedulable, race-free, executable or graphically editable without the corresponding implementation/evidence.

Required checks

For grammar, semantic or source changes run `npm run check`. Use `npm ci` after dependency changes or on a fresh checkout. Add meaningful positive and negative tests for changed language rules, including malformed input when parser recovery is affected. Report failures honestly and fix regressions within the task scope.

For documentation changes run `node scripts/check-docs.mjs` after a build. Complete `swaml` fences are executable fixtures; deliberate fragments should use inline text or a `text` fence.

Working practices

Prefer small, reviewable changes with stable diagnostic codes. Keep every grammar rule/type covered in `spec/kerml-mapping.json`; the documentation checker enforces coverage. Add a decision record for substantial language changes and update the changelog and implementation matrix. Do not silently widen access scopes to make a fixture pass.

Preserve the existing license, unrelated user edits and repository history. Never add credentials or downloaded proprietary standards. Use primary-source links and original analysis. Follow the user's authorization for GitHub changes; local edits alone do not imply permission to publish a release or alter another repository.

Reference execution and publication

Keep the reference simulator portable: file access and source hashing belong to CLI/host adapters. Preserve the documented scheduler, timeout ties, queue identity, copied values, snapshot scope, direct function/state ownership and explicit resource failures. Test observable traces and final stores, including adverse cases. A simulator run is not production scheduling proof or universal verification.

Maintain every current keyword entry in `spec/keywords.json` as the grammar evolves. Regenerate the consolidated Markdown with `npm run docs:build`; do not hand-edit generated `KEYWORD_REFERENCE.md` or `SWAML-1.3.0-Reference.md`. Regenerate vector examples with `python3 scripts/build-graphical-examples.py`. Their scene/SVG sources and model-subject manifest must stay synchronized.

Keep `spec/notation.json` exhaustive over the grammar and keyword inventory. Every graphical concept needs a context, syntax, canonical drawing and constraints; text-only terms need an explicit rationale. Regenerate the catalog and SVG/PNG assets with `scripts/build-notation.py` and run its `--check` mode. Keep all images original and do not import unsupported SysML constructs through notation.

Before publishing an edition, run the complete language gate, figure reproducibility check and PDF build/check described in `docs/PDF_PUBLICATION.md`, then render and inspect every page. Commit synchronized Markdown, PDF, manifest and checksum. Preserve published historical assets. Keep the public specification independent of specific authoring tools.

Requirements and presentation

Use lower camel case keywords, including primitive types and view kinds. Requirements own only immutable typed attributes and optional doc. A declaration owns at most one formal doc; ordinary comments remain separate lexical annotations. satisfy links design to requirements; only verificationCase owns verify. Neither is proof. Views select a stable union of occurrence paths through include/recursive/where; filters never execute behavior. Read ADR 0009 before changing these rules.

Use `of` for case root contexts, satisfaction design sources and compact view subjects. The former `for` and `by` relation spellings require migration; `on` retains activation/location meaning and `as` retains result binding. Read ADR 0010 before changing relation syntax.

Control flow includes bounded while/for/do and copied fixed-array forEach. Interruptible regions require triggered serial flow functions and an owned typed input route; preserve same-activation cancellation, terminal correlation identity, late-reply suppression and committed effects. FV timing is a trace-backed projection with declared contracts shown separately. Read ADR 0011 and the 1.3 migration guide before changing these rules. State changes directly; cite relevant ticket numbers rather than personal attribution.

D. External references

Primary-source and project links cited in the included text. The comparisons identify their research baseline; this list does not imply conformance to any external standard.

1. OMG KerML 1.0
https://www.omg.org/spec/KerML/1.0

2. KerML specification
https://www.omg.org/spec/KerML/1.0/PDF

3. interactive landing page
https://voidaliot.github.io/swaml/#modeling-overview

4. AUTOSAR Classic overview
https://www.autosar.org/standards/classic-platform/

5. AUTOSAR Software Component Template, R24-11
https://www.autosar.org/fileadmin/standards/R24-11/CP/AUTOSAR_CP_TPS_SoftwareComponentTemplate.pdf

6. AUTOSAR Adaptive overview
https://www.autosar.org/standards/adaptive-platform/

7. AUTOSAR Communication Management, R25-11
https://www.autosar.org/fileadmin/standards/R25-11/AP/AUTOSAR_AP_SWS_CommunicationManagement.pdf

8. OMG UML 2.5.1
https://www.omg.org/spec/UML/2.5.1

9. Software Component Template R21-11
https://www.autosar.org/fileadmin/standards/R21-11/CP/AUTOSAR_TPS_SoftwareComponentTemplate.pdf

10. Communication Management R21-11
https://www.autosar.org/fileadmin/standards/R21-11/AP/AUTOSAR_SWS_CommunicationManagement.pdf

11. SysML 2.0 Language, clause 7.19
https://www.omg.org/spec/SysML/2.0/Language/PDF

12. SEI introduction to AADL
https://www.sei.cmu.edu/library/the-architecture-analysis-design-language-aadl-an-introduction/

13. Langium grammar reference
https://langium.org/docs/reference/grammar-language/

14. reference-resolution guide
https://langium.org/docs/learn/workflow/resolve_cross_references/

15. interactive showcase
https://voidaliot.github.io/swaml/#traceability

16. official SysML 2.0 language
https://www.omg.org/spec/SysML/2.0

E. License

MIT License

Copyright (c) 2026 Catalin Ilie

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.